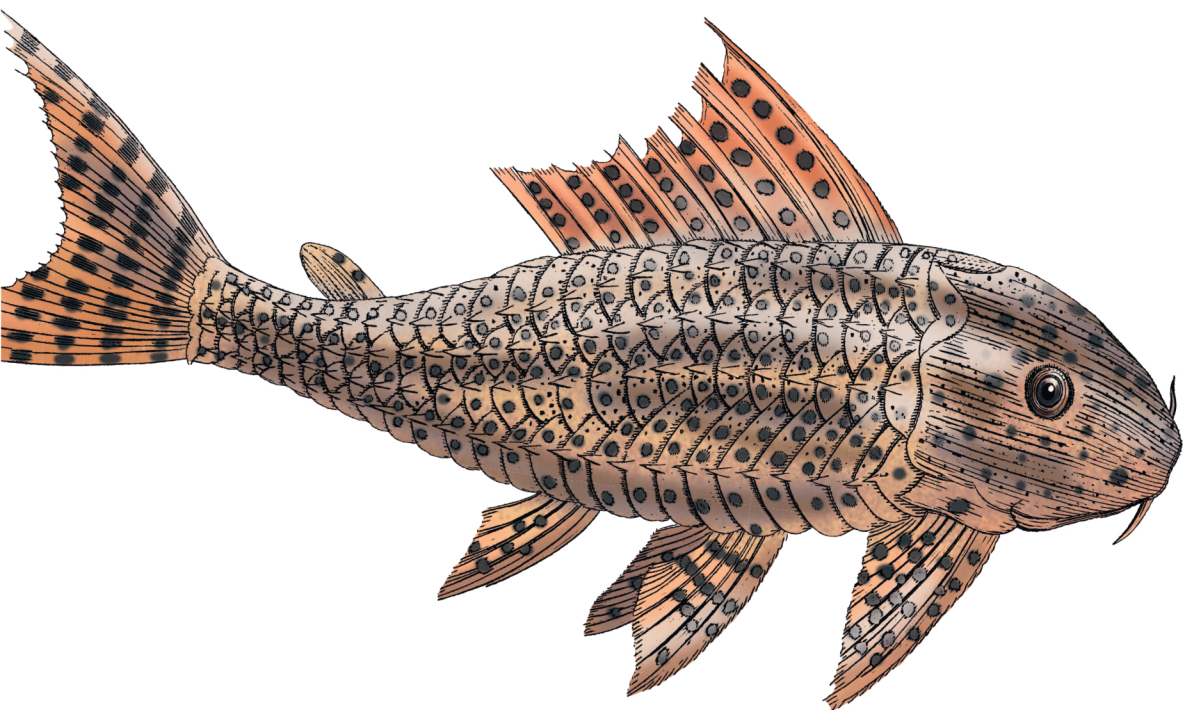


O'REILLY®

Второе
издание

Безопасность контейнеров

Фундаментальный подход к защите
контейнеризированных приложений



SPRINT
book

Лиз Райс

SECOND EDITION

Container Security

*Fundamental Technology Concepts
That Protect Containerized Applications*

Liz Rice

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

ВТОРОЕ ИЗДАНИЕ

Безопасность контейнеров

*Фундаментальный подход к защите
контейнеризированных приложений*

Лиз Райс

ББК 32.988.02-018-07
УДК 004.056.53
Р18

Райс Лиз

Р18 Безопасность контейнеров. Фундаментальный подход к защите контейнеризированных приложений. 2-е изд.. — Астана.: «Спринт Бук», 2026. — 272 с.: ил.
ISBN 978-601-12-8919-1

Контейнеризированные и облачные приложения постепенно становятся базой современной программной инфраструктуры. Глубокое концептуальное понимание их безопасности — задача номер один. Второе издание книги «Безопасность контейнеров» — это строгий, но при этом практический анализ технологий, лежащих в основе контейнерных платформ. В книге разработчики, администраторы и специалисты по безопасности найдут набор моделей для оценки рисков и повышения надежности систем, а также конкретные рекомендации по их применению.

Лиз Райс, признанный эксперт в области безопасности облачных приложений, основывается на фундаментальных принципах, описанных в первом издании, и одновременно учитывает изменения в сфере угроз, современные инструменты и эволюцию таких платформ, как Kubernetes и Linux. Получите твердое понимание архитектурных компонентов контейнеров и поддерживаемых ими примитивов Linux, что поможет системно оценивать угрозы и разрабатывать эффективные стратегии их смягчения.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.988.02-018-07
УДК 004.056.53

Права на издание получены по соглашению с O'Reilly. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 979-8341627703 англ.

Authorized Russian translation of the English edition of Container Security:
2nd Edition ISBN 9798341627703 © 2026 Vertical Shift Ltd.

ISBN 978-601-12-8919-1

This translation is published and sold by permission of O'Reilly Media, Inc.,
which owns or controls all rights to publish and sell the same.

© Перевод на русский язык ТОО «Спринт Бук», 2026

© Издание на русском языке, оформление ТОО «Спринт Бук», 2026

Краткое содержание

Предисловие	12
Глава 1. Угрозы безопасности контейнеров.....	20
Глава 2. Системные вызовы Linux, права доступа и привилегии.....	34
Глава 3. Контрольные группы.....	45
Глава 4. Изоляция контейнеров	53
Глава 5. Виртуальные машины	81
Глава 6. Образы контейнеров.....	91
Глава 7. Безопасность цепочки поставок	107
Глава 8. Программные уязвимости в образах контейнеров	129
Глава 9. Инфраструктура как код и GitOps	148
Глава 10. Усиление изоляции контейнеров	156
Глава 11. Нарушение изоляции контейнеров.....	169
Глава 12. Сетевая безопасность контейнеров.....	190
Глава 13. Защищенное соединение между компонентами	211
Глава 14. Передача в контейнеры секретных данных.....	228
Глава 15. Защита среды выполнения контейнеров.....	238
Глава 16. Контейнеры и десять главных рисков по версии OWASP.....	259
Заключение	265
Приложение. Чек-лист по безопасности.....	266
Об авторе	269
Иллюстрация на обложке.....	270

Оглавление

Предисловие	12
Для кого эта книга.....	13
Структура издания.....	14
Что нового во втором издании	15
Примечание относительно Kubernetes.....	15
Примеры	16
Запуск контейнеров.....	16
Благодарности.....	17
Обратная связь.....	18
Условные обозначения.....	18
Использование примеров кода	18
От издательства	19
О научных редакторах русскоязычного издания	19
Глава 1. Угрозы безопасности контейнеров.....	20
Риски, угрозы и уменьшение их последствий.....	21
Модель угроз для контейнеров	22
Границы зон безопасности.....	26
Мультиарендность	27
Совместно используемые машины	28
Виртуализация	28
Мультиарендность контейнеров.....	30
Экземпляры контейнеров.....	31
Принципы безопасности	31
Минимум привилегий.....	31
Многоуровневая защита	32
Минимальная поверхность атаки	32
Ограничение радиуса поражения	32
Разделение обязанностей	32
Реализация принципов безопасности с помощью контейнеров.....	32
Резюме	33
Глава 2. Системные вызовы Linux, права доступа и привилегии.....	34
Системные вызовы	34
Права доступа к файлам	36
Биты setuid и setgid.....	37
Влияние бита setuid на безопасность.....	40
Привилегии Linux.....	41
Повышение привилегий.....	43
Резюме	44

Глава 3. Контрольные группы	45
Контроллеры контрольных групп	46
Создание и настройка контрольных групп	47
Привязка процесса к контрольной группе	48
Контрольные группы для контейнеров	49
Предотвращение взрыва fork-бомбы	50
Резюме	52
Глава 4. Изоляция контейнеров	53
Пространства имен Linux	54
Изоляция имени хоста	55
Изоляция идентификаторов процессов	57
Изменение корневого каталога	60
Сочетание возможностей пространств имен и изменения корневого каталога	63
Пространство имен монтирования	64
Пространство имен сети	66
Пространство имен пользователей	69
Пространство имен обмена информацией между процессами	72
Пространство имен контрольных групп	73
Пространство имен времени	75
Поды и пространства имен контейнеров в Kubernetes	76
Процессы контейнера с точки зрения хоста	76
Хосты и контейнеры	78
Резюме	79
Глава 5. Виртуальные машины	81
Загрузка компьютера	81
Знакомство с VMM	83
VMM Type 1 (гипервизоры)	83
VMM Type 2	84
Виртуальные машины, работающие в ядре	85
Перехват и эмуляция	85
Обработка инструкций, не поддающихся виртуализации	86
Вложенная виртуализация	87
KubeVirt	87
Изоляция процессов и безопасность	88
Недостатки виртуальных машин	89
Изоляция контейнеров по сравнению с изоляцией виртуальных машин	90
Резюме	90
Глава 6. Образы контейнеров	91
Корневая файловая система и конфигурация образов контейнеров	91
Переопределение настроек в среде выполнения	92
Стандарты OCI	93
Конфигурация образа	95

Сборка образов.....	96
Опасности команды docker build	96
Слои образов	98
Мультиплатформенные образы	102
Хранение образов	103
Создание собственного реестра	103
Сохранение и извлечение	104
Идентификация образов	104
Резюме	106
Глава 7. Безопасность цепочки поставок	107
Программные компоненты в образах контейнеров.....	108
SLSA	109
Спецификация программного обеспечения	110
Минимальные базовые образы.....	112
Безопасность Dockerfile	113
Происхождение Dockerfile	113
Практические рекомендации по безопасности Dockerfile	113
Атаки на машину сборки.....	117
Генерирование SBOM.....	118
Подписывание образов и программных компонентов.....	119
Создание аттестатов	120
Манифесты образов	122
Безопасность развертывания образов.....	125
Развертывание правильного образа	125
Вредоносная конфигурация развертывания системы.....	126
Проверка подлинности подписи и происхождения образа	126
Контроль допуска	127
Резюме	127
Глава 8. Программные уязвимости в образах контейнеров	129
Исследования уязвимостей	129
Уязвимости, исправления и дистрибутивы.....	131
Уязвимости уровня приложения	132
Управление рисками, связанными с уязвимостями	132
Сканирование на уязвимости	133
Установленные пакеты	134
Сканирование образов контейнеров.....	135
Неизменяемые контейнеры.....	136
Регулярное сканирование.....	137
Средства сканирования.....	137
Источники информации	138
Устаревшие источники	138
Не все уязвимости исправляются	139
Передача VEX в сканеры.....	139
Уязвимости подпакетов	139

Различия названий пакетов	139
Статически скомпонованные исполняемые файлы.....	140
Сканирование мультиплатформенных образов	140
Дополнительные возможности сканирования.....	141
Ошибки сканеров	141
Сканирование в конвейере CI/CD	142
Предотвращение запуска образов с уязвимостями.....	144
Обновление образов.....	144
Уязвимости нулевого дня	146
Резюме	147
Глава 9. Инфраструктура как код и GitOps	148
IaC	148
GitOps.....	149
Следствия для безопасности развертывания	151
Рекомендации по обеспечению безопасности GitOps.....	153
Резюме	155
Глава 10. Усиление изоляции контейнеров	156
Механизм seccomp	157
Модуль AppArmor	159
Модуль SELinux	160
Песочница gVisor.....	162
Среда выполнения контейнеров Kata Containers	165
Легковесные микровиртуальные машины	165
Unikernels	167
Резюме	168
Глава 11. Нарушение изоляции контейнеров.....	169
Выполнение контейнеров по умолчанию от имени суперпользователя.....	169
Переопределение идентификатора пользователя.....	171
Без новых привилегий.....	172
Требование выполнения от имени суперпользователя внутри контейнера.....	173
Привилегии root для установки программного обеспечения.....	176
Привилегии для eBPF и модулей ядра	176
Контейнеры, не требующие привилегий суперпользователя.....	178
Флаг --privileged и привилегии	180
Монтирование каталогов с конфиденциальными данными	183
Монтирование сокета Docker.....	185
Совместное использование пространств имен контейнером и его хостом	185
Sidecar-контейнеры.....	186
Развертывание sidecar-контейнеров.....	188
Ограничения sidecar-контейнеров.....	188
Контейнеры для отладки	189
Резюме	189

Глава 12. Сетевая безопасность контейнеров	190
Межсетевые экраны для контейнеров и микросегментация	190
Сетевая модель OSI	192
Отправка IP-пакета	194
IP-адреса контейнеров	195
Сетевая изоляция	197
Маршрутизация на уровнях 3, 4 и правила	197
Утилита iptables	198
eBPF	201
Сетевые политики	202
Реализация политики на уровнях 3 и 4 с использованием iptables	203
Реализация политики на уровнях 3 и 4 с использованием eBPF	204
Политики на уровне 7	205
Программные решения для сетевых политик	206
Сервисная сетка	207
Практические рекомендации для сетевых стратегий	209
Резюме	210
Глава 13. Защищенное соединение между компонентами	211
Защищенные соединения	211
Сертификаты X.509	213
Пары ключей «открытый — закрытый»	213
Центры сертификации	215
Запросы на подписание сертификатов	216
TLS-соединения	217
WireGuard и IPSec	219
Принцип нулевого доверия	220
Защищенные соединения между контейнерами	221
Отзыв сертификатов	222
Сервисные сетки и шифрование трафика	222
SPIFFE	223
Внешний трафик	225
Входящий трафик	225
Исходящий трафик	226
Наблюдаемость сети и журналирование	226
Резюме	227
Глава 14. Передача в контейнеры секретных данных	228
Свойства секретных данных	228
Передача информации в контейнер	229
Хранение секретных данных в образе контейнера	230
Передача секретных данных по сети	230
Передача секретных данных в переменных окружения	231
Передача секретных данных через файлы	232
Секретные данные в Kubernetes	233
Драйвер Secrets Store CSI	234

External Secrets Operator	234
Ротация секретных данных в Kubernetes	234
Секретные данные доступны суперпользователю хоста	236
Резюме	237
Глава 15. Защита среды выполнения контейнеров.....	238
Политики использования контейнерных образов в среде запуска	239
Сетевой трафик	239
Исполняемые файлы	240
Доступ к файлам	241
Идентификаторы пользователей и групп.....	242
Применение ИИ для генерации политик в среде выполнения	242
Технологические решения для безопасности в среде выполнения	243
LD_PRELOAD	243
ptrace	244
seccomp, AppArmor и SELinux	245
Обеспечение безопасности в среде выполнения на основе ядра	245
eBPF для обеспечения безопасности в среде выполнения	247
Инструменты обеспечения безопасности контейнеров в среде выполнения	249
Falco.....	250
Cilium Tetragon	251
Tracee	253
Inspektor Gadget	253
Предотвращение или оповещение	254
Карантин контейнеров.....	256
Меры по снижению уязвимости	256
Резюме	258
Глава 16. Контейнеры и десять главных рисков по версии OWASP.....	259
Ошибки в управлении доступом	259
Недостаточно надежное шифрование	260
Инъекции кода	260
Небезопасный дизайн	261
Неправильные настройки безопасности.....	261
Использование уязвимых или устаревших компонентов.....	262
Взлом аутентификации и идентификации.....	262
Ненадежность цепочки поставок.....	263
Небезопасные журналирование и мониторинг	263
Подделка запросов на стороне сервера.....	264
Резюме	264
Заключение	265
Приложение. Чек-лист по безопасности	266
Об авторе	269
Иллюстрация на обложке	270

Предисловие

Во многих организациях приложения работают в облачных средах, обеспечивая масштабируемость и отказоустойчивость с помощью контейнеров и средств оркестрации. Если вы участник команды Ops, DevOps или даже DevSecOps и отвечаете за настройку подобной среды для своей компании, как гарантировать безопасность развертываемых приложений? Как специалисту по безопасности с опытом использования традиционных систем на основе серверов или виртуальных машин применить свои знания для контейнерного развертывания? И что разработчику облачных приложений следует учесть, чтобы повысить безопасность своих контейнеризированных приложений? В книге описаны ключевые технологии, лежащие в основе контейнеров и облачного программирования. После ее прочтения вы сможете лучше оценить риски для безопасности и решения, подходящие для конкретной среды, а также избежать распространенных ошибок, из-за которых развернутые вами технологии подвергаются опасности.

В книге вы познакомитесь со многими базовыми технологиями и механизмами, широко применяемыми в контейнерных системах, а также с методами их построения в операционной системе Linux. Мы подробно разберем фундаментальные принципы работы контейнеров и способы их взаимодействия и ответим не только на вопрос: *что* происходит в области безопасности контейнеров, но и, главное, *почему*. При написании книги я ставила перед собой цель помочь читателю лучше разобраться в происходящем при развертывании контейнеров. Мне хотелось бы вдохновить вас на создание ментальных моделей, позволяющих вам собственноручно оценить потенциальные риски для безопасности при развертывании.

В книге главным образом рассматриваются так называемые контейнеры приложений, которые в настоящее время используют многие организации для запуска своих приложений в таких системах, как Kubernetes и Docker, а не системные контейнеры наподобие LXC и LXD из проекта Linux Containers Project (<https://linuxcontainers.org/>). В контейнере приложений обычно содержится только код приложения и минимально необходимые зависимости для его запуска, а в среде системных контейнеров выполняется полный дистрибутив Linux, который во многом используется подобно виртуальной машине. Вполне допустимо подключаться к системному контейнеру по SSH. Однако если вы захотите сделать то же самое в отношении контейнера приложений, специалисты по их безопасности не одобряют это (по причинам, изложенным далее в книге). Впрочем, базовые механизмы создания контейнеров для систем

и приложений совпадают: контрольные группы, пространства имен и изменение корневого каталога. Так что фундаментальные сведения, которые вы почерпнете из книги, позволят вам и далее изучать различия в подходах к работе с контейнерами.

Имейте в виду, что существует несколько реализаций «контейнеров», таких как новый проект Apple Containerization, изолирующих рабочие нагрузки с помощью других технологий. К концу книги вы поймете разницу!

Для кого эта книга

Независимо от того, разработчик вы, специалист в области безопасности, оператор или руководитель, если вам интересно дойти до самой сути того, как функционирует что-либо, и нравится проводить время за терминалом Linux, — эта книга для вас.

Если же вы ищете пошаговое руководство по безопасности контейнеров, то, возможно, книга вам не подойдет. Я не верю в существование универсального метода, одинаково хорошо подходящего для любых приложений во всех средах и для всех организаций. Напротив, я хочу помочь вам разобраться, что происходит при запуске приложений в контейнерах и как работают различные механизмы безопасности, чтобы вы могли сами оценить риски.

Как вы узнаете далее, контейнеры основаны на сочетании некоторых функциональных возможностей ядра Linux. Механизмы безопасности контейнеров во многом схожи с механизмами безопасности для хоста Linux (под словом «хост» я понимаю как виртуальные машины, так и физические серверы). Я объясню все нюансы функционирования этих механизмов, а затем продемонстрирую их применение в контейнерах. Опытные системные администраторы могут спокойно пропустить часть разделов и перейти сразу к информации, относящейся к контейнерам.

Вы, наверное, уже заметили, что я несколько раз упомянула Linux, но есть же и другие операционные системы! Я буду время от времени обращаться к ним, однако основное внимание в книге уделяется контейнерам Linux, работающим на хостах Linux.

Я предполагаю, что вы хотя бы поверхностно знакомы с контейнерами и, возможно, хотя бы немного экспериментировали с Docker и Kubernetes. А также понимаете, как минимум в общих чертах, выражения вроде «извлечь образ контейнера из реестра» или «запустить контейнер», даже если не знаете в точности, что происходит «под капотом» при подобных действиях. Я не жду от вас знаний нюансов работы контейнеров, по крайней мере до того, как вы прочтаете книгу.

Структура издания

Мы начнем с главы 1, где рассмотрим модели угроз и векторы атак, которые встречаются при развертывании контейнеров, а также нюансы, отличающие безопасность контейнеров от безопасности развертывания обычных приложений. Цель оставшейся части книги — помочь вам разобраться в контейнерах и связанных с ними угрозах безопасности, равно как и в защите от них.

Прежде чем заняться собственно безопасностью контейнеров, необходимо разобраться, как они работают. В главе 2 описываются основные механизмы Linux, в частности системные вызовы и привилегии, используемые для контейнеров. Далее, в главах 3 и 4, мы подробно разберем компоненты Linux, из которых состоят контейнеры. Это позволит понять, чем в действительности являются контейнеры и насколько они изолированы друг от друга. А в главе 5 мы сравним их степень изоляции с изоляцией виртуальных машин.

В главе 6 вы узнаете о содержимом образов контейнеров и получите некоторые практические рекомендации по обеспечению их безопасности. В главе 7 мы обсудим передовые практики обеспечения безопасности цепочки поставок, гарантирующие неизменность образов контейнеров и их содержимого. В главе 8 вы научитесь определять известные программные уязвимости контейнеров. Такой подход опирается на принцип неизменности контейнеров, а в главе 9 мы рассмотрим некоторые преимущества безопасности, возникающие при дальнейшем развитии идеи неизменности, например применение подхода GitOps к развертыванию контейнеров и их конфигураций.

В главе 10 мы рассмотрим некоторые дополнительные меры безопасности Linux, выходящие за рамки базовой реализации, представленной в главе 4, позволяющие усилить безопасность контейнеров, а также альтернативные способы изоляции контейнеров друг от друга. В главе 11 обсудим способы нарушения изоляции контейнеров в результате опасных, хотя и очень распространенных ошибок конфигурации.

Далее перейдем к вопросам взаимодействия контейнеров. Глава 12 посвящена методам обмена информацией между контейнерами и улучшению безопасности с помощью соединений между ними. В главе 13 объясняются основы ключей и сертификатов, благодаря которым контейнеризированные компоненты могут идентифицировать друг друга и устанавливать между собой защищенные соединения. Хотя их использование в случае контейнеров не отличается от любых других компонентов, эта тема включена в книгу, поскольку в распределенных системах ключи и сертификаты часто вызывают затруднения. В главе 14 мы увидим, как безопасно (и как не столь безопасно) передавать сертификаты и прочие учетные данные в контейнеры в среде выполнения.

В главе 15 мы поговорим о способах, позволяющих предотвращать атаки в среде выполнения благодаря возможностям контейнеров.

Наконец, в главе 16 мы рассмотрим список из десяти основных рисков для безопасности, опубликованный открытым проектом обеспечения безопасности веб-приложений (Open Web Application Security Project, OWASP), а также подходы к их устранению, характерные для контейнеров. Кстати, некоторые угрозы безопасности устраняются одинаково — независимо от того, работает ли ваше приложение в контейнере или нет.

Что нового во втором издании

С момента выхода первого издания книги в экосистеме контейнеров произошли значительные изменения. В 2020 году кибератака на SolarWinds привлекла внимание к цепочке поставок программного обеспечения, что привело к формированию целого направления — безопасности цепочки поставок. Термин GitOps впервые был упомянут в 2018 году, а широкое распространение получил в последние несколько лет в немалой степени потому, что организации начали использовать «раннеры» (runners — исполнители заданий, обычно временные виртуальные машины), предоставляемые GitHub и другими платформами репозитория, в рамках современного подхода к непрерывной интеграции и непрерывной поставке (CI/CD). Технология eBPF стала ведущей для инструментов, обеспечивающих безопасность среды выполнения контейнеров, и сетевых взаимодействий между ними. Даже фундаментальные основы, на которых базируются контейнеры, претерпели изменения: они по-прежнему используют пространства имен Linux, но пространство имен пользователей теперь применяется намного чаще, контейнеры без привилегий суперпользователя (root-прав) стали реальностью, а экосистема перешла на cgroups v2.

Сообщества, занимающиеся облачными технологиями и контейнерами, способствовали распространению знаний о передовых методах обеспечения безопасности, поэтому некоторые из наиболее опасных приемов (например, установка пакетов в работающий контейнер), к счастью, встречаются намного реже, чем раньше. Тем не менее они все еще описываются в книге, потому что крайне важно, чтобы мы не вернулись к их использованию!

И, пожалуй, самое заметное изменение с момента выхода первого издания: Kubernetes прочно утвердился в роли доминирующего инструмента оркестрации.

Примечание относительно Kubernetes

Используемые сегодня контейнеры работают в основном под управлением Kubernetes (<https://kubernetes.io/>). Он автоматизирует процесс выполнения различных рабочих нагрузок в кластере машин, и в некоторых разделах я исхожу из того, что в общих чертах эта идея вам понятна. В целом я старалась сосредоточиться на понятиях уровня контейнеров — «плоскости данных» (data plane) в Kubernetes.

Поскольку рабочие нагрузки Kubernetes выполняются в контейнерах, эта книга в определенной степени имеет отношение и к безопасности Kubernetes, однако ни в коем случае не является исчерпывающим руководством по защите Kubernetes или облачных инфраструктур в целом. Актуальный список вопросов безопасности вы найдете в документации Kubernetes (<https://kubernetes.io/docs/concepts/security>), включая настройки и параметры компонентов слоя управления, которые выходят за рамки книги.

Однако, несмотря на повсеместное распространение Kubernetes, это далеко не единственный вариант. Широко используются также управляемые контейнерные платформы, такие как AWS Elastic Container Service, AWS Fargate, Azure Container Instances и Google Cloud Run. Кроме того, немалую популярность завоевали Docker и Podman, особенно в локальных средах разработки, конвейерах непрерывной интеграции (Continuous Integration, CI) и небольших развертываниях.

Примеры

В книге вы найдете множество примеров, и я рекомендую вам поэкспериментировать с ними.

В этих примерах я предполагаю, что вы хорошо знакомы с основными утилитами командной строки Linux, наподобие `ps` и `grep`, а также с основными операциями, необходимыми для запуска контейнерных приложений с помощью таких утилит, как `kubectl` или `docker`. Первый из этих наборов инструментов поможет нам пояснить очень многое из того, что происходит при использовании второго!

Чтобы воспроизвести примеры, вам понадобится доступ к машине с Linux или соответствующей виртуальной машине. Я создавала примеры с помощью виртуальной машины Ubuntu 24.04. Аналогичные результаты можно получить на различных дистрибутивах Linux, а также виртуальных машин от любого поставщика облачных сервисов.

Запуск контейнеров

Для многих пользователей основной (а возможно, и единственный) опыт использования контейнеров напрямую связан с Docker. Эта платформа сильно упростила работу с контейнерами за счет набора удобных утилит для разработчиков. Управлять контейнерами и их образами из командной строки позволяет команда `docker`.

Однако сама утилита `docker` представляет собой лишь тонкую прослойку, которая обращается к демону Docker, выполняющему всю основную работу. «Под капотом» `docker` имеется компонент `containerd` (<https://containerd.io>),

который компания Docker в 2017 году безвозмездно передала фонду Cloud Native Computing Foundation (CNCF) (<https://cncf.io/>).

Компонент `containerd` проверяет наличие требуемого образа контейнера, после чего вызывает компонент `runc`, фактически создающий объект контейнера. При желании контейнер можно запустить, вызвав `containerd` или даже `runc` напрямую.

В Kubernetes используется интерфейс Container Runtime Interface (CRI), позволяющий выбрать реализацию среды выполнения контейнеров. Сегодня чаще всего применяются упомянутый выше `containerd` и CRI-O (<https://cri-o.io/>), который, прежде чем его безвозмездно передали CNCF, был частью Red Hat.

Утилита командной строки `docker` — лишь один из вариантов управления контейнерами и образами. Существуют и другие возможности запуска контейнеров приложений, наподобие тех, что обсуждаются в книге. Например, утилита `podman` из дистрибутива Red Hat, изначально предназначенная для устранения зависимости от компонента-демона и необходимости прав суперпользователя для запуска.

В примерах, представленных в книге, используется множество разнообразных утилит для работы с контейнерами, чтобы показать, что значительная часть функциональности многих реализаций контейнеров совпадает.



Во время моей работы над книгой компания Apple запустила свой проект по контейнеризации. В нем используются те же форматы образов контейнеров, что и в Docker (о котором рассказывается в главе 6), и имеется много общего в интерфейсе командной строки.

Однако, как вы увидите в главе 10, для изоляции контейнеров друг от друга там применяется другой механизм. По этой причине некоторые примеры из книги будут работать иначе, если запустить их с использованием технологии контейнеризации Apple.

Благодарности

Я благодарна множеству людей, которые помогали мне и поддерживали, пока я работала над книгой:

- моим редакторам из издательства O'Reilly, Рите Фернандо (Rita Fernando; второе издание) и Вирджинии Вилсон (Virginia Wilson; первое издание), державшим весь процесс под контролем и следившим за качеством книги;
- научным редакторам за их вдумчивые замечания и конструктивную обратную связь: Ахилу Белю (Akhil Behl), Алексу Поллитту (Alex Pollitt), Эндрю Мартину (Andrew Martin), Эрику Ст. Мартину (Erik St. Martin), Джеду Салазару (Jed Salazar), Филу Эстесу (Phil Estes), Рани Оснату (Rani Osnat) и Роберту П. Дж. Дею (Robert P. J. Day);

- моим коллегам из Isovalent, бывшим коллегам из Aqua Security и бесчисленным участникам сообщества, у которых я за эти долгие годы многому научилась;
- Филу Перлу (Phil Pearl) — моему мужу, самому важному критику и наставнику и в то же время моему лучшему другу.

Обратная связь

В дополнение к книге существует сайт containersecurity.tech. Не стесняйтесь сообщать там о найденных проблемах и о том, что бы вы хотели увидеть в следующих изданиях.

Условные обозначения

В книге используются следующие условные обозначения.

Курсив

Курсивом выделены новые термины.

Моноширинный шрифт

Используется для листингов программ, а также внутри абзацев для обозначения таких элементов, как переменные и функции, базы данных, типы данных, переменные окружения, операторы и ключевые слова, имена файлов и их расширений.

Моноширинный курсив

Показывает текст, который должен быть заменен значениями, введенными пользователем, или значениями, определяемыми контекстом.

Шрифт без засечек

Используется для обозначения URL, адресов электронной почты, названий кнопок, каталогов.



Этот рисунок указывает на общее примечание.

Использование примеров кода

Дополнительные материалы (примеры кода, упражнения и т. д.) доступны для скачивания по адресу <https://containersecurity.tech/>. На онлайн-платформе O'Reilly есть изолированная среда, где можно запустить примеры, а также размещены вопросы для проверки усвоения материала.

Если при использовании примеров кода у вас возникнут технические вопросы или проблемы, то, пожалуйста, напишите нам на электронную почту по адресу bookquestions@oreilly.com.

Помочь вам делать вашу работу — вот цель создания книги. Если к ней прилагается какой-либо пример кода, то вы можете задействовать его в ваших программах и документации. Обращаться к нам за разрешением нет необходимости, разве что вы копируете значительную часть кода в коммерческих целях. Так, написание программы, в которой используется несколько фрагментов кода из книги, не требует отдельного разрешения, в отличие от продажи или распространения компакт-дисков с примерами из книг O'Reilly. Вы можете свободно цитировать книгу с примерами кода, отвечая на некий вопрос. А вот если хотите включить существенную часть приведенного здесь кода в документацию своего программного продукта, то вам следует связаться с нами.

Мы приветствуем, хотя и не требуем ссылки на первоисточник.

Если вам кажется, что ваше обращение с примерами кода выходит за рамки правомерного использования или условий, перечисленных выше, то можете обратиться к нам по адресу permissions@oreilly.com.

От издательства

Мы выражаем огромную благодарность компании Orion soft за помощь в работе над русскоязычным изданием книги и вклад в повышение качества переводной литературы.

Ваши замечания, предложения, вопросы отправляйте по адресу comp@sprintbook.kz (издательство SprintBook, компьютерная редакция).

Мы будем рады узнать ваше мнение!

О научных редакторах русскоязычного издания

Сергей Задорожный — руководитель Platform Engineering и Enabling Team в банке «Центр-инвест». Внедряет DevSecOps-практики, AI в финтех. Спикер конференций, член программных комитетов DevOps, Dump и Merge. Автор курсов по DevOps, DevSecOps от Яндекс Практикума.

Сергей Метелёв — ведущий системный инженер компании Orion soft. Имеет обширный опыт внедрения DevOps-практик в различных командах разработки: от автоматизации CI-процессов и сборки кода в контейнеры до мониторинга межсервисного взаимодействия. Участвовал в запуске и развитии региональной медицинской информационной системы под Kubernetes, а также в модернизации и расширении распределенной инфраструктуры в международной финтехкоманде.

Угрозы безопасности контейнеров

За последнее десятилетие масштабы применения контейнеров резко возросли. Идеи, лежащие в основе контейнеризации, существовали задолго до появления Docker. Но большинство наблюдателей сходятся во мнении, что именно платформа Docker с ее удобными утилитами командной строки дала толчок популярности контейнеров среди сообщества разработчиков после ее выхода в 2013 году.

У контейнеров много достоинств. Как гласит рекламный слоган Docker, они позволяют «создать один раз — выполнять где угодно». Это достигается благодаря тому, что приложение упаковывается в пакет со всеми своими зависимостями и изолируется от остальной части системы, на которой оно работает. У контейнеризированного приложения есть все необходимое для работы, и его легко оформить в виде образа контейнера, который будет одинаково запускаться на моем или вашем ноутбуке, в облаке и на сервере в центре обработки данных (ЦОД)¹.

Следствие этой изоляции — возможность параллельного выполнения нескольких различных контейнеров, которые не будут мешать друг другу. До появления контейнеров мешанина зависимостей могла легко превратиться в настоящий кошмар для разработчиков, когда двум приложениям требовались различные версии одних и тех же пакетов. Самым простым решением в такой ситуации было запускать приложения на отдельных машинах. Контейнеры позволяют изолировать зависимости друг от друга, благодаря чему выполнение нескольких приложений на одном сервере не доставляет никаких проблем. Вскоре стало очевидно, что контейнеризация дает возможность запускать множество приложений на одном хосте (будь то виртуальная машина или реальный сервер), не беспокоясь о зависимостях.

Следующим логичным шагом стало распределение контейнеризированных приложений по кластеру серверов. Оркестраторы наподобие Kubernetes автоматизируют этот процесс до такой степени, что больше не нужно вручную устанавливать

¹ Однако есть ограничения по архитектуре процессоров, о которых не следует забывать. К примеру, контейнер, собранный под `arm64`, не запустится под `x86_64`, так как у систем разные наборы инструкций. Более подробно об этом будет рассказано в главе 6. — *Примеч. науч. ред.*

приложения на конкретных машинах, достаточно сообщить оркестратору, какие контейнеры необходимо запустить, и он сам найдет подходящее место для каждого из них.

С точки зрения безопасности контейнеризованная среда во многом схожа с традиционным развертыванием. Нарушители пытаются похитить данные, или изменить поведение системы, или использовать вычислительные ресурсы других людей для майнинга криптовалюты. При переходе к контейнерам ничего из этого не меняется. Но контейнеры существенно меняют способ выполнения приложений, что приводит к иному набору рисков (рис. 1.1).

Как показано на рис. 1.1, приложение и его зависимости работают внутри контейнера, который управляется оркестратором, таким как Kubernetes или AWS Elastic Container Service. Контейнер может работать непосредственно на реальном физическом хосте или внутри виртуальной машины (virtual machine, VM), которая, в свою очередь, запущена на физическом оборудовании. Главы 3 и 4 помогут вам понять, как контейнер изолирует приложение от хоста и от других контейнеров, а в главе 5 рассматриваются совершенно иные механизмы изоляции, используемые в виртуальных машинах. Понимание этих различий имеет решающее значение для эффективного анализа возможных рисков и угроз для контейнеризированных развертываний, а также для выбора мер по их смягчению и обеспечению безопасности контейнеров.



Рис. 1.1. Контейнеризованное приложение

Риски, угрозы и уменьшение их последствий

Риск (risk) — потенциальная проблема, а также ее возможные последствия.

Угроза (threat) — путь реализации этого риска.

Смягчение последствий (mitigation) — контрмеры, с помощью которых можно предотвратить угрозу или по крайней мере снизить вероятность ее успешной реализации.

Скажем, существует риск, что кто-то украдет из вашего дома ключи от вашей же машины и уедет на ней. Угрозами в данном случае будут различные способы кражи ключей: разбить окно и дотянуться до них; просунуть удочку через щель для почты; постучать в дверь и отвлечь вас, пока сообщник быстро проскользнет внутрь и схватит ключи. Контрмерой может быть простое правило — не хранить ключи на видном месте.

В зависимости от организации риски могут сильно различаться. Основной риск для банка, хранящего клиентские деньги, — их кража. Для интернет-магазина основная «головная боль» — мошеннические транзакции. Ведущий личный блог пользователь может бояться, например, что кто-то взломает его учетную запись, выдаст себя за него и начнет публиковать неподобающие комментарии. В разных странах законодательство о защите персональной информации имеет свои особенности, поэтому различается и риск утечки личных данных пользователей: в одних странах риски «лишь» репутационные, в то время как в Европе Общий регламент защиты персональных данных (General Data Protection Regulation, GDPR) допускает штрафы до 4 % общего дохода компании (<https://oreil.ly/guQg3>).

Поскольку риски весьма разнятся, то сильно различается и относительная значимость потенциальных угроз, равно как и соответствующий набор мер по их предотвращению. В основе управления рисками лежит процесс их систематизации, перечисления возможных угроз, расстановки их по приоритетам и выбора способов противодействия им. *Моделирование угроз* (threat modeling) — процесс выявления и перечисления потенциальных угроз системе. За счет планомерного анализа ее компонентов и вероятных векторов атаки модель угроз помогает определить наиболее уязвимые для атак места.

Единой универсальной модели угроз не существует: все зависит от рисков, конкретной среды, организации и запускаемых приложений. Тем не менее можно выделить ряд потенциальных угроз, характерных для большинства, если не всех, контейнерных развертываний.

Модель угроз для контейнеров

В частности, модель угроз можно рассматривать с точки зрения ее участников, в число которых могут входить:

- *внешние нарушители* (external attackers), пытающиеся получить доступ к развернутой системе извне;
- *внутренние нарушители* (internal attackers), сумевшие проникнуть в какую-то часть развернутой системы;
- *недобросовестные сотрудники* (malicious internal actors), например разработчики и администраторы с определенным уровнем привилегий доступа к развернутой системе;
- *невнимательные сотрудники* (inadvertent internal actors), которые могут непреднамеренно вызывать проблемы;
- *процессы приложений* (application processes), которые не являются людьми-злоумышленниками, тем не менее имеют определенный программный доступ к системе.

У каждого из этих источников угроз есть определенный набор прав доступа, который необходимо учитывать.

- Какой уровень доступа к системе есть у них через учетные данные? Например, обладают ли они доступом к пользовательским учетным записям на машинах хостов, где работает развернутая система?
- Какие привилегии у них есть в системе? В Kubernetes этот пункт относится к настройкам управления доступом для всех пользователей, в том числе анонимных, на основе ролей. Права доступа могут дополнительно контролироваться механизмами политик или инструментами безопасности в среде выполнения. В общедоступных облачных средах права доступа также зависят от политик управления идентификацией и доступом (identity and access management, IAM).
- Какие права доступа к сети у них есть? Например, какие части системы включены в виртуальное частное облако (Virtual Private Cloud, VPC)? Действуют ли политики сетевой безопасности для ограничения доступа?

Существует несколько возможных путей атаки на развернутую контейнеризированную систему, и, чтобы их систематизировать, можно, например, проанализировать потенциальные векторы атак на каждом из этапов жизненного цикла контейнера (рис. 1.2).

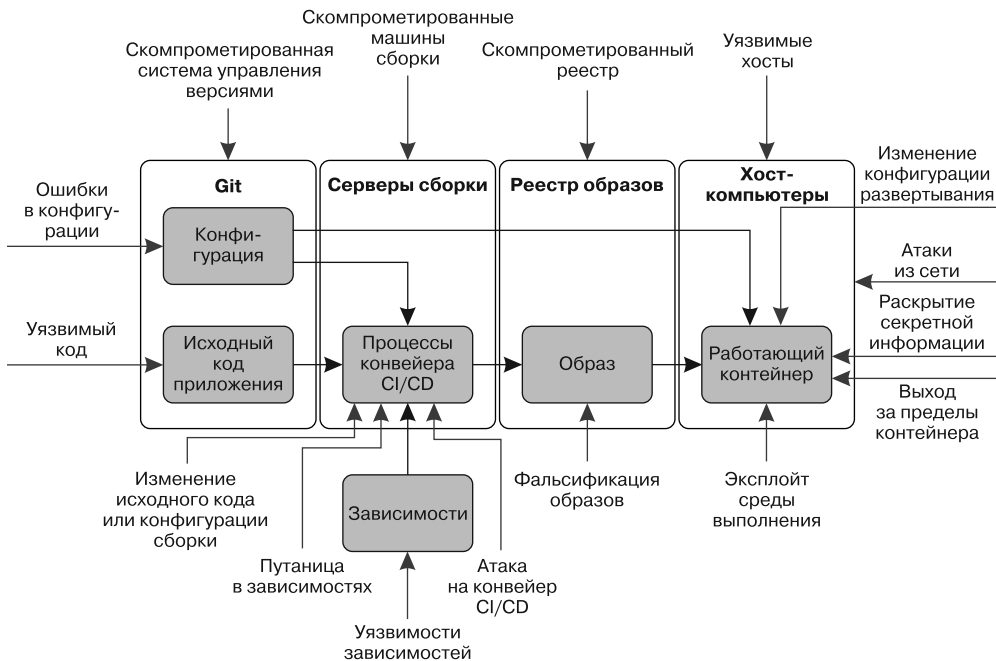


Рис. 1.2. Векторы атак на контейнеры

Рассмотрим эти векторы более подробно.

Уязвимости в коде и зависимостях

Жизненный цикл контейнера начинается с написания разработчиком кода приложения. В этом коде, равно как и в его зависимостях, могут содержаться изъяны (уязвимости). Существуют тысячи известных уязвимостей, которыми (если они есть в приложении) могут воспользоваться злоумышленники. Чтобы избежать запуска контейнеров с известными уязвимостями, образы контейнеров необходимо проверять с помощью специальных средств анализа, о которых пойдет речь в главе 8. Причем делать это нужно регулярно, поскольку уязвимости постоянно обнаруживаются даже в уже существующем коде. В процессе анализа должны также выявляться случаи, когда контейнеры используют устаревшие пакеты, требующие установки патчей безопасности. Кроме того, существуют средства проверки, способные выявлять встроенное в образы вредоносное программное обеспечение (ПО).

Некорректно настроенные образы контейнеров

Написанный код встраивается в образ контейнера. В ходе сборки образа контейнера возникает множество возможностей для внедрения уязвимостей, которые впоследствии могут быть использованы для атаки на работающий контейнер. В их число входит запуск контейнера от имени суперпользователя, в результате чего у него оказывается больше привилегий на хосте, чем ему действительно нужно. Больше информации об этом — в главе 6.

Атаки на цепочку поставок и систему сборки

Исходный код и конфигурация сборки поступают на вход процесса сборки, который создает образ контейнера. Собранный образ контейнера сохраняется в реестре, откуда извлекается перед запуском. Если злоумышленник сумеет изменить исходный код, повлиять на сборку образа контейнера, подменить образ контейнера в реестре или добиться загрузки неправильного образа, он сможет вставить вредоносный код, который потом будет запущен в рабочей среде. Как убедиться, что для сборки образа используются именно тот исходный код и зависимости? Как гарантировать соответствие извлекаемого образа тому, который был ранее помещен в реестр? Не внесли ли злоумышленники в него изменения? Любой субъект, способный повлиять на образ в промежутке между сборкой и развертыванием, сможет выполнить произвольный код в вашей системе. Об этих атаках на «цепочку поставок» подробнее рассказывается в главе 7.

Некорректно настроенные контейнеры

Как мы обсудим в главе 11, контейнер можно запустить с настройками, в результате которых у них появляются ненужные, а порой и незапланированные привилегии. Скачав файлы конфигурации YAML из Интернета, не запускайте их, пока не убедитесь в отсутствии небезопасных настроек!

Уязвимые хосты

Контейнеры выполняются на хостах, и важно убедиться, что на них не будет запущен уязвимый код (например, старые версии компонентов механизма оркестрации с известными уязвимостями). Имеет смысл уменьшить количество установленного на каждом хосте ПО, чтобы сократить поверхность атаки. Кроме того, необходимо правильно настроить хосты, следуя практическим рекомендациям по обеспечению безопасности. Все это обсуждается в главе 4. Желательно также ограничить доступ пользователей к хостам. Для этого можно, например, использовать подход GitOps, о котором рассказывается в главе 9, — в этом случае весь код и конфигурация хранятся в системе управления версиями и обновляются с помощью автоматизированного процесса, а не вручную.

Незащищенная сеть

Контейнерам обычно требуется взаимодействовать друг с другом или с внешними системами. В главе 12 рассказывается, как работает сеть в контейнерах, а в главе 13 — как устанавливать защищенные соединения между компонентами.

Раскрытие секретных данных

Для взаимодействий с другими компонентами системы приложениям часто требуются учетные данные, токены или пароли. При развертывании в контейнере эти секретные значения необходимо передавать в контейнеризированный код. В главе 14 вы узнаете, что у этой задачи существует несколько вариантов решения и степень их безопасности может существенно различаться.

Эксплойты приложения в среде выполнения

Широко используемые среды выполнения контейнеров, включая `containerd` и `CRI-O`, уже хорошо зарекомендовали себя на практике, но не исключено, что в них все же остаются программные ошибки, которые позволят вредоносному коду, работающему внутри контейнера, просочиться за пределы контейнера в пространство хостовой операционной системы. Аналогично время от времени обнаруживаются уязвимости в ядре, которые могут позволить выход из контейнера или повышение привилегий. В главе 4 вы узнаете об изоляции, предназначенной для ограничения кода приложения рамками контейнера. Ущерб от выхода за пределы контейнера для ряда приложений может быть столь велик, что имеет смысл применить более эффективные механизмы изоляции, например те, о которых говорится в главе 10. А в главе 15 мы рассмотрим инструменты безопасности, помогающие обнаружить и предотвратить выход за пределы контейнера, а также выявить эксплойты в среде выполнения (рантайме).

Некоторые векторы атак, показанные на рис. 1.2, не относятся к теме книги.

- Исходный код обычно хранится в репозиториях, потенциально доступных для атак, цель которых — взлом приложения. Аналогично образы контейнеров

хранятся в реестрах. Необходимо обеспечить должный контроль доступа пользователя к репозиторию исходного кода и реестру образов.

- Хосты соединены по сети, зачастую с использованием VPC и, как правило, подключенной к Интернету. Как и при стандартном развертывании, необходимо защитить физические хосты (или виртуальные машины) от злоумышленников. Безопасные настройки сети, использование межсетевого экрана, а также IAM актуальны в облачном развертывании так же, как и в локальном. Эти меры защиты также относятся и к машинам сборки, на которых компилируется код и создаются образы контейнеров.
- Контейнеры обычно работают под управлением оркестратора. Недостаточная безопасность настроек оркестратора или отсутствие должного контроля над доступом с правами администратора открывают злоумышленникам дополнительные возможности для вмешательства в процесс развертывания.



Помимо этой книги, оценить потенциальные угрозы для ваших операций вам помогут следующие ресурсы.

- Компания Microsoft опубликовала хорошее обобщенное описание угроз, тактик и процедур, которые злоумышленники могут использовать против Kubernetes, в документе *Threat Matrix for Kubernetes* (<https://microsoft.github.io/Threat-Matrix-for-Kubernetes>).
- Исследователи из Сингапурского университета технологий и дизайна и Цюрихского технологического института опубликовали статью под названием *Threat Modeling and Security Analysis of Containers: A Survey* (<https://arxiv.org/pdf/2111.11475>).

Границы зон безопасности

Границы зон безопасности (иногда называемые границами доверия) между частями системы означают, что наборы прав доступа в этих частях отличаются. Иногда границы задаются в ходе администрирования. Например, в системах под управлением Linux системный администратор может установить границы зон безопасности, определяющие, какие файлы доступны пользователю, просто изменив группы, в которых состоит данный пользователь. Если вы забыли, как работает система прав доступа к файлам в Linux, в главе 2 будет краткое напоминание.

Иногда можно услышать, что «контейнеры не являются зоной безопасности». Это не совсем так. В действительности контейнер представляет собой зону безопасности — просто не самую надежную! Предполагается, что код приложения выполняется внутри контейнера и не имеет доступа к коду или к данным за его пределами, за исключением случаев, когда ему явно не предоставлено такое разрешение (например, путем подключения внешнего тома). Как вы узнаете далее, защита, обеспечиваемая контейнером, относительно слаба, поэтому вам

потребуется провести дополнительные границы, чтобы гарантировать безопасность ваших приложений.

Чем строже граница зон безопасности между злоумышленником и его целью (например, данными пользователей), тем сложнее ему достичь этой цели.

Векторы атак, ранее описанные в разделе «Модель угроз для контейнеров», могут использоваться комплексно в целях проникновения через несколько границ зон безопасности. Приведу следующие примеры.

- Злоумышленник может обнаружить, что уязвимость в одной из зависимостей приложения позволяет ему удаленно выполнять код внутри контейнера.
- Допустим, у взломанного компьютера нет прямого доступа к каким-либо ценным данным. Злоумышленнику необходимо найти способ попасть в другой контейнер или на хост. Один из вариантов — уязвимость, позволяющая выйти за рамки контейнера; другой — небезопасная конфигурация контейнера. Если злоумышленнику удастся воспользоваться одним из этих способов, он получит доступ к хосту.
- Следующий шаг — поиск возможности получить на хосте привилегии суперпользователя, что может оказаться тривиальной задачей, если код приложения выполняется от имени суперпользователя внутри контейнера, как вы увидите в главе 4.
- Имея привилегии суперпользователя на физическом хосте, злоумышленник способен получить доступ ко всему, что открывается с хоста или из любого контейнера, запущенного на этом хосте.

Добавление новых границ безопасности и усиление существующих значительно усложняет жизнь взломщиков.

При построении модели угроз важно учитывать, что атака может быть совершенна изнутри среды, в которой работают ваши приложения. При развертывании в облаке отдельные ресурсы порой используются совместно с другими пользователями и их приложениями. Совместное использование ресурсов компьютера называется *мультиарендностью* (multitenancy) и серьезно влияет на модель угроз.

Мультиарендность

В мультиарендной среде различные пользователи — *арендаторы* (tenants) — выполняют задания на общем оборудовании. (Термин «мультиарендность», или «мультиитенантность» можно также встретить в контексте программных приложений, где он означает нескольких пользователей, работающих с одним экземпляром программы, однако в нашем случае совместно используемым будет лишь аппаратное обеспечение.) В зависимости от владельцев этих различных заданий, а также степени взаимного доверия арендаторов могут

понадобиться более жесткие границы между ними, чтобы предотвратить их возможное негативное влияние друг на друга.

Мультиарендность — идея, появившаяся еще во времена мейнфреймов в 1960-х, когда пользователи арендовали ресурсы процессора (CPU), память и место на диске на совместно используемой машине. Эта концепция не так уж сильно отличается от современных общедоступных облачных сервисов, например Amazon AWS, Microsoft Azure и Google Cloud Platform, в которых пользователи также арендуют вычислительные ресурсы — время CPU, оперативную память, дисковое пространство — наряду с прочими возможностями и управляемыми сервисами. С тех пор как в 2006 году в Amazon AWS появилась служба EC2, стало возможно арендовать экземпляры виртуальных машин, запущенные на стойках серверов в центрах обработки данных, разбросанных по всему миру. На одном физическом сервере может работать несколько VM, а вы, как пользователь, управляющий группой таких VM, можете не знать, кто управляет соседними VM.

Совместно используемые машины

Бывают и ситуации, когда на одной (возможно, виртуальной) машине Linux работают сразу несколько пользователей. Такой вариант истинной мультиарендности часто встречается в университетах, где пользователи не доверяют друг другу и, говоря откровенно, администраторы не доверяют пользователям. В подобной среде у каждого пользователя есть своя учетная запись, а доступ ограничивается средствами операционной системы (ОС) Linux, чтобы, например, пользователь мог изменять только файлы в своих каталогах. Представьте себе, какой бы хаос возник, если бы студенты читали или, хуже того, редактировали файлы своих однокурсников.

Как вы увидите в главе 4, все контейнеры, действующие на одном хосте, используют одно и то же ядро. Если на машине запущен демон Docker, то любой пользователь, имеющий право выполнять команды `docker`, по сути, обладает `root`-правами, и вряд ли администратору следует предоставлять не доверенным пользователям такие возможности.

В корпоративной и особенно в облачной среде подобные совместно используемые машины встречаются реже. Обычно пользователи (или команды, доверяющие друг другу) задействуют собственные ресурсы, выделенные им в виде виртуальных машин.

Виртуализация

В целом считается, что виртуальные машины достаточно надежно изолированы друг от друга, то есть маловероятно, что пользователи, работающие на соседних виртуальных машинах, смогут вмешаться в вашу систему или

увидеть, что в ней происходит. Как достигается подобная изоляция, можно узнать в главе 5. Более того, в соответствии с общепринятым определением (<https://en.wikipedia.org/wiki/Multitenancy>) виртуализация вообще не означает мультиарендность. Мультиарендность (или мультитенантность) — это когда различные группы пользователей совместно работают с одним экземпляром программного обеспечения. В случае виртуализации у пользователей нет доступа к гипервизору, управляющему их виртуальными машинами, поэтому фактически они не используют совместно никаких программ.

Впрочем, изоляция виртуальных машин отнюдь не идеальна. Пользователи и раньше жаловались на проблемы с «шумными соседями», когда совместное использование реальной машины может привести к неожиданным колебаниям производительности. Компания Netflix одной из первых начала применять общедоступные облачные сервисы. В 2010 году в блоге компании была опубликована статья (<https://oreil.ly/8zHGy>), в которой говорилось, что создаваемые системы могли осознанно прекратить выполнение подзадачи, если она выполнялась слишком медленно. В то же время другие специалисты утверждают, что проблема «шумных соседей» не столь серьезна (<https://oreil.ly/iE4qE>).

Известны также случаи, когда уязвимости в программном обеспечении приводили к нарушению границ между виртуальными машинами. Поставщики облачных сервисов принимают дополнительные меры для защиты от таких уязвимостей на уровне гипервизора.

Для некоторых приложений и организаций (особенно правительственных, финансовых и медицинских) последствия несанкционированного доступа могут быть настолько серьезными, что требуется полное физическое разделение. Чтобы гарантировать полную изоляцию рабочих заданий, можно использовать частное облако, работающее в вашем собственном центре обработки данных или под управлением выбранного вами поставщика сервисов. Частные облака иногда подразумевают дополнительные меры защиты, например более строгие проверки анкетных данных персонала, имеющего доступ к ЦОД.



Компания Google предоставляет подробную документацию по мерам безопасности своих ЦОД (<https://datacenters.google/advancing-security>).

Многие поставщики облачных сервисов предлагают варианты VM, при которых гарантируется, что одну реальную машину использует только один клиент. Кроме того, можно арендовать выделенные физические серверы, обслуживаемые поставщиками облачных сервисов. В обоих сценариях вы полностью избегаете проблемы «шумных соседей» и получаете более надежную изоляцию между реальными машинами.

Неважно, арендуете вы физические или виртуальные машины в облаке либо используете собственные серверы, — при работе с контейнерами может возникнуть необходимость учесть границы зон безопасности между различными группами пользователей.

Мультиарендность контейнеров

В главе 4 вы увидите, что изоляция контейнеров не так строга, как изоляция виртуальных машин. Хотя это зависит от профиля рисков, вряд ли стоит размещать контейнеры на одной машине с пользователями или организациями, которым вы не доверяете. Даже если все контейнеры на ваших машинах запускаются лично вами или доверенными лицами, все равно следует учитывать человеческий фактор и предусмотреть меры, которые не позволят контейнерам влиять друг на друга.

В Kubernetes можно использовать *пространство имен* (namespace), чтобы разделить кластер компьютеров на части между различными людьми, командами или приложениями.



«Пространство имен» — термин с несколькими значениями. В Kubernetes пространство имен представляет собой высокоуровневую абстракцию, позволяющую разделить ресурсы кластера и применить к ним различные политики доступа. В Linux же пространство имен — это низкоуровневый механизм изоляции ресурсов машины, доступных процессу. Более подробно пространства имен будут описаны в главе 4.

Чтобы указать, какие пользователи и компоненты могут обращаться к тем или иным пространствам имен Kubernetes, используется механизм управления доступом на основе ролей (role-based access control, RBAC). Описание подробностей его настройки выходит за рамки книги¹, однако важно отметить, что RBAC позволяет контролировать только действия, производимые через Kubernetes API. Контейнеры приложений в подах Kubernetes, работающих на одном хосте, защищены друг от друга лишь с помощью механизмов изоляции контейнеров, описанных в книге, даже если они находятся в разных пространствах имен. Если злоумышленнику удастся выйти за пределы контейнера на хост, то границы пространств имен Kubernetes никак не помешают ему влиять на другие контейнеры.

¹ Если вам интересна эта тема, можете ознакомиться с ней в книге: *Маданпарамбат Г., Маккендрик Р. Kubernetes. Полное руководство по развертыванию и управлению Kubernetes в облачных и локальных средах. 2-е изд. — Примеч. науч. ред.*

Экземпляры контейнеров

Такие облачные сервисы, как Amazon AWS, Microsoft Azure и Google Cloud Platform, предоставляют множество *управляемых сервисов* (managed services), позволяющих пользователю арендовать программное обеспечение, хранилище и другие компоненты без необходимости самостоятельно устанавливать или обслуживать их. Классическим примером является Amazon Relational Database Service (RDS): с его помощью можно легко развернуть базу данных (БД), основанную на таком популярном ПО, как PostgreSQL, а для резервного копирования данных достаточно поставить одну галочку (и оплатить счет, конечно).

Управляемые сервисы распространились и на мир контейнеров. С помощью сервисов Azure Container Instances и AWS Fargate можно запускать контейнеры, не задумываясь о том, на каком компьютере (или VM) они будут работать.

Это снимает с наших плеч немалый груз администрирования и позволяет легко масштабировать развертываемую систему. Однако (в теории) экземпляры контейнеров разных пользователей могут располагаться на одной VM. Уточните этот момент у вашего поставщика облачных услуг на всякий случай.

Теперь вы уже знакомы со многими потенциальными угрозами для развертываемых вами систем. Прежде чем двигаться дальше, я бы хотела поговорить об основных принципах безопасности, которые пригодятся при выборе утилит и средств защиты, требуемых при развертывании.

Принципы безопасности

Это общие рекомендации, которые обычно считаются разумным подходом независимо от того, о безопасности чего идет речь.

Минимум привилегий

Принцип наименьших привилегий предполагает ограничение прав доступа пользователя или компонента до минимума, необходимого для выполнения задачи. Например, для микросервиса, осуществляющего поиск товара в приложении электронной коммерции, данный принцип означает, что учетная запись этого микросервиса должна предоставлять ему доступ к части базы данных только для чтения. Ему не требуется, скажем, информация о пользователях или платежах, равно как нет нужды записывать в БД информацию о товарах.

Многоуровневая защита

Как вы увидите позже, есть множество способов повысить безопасность развертываемой системы и работающих в ней приложений. Принцип многоуровневой защиты гласит, что защита должна состоять из нескольких слоев. Если злоумышленник сумеет преодолеть один из них, то следующий должен помешать ему нанести вред развернутой системе или похитить данные.

Минимальная поверхность атаки

Как правило, чем больше система, тем выше вероятность существования в ней уязвимостей. Затруднить атаки на нее можно путем упрощения системы, в том числе:

- сокращения количества точек доступа за счет минимизации и упрощения интерфейсов;
- ограничения количества пользователей и компонентов, имеющих доступ к сервису;
- сокращения объема кода.

Ограничение радиуса поражения

Концепция разделения средств управления безопасностью на небольшие изолированные части гарантирует, что даже при худшем варианте развития событий ущерб будет ограничен. Контейнеры прекрасно подходят для реализации этого принципа, ведь они сами могут выступать в роли зоны безопасности за счет разделения архитектуры проекта на несколько экземпляров микросервисов.

Разделение обязанностей

С принципами наименьших привилегий и ограничения радиуса поражения тесно связана идея разделения обязанностей. Она предполагает, чтобы различные компоненты/люди отвечали за как можно меньшую часть всей системы. Этот подход ограничивает потенциальный ущерб от действий одного привилегированного пользователя, поскольку выполнение некоторых операций требует полномочий более чем одного человека.

Реализация принципов безопасности с помощью контейнеров

Как вы увидите далее, благодаря контейнерам можно применить все эти принципы безопасности.

Минимум привилегий

Различным контейнерам можно назначать разные наборы привилегий, определяя каждому лишь те права, которые необходимы для выполнения поставленной задачи.

Многоуровневая защита

Контейнеры — это еще один рубеж для обеспечения безопасности.

Минимальная поверхность атаки

Разбиение монолитного приложения на простые микросервисы создает между ними «чистые» интерфейсы, позволяющие при тщательном проектировании снизить сложность системы, а значит, и уменьшить поверхность атаки. С другой стороны, добавление дополнительного слоя оркестрации контейнеров само по себе создает новую поверхность для атаки.

Ограничение радиуса поражения

В случае взлома контейнеризированного приложения средства безопасности помогают локализовать атаку внутри контейнера и не позволить ей затронуть остальную систему.

Разделение обязанностей

Если передавать права и учетные данные только в те контейнеры, которым они нужны, то утечка одного набора секретных данных не обязательно будет означать потерю их всех.

Все описанные преимущества хороши, но только в теории. На практике их легко свести на нет ненадежной конфигурацией системы, плохой организацией образов контейнера или небезопасными методами работы. К концу книги у вас будет достаточно знаний, чтобы избежать типичных ошибок безопасности при развертывании контейнеризированной системы.

Резюме

Мы в общих чертах рассмотрели виды атак, способных повлиять на развертывание контейнеров, а также принципы безопасности, применяемые для защиты от этих атак. В дальнейших главах подробно рассмотрим механизмы, лежащие в основе контейнеров, чтобы вы могли понять, как реализовать эти принципы, используя утилиты безопасности и лучшие практики.

ГЛАВА 2

Системные вызовы Linux, права доступа и привилегии

В большинстве случаев контейнеры запускаются на компьютере, работающем под управлением Linux. Поэтому желательно разобраться в ее базовых возможностях, чтобы понимать, как они влияют на безопасность, в частности, применительно к контейнерам. В этой главе мы рассмотрим системные вызовы, права доступа к файлам, привилегии, а в заключение — повышение привилегий (*capabilities*). Если вы уже знакомы с этими понятиями, то можете сразу перейти к главе 3.

Эти понятия жизненно важны, поскольку *контейнеры запускают процессы Linux, видимые с хоста*. Контейнеризированный процесс использует системные вызовы и требует таких же прав доступа и привилегий, что и обычный. Но контейнеры позволяют иначе управлять тем, как эти права назначаются — либо во время выполнения, либо на этапе сборки образа контейнера, что существенно влияет на безопасность.

Системные вызовы

Приложения выполняются в так называемом *пользовательском пространстве* (user space), уровень привилегий которого ниже, чем у ядра ОС. Чтобы обратиться к файлу, передать данные по сети или даже узнать время суток, приложению приходится просить ядро выполнить соответствующее действие. Программный интерфейс, с помощью которого код из пользовательского пространства выполняет подобные запросы к ядру, называется интерфейсом *системных вызовов* (system call).

Существует более 400 различных системных вызовов, причем их число зависит от версии ядра Linux. Вот несколько примеров.

`read`

Чтение данных из файла.

`write`

Запись данных в файл.

`open`

Открытие файла для последующего чтения или записи.

`execve`

Запуск исполняемой программы.

`chown`

Изменение владельца файла.

`clone`

Создание нового процесса.

Разработчикам приложений редко приходится напрямую иметь дело с системными вызовами, поскольку обычно они обернуты в высокоуровневые программные абстракции. Вероятно, самая низкоуровневая абстракция, с которой может столкнуться разработчик приложений, — библиотека `glibc` или `musl` либо пакет `syscall` языка `Golang`. Однако на практике они обычно также обернуты в более высокоуровневые слои абстракций.



Если вы хотите подробнее узнать о системных вызовах, можете посмотреть мое выступление «Руководство по системным вызовам для начинающих» (A Beginner's Guide to Syscalls) (<https://oreil.ly/HrZzJ>), доступное на образовательной платформе издательства O'Reilly.

В коде приложения системные вызовы применяются совершенно одинаково, вне зависимости от того, запущено оно в контейнере или нет. Но, как вы увидите далее, тот факт, что все контейнеры на одном хосте используют одно и то же ядро — то есть обращаются к нему через системные вызовы, — имеет серьезные последствия для безопасности.

Не всем приложениям необходимы системные вызовы, поэтому — следуя принципу минимума привилегий — в Linux предусмотрены средства защиты, позволяющие ограничить набор системных вызовов, доступных различным программам. В главе 10 вы увидите, как это реализуется в контексте контейнеров.

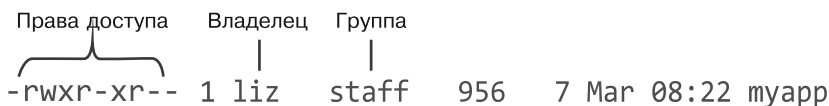
К вопросу пользовательского пространства и привилегий уровня ядра мы вернемся в главе 5. А пока обсудим, как в Linux происходит управление правами доступа к файлам.

Права доступа к файлам

В любой системе на основе Linux, независимо от того, используете ли вы контейнеры, права доступа к файлам — краеугольный камень безопасности. Существует известное выражение: «Все в Linux является файлом» (https://en.wikipedia.org/wiki/Everything_is_a_file). Код приложений, данные, конфигурационные сведения, журналы и т. д. — все хранится в файлах. Даже физические устройства, например мониторы и принтеры, представлены в системе в виде файлов. Права доступа к файлам определяют, какие пользователи могут обращаться к ним и какие действия выполнять над ними. Такой механизм управления доступом иногда называют *избирательным управлением доступом* (discretionary access control, DAC)¹.

Разберем это понятие более подробно.

Если вы хоть немного работали в терминале Linux, то наверняка вам пришлось использовать команду `ls -l`, чтобы получить информацию о файлах и их атрибутах (рис. 2.1).



```

Права доступа  Владелец  Группа
┌───┴───┐
-rw-r--r--  1 liz    staff    956    7 Mar 08:22 myapp
  
```

Рис. 2.1. Пример прав доступа к файлу в Linux

На рис. 2.1 показан файл с именем `myapp`, владельцем которого является пользователь `liz`, а связанной группой — `staff`. Атрибуты прав доступа определяют, какие действия пользователи могут выполнять с этим файлом в зависимости от их роли. В данном примере можно видеть девять символов, соответствующих атрибутам прав доступа, которые следует рассматривать как группы по три символа:

- первая группа из трех символов описывает права доступа для владельца данного файла (здесь — пользователя `liz`);
- вторая группа описывает права доступа для членов группы, с которой ассоциирован данный файл (в данном случае `staff`);
- последние три символа отражают, что могут делать с этим файлом прочие пользователи (не `liz` и не члены группы `staff`).

В зависимости от того, установлены ли биты `r`, `w` и `x`, пользователи могут производить над этим файлом три действия: чтение, запись и выполнение

¹ В русскоязычной литературе встречаются также варианты «разграничительное управление доступом» и «дискреционное управление доступом». — *Примеч. пер.*

соответственно. Три символа в каждой группе отражают, установлены или не установлены биты, демонстрируя таким образом, какие из трех указанных действий разрешены — тире означает, что бит не установлен.

В нашем примере производить запись в файл может только его владелец, поскольку бит `w` установлен лишь в первой группе символов, соответствующей правам доступа владельца. Выполнять файл может как владелец, так и любой из членов группы `staff`, а читать — все пользователи, поскольку бит `r` установлен во всех трех группах символов.



Если вы хотите больше узнать о правах доступа в Linux и о том, как их можно изменять с помощью команды `chmod`, обратитесь к этой статье: <https://www.linuxjournal.com/content/mastering-linux-file-permissions-and-ownership>.

Вероятно, что вы уже знакомы с битами `r`, `w` и `x`. Однако это еще не все. На права доступа влияют также специальные биты `setuid`, `setgid` и *бит закрепления* (sticky bit). С точки зрения безопасности первые два из них имеют особое значение, поскольку с их помощью процессы могут получать дополнительные права, которыми затем способны воспользоваться злоумышленники в своих целях.

Биты `setuid` и `setgid`

В обычной ситуации, когда пользователь запускает файл, процесс наследует его числовой идентификатор. Если же у файла установлен бит `setuid`, то процесс будет выполняться с идентификатором пользователя владельца файла. Посмотрим, как это работает, на примере копии исполняемого файла `sleep`:

```
$ ls -l $(which sleep)
-rwxr-xr-x 1 root 35336 Apr  5 2024 /usr/bin/sleep
$ cp /usr/bin/sleep ./mysleep
$ ls -l mysleep
-rwxr-xr-x 1 liz liz 35336 May  7 10:50 mysleep
```

Как показывает вывод команды `ls`, исходная версия команды принадлежит суперпользователю (пользователь `root`), а ее копия — моему пользователю `liz`, под которым я выполнила команду `cp`. Запустите эту копию, выполнив команду `./mysleep 10`, и во втором терминале посмотрите на работающий процесс. Число `10` означает, что у вас есть на это 10 с, прежде чем процесс завершится (чтобы сделать вывод более понятным, я убрала из него несколько строк):

```
$ ps -fc mysleep
UID      PID      PPID  C  STIME TTY          TIME CMD
liz      37920    37876  0  06:07 pts/0      00:00:00 ./mysleep 10
```

В этом случае процесс выполняется от имени пользователя `liz`. Теперь изменим атрибуты исполняемого файла так, чтобы он принадлежал суперпользователю, и заодно установим бит `setuid`:

```
$ sudo chown root ./mysleep
$ sudo chmod +s ./mysleep
$ ls -l mysleep
-rwsr-sr-x 1 root liz 35336 May  7 10:50 mysleep
```

Теперь снова выполните `./mysleep 10` с привилегиями обычного пользователя и посмотрите в другом терминале на запущенные процессы:

```
$ ps -fc mysleep
UID          PID     PPID  C  STIME TTY          TIME CMD
root         37940    37876  0  06:09 pts/0    00:00:00 ./mysleep 10
```

Теперь процесс `mysleep` выполняется от имени суперпользователя, который назначен владельцем исполняемого файла. Это означает, что процесс обладает всеми привилегиями суперпользователя.

Обычно с помощью этого бита программе предоставляются привилегии, необходимые ей для работы, но недоступные обычным пользователям.

PING И SETUID

Классический пример, где может пригодиться бит `setuid`, — команда `ping`, которой требуется право на открытие сетевых сокетов, чтобы отправлять и принимать ICMP-сообщения (Internet Control Message Protocol — протокол межсетевых управляющих сообщений).

Администратор может быть не против того, чтобы его пользователи применяли утилиту `ping`, но едва ли он разрешит им открывать сокет прямого доступа для других целей. Поэтому для исполняемого файла `ping` обычно сразу устанавливается бит `setuid`, и владельцем его является суперпользователь, так что команда `ping` может использовать привилегии, обычно доступные лишь суперпользователю.

С появлением сокетов ICMP в версии ядра 5.6 в этом больше нет необходимости. Процессам разрешено открывать сокет такого типа, предназначенные исключительно для передачи и приема ICMP-сообщений, что позволяет запускать команду `ping` без каких-либо дополнительных привилегий.

На момент написания книги большинство дистрибутивов не задействуют механизм ICMP-сокетов в своих версиях `ping`. Вместо этого файлу `ping` предоставляется привилегия (capability) `CAP_NET_RAW`. Мы рассмотрим ее в разделе «Привилегии Linux» далее в этой главе.

Еще до примера с `mysleep` вы видели, как работает `setuid`. Он используется командой `sudo`, которая представлена исполняемым файлом, принадлежащим суперпользователю:

```
$ ls -l $(which sudo)
-rwsr-xr-x 1 root root 277936 Apr  8 2024 /usr/bin/sudo
```

Установка бита `setuid` в команде `sudo` означает, что исполняемый файл будет запускаться от имени суперпользователя, что соответствует виденному ранее в выводе команды `ps`.



После запуска `sudo` проверяет, есть ли у пользователя, вызвавшего команду, право на ее выполнение, сверяясь с содержимым файла `sudoers` или используя другой настроенный механизм политики безопасности. Чтобы узнать об этом подробнее, обращайтесь к справке `man sudo`.

Как было показано на примере `mysleep`, исполняемый файл с установленным битом `setuid` позволяет пользователю повысить свои привилегии. Однако иногда результаты могут отличаться от ожидаемых! Давайте посмотрим, что получится, если установить бит `setuid` для копии `bash`:

```
$ cp $(which bash) ./mybash
$ sudo chown root ./mybash
$ sudo chmod +s ./mybash
$ ls -l mybash
-rwsr-sr-x 1 root liz 1446024 May  7 15:33 mybash
```

Поскольку этот исполняемый файл принадлежит суперпользователю и у него установлен бит `setuid`, логично предположить, что при его запуске процесс будет работать с привилегиями суперпользователя `root`. Но взгляните, что получится, если попытаться запустить этот файл:

```
$ ./mybash
mybash-5.2$ whoami
liz
```

Как видите, процесс выполняется *не* от имени `root`, хотя бит `setuid` установлен и владельцем файла является `root`. Что произошло? Дело вот в чем: в современных версиях `bash` (и в некоторых других интерпретаторах, таких как `python`, `node` и `ruby`) исполняемый файл запускается от имени `root`, после чего явным образом задает только нужные ему привилегии и сбрасывает свой идентификатор пользователя (UID), возвращая его к UID исходного пользователя, чтобы избежать потенциального повышения привилегий.



Если вы хотите подробнее изучить этот механизм, можно воспользоваться утилитой **strace**, которая позволяет просматривать системные вызовы, выполняемые **bash** (или **mybash**). Для начала определите идентификатор процесса (PID) вашей командной оболочки (например, командой **echo \$\$**), после чего в другом терминале, запущенном от имени суперпользователя, выполните:

```
$ sudo strace -f -p <ID процесса командной оболочки> \
-e trace=execve,getuid,setresuid,setuid
```

В результате вы увидите трассировку всех системных вызовов, выполненных из первой командной оболочки, включая все запущенные в ней исполняемые файлы. Найдите системный вызов **setresuid()** или **setuid()**, сбрасывающий идентификатор пользователя.

Лишь очень немногие исполняемые файлы сбрасывают UID подобным образом. Можете использовать копию исполняемого файла команды **sleep**, упомянутой ранее в этой главе, чтобы посмотреть на более традиционное поведение **setuid**.

Проведя эти эксперименты с битом **setuid**, мы можем приступить к оценке его потенциального влияния на безопасность.

Влияние бита **setuid** на безопасность

Бит **setuid** позволяет пользователю действовать так, будто он является другим пользователем, что может дать ему доступ к файлам, программам, а также привилегии, которыми он не должен обладать. Поскольку **setuid** открывает опасный путь к повышению привилегий, некоторые сканеры образов контейнеров (рассматриваются в главе 8) сообщают о наличии файлов с установленным битом **setuid**.

Как вы уже видели, современные версии **bash**, а также большинство командных оболочек и интерпретаторов сбрасывают UID, чтобы исключить возможность простого повышения привилегий. Кроме того, можно запретить использование бита **setuid** внутри контейнера с помощью параметра **--security-opt no-newprivileges** в команде **docker run** (мы еще поговорим об этом в главе 4). Однако это не помешает злоумышленнику записать исполняемый файл с битом **setuid** и принадлежащий пользователю **root** в смонтированный каталог (<https://oreil.ly/viKwm>) на хосте. Пример такого сценария вы найдете в каталоге **chapter2/setuid** в репозитории на GitHub (<https://github.com/lizrice/container-security>). Возможность монтирования томов на хосте может привести к самым разным атакам, и мы подробнее поговорим об этом в главе 11.

Бит **setuid** ведет свою историю еще с тех времен, когда механизм привилегий был гораздо более простым: у процесса либо были **root**-права, либо нет. Бит **setuid** служил механизмом предоставления дополнительных

привилегий не суперпользователям. В версии 2.2 ядра Linux появилась возможность более детально контролировать эти параметры с помощью *привилегий* (capabilities).

Привилегии Linux

В современном ядре Linux существует более 40 привилегий. Они назначаются потокам выполнения и определяют, имеет ли данный поток право производить конкретные действия. Например, чтобы выполнять привязку к какому-либо порту с номером ниже 1024, потоку требуется привилегия `CAP_NET_BIND_SERVICE`. Привилегия `CAP_SYS_BOOT` существует для того, чтобы произвольные исполняемые файлы не имели права перезагружать систему. Для загрузки и выгрузки модулей ядра служит `CAP_SYS_MODULE`, а для загрузки программ eBPF — `CAP_BPF`.



Подробную информацию о привилегиях можно получить на машине под управлением Linux с помощью команды `man capabilities`.

Я уже упоминала, что утилита `ping` использует привилегию `CAP_NET_RAW`, чтобы получить возможность открывать сетевой сокет прямого доступа.

Привилегии назначаются как файлам, так и процессам. Просмотреть привилегии, присвоенные файлу, можно с помощью команды `getcap`¹:

```
$ getcap $(which ping)
/usr/bin/ping cap_net_raw=ep
```

Посмотреть список привилегий процесса позволяет команда `getpcaps`. Например, в Ubuntu демон `journald` имеет несколько привилегий:

```
$ getpcaps $(pgrep journal)
307: cap_chown,cap_dac_override,cap_dac_read_search,cap_fowner,cap_setgid,
cap_setuid,cap_sys_ptrace,cap_sys_admin,cap_audit_control,cap_mac_override,
cap_syslog,cap_audit_read=ep
```

Многие процессы вообще не обладают никакими привилегиями. Например, моя текущая командная оболочка:

```
$ getpcaps $$
22355: =
```

В прошлом команда `getpcaps` предполагала, что если процесс запущен от имени `root`, то он обладает всеми привилегиями, и поэтому просто отображала весь

¹ В обновленных версиях ОС вывод будет отличаться, так как `ping` был переведен на использование параметра ядра `net.ipv4.ping_group_range`. — *Примеч. науч. ред.*

список. В настоящее время `getpcaps` и другие инструменты уже не делают такого предположения, поэтому процессы, запущенные от имени `root`, обычно отображаются без каких-либо привилегий.

Вы уже видели, что исполняемый файл команды `ping` имеет привилегию `CAP_NET_RAW`. Теперь давайте посмотрим, какие привилегии получает процесс при запуске этого файла. Запустим команду `ping` в одном терминале:

```
$ ping 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.162 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.188 ms
...
```

В другом терминале получите идентификатор процесса `ping` и проверьте его привилегии:

```
$ getpcaps $(pgrep ping)
50394: =
```

Возникает вопрос: как `ping` смог открыть сокет, если у процесса нет необходимой для этого привилегии `CAP_NET_RAW`? И почему у него нет этой привилегии, если ранее мы видели, что она назначена исполняемому файлу?

Ответ можно найти, выполнив трассировку системных вызовов с помощью `strace`, как уже мы делали это, изучая поведение `bash`. Завершите команду `ping` и получите PID оболочки с помощью `echo $$`. Во втором терминале введите:

```
$ sudo strace -f -p <ID процесса оболочки> \
-e trace=capget,capset,socket
```

Вы увидите, что современная версия `ping` получает и намеренно отбрасывает привилегию `CAP_NET_RAW` сразу после открытия сокета, когда надобность в привилегии отпадает (несколько вызовов и деталей, не относящихся к делу, опущены для ясности):

```
capget({version=_LINUX_CAPABILITY_VERSION_3, pid=0},      ❶
{effective=0, permitted=1<<CAP_NET_RAW, inheritable=0}) = 0
capset({version=_LINUX_CAPABILITY_VERSION_3, pid=0},      ❷
{effective=1<<CAP_NET_RAW, permitted=1<<CAP_NET_RAW, inheritable=0}) = 0
socket(AF_INET, SOCK_RAW, IPPROTO_ICMP) = 3               ❸
socket(AF_INET6, SOCK_RAW, IPPROTO_ICMPV6) = 4
capget({version=_LINUX_CAPABILITY_VERSION_3, pid=0},
{effective=1<<CAP_NET_RAW, permitted=1<<CAP_NET_RAW, inheritable=0}) = 0
capset({version=_LINUX_CAPABILITY_VERSION_3, pid=0},      ❹
{effective=0, permitted=1<<CAP_NET_RAW, inheritable=0})
```

❶ Процесс проверяет, присутствует ли `CAP_NET_RAW` в наборе доступных привилегий. Код возврата 0 указывает, что такая привилегия имеется.

- ❷ Для активизации привилегии в этом процессе используется системный вызов `capset`.
- ❸ Затем процесс открывает сокеты для IPv4 и IPv6.
- ❹ И после того, как сокеты будут открыты, вновь вызывается функция `capset` для деактивации данной привилегии.

К тому моменту, когда мы с помощью `getpcaps` проверили процесс, `CAP_NET_RAW` уже не действует.



Подробнее о влиянии друг на друга привилегий файлов и процессов говорится в статье Эдриена Моата (Adrian Mouat) *Linux capabilities in practice* (<https://oreil.ly/6uin9>).

Исходя из принципа минимума привилегий имеет смысл предоставлять процессам только необходимые для их работы возможности. При запуске контейнера можно управлять привилегиями, как вы увидите в главе 11. Теперь, познакомившись с основными идеями прав доступа и привилегий в Linux, мы можем перейти к обсуждению повышения привилегий.

Повышение привилегий

Термин «повышение привилегий» (privilege escalation) означает выход за пределы имеющихся возможностей для выполнения действий, которые не были разрешены изначально. Для повышения привилегий злоумышленник может воспользоваться, например, системной уязвимостью или некорректными настройками и предоставить себе дополнительные права.

Зачастую злоумышленники начинают действовать от лица непривилегированного пользователя и пытаются получить на машине `root`-права. Распространенный метод повышения привилегий — найти запущенную от имени `root` программу и задействовать ее известные уязвимости. Так, ПО веб-сервера может включать уязвимость, позволяющую злоумышленникам удаленно выполнять код, например уязвимость Struts¹. Если веб-сервер запущен от имени суперпользователя, то весь код, удаленно выполняемый атакующим, будет

¹ В первом издании я упоминала критическую уязвимость удаленного выполнения кода, обнаруженную в 2018 году (<https://oreil.ly/wO3Cr>) и получившую широкое освещение в прессе. С тех пор были выявлены и другие (<https://oreil.ly/adsJ>) серьезные проблемы в Struts (<https://oreil.ly/pAfE3>), что является хорошим примером того, как уязвимости продолжают обнаруживаться в широко используемых программных пакетах и почему важно постоянно обновлять зависимости!

запущен с полномочиями суперпользователя. Поэтому ПО желательно запускать от имени непривилегированных пользователей, когда это возможно.

Как вы узнаете далее, по умолчанию *контейнеры запускаются от имени суперпользователя*. Это означает, что приложения, запущенные в контейнерах, с гораздо большей вероятностью, чем в обычной машине под управлением Linux, будут выполняться от имени суперпользователя. Злоумышленнику, захватившему контроль над процессом внутри контейнера, конечно, все равно придется найти способ выйти за рамки контейнера. Однако если у него это получится, то он сразу получит **root**-права на хосте, не прибегая к какому-либо дальнейшему повышению привилегий. Все это более подробно обсуждается в главе 11.

Но даже если контейнер запущен не от имени **root**, все равно остается возможность повысить привилегии на основе механизмов прав доступа Linux, которые обсуждались ранее в главе:

- образов контейнеров, содержащих исполняемые файлы с установленным битом **setuid**;
- предоставления дополнительных привилегий контейнеру, запущенному от имени не суперпользователя.

Далее в книге вы узнаете, с помощью каких подходов можно уменьшить последствия этих проблем.

Резюме

Из этой главы вы узнали (или вспомнили) об основных механизмах Linux, необходимых для дальнейшей работы с книгой. Описанные механизмы играют важную роль в безопасности; все средства безопасности контейнеров, представленные далее, основаны на этих базовых элементах.

Разобравшись с основными средствами безопасности Linux, мы можем обсудить механизмы, лежащие в основе контейнеров, чтобы вам стало понятно, почему **root** на хосте и в контейнере — это, по сути, одно и то же.

Контрольные группы

В этой главе вам предстоит познакомиться с одним из главных «кирпичиков» создания контейнеров — *контрольными группами* (control groups, cgroups).

Контрольные группы служат для ограничения доступных группе процессов ресурсов, например памяти, CPU и ресурсов сетевого ввода/вывода. В контейнерах они используются для контролируемого распределения ресурсов между различными рабочими нагрузками. С точки зрения безопасности корректно настроенная контрольная группа позволяет предотвратить ситуацию, когда один процесс влияет на работу других, захватывая все доступные ресурсы, например потребляя весь CPU или память и лишая ресурсов другие приложения. Кроме того, можно ограничить общее число процессов внутри контрольной группы. Это удобная техника для защиты от так называемых fork-бомб, о которых мы поговорим в конце главы.

В главе 4 во всех подробностях будет показано, что контейнеры выполняются как обычные процессы Linux, вследствие чего контрольные группы можно использовать для ограничения ресурсов, доступных каждому из контейнеров.



Большинство современных дистрибутивов Linux используют cgroups версии 2, которая имеет некоторые улучшения по сравнению с первоначальной реализацией, широко применявшейся в период, когда контейнеры только начинали набирать популярность. Именно cgroups v2 сейчас используется в Kubernetes и во всех популярных средах выполнения контейнеров, поэтому далее речь пойдет о ней. Однако в более ранних изданиях, посвященных теме контейнеров, можно встретить упоминание о версии 1.

Основное отличие заключается в том, что в версии 2 используется единая, унифицированная иерархия для управления всеми поддерживаемыми типами ресурсов, тогда как в предыдущей версии для каждого типа управляемых ресурсов существовали отдельные иерархии.

Если вы занимаетесь системным администрированием серверов Linux, то у вас могут быть свои причины создавать контрольные группы и управлять ими

вручную. Однако при использовании контейнеров среда выполнения берет эту роль на себя. Как вы увидите далее, от нас потребуется только указать, какие ресурсы нужно выделить тем или иным рабочим нагрузкам. А теперь давайте подробнее разберемся, как именно cgroups используются для ограничения ресурсов для контейнеров.

Контроллеры контрольных групп

Контрольные группы представлены в файловой системе Linux через точку монтирования `/sys/fs/cgroup`. Управление cgroups осуществляется путем чтения и записи файлов и каталогов, расположенных в этой точке монтирования. Давайте посмотрим, что имеется в этом каталоге:

```
root@vm:/sys/fs/cgroup# ls
cgroup.controllers      io.pressure
cgroup.max.depth        io.prio.class
cgroup.max.descendants    io.stat
cgroup.pressure          memory.numa_stat
cgroup.procs            memory.pressure
cgroup.stat             memory.reclaim
cgroup.subtree_control  memory.stat
cgroup.threads          memory.zswap.writeback
cpu.pressure            misc.capacity
cpu.stat               misc.current
cpu.stat.local         misc.peak
cpuset.cpus.effective  proc-sys-fs-binfmt_misc.mount
cpuset.cpus.isolated   sys-fs-fuse-connections.mount
cpuset.mems.effective  sys-kernel-config.mount
dev-hugepages.mount     sys-kernel-debug.mount
dev-mqueue.mount        sys-kernel-tracing.mount
init.scope             system.slice
io.cost.model          user.slice
io.cost.qos
```

Как вы увидите далее, чтобы получить новую контрольную группу, нужно создать новый каталог, в котором, в свою очередь, следует организовать дочерние контрольные группы и тем самым сформировать иерархию контрольных групп.

Файл `cgroup.controllers` содержит список контроллеров cgroup, доступных на компьютере:

```
root@vm:/sys/fs/cgroup# cat cgroup.controllers
cpuset cpu io memory hugetlb pids rdma misc
```

Каждый контроллер управляет определенным типом ресурсов, которые могут потребоваться процессам. Например, контроллер `cpu` управляет использованием CPU процессами в cgroup, а контроллер `memory` управляет доступной им памятью.

Чтобы изменения вступили в силу, контроллеры необходимо активизировать, записав имя контроллера в файл `cgroup.subtree_control`. При этом контроллер будет активизирован для `cgroup`, только если он активен в родительской группе. Каждый запущенный процесс Linux является членом ровно одной контрольной группы, а идентификаторы процессов, принадлежащих к группе, перечислены в файле `cgroup.procs`.

Создание и настройка контрольных групп

Создание подкаталога внутри `/sys/fs/cgroup` приводит к формированию контрольной группы, а ядро Linux автоматически заполняет его различными файлами для параметров и статистики этой группы.

```
root@vm:/sys/fs/cgroup# mkdir liz
root@vm:/sys/fs/cgroup# ls liz
cgroup.controllers
cgroup.events
cgroup.freeze
...
pids.max
pids.peak
rdma.current
rdma.max
```

Подробное описание назначения каждого из этих файлов выходит за рамки книги, однако одни файлы содержат параметры для управления ограничениями ресурсов контрольной группы, а другие — показатели текущего использования ресурсов в данной контрольной группе. Например, нетрудно догадаться, что файл `memory.current` показывает, сколько памяти в настоящий момент использует группа. А максимальный объем памяти, доступный группе, задается файлом `memory.max`:

```
root@vm:/sys/fs/cgroup/liz# cat memory.max
max
```

Значение `max` в выводе означает, что использование памяти для этой группы процессов не ограничено — это значение по умолчанию, если ограничение не задано. Если процессу не задать лимит доступной памяти, он может оставить без ресурсов другие службы на том же хосте. Такое обычно происходит либо в результате утечки памяти в приложении, либо в результате атаки на истощение ресурсов (https://en.wikipedia.org/wiki/Resource_exhaustion_attack), при которой злоумышленник намеренно использует утечку памяти, чтобы занять всю доступную память.

Установив ограничения на объем потребляемых ресурсов, можно уменьшить последствия подобных атак и обеспечить нормальную работу остальных процессов.

Чтобы задать ограничение для контрольной группы, достаточно записать значение в файл, соответствующий ограничиваемому параметру. Давайте ограничим объем доступной памяти в байтах для только что созданной контрольной группы:

```
root@vm:/sys/fs/cgroup/memory/liz# echo 100000 > memory.max
```

После этого значение параметра `memory.max` будет соответствовать заданному лимиту — вероятно, округленному (в меньшую сторону) до величины, кратной размеру страницы:

```
root@vm:/sys/fs/cgroup/memory/liz# cat memory.max
98304
```

Теперь вы знаете, как задавать ограничения. Однако остается еще один важный элемент механизма `cgroups` — распределение процессов по контрольным группам.

Привязка процесса к контрольной группе

Как уже упоминалось, процессы, принадлежащие контрольной группе, перечислены в файле `cgroup.procs`. Сразу после создания новая контрольная группа не содержит процессов, то есть данный файл будет пустым.

В момент запуска процесс присоединяется к группе своего родителя. Однако его можно переместить в другую контрольную группу, просто записав PID в файл `cgroup.procs` той группы, к которой нужно его присоединить. В следующем примере 29903 — это PID командной оболочки:

```
root@vm:/sys/fs/cgroup/memory/liz# echo 29903 > cgroup.procs
root@vm:/sys/fs/cgroup/memory/liz# cat cgroup.procs
29903
root@vm:/sys/fs/cgroup/memory/liz# cat /proc/29903/cgroup
0::liz
```

Командная оболочка теперь относится к контрольной группе `liz`, а объем доступной ей памяти ограничен значением чуть меньше 100 Кбайт. Не так уж много, поэтому даже попытка выполнить `ls` в данной командной оболочке приведет к превышению этого ограничения:

```
$ ls
killed
```

При превышении ограничения на память выполнение процесса прекращается.

Контрольные группы для контейнеров

Ранее было сказано, что управление контрольными группами осуществляется через изменение файлов в файловой системе `cgroup` для соответствующего типа ресурса. Несложно заметить, как этот механизм используется на практике в Docker.



Чтобы повторить приведенные примеры, вам понадобится Docker, работающий непосредственно на (виртуальной) машине Linux. Docker для Mac/Windows работает внутри VM. Это значит (как вы увидите в главе 5), что эти примеры у вас работать не будут, поскольку демон Docker и контейнеры выполняются на отдельном ядре в этой VM.

Поскольку большинство дистрибутивов Linux используют `systemd` и именно драйвер `systemd` является рекомендуемым для `cgroups` в Kubernetes, в моих примерах предполагается применение драйвера `systemd`. Некоторые читатели могут столкнуться с альтернативным драйвером `cgroupfs`, но для данной книги единственное существенное различие между ними заключается в том, что контрольные группы создаются в другом месте в файловой системе `cgroup`.

Docker автоматически создает контрольную группу для каждого контейнера, иерархия которой выглядит так:

```
/sys/fs/cgroup/system.slice/  
└─ docker-<container_id>.scope/  
    ├── cpu.max  
    ├── memory.max  
    ├── pids.max  
    └── cgroup.procs  
    ...
```

Часть `system.slice` в иерархии указывает, что управление контрольными группами осуществляется с помощью драйвера `systemd`.

В следующем примере контейнер запускается в фоновом режиме с ограничением объема памяти в 100 Мбайт (104 857 600 байт). Как можно заметить, для соблюдения этого ограничения Docker применяет механизм контрольных групп. Контейнер будет находиться в режиме ожидания достаточно долго, чтобы вы смогли увидеть его контрольную группу:

```
root@vm:~# docker run --rm --memory 100M -d alpine sleep 10000  
68fb008c5fd3f9067e1aa245b4522a9f3675720d8953371ecfcf2e9faf91b8a0  
root@vm:/sys/fs/cgroup# ls system.slice/docker-68fb...2e9faf91b8a0.scope  
cgroup.controllers  
cgroup.events  
cgroup.freeze  
...
```

Проверим лимит памяти для этого контейнера и ее текущий используемый объем:

```
root@vm:/sys/fs/cgroup# cat system.slice/docker-68fb...scope/memory.max
104857600
root@vm:/sys/fs/cgroup# cat system.slice/docker-68fb...scope/memory.current
462848
```

Можно также убедиться, что наш процесс бездействия является членом этой контрольной группы:

```
root@vm:/sys/fs/cgroup# cat system.slice/docker-68fb...scope/cgroup.procs
19824
root@vm:/sys/fs/cgroup# pgrep sleep
19824
```

В Kubernetes вы можете устанавливать лимиты памяти и CPU для подов и отдельных контейнеров (<https://kubernetes.io/docs/concepts/configuration/manage-resources-containers>) через спецификацию пода. Если вы проверите конфигурацию cgroup на хосте после того, как под запущен, то увидите, что эти лимиты преобразованы в настройки cgroup.

Предотвращение взрыва fork-бомбы

Fork-бомба — это программа, быстро создающая процессы, которые, в свою очередь, тоже создают процессы, что приводит к экспоненциальному росту потребления ресурсов и в итоге парализует работу машины. Далее я покажу, как воспроизвести такой сценарий, но из соображений безопасности не запускайте fork-бомбу в системе, которой вы не можете пожертвовать ради любопытства¹!



Если не хотите рисковать, то посмотрите видеозапись моего доклада, сделанного несколько лет назад (<https://www.youtube.com/watch?v=MhV6cWjvQjM>), — там вы найдете демонстрацию.

Ранее в этой главе я создала контрольную группу с именем `liz` и установила ограничение на использование памяти. Давайте снимем ограничение на память и вместо этого определим максимальное количество процессов, которые могут быть запущены в группе:

```
root@vm:/sys/fs/cgroup/liz# echo max > memory.max
root@vm:/sys/fs/cgroup/liz# echo 20 > pids.max
```

¹ Можно безопасно протестировать fork-бомбу, если создать образ с ней, затем запустить контейнер из этого образа, а в новой сессии терминала выполнить `docker stats` для отслеживания количества процессов. Например: `docker run --rm -d --name forkbomb --pids-limit 1000 forkbomb && docker stats forkbomb`. — *Примеч. науч. ред.*

Добавим текущую оболочку в группу:

```
root@vm:/sys/fs/cgroup/liz# echo $$ > cgroup.procs
```

Проверим количество процессов:

```
root@vm:/sys/fs/cgroup/liz# cat pids.current
2
```

Как видно, в этой группе всего два процесса: первый — это оболочка, добавленная явно путем записи ее идентификатора процесса в файл `cgroup.procs`. Поскольку вновь создаваемые процессы наследуют группу своего родителя, второй наблюдаемый процесс — это программа `cat`, работающая внутри оболочки.

Теперь запустим fork-бомбу — эта команда будет работать только в оболочке `bash`:

```
root@vm:/sys/fs/cgroup/liz# :(){ :|:& };;
```

Вскоре вы увидите, как терминал заполняется сообщениями вида `bash: fork: retry: Resource temporarily unavailable`. Это происходит потому, что процессы не могут быть созданы из-за установленных ограничений. Появление записей в терминале может раздражать, однако другие процессы на машине по-прежнему смогут нормально работать. Если бы fork-бомба продолжала создавать новые процессы, вы бы увидели, как другие операции останавливаются.

Остановить fork-бомбу можно с помощью функции `cgroup.kill`. К сожалению, оболочка, в которой запущена fork-бомба, тоже падет жертвой этой операции, потому что она завершит все процессы в группе:

```
root@vm:/sys/fs/cgroup/liz# echo 1 > cgroup.kill
```

Давайте разберем, как работает fork-бомба `:(){ :|:& };;`, потому что это очень любопытный синтаксический прием, хотя и не имеющий прямого отношения к безопасности контейнеров:

- `:() { ... }` определяет функцию с именем `:` (да, двоеточие — допустимое имя функции в Bash);
- тело функции — `:|:&`. Каждый символ `:` обозначает вызов функции, то есть функция вызывает саму себя и передает результат в другой вызов самой себя, при этом символ `&` запускает этот второй вызов в фоновом режиме. Передача вывода функции приводит к ее запуску в новом процессе, как и запуск процесса в фоне. В итоге каждый вызов функции `:` запускает два процесса, каждый из которых, в свою очередь, выполняет функцию `:`;
- символ `;` завершает определение функции;
- последнее двоеточие вызывает только что определенную функцию и запускает каскад создания процессов. Процессы продолжают создаваться, пока не будет достигнуто ограничение на число процессов в контрольной группе.

Вместо того чтобы напрямую работать с `cgroups`, лимит процессов можно задать с помощью параметра `--pids-limit` в команде `docker run`. На момент написания книги в Kubernetes существовала возможность настроить максимальное количество процессов на под (`pod`) (<https://kubernetes.io/docs/concepts/policy/pid-limiting>) через параметр Kubelet (соответственно ограничение будет применяться ко всем подам на узле, а не отдельно к каждому).

Резюме

Контрольные группы ограничивают ресурсы, доступные различным Linux-процессам. Чтобы воспользоваться возможностями контрольных групп, не обязательно применять контейнеры, но Docker и прочие среды выполнения контейнеров предоставляют удобный интерфейс для работы с ними: задавать лимит по ресурсам можно прямо при запуске контейнера, а контрольные группы обеспечивают соблюдение этих ограничений.

Ограничение ресурсов помогает защититься от целого класса атак, нацеленных на нарушение работы развернутой системы путем чрезмерного потребления ресурсов, лишая другие приложения, таким образом, законного доступа к этим ресурсам. Поэтому рекомендуется задавать ограничения на память и ресурсы CPU при запуске контейнеризированных приложений.

Теперь, разобравшись с ограничением ресурсов в контейнерах, можно перейти к другим механизмам, составляющим технологию контейнеров: пространствам имен и изменению корневого каталога, описанным в главе 4.

Изоляция контейнеров

Именно в этой главе вы узнаете, как на самом деле работают контейнеры! Очень важно понимать, насколько они изолированы друг от друга и от физического хоста. Вы самостоятельно сможете оценить прочность границ безопасности, окружающих контейнеры.

Если вы когда-нибудь выполняли команду `docker exec -ti <образ> bash`, то наверняка знаете, что контейнер изнутри очень похож на виртуальную машину. Если получить доступ к командной оболочке контейнера и в ней выполнить команду `ps`, то вы увидите только процессы, работающие внутри контейнера. У контейнера свой сетевой стек и на первый взгляд собственная файловая система с корневым каталогом, никак не связанным с корневым каталогом физического хоста. Контейнеры можно запускать с ограниченными ресурсами, например с фиксированным объемом оперативной памяти или с использованием лишь части доступных процессоров. Сделать все это позволяют возможности Linux, которые мы более подробно обсудим в данной главе.

Однако, как бы ни были похожи контейнеры и виртуальные машины по типу изоляции, важно отдавать себе отчет, что контейнеры — это *не* виртуальные машины, и в главе 5 мы обсудим их различия. По моему опыту, без глубокого понимания их различий совершенно невозможно оценить степень эффективности обычных мер безопасности контейнеров и разобраться, где необходимо применить специализированные средства.

Вы увидите, что для контейнеров используются такие механизмы Linux, как пространства имен и `chroot`, а также контрольные группы (`cgroups`), которые мы обсуждали в главе 3. Имея четкое представление о соответствующих концепциях, вы узнаете, насколько хорошо защищены ваши приложения, запущенные внутри контейнеров.

И хотя основные принципы этих механизмов относительно просты, они могут взаимодействовать с другими функциями ядра Linux довольно запутанным образом. Именно тонкие нюансы взаимодействия пространств имен, привилегий

и файловых систем играют главную роль в понимании уязвимостей, позволяющих выйти за рамки контейнера (например, CVE-2019-5736 (<https://oreil.ly/NtcRv>) — серьезной уязвимости, обнаруженной как в `runc`, так и в `LXC`).

Пространства имен Linux

Контрольные группы позволяют контролировать ресурсы, доступные процессу, а *пространства имен* (namespaces) — ресурсы, которые он видит. Помещая процесс в пространство имен, можно ограничить такие ресурсы.

Пространства имен ведут свою историю от операционной системы Plan 9 (<https://oreil.ly/BCi9W>). В те времена у большинства ОС было одно общее «пространство имен» файлов. Системы Unix позволяли монтировать файловые системы, но лишь в единое для всей системы пространство имен файлов. В Plan 9 все процессы входили в одну из групп процессов со своим «пространством имен» — иерархией файлов (и файлоподобных объектов), доступных данной группе процессов. Каждая группа процессов могла монтировать собственный набор файловых систем, не видя файловых систем друг друга.

Первое пространство имен появилось в ядре Linux в версии 2.4.19 в 2002 году. Это было пространство имен монтирования с функциональностью, аналогичной операционной системе Plan 9. На сегодняшний день Linux поддерживает несколько типов пространств имен, которым соответствуют такие ресурсы:

- система разделения времени Unix (Unix Timesharing System, UTS) — звучит довольно сложно, но фактически это пространство имен служит для изоляции имени хоста и доменного имени, которые видит процесс;
- идентификаторы процессов (PID);
- точки монтирования;
- сеть;
- идентификаторы пользователей и групп;
- обмен информацией между процессами (inter-process communication, IPC)¹;
- контрольные группы;
- время.

Процесс в Linux всегда принадлежит ровно одному пространству имен каждого типа. При запуске системы Linux в ней существует одно пространство имен

¹ В русскоязычной литературе нередко можно также встретить термин «межпроцессное взаимодействие». — *Примеч. пер.*

каждого типа, но, как вы увидите, можно создавать дополнительные пространства имен и распределять по ним процессы. Просмотреть пространства имен на машине можно с помощью команды `lsns`:

```
$ lsns
      NS TYPE      NPROCS      PID USER COMMAND
4026531834 time          3 848409 liz -bash
4026531835 cgroup        3 848409 liz -bash
4026531836 pid           3 848409 liz -bash
4026531837 user          3 848409 liz -bash
4026531838 uts           3 848409 liz -bash
4026531839 ipc           3 848409 liz -bash
4026531840 net            3 848409 liz -bash
4026531841 mnt            3 848409 liz -bash
```

На первый взгляд все выглядит идеально: по одному пространству имен для каждого из вышеупомянутых типов. К сожалению, это неполная картина! В справке (`man`) (<https://man7.org/linux/man-pages/man8/lsns.8.html>) по команде `lsns` говорится, что утилита «читает информацию непосредственно из файловой системы `/proc` и для пользователей без прав `root` может возвращать не всю информацию». Пользователю `root` видны дополнительные пространства имен, но они не представляют особого интереса, пока вы не начнете создавать свои пространства. Я упомянула об этом, только чтобы подчеркнуть, что запускать `lsns` необходимо от имени `root` (или задействовать `sudo`), чтобы видеть полную картину.

Теперь посмотрим, как с помощью пространств имен создать нечто по поведению напоминающее *контейнер*.



В примерах из этой главы для создания контейнера используются инструкции командной оболочки Linux. Инструкции по созданию контейнера на языке программирования Go можно найти по адресу <https://github.com/lizrice/containers-from-scratch>.

Изоляция имени хоста

Начнем с пространства имен для системы разделения времени Unix (UTS). Как уже упоминалось, она охватывает имя хоста и доменное имя. Поместив процесс в отдельное пространство имен UTS, можно изменить имя хоста для этого процесса независимо от имени хоста самой (виртуальной) машины, на которой он запущен.

Узнать имя хоста в Linux можно с помощью команды:

```
$ hostname
myhost
```

Большинство (если не все) систем контейнеров присваивают каждому контейнеру случайный ID. По умолчанию этот идентификатор используется в качестве имени хоста. Убедиться в этом можно, запустив контейнер и подключившись к нему через командную оболочку. Вот как это сделать в Docker:

```
$ docker run --rm -it --name hello ubuntu bash
root@cdf75e7a6c50:/$ hostname
cdf75e7a6c50
```

Кстати, из этого примера видно, что, даже если присвоить в Docker контейнеру название (в данном случае я задала `--name hello`), оно не используется в качестве имени хоста контейнера.

Имя хоста контейнера может отличаться, поскольку Docker создает его с собственным пространством имен UTS. Тот же результат можно получить, воспользовавшись командой `unshare` для создания процесса с отдельным пространством имен UTS.

Как описано на соответствующей странице справки (`man unshare`), `unshare` позволяет «запускать программы с пространствами имен, отдельными от родительского процесса». Разберемся подробнее, что это означает. При «запуске программы» ядро создает новый процесс и выполняет в нем программу. Это происходит в контексте текущего — *родительского* — процесса, а новый процесс при этом называется *дочерним*. Слово *unshare* («прекратить совместный доступ») означает, что дочерний процесс получает собственное пространство имен, а не делит его с родительским.

Попробуем это на практике. Вам понадобятся привилегии суперпользователя, поэтому в начале строки используется `sudo`:

```
liz@myhost:~$ sudo unshare --uts sh
$ hostname
myhost
$ hostname experiment
$ hostname
experiment
$ exit
liz@myhost:~$ hostname
myhost
```

Данный пример запускает командную оболочку `sh` в новом процессе с новым пространством имен UTS. Все программы в ней наследуют ее пространства имен. Когда вы выполняете команду `hostname`, она запускается в новом пространстве имен UTS, изолированном от пространства имен UTS хоста.

Если открыть еще одно окно терминала на том же хосте перед командой `exit`, то можно убедиться, что имя хоста не поменялось для (виртуальной) машины в целом. Изменение имени хоста на основном хосте не влияет на имя хоста, которое видит процесс, заключенный в новое пространство имен, и наоборот.

Это ключевой элемент работы контейнеров. Благодаря пространствам имен они получают набор ресурсов (в данном случае имя хоста), независимых от хоста и прочих контейнеров. Но процесс все равно выполняется тем же самым ядром Linux. Это имеет последствия для безопасности, о которых я расскажу чуть позже. А пока рассмотрим еще один пример пространства имен, которое предоставляет контейнерам возможность получить собственное видение запущенных процессов.

Изоляция идентификаторов процессов

Выполнив команду `ps` в контейнере Docker, вы увидите только процессы, работающие внутри него, и ни одного процесса, запущенного на хосте:

```
$ docker run --rm -it --name hello ubuntu bash
root@cdf75e7a6c50:/$ ps -eaf
UID          PID  PPID  C  STIME TTY          TIME CMD
root           1      0  0  18:41 pts/0    00:00:00 bash
root          10      1  0  18:42 pts/0    00:00:00 ps -eaf
```

Это происходит благодаря пространству имен PID, которое ограничивает набор видимых идентификаторов процессов. Попробуйте снова выполнить команду `unshare`, однако на сей раз укажите, что вам нужно новое пространство имен PID, добавив флаг `--pid`:

```
liz@myhost:~$ sudo unshare --pid sh
$ whoami
root
$ whoami
sh: 2: Cannot fork
$ whoami
sh: 3: Cannot fork
$ ls
sh: 4: Cannot fork
```

Не слишком успешно: после первой команды `whoami` не получается выполнить ни одной команды! Но в выведенных результатах есть несколько интересных деталей.

Первый процесс под `sh` сработал нормально, но все команды после него завершились неудачей из-за невозможности создать копию процесса. Ошибка выводится в формате `<команда>: <ID_процесса>: <сообщение>`, и, как видите, идентификатор процесса всякий раз увеличивается на единицу. Исходя из данной последовательности, можно предположить, что первая команда `whoami` выполняется в процессе с ID 1, а значит, пространство имен PID все же работает, однако нумерация идентификаторов процессов начинается заново. Однако пользы от этого немного, раз нельзя запустить больше одного процесса!

Ключ к решению проблемы можно найти в описании флага `--fork` в справке для команды `unshare`: «Копию заданной программы следует запускать как дочерний процесс `unshare`, а не непосредственно. Это может потребоваться после создания нового пространства имен `PID`».

Можете поэкспериментировать с этим, выполнив команду `ps`, чтобы просмотреть иерархию из другого окна терминала:

```
liz@myhost:~$ ps fa
  PID TTY          STAT       TIME COMMAND
...
 924537 pts/0    Ss          0:00 -bash
 924718 pts/0    S+          0:00  \_ sudo unshare --pid sh
 924719 pts/1    Ss          0:00      \_ sudo unshare --pid sh
 924720 pts/1    S+          0:00          \_ sh
```

Процесс `sudo` создает собственную копию для мониторинга, а затем выполняет команду `unshare`. Процесс `sh` при этом является дочерним не для `unshare`, а для (второго) процесса `sudo`.



Если вы используете версию `sudo` 1.9.14 или новее, то увидите два процесса `sudo`, как в данном примере, но при использовании старых версий будет только один. На странице руководства для `sudo` говорится, что в более свежих версиях эта утилита запускает еще один процесс в новом псевдотерминале (в моем примере в `pts/1`), который будет осуществлять мониторинг, а потом создаст второй процесс для запуска заданной команды. Проверить версию на своем компьютере вы можете командой `sudo -V`.

Теперь попробуйте выполнить команду `unshare` с параметром `--fork`:

```
liz@myhost:~$ sudo unshare --pid --fork sh
$ whoami
root
$ whoami
root
```

Явный прогресс: мы теперь можем выполнить более одной команды без возникновения ошибки `Cannot fork`. Если посмотреть на иерархию во втором окне терминала, то вы увидите существенное отличие:

```
liz@myhost:~$ ps fa
  PID TTY          STAT       TIME COMMAND
...
 924537 pts/0    Ss          0:00 -bash
 925113 pts/0    S+          0:00  \_ sudo unshare --pid --fork sh
 925114 pts/1    Ss          0:00      \_ sudo unshare --pid --fork sh
 925115 pts/1    S           0:00          \_ unshare --pid --fork sh
 925116 pts/1    S+          0:00              \_ sh
...
```

При использовании параметра `--fork` оболочка `sh` запускается как потомок `unshare`, поэтому внутри нее допускается выполнять сколько угодно дочерних команд. Результаты выполнения `ps` могут вас удивить, учитывая тот факт, что эта командная оболочка находится в отдельном пространстве идентификаторов процессов:

```
liz@myhost:~$ sudo unshare --pid --fork sh
$ ps -eaf
UID          PID    PPID  C STIME TTY          TIME CMD
root          1        0  0 Jul22 ?           00:00:09 /sbin/init
root          2        0  0 Jul22 ?           00:00:00 [kthreadd]
root          3        2  0 Jul22 ?           00:00:00 [pool_workqueue_release]
...еще много строк для разных процессов...
root        13172   12943  0 13:03 pts/2       00:00:00 unshare --pid --fork sh
root        13173   13172  0 13:03 pts/2       00:00:00 sh
root        13174   13173  0 13:03 pts/2       00:00:00 ps -eaf
```

Как видно, команда `ps` по-прежнему отображает все процессы всей системы, хотя и работает внутри нового пространства имен идентификаторов процессов. Чтобы `ps` вела себя так, как в контейнере `Docker`, нового пространства имен `PID` недостаточно. Причина этого изложена на странице справки команды `ps`: «Работа команды `ps` основана на чтении виртуальных файлов в `/proc`».

Посмотрим на каталог `/proc`, чтобы понять, о каких виртуальных файлах идет речь. В вашей системе список будет похожим, но не идентичным, поскольку набор работающих в ней процессов отличается:

```
liz@myhost:~$ ls /proc
1      29      492628 64      acpi      loadavg
10     29375   492642 65      bootconfig locks
10927  29451   492656 7       buddyinfo mdstat
1181   3       492664 72      bus       meminfo
... еще множество строк ...
```

Все каталоги в `/proc`, у которых имена состоят из цифр, соответствуют идентификаторам процессов, и внутри каждого содержится множество интересной информации о соответствующем процессе. Например, `/proc/<pid>/exe` — это символическая ссылка на исполняемый файл, запущенный в рамках конкретного процесса, как видно из данного примера:

```
liz@myhost:~$ ps
  PID TTY          TIME CMD
28441 pts/1    00:00:00 bash
28558 pts/1    00:00:00 ps

liz@myhost:~$ ls /proc/28441
arch_status  fdinfo      ns          smaps_rollup
attr         gid_map     numa_maps   stack
autogroup    io          oom_adj     stat
auxv         ksm_merging_pages oom_score   statm
```

cgroup	ksm_stat	oom_score_adj	status
clear_refs	latency	pagemap	syscall
cmdline	limits	patch_state	task
comm	loginuid	personality	timens_offsets
coredump_filter	map_files	projid_map	timers
cpu_resctrl_groups	maps	root	timerslack_ns
cpuset	mem	sched	uid_map
cwd	mountinfo	schedstat	wchan
environ	mounts	sessionid	
exe	mountstats	setgroups	
fd	net	smaps	

```
liz@myhost:~$ ls -l /proc/28441/exe
lrwxrwxrwx 1 liz 0 Oct 10 13:32 /proc/28441/exe -> /usr/bin/bash
```

Вне зависимости от того, в каком пространстве имен PID выполняется процесс, команда `ps` будет искать информацию о запущенных процессах в каталоге `/proc`. Чтобы `ps` возвращала данные только о процессах, запущенных внутри нового пространства имен, нужна отдельная копия каталога `/proc`, в которую ядро могло бы записывать сведения о процессах, заключенных в отдельное пространство имен. А поскольку каталог `/proc` располагается в корне дерева каталогов, то это означает необходимость изменения корневого каталога.

Изменение корневого каталога

Изнутри контейнера видна не вся файловая система хоста, а только ее подмножество, поскольку при создании контейнера корневой каталог изменяется.

Изменить корневой каталог в Linux можно с помощью команды `chroot`. Фактически она перемещает корневой каталог текущего процесса в другую точку в файловой системе. После выполнения `chroot` доступ ко всем каталогам, располагавшимся в иерархии файлов выше нового корневого каталога, теряется, поскольку подняться выше корня файловой системы невозможно (рис. 4.1).

Файлы на хосте:

```
/some/directory/on/host/file1
/some/directory/on/host/dir1/file2
```

Процесс с корнем, перенесенным в

/some/directory/on/host, видит:

```
/file1
/dir1/file2
```

Рис. 4.1. Изменение корневого каталога, после чего процессу становится видна лишь часть файловой системы

Описание на странице справки для команды `chroot` гласит: «Выполняет команду, назначив корневым каталогом *новый_корневой_каталог*. [...] Если команда не задана, то выполняет `${командная_оболочка} -i` (по умолчанию: `/bin/sh -i`)».

Иными словами, `chroot` не просто меняет корневой каталог, но и выполняет указанную команду, а если другая команда не задана, то запускает командную оболочку.

Создайте новый каталог и попробуйте изменить корневой на него с помощью команды `chroot`:

```
liz@myhost:~$ mkdir new_root
liz@myhost:~$ sudo chroot new_root
chroot: failed to run command '/bin/bash': No such file or directory
liz@myhost:~$ sudo chroot new_root ls
chroot: failed to run command 'ls': No such file or directory
```

Не работает! Дело в том, что в новом корневом каталоге нет подкаталога `bin`, из-за чего невозможно запустить командную оболочку `/bin/bash`. По той же причине не удастся найти и команду `ls`. Для запуска любых команд необходимо, чтобы соответствующие файлы были доступны внутри нового корневого каталога. Именно так работает «настоящий» контейнер: из образа контейнера создается экземпляр контейнера, который включает видимую контейнером файловую систему. Если внутри нее нет нужного исполняемого файла, то контейнер не сможет его найти и выполнить.

Попробуем запустить Alpine Linux в контейнере. Alpine — минималистичный дистрибутив Linux, предназначенный как раз для контейнеров. Для начала необходимо скачать файловую систему¹:

```
liz@myhost:~$ mkdir alpine
liz@myhost:~$ cd alpine
liz@myhost:~/alpine$ curl -o alpine.tar.gz https://dl-cdn.alpinelinux.org/
alpine/v3.22/releases/${uname -m}/alpine-minirootfs-3.22.1-${uname -m}.tar.gz
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 3425k 100 3425k 0 0 22.1M 0 --:--:-- --:--:-- --:--:-- 22.3M
liz@myhost:~/alpine$ tar xvf alpine.tar.gz
```

¹ Параметр `$(uname -m)` здесь определяет архитектуру. Например, на машине с процессором ARM он примет значение `aarch64`, а на машине с процессором Intel — значение `x86_64`. Вам также нужно найти последнюю версию Alpine (<https://alpinelinux.org/releases>). На момент написания книги это была версия 3.22.1, но, вероятно, к тому времени, как вы будете читать эти строки, она уже изменится.

После этого у вас появится копия файловой системы Alpine внутри созданного нами каталога `alpine`. Удалите архив и перейдите обратно в родительский каталог:

```
liz@myhost:~/alpine$ rm alpine.tar.gz
liz@myhost:~/alpine$ cd ..
```

Содержимое файловой системы можно посмотреть с помощью команды `ls alpine`. Оно похоже на корневой каталог обычной файловой системы Linux: здесь есть каталоги `bin`, `lib`, `var`, `tmp` и т. д.

Теперь, когда дистрибутив Alpine распакован, можно переместиться в каталог `alpine`, используя команду `chroot`, указав при этом команду, существующую внутри иерархии этого каталога.

Здесь есть некоторые нюансы: исполняемый файл должен располагаться по системному пути нового процесса. Данный процесс наследует среду родительского процесса, включая переменную окружения `PATH`. Подкаталог `bin` в каталоге `alpine` становится каталогом `/bin` нового процесса. Поэтому если ваш обычный системный путь включает каталог `/bin`, то исполняемый файл `ls` можно запустить из этого каталога, не задавая путь к нему явным образом:

```
liz@myhost:~$ sudo chroot alpine ls
bin  etc  lib  mnt  proc  run  srv  tmp  var
dev  home media opt  root  sbin  sys  usr
liz@myhost:~$
```

Обратите внимание: этот новый корневой каталог предназначен только для дочернего процесса (в нашем случае процесса, выполняющего команду `ls`). По его завершении контроль возвращается родительскому процессу. Если запустить командную оболочку как дочерний процесс, она не завершится сразу, и это позволит лучше рассмотреть эффект от смены корневого каталога:

```
liz@myhost:~$ sudo chroot alpine sh
/ $ ls
bin  etc  lib  mnt  proc  run  srv  tmp  var
dev  home media opt  root  sbin  sys  usr
/ $ whoami
root
/ $ exit
liz@myhost:~$
```

Если же попытаться запустить командную оболочку `bash`, то ничего не получится. Причина в том, что в дистрибутиве Alpine она отсутствует, поэтому ее нет и внутри нового корневого каталога. Если бы вы проделали тот же эксперимент с файловой системой, например Ubuntu, включающей `bash`, все бы сработало корректно.

Подведем итог: команда `chroot` буквально «подменяет корневой каталог» процесса. После такой операции процесс (и все его дочерние процессы) сможет обращаться только к файлам и каталогам, которые находятся ниже в иерархии этого нового корневого каталога.



Помимо `chroot`, существует и системный вызов `pivot_root`. В текущей главе не играет роли, какой из них использовать; главное, что у контейнера должен быть свой корневой каталог. В приведенных примерах я задействовала команду `chroot`, поскольку она немного проще и привычнее для многих.

С точки зрения безопасности у `pivot_root` есть несколько преимуществ по сравнению с `chroot`, поэтому на практике в реализации среды выполнения контейнеров обычно применяется именно она. Основное различие состоит в том, что `pivot_root` использует пространство имен монтирования: старый корневой каталог отключается и становится недоступным внутри пространства имен монтирования. Системный же вызов `chroot` работает иначе, поэтому старый корневой каталог остается доступным через точки монтирования.

Вот каким образом можно создать для контейнера собственную корневую файловую систему. Мы обсудим это подробнее в главе 6, а пока посмотрим, как благодаря подобной отдельной корневой файловой системе ядро Linux показывает контейнеру лишь ограниченное множество ресурсов, заключенных в отдельное пространство имен.

Сочетание возможностей пространств имен и изменения корневого каталога

До сих пор мы рассматривали пространства имен и изменение корневого каталога как нечто отдельное, однако их можно объединить, выполнив команду `chroot` в новом пространстве имен:

```
liz@myhost:~$ sudo unshare --pid --fork chroot alpine sh
/ $ ls
bin    etc    lib    mnt    proc   run    srv    tmp    var
dev    home  media  opt    root   sbin   sys    usr
```

Как отмечалось в разделе «Изоляция идентификаторов процессов», наличие собственного корневого каталога позволяет контейнеру создать каталог `/proc`, независимый от каталога `/proc` хоста. Чтобы он заполнялся информацией о процессах, его необходимо смонтировать в виде псевдофайловой системы типа `proc`. Сочетание пространства имен идентификаторов процессов

и независимого каталога `/proc` позволит команде `ps` выводить только процессы внутри пространства имен PID:

```
/ $ mount -t proc proc proc
/ $ ps
PID  USER      TIME  COMMAND
  1   root        0:00   sh
  5   root        0:00   ps
```

Получилось! Не так просто, как изолировать имя хоста контейнера, но, комбинируя создание пространства имен идентификаторов процессов, изменение корневого каталога и монтирование псевдофайловой системы для обработки информации о процессах, мы смогли ограничить «поле зрения» контейнера, поэтому ему теперь видны лишь его собственные процессы.

Если этот пример показался вам сложным, то вам наверняка будет интересно узнать, что команда `unshare` имеет параметр `--mount-proc`, упрощающий задачу:

```
liz@myhost:~$ sudo unshare --pid --fork --mount-proc bash
root@myhost:/home/liz# ps
  PID TTY          TIME CMD
    1 pts/4        00:00:00 bash
    8 pts/4        00:00:00 ps
```

Это далеко не все пространства имен. Посмотрим на пространство имен монтирования.

Пространство имен монтирования

Обычно нежелательно, чтобы точки монтирования файловых систем внутри контейнера совпадали с точками монтирования файловых систем хоста. Разделить их можно, если создать для контейнера его собственное пространство имен монтирования.

Вот пример создания простой связанной точки монтирования для процесса с отдельным пространством имен монтирования:

```
liz@myhost:~$ sudo unshare --mount sh
$ mkdir source
$ touch source/HELLO
$ ls source
HELLO
$ mkdir target
$ ls target
$ mount --bind source target
$ ls target
HELLO
```


После формирования такой связанной точки монтирования содержимое каталога `source` становится доступно в `target`. Всего точек монтирования (при взгляде изнутри этого процесса), наверное, будет немало, но если вы выполнили предыдущий пример, то сможете найти `target` с помощью такой команды:

```
$ findmnt target
TARGET      SOURCE                                FSTYPE OPTIONS
/home/liz/target
            /dev/sda1[/home/liz/source] ext4    rw,discard,
            errors=remount-ro, commit=30
```

Для хоста она невидима. Это легко проверить, выполнив ту же команду в другом окне терминала и убедившись, что она ничего не возвращает.

Попробуйте снова выполнить `findmnt` внутри пространства имен монтирования, но на этот раз без каких-либо параметров, — вы получите длинный список точек монтирования. Возможность для контейнера видеть все точки монтирования на хосте может показаться нелогичной. Очень похоже на ситуацию с пространством имен идентификаторов процессов: ядро использует каталог `/proc/<PID>/mounts`, чтобы обмениваться информацией о точках монтирования для каждого процесса. Если создать процесс, имеющий отдельное пространство имен монтирования, но использующий каталог `/proc` хоста, то окажется, что его файл `/proc/<PID>/mounts` содержит все существующие точки монтирования хоста. (Получить список точек монтирования можно командой `cat`.)

Чтобы получить полностью изолированный набор точек монтирования для контейнеризированного процесса, необходимо объединить создание нового пространства имен монтирования с новой корневой файловой системой и новой точкой монтирования `proc`:

```
liz@myhost:~$ sudo unshare --mount chroot alpine sh
/ $ mount -t proc proc proc
/ $ mount
proc on /proc type proc (rw,relatime)
/ $ mkdir source
/ $ touch source/HELLO
/ $ mkdir target
/ $ mount --bind source target
/ $ mount
proc on /proc type proc (rw,relatime)
/dev/root on /target type ext4 (rw,relatime,discard,errors=remount-ro,commit=30)
```

В дистрибутиве Alpine Linux команда `findmnt` отсутствует, поэтому в данном примере список точек монтирования генерируется с помощью команды `mount` без параметров. (Если вы скептически относитесь к подобной замене, то попробуйте выполнить предыдущий пример с командой `mount` вместо `findmnt` и убедитесь, что результаты не отличаются.)

Наверное, вам знакома идея монтирования каталогов хоста к контейнеру с помощью команды `docker run -v <каталог_хоста>:<каталог_контейнера> ...`. Чтобы реализовать это, после создания корневой файловой системы для контейнера сначала создается каталог `target` в контейнере, а затем производится связанное монтирование каталога `source` хоста в `target`. А поскольку у каждого контейнера свое пространство имен монтирования, то каталоги хоста, смонтированные подобным образом, не видны из других контейнеров.



Если вы создадите монтирование, видимое с хоста, оно не очистится автоматически при завершении процесса контейнера. Его необходимо удалять явно через команду `umount`. То же относится и к псевдофайловым системам `/proc`. Никакого вреда от них нет, но если вы предпочитаете поддерживать порядок, можете удалить их командой `umount proc`. При этом система не позволит отключить последнюю точку монтирования `/proc`, используемую хостом.

Пространство имен сети

Благодаря пространству имен сети контейнер получает независимое представление о сетевых интерфейсах и таблицах маршрутизации. Посмотреть это представление для процесса, созданного с отдельным пространством имен сети, можно с помощью команды `lsns`:

```
liz@myhost:~$ sudo lsns -t net
      NS TYPE NPROCS      PID USER      NETNSID NSFS  COMMAND
4026531840 net       126        1 root  unassigned  /sbin/init

liz@myhost:~$ sudo unshare --net bash
$ lsns -t net
      NS TYPE NPROCS      PID USER      NETNSID NSFS  COMMAND
4026531840 net       125        1 root  unassigned  /sbin/init
4026532277 net         2    28586 root  unassigned  bash
```

В выводе отображается информация обо всех пространствах имен сети на этом хосте и связанных с ними PID. Я воспользуюсь идентификатором 28586 процесса `bash` чуть позже.



Вам может встретиться команда `ip netns`, но здесь она не пригодится. Команда `unshare --net` создает анонимное пространство имен сети, а такие пространства `ip netns list` не выводит.

Если поместить процесс в отдельное пространство имен сети, то ему будет доступен лишь петлевой интерфейс:

```
# ip a
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

Имея же только петлевой интерфейс, контейнер не сможет взаимодействовать с внешним окружением. Чтобы обеспечить ему доступ к внешнему миру, необходимо создать виртуальный интерфейс Ethernet, точнее говоря, пару виртуальных интерфейсов Ethernet, играющих роль двух концов воображаемого кабеля, который соединяет пространство имен сети контейнера с пространством имен сети по умолчанию.

Создать пару виртуальных интерфейсов Ethernet можно от имени суперпользователя во втором окне терминала, указав два анонимных пространства имен с соответствующими идентификаторами процессов, вот так:

```
liz@myhost:~$ sudo ip link add ve1 netns 28586 type veth peer name ve2 netns 1
```

- Команда `ip link add` сообщает, что мы хотим добавить сетевое подключение.
- `ve1` — название одного из «концов» виртуального Ethernet-«кабеля».
- `netns 28586` указывает, что этот конец «подключается» в пространство имен сети, соответствующее процессу с идентификатором `28586` (согласно выводу команды `lsns -t net` в примере в начале данного раздела).
- `type veth` означает, что речь идет о виртуальной паре Ethernet.
- `peer name ve2` задает название второго «конца кабеля».
- `netns 1` указывает, что этот конец «подключается» в пространство имен сети, соответствующее процессу с идентификатором `1`.

Теперь виртуальный интерфейс Ethernet `ve1` будет виден внутри процесса «контейнера»:

```
# ip a
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ve1@if3: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group ...
   link/ether 7a:8a:3f:ba:61:2c brd ff:ff:ff:ff:ff:ff link-netnsid 0
```

Соединение находится в состоянии `DOWN` (выключено), поэтому его необходимо включить перед использованием. Причем сделать это необходимо с обоими концами соединения.

Сначала включаем `ve2` на хосте:

```
liz@myhost:~$ sudo ip link set ve2 up
```

Затем включите `ve1` внутри контейнера — после этого соединение должно перейти в состояние `UP`:

```
# ip link set ve1 up
# ip a
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

```
2: ve1@if3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
...
    link/ether 7a:8a:3f:ba:61:2c brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::788a:3fff:feba:612c/64 scope link
        valid_lft forever preferred_lft forever
```

Как видите, этому интерфейсу в контейнере автоматически назначен IPv6-адрес. Давайте проверим доступность IPv6-адреса в контейнере для хоста:

```
liz@myhost:~$ ping6 fe80::788a:3fff:feba:612c%ve2
PING fe80::788a:3fff:feba:612c%ve2 (fe80::788a:3fff:feba:612c) 56 data bytes
64 bytes from fe80::788a:3fff:feba:612c%ve2: icmp_seq=1 ttl=64 time=0.160 ms
64 bytes from fe80::788a:3fff:feba:612c%ve2: icmp_seq=2 ttl=64 time=0.052 ms
64 bytes from fe80::788a:3fff:feba:612c%ve2: icmp_seq=3 ttl=64 time=0.072 ms
```

В отличие от IPv6 адреса IPv4 не присваиваются интерфейсам автоматически, поэтому, чтобы получить возможность отправлять IPv4-трафик через виртуальное Ethernet-соединение, необходимо определить IPv4-адрес на обоих концах. В контейнере:

```
# ip addr add 192.168.1.100/24 dev ve1
```

Теперь ядро знает, что трафик из этого пространства имен сети к любому адресу `192.168.1.*` должен проходить через интерфейс `ve1`, используя исходящий IP-адрес `192.168.1.100`. Помимо прочего, в таблицу маршрутизации в контейнере будет добавлен IP-маршрут:

```
# ip route
192.168.1.0/24 dev ve1 proto kernel scope link src 192.168.1.100
```

В начале раздела уже упоминалось, что пространство имен сети изолирует как интерфейсы, так и таблицу маршрутизации, поэтому данная информация о маршрутизации не зависит от таблицы маршрутизации IP на хосте. На текущем этапе контейнер может отправлять трафик только по адресам подсети `192.168.1.0/24`.

Необходимо также добавить адрес на хосте:

```
liz@myhost:~$ sudo ip addr add 192.168.1.200/24 dev ve2
```

Для проверки можно выполнить `ping` к удаленному концу подключения из контейнера:

```
# ping 192.168.1.200
PING 192.168.1.200 (192.168.1.200) 56(84) bytes of data.
64 bytes from 192.168.1.200: icmp_seq=1 ttl=64 time=0.355 ms
64 bytes from 192.168.1.200: icmp_seq=2 ttl=64 time=0.035 ms
^C
```

В главе 12 мы более подробно изучим вопросы передачи данных по сети и сетевую безопасность контейнеров.

Пространство имен пользователей

Благодаря пространству имен пользователей процессы получают свое представление идентификаторов пользователей и групп. Подобно идентификаторам процессов, пользователи и группы остаются на хосте, но могут получать другие идентификаторы. Основным достоинством этого подхода является возможность связать суперпользователя с идентификатором 0 в контейнере с какой-либо непривилегированной учетной записью на хосте. С точки зрения безопасности преимущества огромны, ведь ПО может запускаться от имени суперпользователя внутри контейнера, но злоумышленник, сумевший выбраться за пределы контейнера на хост, получит доступ лишь к непривилегированной учетной записи, отличной от `root`. Как вы увидите в главе 11, контейнер довольно легко настроить неправильно, так что атакующему будет нетрудно выйти из него на хост. Использование пространств имен пользователей означает, что одна ошибка конфигурации уже не приводит к захвату всей системы.



Поддержка пространств имен пользователей включена по умолчанию, начиная с Kubernetes 1.33, но для этого требуется ядро Linux версии 6.3 или выше (<https://kubernetes.io/docs/concepts/workloads/pods/user-namespaces/#before-you-begin>). Этот механизм также поддерживается в последних версиях сред выполнения контейнеров, таких как `containerd` и `runc`, а в Docker его можно включить с помощью флага `--userns-remap` в настройках демона (<https://docs.docker.com/engine/security/userns-remap>).

По сути, для создания новых пространств имен необходимы привилегии суперпользователя (именно поэтому демон Docker выполняется от имени суперпользователя), но пространство имен пользователей — исключение:

```
liz@myhost:~$ unshare --user bash
nobody@myhost:/home/liz$ id
uid=65534(nobody) gid=65534(nogroup) groups=65534(nogroup)
nobody@myhost:/home/liz$ echo $$
31196
```

Я воспользуюсь идентификатором процесса, полученным командой `echo $$`, чуть позже. А пока обратите внимание, что внутри нового пространства имен пользователей текущий пользователь определяется как `nobody`. Необходимо задать соответствие идентификаторов пользователей внутри и вне пространства имен, как показано на рис. 4.2.

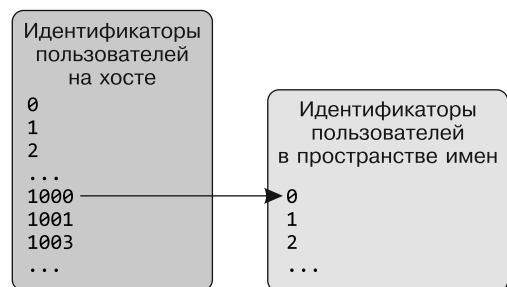


Рис. 4.2. Привязка непривилегированной учетной записи на хосте к учетной записи суперпользователя в контейнере

Упомянутое соответствие описано в файле `/proc/<PID>/uid_map`, который можно редактировать с привилегиями пользователя `root` (на хосте). В этом файле три поля:

- наименьший идентификатор для отображения с точки зрения дочернего процесса;
- наименьший идентификатор, которому он должен соответствовать на хосте;
- количество отображаемых идентификаторов.

Рассмотрим следующий пример. На моей машине пользователь `liz` имеет идентификатор `1001`. Чтобы связать `liz` с идентификатором суперпользователя внутри дочернего процесса, в первых двух полях должны содержаться значения `0` и `1001`. Последнее поле может содержать `1`, если нужно связать только один идентификатор (например, если внутри контейнера нужен лишь один пользователь). Вот команда, которую я выполнила во втором окне терминала, чтобы настроить это соответствие:

```
liz@myhost:~$ sudo echo '0 1001 1' > /proc/31196/uid_map
```

Таким образом, соответствующий процесс получил привилегии суперпользователя внутри пространства имен пользователей. Не удивляйтесь, что в командной строке `bash` по-прежнему отображается `nobody`; здесь значение обновится только после повторного выполнения сценариев, запускаемых при открытии новой командной оболочки (например, `~/.bash_profile`):

```
nobody@myhost:/home/liz$ id
uid=0(root) gid=65534(nogroup) groups=65534(nogroup)
```

Аналогично задается соответствие идентификаторов групп, используемых внутри дочернего процесса, с помощью файла `/proc/<PID>/gid_map`.

Теперь процесс выполняется с идентификатором пользователя `root`, и в более старых версиях Linux этого было достаточно для получения полного набора привилегий суперпользователя. Но в ядре 5.8 данный порядок претерпел существенные изменения: теперь пользователь `root` внутри дочернего процесса не получит автоматически привилегии суперпользователя на хосте — он становится просто «суперпользователем в ограниченном пространстве имен». Посмотрим, как выглядит этот идентификатор с точки зрения хоста. Для начала попробуем выполнить команду `sleep`:

```
nobody@myhost:/home/liz$ sleep 100
```

Обратите внимание, что строка приглашения не обновилась и не отражает доступность привилегий суперпользователя.

Посмотрим, как этот процесс выглядит во втором терминале:

```
liz@myhost:~$ ps -fc sleep
UID          PID    PPID  C  STIME TTY          TIME CMD
liz          84714    84272  0  17:33 pts/0    00:00:00 sleep 100
```

С точки зрения хоста команда `sleep` выполняется от имени непривилегированного пользователя `liz`, хотя внутри пространства имен пользователей он выглядит как суперпользователь. Такой пользователь не обладает привилегией `CAP_SYS_ADMIN`, необходимой, например, для создания нового пространства имен `UTS`:

```
nobody@myhost:~$ unshare --uts
unshare: unshare failed: Operation not permitted
```

В разделе «Изоляция имени хоста» я упоминала, что для успешного выполнения команды `unshare --uts` необходимы `root`-права. Без них команда завершится ошибкой `Operation not permitted` (Операция запрещена) по той же самой причине, что и в нашем примере, — из-за отсутствия привилегии `CAP_SYS_ADMIN`. Однако ядро допускает возможность создания других пространств имен наряду с пространством имен пользователей. Обычный, непривилегированный пользователь на хосте может выполнить такую команду:

```
liz@myhost:~$ unshare --user --uts sleep 100
```

Найдем идентификатор процесса команды `sleep` в другом терминале и проверим его пространства имен:

```
liz@myhost:~$ lsns -p 87982
      NS TYPE      NPROCS   PID USER COMMAND
4026531834 time          4 15244 liz  -bash
4026531835 cgroup          4 15244 liz  -bash
4026531836 pid            4 15244 liz  -bash
4026531839 ipc            4 15244 liz  -bash
4026531840 net              4 15244 liz  -bash
4026531841 mnt              4 15244 liz  -bash
4026532267 user             1 87982 liz  |sleep 100
4026532306 uts             1 87982 liz  |sleep 100
```

Как видите, процесс `sleep` унаследовал большинство пространств имен, но получил собственные пространства имен пользователей и `UTS`.

Иначе говоря, непривилегированный пользователь все-таки может создавать другие пространства имен! Эта возможность кажется весьма удобной, но в таком виде практической пользы от нее мало. Пользователю не предоставляется никаких дополнительных возможностей, поэтому попытка изменить имя хоста завершится неудачей:

```
liz@myhost:~$ unshare --user --uts hostname hello
hostname: you must be root to change the host name
```

Посмотрим, что произойдет, если сделать то же самое от имени привилегированного пользователя:

```
liz@myhost:~$ sudo unshare --user --uts bash
nobody@myhost:/home/liz$ hostname new
hostname: you must be root to change the host name
nobody@myhost:/home/liz$ id
uid=65534(nobody) gid=65534(nogroup) groups=65534(nogroup)
```

Внутри нового пространства имен пользователь идентифицируется как `nobody`. Чтобы UID 0 на хосте соответствовал UID 0 внутри пространства имен пользователей, необходимо явно выполнить привязку. С этой целью во втором терминале можно создать файл `uid_map`, подобно тому как мы делали это ранее, или использовать удобный флаг `--map-root-user` в команде `unshare`:

```
liz@myhost:~$ sudo unshare --user --uts --map-root-user bash
root@myhost:/home/liz# id
uid=0(root) gid=65534(nogroup) groups=65534(nogroup)
root@myhost:/home/liz# cat /proc/$$/uid_map
    0      0      1
root@myhost:/home/liz# hostname new
root@myhost:/home/liz# hostname
new
```

Если контейнеры запускать от имени суперпользователя, то получить права `root` внутри контейнера очень просто, и так же просто настроить контейнер с пространством имен пользователей так, чтобы он не получал права `root` автоматически. Это повышает безопасность, поскольку количество контейнеров, которым необходимы привилегии «настоящего» суперпользователя (то есть `root` с точки зрения хоста), невелико.

Однако запускать контейнеры от имени непривилегированного пользователя и затем получать права `root` внутри контейнера немного сложнее. Такие контейнеры называют *rootless-контейнерами*, и мы рассмотрим их в главе 11.

Пространство имен обмена информацией между процессами

В Linux различные процессы могут обмениваться информацией с помощью разделяемой (совместно используемой) области памяти либо очереди сообщений. Чтобы два процесса могли пользоваться одними и теми же идентификаторами этих механизмов, они должны входить в одно пространство имен обмена информацией между процессами (inter-process communications, IPC).

Как правило, *нежелательно*, чтобы контейнеры могли обращаться к разделяемой памяти друг друга, поэтому им назначаются отдельные пространства имен IPC.

Для примера создайте блок разделяемой памяти и посмотрите на текущее состояние IPC с помощью команды `ipcs`:

```
$ ipcmk -M 1000
Shared memory id: 0
$ ipcs
```

----- Message Queues -----						
key	msqid	owner	perms	used-bytes	messages	
----- Shared Memory Segments -----						
key	shmid	owner	perms	bytes	nattch	status
0x74e9655a	0	root	644	1000	0	

----- Semaphore Arrays -----				
key	semid	owner	perms	nsems

В этом примере созданный новый блок разделяемой памяти (идентификатор которого указан в столбце `shmid`) выводится последним в блоке **Shared Memory Segments** (Сегменты разделяемой памяти). Процесс с отдельным пространством имен IPC не видит ни одного из этих объектов IPC:

```
$ sudo unshare --ipc sh
$ ipcs
```

----- Message Queues -----						
key	msqid	owner	perms	used-bytes	messages	
----- Shared Memory Segments -----						
key	shmid	owner	perms	bytes	nattch	status

----- Semaphore Arrays -----				
key	semid	owner	perms	nsems

Пространство имен контрольных групп

Пространство имен контрольных групп чем-то напоминает `chroot` для файловой системы контрольных групп: оно не позволяет процессу видеть конфигурацию контрольной группы, которая в иерархии каталогов контрольных групп расположена выше, чем его собственная контрольная группа.



В этом разделе предполагается использование контрольных групп версии 2. Чтобы вспомнить, как они работают, обратитесь к главе 2.

Посмотреть на пространство имен контрольных групп в действии можно, сравнив содержимое `/proc/self/cgroup` вне и внутри пространства имен контрольных групп:

```
liz@myhost:~$ cat /proc/self/cgroup
0::/user.slice/user-1001.slice/session-357.scope

liz@myhost:~$ sudo unshare --cgroup bash
root@myhost:/home/liz# cat /proc/self/cgroup
0::/
```

Процесс видит корневую контрольную группу. Однако он имеет полный доступ к корневой файловой системе, поэтому просмотр `/sys/fs/cgroup` показывает иерархию контрольных групп хоста. Например, взглянув на содержимое файла `/sys/fs/cgroup/cgroup.procs`, мы увидим множество процессов, которые не имеют никакого отношения к данному процессу и его собственной контрольной группе. Похоже, что контейнеру требуется собственная файловая система `/proc`, чтобы получить корректное представление о процессах в своем пространстве имен идентификаторов процессов. Ему также нужна собственная версия `/sys/fs/cgroup`. Для решения проблемы, как и прежде, необходимо создать пространство имен монтирования и изменить корневой каталог. В этом примере я использую корневую файловую систему Alpine, которую мы создали ранее:

```
liz@myhost:~$ sudo unshare --cgroup --mount chroot alpine sh
# mkdir -p /sys/fs/cgroup
# mount -t cgroup2 none /sys/fs/cgroup
# ls /sys/fs/cgroup
```

cgroup.controllers	cpu.stat.local	memory.reclaim
cgroup.events	cpu.uclamp.max	memory.stat
cgroup.freeze	cpu.uclamp.min	memory.swap.current
cgroup.kill	cpu.weight	memory.swap.events
cgroup.max.depth	cpu.weight.nice	memory.swap.high
cgroup.max.descendants	io.pressure	memory.swap.max
cgroup.pressure	memory.current	memory.swap.peak
cgroup.procs	memory.events	memory.zswap.current
cgroup.stat	memory.events.local	memory.zswap.max
cgroup.subtree_control	memory.high	memory.zswap.writeback
cgroup.threads	memory.low	pids.current
cgroup.type	memory.max	pids.events
cpu.idle	memory.min	pids.events.local
cpu.max	memory.numa_stat	pids.max
cpu.max.burst	memory.oom.group	pids.peak
cpu.pressure	memory.peak	
cpu.stat	memory.pressure	

Псевдофайловая система контрольных групп заполнена всей необходимой информацией, но остается одна проблема: перечисленные здесь PID отражают

представление процессов на хосте. Для корректной работы также необходимо создать пространство имен идентификаторов процессов:

```
liz@myhost:~$ sudo unshare --mount --pid --fork --cgroup bash
root@myhost:/home/liz# mount -t proc proc alpine/proc
root@myhost:/home/liz# mount -t cgroup2 none alpine/sys/fs/cgroup
root@myhost:/home/liz# chroot alpine sh
# ps
PID   USER      TIME  COMMAND
    1   root          0:00  bash
    22   root          0:00  sh
    23   root          0:00  ps
# cat /sys/fs/cgroup/cgroup.procs
0
0
0
0
0
0
1
22
26
```

Результат выглядит вполне ожидаемо, но что означают все эти нули в файле `cgroup.procs`? Ответ прост: это процессы в данной контрольной группе, которые находятся за пределами пространства имен идентификаторов процессов. Дочерний процесс имеет собственное представление о контрольных группах, но по-прежнему является членом контрольной группы родительского процесса. Родительский процесс может создать контрольную группу для дочернего, добавив новый каталог в `/sys/fs/cgroup` и записав PID дочернего процесса в файл `cgroup.procs`.

Пространство имен времени

Пространство имен времени позволяет процессу менять свои значения `CLOCK_MONOTONIC` и `CLOCK_BOOTTIME`, чтобы создать впечатление, что система загрузилась в другое время. Этот механизм предназначен для бесшовной миграции процессов между системами, чтобы таймеры и задержки могли продолжить работу с того места, где они остановились, а также для воспроизведения проблем, зависящих от времени, если значение переменной формируется на основе текущего времени.

Однако представьте, какая может возникнуть путаница, если каждый контейнер в распределенной системе будет иметь свое представление о времени! Во-первых, одна только попытка согласовать журналы и метрики между контейнерами стала бы весьма сложной задачей! Во-вторых, у злоумышленника

появится возможность замаскировать свою вредоносную активность, создав видимость, что она имела место в прошлом или в будущем. По этой причине ни одна из известных мне систем поддержки контейнеров не использует пространство имен времени.

К этому моменту мы познакомились со всеми типами пространств имен и узнали, как их использовать совместно с `chroot` для изоляции представления процессом своего окружения. В сочетании с информацией о контрольных группах, приведенной в главе 3, этого должно быть достаточно для понимания всего, что требуется для создания того, что мы называем *контейнером*.

Поды и пространства имен контейнеров в Kubernetes

В Kubernetes *подом* (pod) называют группу из одного или нескольких контейнеров. У каждого контейнера есть собственные корневые файловые системы, благодаря чему они могут иметь отдельные наборы зависимостей. Однако контейнеры не полностью изолированы друг от друга с помощью пространств имен:

- контейнеры в одном поде используют общее пространство имен сети, что дает им один и тот же IP-адрес и позволяет обмениваться данными через петлевой интерфейс;
- контейнеры в одном поде используют общее пространство имен IPC и способны обмениваться данными друг с другом через сокеты домена UNIX и разделяемую память;
- при необходимости под можно настроить так, чтобы он совместно использовал пространство имен Process ID, позволяя контейнерам видеть процессы друг друга и отправлять им сигналы.

Прежде чем перейти к главе 5, имеет смысл взглянуть на контейнер с точки зрения хоста, на котором он работает.

Процессы контейнера с точки зрения хоста

Хотя мы называем их контейнерами, правильнее было бы говорить «контейнеризированные процессы». Контейнер по-прежнему представляет собой обычный процесс Linux, работающий на хосте. Однако этот процесс видит лишь ограниченную часть этого хоста, имеет доступ только к некоторой области файловой системы и, возможно, к набору ресурсов, ограниченному контрольными группами. Поскольку контейнер — всего лишь процесс, он существует

в контексте ОС хоста и задействует одно и то же ядро с хостом, как показано на рис. 4.3.

В главе 5 мы сравним этот подход с виртуальными машинами, но прежде более подробно рассмотрим, насколько контейнеризированный процесс действительно изолирован от хоста и от остальных контейнеризированных процессов на том же хосте. Для этого проведем несколько экспериментов над контейнером Docker.

Запустите процесс командной оболочки в контейнере на основе Ubuntu (или другого дистрибутива Linux, который вам по душе) и выполните в ней команду `sleep` на 1000 с. Обратите внимание, что данная команда выполняется как процесс внутри контейнера. При нажатии клавиши `Enter` после ввода команды `sleep` Linux создает копию текущего процесса с новым PID и запускает исполняемый файл `sleep` в этом процессе.

Процесс `sleep` можно перевести в фоновый режим (нажмите `Ctrl+Z` для приостановки процесса, а затем выполните `bg %1`). Теперь выполните `ps` внутри контейнера, чтобы посмотреть на тот же процесс с точки зрения контейнера:

```
$ docker run --rm -it ubuntu bash
root@ab6ea36fce8e:/$ sleep 1000
^Z
[1]+  Stopped                  sleep 1000
root@ab6ea36fce8e:/$ bg %1
[1]+  sleep 1000 &
root@ab6ea36fce8e:/$ ps
  PID TTY          TIME CMD
   1 pts/0        00:00:00 bash
  10 pts/0        00:00:00 sleep
  11 pts/0        00:00:00 ps
root@ab6ea36fce8e:/$
```

Пока команда `sleep` выполняется, откройте второй терминал на том же хосте и посмотрите на этот же приостановленный процесс с точки зрения хоста:

```
$ ps -C sleep
  PID TTY          TIME CMD
30591 pts/0        00:00:00 sleep
```

Параметр `-C sleep` указывает, что нас интересуют только процессы, в которых запущен исполняемый файл `sleep`.

У контейнера есть собственное пространство имен PID, поэтому вполне естественно, что идентификаторы его процессов меньше в числовом выражении,

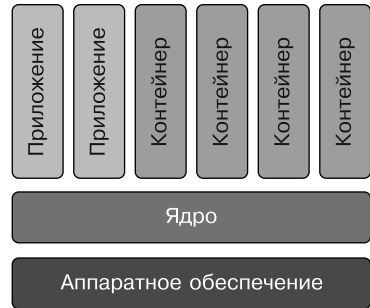


Рис. 4.3. Контейнеры используют одно ядро с хостом

и именно это мы и наблюдаем в выводе команды `ps`, выполненной в контейнере. Однако с точки зрения хоста тот же самый процесс `sleep` имеет другой, больший идентификатор. В предыдущем примере отображается только один процесс — с идентификатором `30591` на хосте и `10` в контейнере. (Конкретный номер на хосте может отличаться в зависимости от того, какие еще процессы выполняются сейчас или запускались ранее на этой машине, но, вероятно, число будет намного больше.)

Чтобы лучше понимать контейнеры и обеспечиваемый ими уровень изоляции, важно знать следующее: хотя идентификаторы процессов и отличаются, они оба указывают на *один и тот же процесс*. Просто с точки зрения хоста числовому идентификатору процесса присвоен другой, более высокий номер.

То, что процессы контейнера видны с хоста, — одно из фундаментальных различий контейнеров и виртуальных машин. Злоумышленник, получивший доступ к хосту, может видеть и влиять на *все работающие на этом хосте процессы*, особенно если он обладает привилегиями суперпользователя. Как вы увидите в главе 11, существует немало на удивление простых способов непреднамеренно предоставить злоумышленнику возможность выйти за пределы взломанного контейнера.

Хосты и контейнеры

Как вы уже заметили, контейнеры и их хосты используют общее ядро, что определенным образом влияет на практические рекомендации по хостам, на которых работают контейнеры. Если хост скомпрометирован, все расположенные на нем контейнеры оказываются под угрозой, особенно когда злоумышленник получает полномочия суперпользователя или иные привилегии (например, членство в группе `docker`, участники которой могут управлять контейнерами, если в качестве среды выполнения используется `Docker`).

Настоятельно рекомендуется запускать контейнерные приложения на выделенных хостах (неважно, на виртуальных машинах или физических), в основном из соображений безопасности.

- Применение оркестратора для работы контейнеров означает, что людям (почти) не нужен доступ к хостам. Если никакие другие приложения не используются на этих машинах, то достаточно очень маленького набора учетных записей пользователей на хостах. Управлять ими проще, а попытки неавторизованного входа легче обнаружить.
- В качестве операционной системы хоста для запуска контейнеров Linux можно использовать любой дистрибутив Linux, но существует несколько «облегченных» сборок, специально предназначенных для запуска контейнеров. Они включают только необходимые для работы контейнеров ком-

поненты и за счет этого уменьшают поверхность атаки на хост. В их числе: Flatcar, Talos и Bottlerocket. Чем меньше компонентов установлено на хосте, тем ниже вероятность существования в них уязвимостей (см. главу 8).

- Все хосты в кластере могут использовать одну конфигурацию, без каких-либо требований, относящихся к конкретным приложениям. Это упрощает автоматизацию подготовки хостов и позволяет рассматривать их как неизменяемые. Если хосту требуется обновление, то нужно не устанавливать на нем патчи, а просто исключить его из кластера и заменить новой машиной. Неизменяемость машин упрощает выявление вторжений.

Мы вернемся к преимуществам неизменяемости в главе 8.

Использование «облегченных» операционных систем сокращает набор возможных настроек, но не исключает их полностью. Например, на каждом хосте будет работать среда выполнения контейнеров (возможно, `containerd`) плюс оркестратор (скажем, `kubelet` Kubernetes). Эти компоненты имеют множество параметров, часть которых связана с безопасностью. Центр по интернет-безопасности (Center for Internet Security, CIS) (<https://cisecurity.org/>) публикует оценки эффективности практических рекомендаций по настройке и выполнению различных программных компонентов, включая Docker, Kubernetes и Linux.

В корпоративной среде имеет смысл использовать решение для обеспечения безопасности контейнеров, которое бы также защищало хосты, сообщая об уязвимостях и потенциально небезопасных значениях настроек. Кроме того, не помешает журналирование и оповещение о входах и попытках входа в систему на уровне хоста.

Резюме

Поздравляю! Раз вы дошли до конца этой главы, то должны уже хорошо представлять себе, чем в действительности является контейнер. Мы рассмотрели три основных механизма ядра Linux, с помощью которых можно ограничить доступ процессов к ресурсам хоста:

- пространства имен служат для ограничения видимых процессу контейнера ресурсов, например изоляции предоставляемого контейнеру набора PID;
- изменение корневого каталога ограничивает набор файлов и каталогов, доступных контейнеру;
- контрольные группы позволяют управлять ресурсами, которыми контейнер может пользоваться.

Как вы видели в главе 1, изоляция рабочих нагрузок друг от друга — важный аспект безопасности контейнеров. Теперь вам уже должно быть ясно, что все

контейнеры на конкретном хосте (виртуальной машине или реальном сервере) задействуют одно и то же ядро. Конечно, нечто похожее происходит и в многопользовательской системе, где различные пользователи могут подключаться к одной машине и запускать приложения. Однако в такой системе администраторы обычно ограничивают полномочия всех пользователей, и, уж конечно, не предоставляют им всех привилегий суперпользователя. Контейнеры же — по крайней мере на момент написания книги — по умолчанию запускаются от имени `root` и полагаются на границы изоляции, обеспечиваемые пространствами имен, измененным корневым каталогом и контрольными группами, чтобы один контейнер не мог вмешиваться в работу другого.

В главе 10 мы обсудим варианты укрепления границ безопасности вокруг контейнеров, а пока подробнее изучим, как работают виртуальные машины. Благодаря этому мы сможем сравнить уровень изоляции контейнеров и виртуальных машин, особенно с точки зрения безопасности.

Виртуальные машины

Контейнеры очень часто сравнивают с виртуальными машинами, особенно в плане изоляции. Поэтому важно четко понимать, как функционируют виртуальные машины, чтобы говорить о различиях между ними и контейнерами. Это особенно важно для оценки границ зон безопасности приложений, запущенных в контейнерах или различных виртуальных машинах. Оценивая преимущества контейнеров с точки зрения безопасности, отнюдь не лишним будет знать, чем они отличаются от виртуальных машин.

Впрочем, все не настолько однозначно. Как вы увидите в главе 10, существует несколько специализированных инструментов, усиливающих изоляцию контейнеров, приближая их к виртуальным машинам. Но чтобы понимать все плюсы и минусы подобных подходов с точки зрения безопасности, лучше начать с изучения разницы между VM и «обычным» контейнером.

Принципиальное различие между ними состоит в том, что в виртуальной машине выполняется полная копия операционной системы, включая ядро, в то время как контейнер использует общее ядро совместно с хостом. Чтобы понять, что это значит, необходимо иметь представление в том, как гипервизор (Virtual Machine Monitor, VMM) создает их и управляет ими. Для начала обсудим, что происходит при загрузке компьютера.

Загрузка компьютера

Представьте физический сервер: с несколькими CPU, оперативной памятью и сетевыми интерфейсами. Когда компьютер включается, запускается специальная программа — BIOS (Basic Input Output System — базовая система ввода/вывода). Она проверяет объем доступной оперативной памяти, находит сетевые интерфейсы и прочие устройства, например мониторы, клавиатуру, подключенные накопители и т. д.

На самом деле значительная часть этой функциональности сегодня перешла в UEFI (Unified Extensible Firmware Interface — универсальный расширяемый интерфейс прошивки), но для простоты будем считать его современным BIOS.

Опознав аппаратное обеспечение, система запускает загрузчик, который загружает и выполняет код ядра ОС — Linux, Windows или какой-либо другой. Как вы видели в главе 2, код ядра работает на более высоком уровне привилегий, чем код приложения. Такие привилегии позволяют ему взаимодействовать с памятью, сетевыми интерфейсами и т. д., в то время как приложения, работающие в пользовательском пространстве, не способны делать этого напрямую.

В процессорах архитектуры x86 уровни привилегий организованы в *кольца* (rings), из которых кольцо 0 имеет наивысшие привилегии, а кольцо 3 — наименьшие. В большинстве операционных систем в обычных условиях (без виртуальных машин) ядро работает на уровне кольца 0, а код пользовательского пространства — на уровне кольца 3, как показано на рис. 5.1.

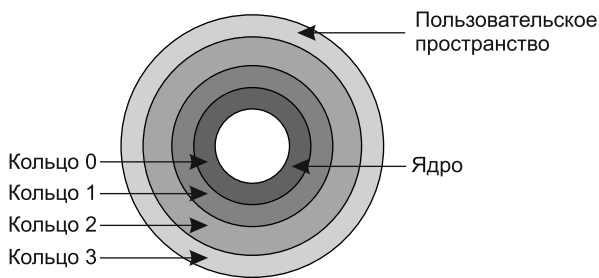


Рис. 5.1. Кольца привилегий

Код ядра (как и любой другой) выполняется на CPU в виде машинных инструкций, которые могут включать привилегированные инструкции для обращения к памяти, запуска потоков выполнения и т. д. Подробный обзор всего, что может и будет происходить при инициализации ядра, выходит за рамки книги, но фактически цель состоит в монтировании корневой файловой системы, настройке передачи данных по сети и запуске всех системных демонов. Если у вас появится желание узнать больше, то на GitHub можно найти множество информации, предоставленной Алексом Кулешовым (Alex Kuleshov), о внутреннем устройстве ядра Linux, включая описание процесса начальной загрузки (<https://github.com/0xAX/linux-insides>).

Завершив инициализацию, ядро начинает запускать программы в пользовательском пространстве. Оно отвечает за управление всем, что необходимо этим программам. Ядро запускает, контролирует и распределяет потоки выполнения, в которых работают эти программы, и отслеживает эти потоки в собственных структурах данных, соответствующих процессам. Одна из важных сторон функциональности ядра — управление памятью. Ядро выделяет процессам память и следит за тем, чтобы блоки памяти одних процессов были недоступны для других.

Знакомство с VMM

Как вы только что видели, в обычном случае ядро напрямую управляет ресурсами машины. В мире же виртуальных машин роль первого слоя управления ресурсами играет VMM (Virtual Machine Monitor, или гипервизор), распределяющий ресурсы между виртуальными машинами. У каждой VM — свое ядро.



Встречается также термин «диспетчер виртуальных машин». Под ним обычно подразумевается пользовательский графический интерфейс или инструмент командной строки, используемый для управления виртуальными машинами, а не компонент, обеспечивающий виртуализацию. Один из известных примеров — `virt-manager`, клиент `vSphere`. Разработчикам, запускающим виртуальные машины Linux в macOS, известен удобный инструмент командной строки `lima`, который предназначен для управления виртуальными машинами, а виртуализацию обеспечивает `QEMU` (Quick Emulator).

Для каждой управляемой виртуальной машины VMM выделяет некоторое количество оперативной памяти и ресурсов CPU, настраивает виртуальные сетевые интерфейсы и прочие виртуальные устройства, после чего запускает гостевое ядро с доступом к этим ресурсам.

На обычном сервере BIOS передает ядру подробную информацию о доступных ресурсах компьютера; в случае с виртуальной машиной VMM делит эти ресурсы и передает гостевым ядрам информацию лишь о той части ресурсов, которые им предоставляются. Ядро гостевой ОС считает, что имеет прямой доступ к физической памяти и устройствам, хотя на самом деле взаимодействует лишь с предоставляемой VMM абстракцией.

VMM отвечает за то, чтобы гостевая ОС и ее приложения не нарушали границы выделенных им ресурсов. Например, если такой системе выделен участок памяти на хосте, ей будет отказано в доступе к памяти вне этого участка.

Существует два основных вида VMM, часто называемых (не слишком изобретательно) Type 1 и Type 2. И конечно, между ними есть небольшая «серая зона»!

VMM Type 1 (гипервизоры)

В обычных системах ядро ОС (например, Linux или Windows) запускает загрузчик. В чистой среде виртуальной машины Type 1 вместо него выполняется специальная программа VMM уровня ядра.

VMM Type 1 также называют *гипервизорами* (hypervisors). В их числе, например, Hyper-V (<https://oreil.ly/FsXVi>), Xen (<https://xenproject.org/>) и ESX/ESXi

(https://en.m.wikipedia.org/wiki/VMware_ESX). Как показано на рис. 5.2, гипервизоры выполняются непосредственно на аппаратном обеспечении (на «голом железе»), без какой-либо ОС в качестве прослойки.

Говоря об «уровне ядра», я подразумеваю, что гипервизор работает в кольце 0. (Это верно до тех пор, пока мы не начнем рассматривать аппаратную виртуализацию далее в этой главе, но сейчас просто предположим, что в кольце 0.) Ядро гостевой ОС работает в кольце 1 (рис. 5.3), поэтому обладает меньшими привилегиями, чем гипервизор.



Рис. 5.2. Диспетчер виртуальных машин Type 1 — гипервизор

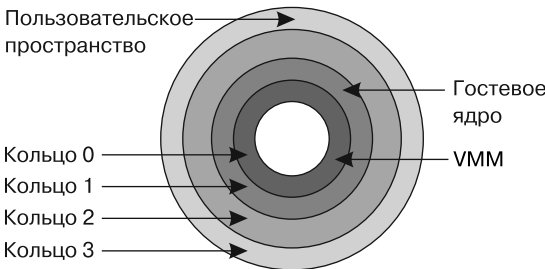


Рис. 5.3. Кольца привилегий при использовании гипервизора

VMM Type 2

Виртуальные машины, которые вы запускаете на своем ноутбуке или настольном компьютере, например, с помощью таких программ, как QEMU (<https://en.m.wikipedia.org/wiki/QEMU>), Parallels (<https://parallels.com>) или VirtualBox (<https://www.virtualbox.org/>), относятся к VMM Type 2 (работают поверх ОС хоста). Например, если ваш ноутбук работает под управлением macOS, то на нем выполняется ядро macOS. Вы можете установить программу QEMU и использовать ее для запуска гостевых виртуальных машин, которые будут сосуществовать с операционной системой хоста. Гостевые виртуальные машины могут работать под управлением совершенно другой ОС, например Linux или Windows. На рис. 5.4 показано, как сосуществуют гостевая и основная ОС.

Задумайтесь на минутку, что значит запустить, скажем, Linux внутри macOS. По определению это означает, что должен существовать отдельный экземпляр ядра Linux, отличный от ядра macOS хоста.

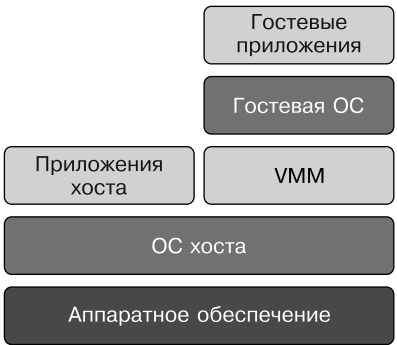


Рис. 5.4. Диспетчер виртуальных машин Type 2

У приложения VMM есть компоненты пользовательского пространства, с которыми вы можете взаимодействовать как пользователь, но оно также устанавливает привилегированные компоненты для виртуализации. В данной главе я расскажу вам более подробно, как это происходит.

Виртуальные машины, работающие в ядре

Я говорила, что границы между VMM Type 1 и Type 2 будут довольно размытыми. В Type 1 гипервизор запускается непосредственно на «голом железе»; в Type 2 VMM выполняется в пользовательском пространстве на ОС хоста. Но что, если запустить диспетчер виртуальных машин в ядре операционной системы хоста?

Именно так и происходит в модуле ядра Linux под названием KVM (Kernel-based Virtual Machine), как показано на рис. 5.5.

Обычно KVM относят к гипервизорам Type 1, поскольку гостевая ОС не должна обращаться к операционной системе хоста, но мне подобная классификация кажется слишком упрощенной.

KVM часто используется совместно с QEMU, который мы отнесли к гипервизорам Type 2. Эмулятор QEMU динамически транслирует системные вызовы гостевой ОС в системные вызовы операционной системы хоста. Стоит упомянуть, что QEMU может полноценно задействовать возможности аппаратного ускорения, предоставляемые KVM.

Для виртуализации VMM применяют схожие технологии, независимо от того, относятся ли они к Type 1, Type 2 или занимают промежуточное положение. Основная их идея называется *trap-and-emulate* («перехват и эмуляция»), хотя, как мы увидим, процессоры архитектуры x86 создают некоторые сложности при реализации этой идеи.

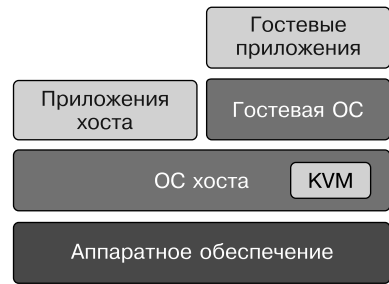


Рис. 5.5. KVM

Перехват и эмуляция

Часть инструкций CPU — *привилегированные* (privileged) в том смысле, что могут выполняться только в кольце 0. Если такая инструкция запускается в иных кольцах, это приводит к *системному прерыванию* (trap). Системное прерывание можно сравнить с исключением (exception) в ПО, которое вызывает срабатывание обработчика ошибок: процессор передает управление обработчику в коде, выполняющемся в кольце 0.

Если VMM работает с привилегиями кольца 0, а код ядра гостевой ОС — с более низкими, то привилегированная инструкция, запущенная гостевой ОС, может привести к вызову обработчика из VMM для ее эмуляции. Благодаря этому VMM гарантирует, что гостевые ОС не смогут вмешаться в работу друг друга через привилегированные инструкции.

Однако привилегированные инструкции — лишь часть проблемы. Инструкции CPU, способные влиять на ресурсы машины, называются *инструкциями, чувствительными к виртуализации* (sensitive instructions). VMM приходится обрабатывать эти инструкции вместо гостевой ОС, поскольку только VMM действительно видит все ресурсы машины. Существует еще один класс чувствительных к виртуализации инструкций, ведущих себя по-разному при выполнении в кольце 0 или в кольцах с меньшими привилегиями. Обрабатывать эти инструкции приходится опять-таки VMM, поскольку код гостевой ОС рассчитан на привилегии кольца 0.

Жизнь разработчиков VMM весьма бы упростилась, если бы все чувствительные к виртуализации инструкции были привилегированными, ведь тогда достаточно было бы просто написать обработчики системных прерываний для всех таких инструкций. К сожалению, не все чувствительные к виртуализации инструкции архитектуры x86 являются привилегированными, поэтому диспетчерам виртуальных машин приходится использовать различные методики для их обработки. Чувствительные к виртуализации, но непривилегированные инструкции считаются неподдающимися виртуализации.

Обработка инструкций, не поддающихся виртуализации

Существует несколько методик обработки подобных инструкций.

- Один из вариантов — *двоичная трансляция* (binary translation). VMM выявляет и переписывает в режиме реального времени все непривилегированные, чувствительные к виртуализации инструкции. Это сложная (ресурсоемкая) задача, и современные процессоры архитектуры x86 поддерживают аппаратную виртуализацию, которая упрощает двоичную трансляцию.
- Еще один вариант — *паравиртуализация* (paravirtualization). Вместо динамического изменения инструкций гостевой ОС эта система модифицируется так, чтобы она не могла использовать набор не поддающихся виртуализации инструкций, фактически выполняя системные вызовы к гипервизору. Эта методика применяется, в частности, в гипервизоре Xen.

- *Аппаратная виртуализация* (например, технология VT-x Intel) позволяет гипервизорам работать на новом уровне, отличающемся дополнительными привилегиями, известном как *режим VMX root*, который, по сути, представляет собой кольцо –1. Благодаря этому ядра гостевых ОС виртуальной машины могут работать в кольце 0 (то есть в режиме VMX non-root), как если бы они были операционной системой хоста.

Вложенная виртуализация

Запуск одной VM внутри другой называется *вложенной виртуализацией*. Нередко можно увидеть VMM Type 2, работающий на виртуальной машине, действующей под управлением VMM Type 1 на «голом железе». Производительность может значительно снизиться, особенно вложенных VMM Type 2, а такие функции, как сквозная передача устройств или ускорение GPU, могут быть недоступны.

KubeVirt

Проект KubeVirt (<https://kubevirt.io>) позволяет запускать виртуальные машины внутри контейнеров в поде Kubernetes. Когда я работала над первым изданием книги, эта идея была больше похожа на научный эксперимент и не находила практического применения, однако теперь она стала использоваться чаще — в основном как шаг для переноса рабочих нагрузок на основе виртуальных машин в облачные среды.

KubeVirt использует виртуализацию KVM, поэтому гостевая VM в контейнере фактически работает непосредственно на ядре хоста. Если на самом деле хост является виртуальной машиной — например, узлом Kubernetes, выполняющимся в публичном облаке, — то это случай вложенной виртуализации, которую, возможно, потребуется явно настраивать для конкретного экземпляра (и которая может быть доступна не для всех типов экземпляров).



Если вы хотите узнать больше о том, как работает виртуализация, то рекомендую ознакомиться со статьей (<https://oreil.ly/Vg5H1>) Кита Адамса (Keith Adams) и Оле Агесена (Ole Agesen), в которой приводится подробный сравнительный анализ и объясняется, как аппаратные улучшения позволяют повысить производительность.

Теперь, когда вы уже четко представляете себе, как создаются и управляются виртуальные машины, поговорим о том, что это означает с точки зрения изоляции процессов (приложений) друг от друга.

Изоляция процессов и безопасность

Первостепенная задача в сфере безопасности — обеспечение надежной изоляции приложений друг от друга. Если мое приложение может читать выделенную вашему приложению область памяти, значит, у меня есть доступ к вашим данным.

Самый надежный вид изоляции — физическая. Если наши приложения работают на совершенно отдельных компьютерах, то мой код никак не сможет получить доступ к памяти вашего приложения.

Как мы только что говорили, ядро отвечает за управление процессами его пользовательского пространства, включая выделение им памяти. Именно ядро обеспечивает, что одни приложения не могут получить доступ к памяти других. Если злоумышленник найдет уязвимость в механизме управления памятью ядра, он может воспользоваться ей, чтобы получить несанкционированный доступ к памяти. И хотя ядро прошло проверку временем и практикой, оно также очень сложно устроено и постоянно развивается. На момент написания книги мне неизвестно о каких-либо серьезных изъянах в изоляции ядра, однако я не могу поручиться за то, что какие-нибудь проблемы не обнаружатся в будущем.

Иногда подобные уязвимости возникают из-за усложнения нижележащего аппаратного обеспечения. В последние годы производители CPU разработали технологию упреждающей обработки, при которой процессор «забегает вперед» инструкций, выполняемых в настоящий момент, и предугадывает результаты, чтобы понять, нужно ли на самом деле выполнять ветку кода. Это позволило добиться более высокого быстродействия и вместе с тем открыло дорогу для широко известных уязвимостей Spectre и Meltdown.

Наверное, вам интересно, почему считается, что гипервизоры изолируют виртуальные машины лучше, чем ядро свои процессы. В конце концов, гипервизоры тоже управляют памятью и доступом к устройствам и отвечают за разделение виртуальных машин. Действительно, ошибки в гипервизоре могут привести к серьезным проблемам с изоляцией виртуальных машин. Разница в том, что стоящая перед гипервизорами задача значительно проще. В ядре процессы пользовательского пространства в определенной степени «видят» друг друга; вот простой пример: выполнив `ps`, вы увидите запущенные на той же машине процессы. При наличии соответствующих прав можно получить сведения об этих процессах через каталог `/proc`. Вы можете намеренно распределять общую память между несколькими процессами через механизм ИРС и собственно совместно используемую память. Все эти механизмы, позволяющие одному процессу на законных основаниях получать информацию о другом, ослабляют изоляцию, поскольку всегда существует риск ошибки, которая откроет подобный доступ в неожиданных или непредусмотренных обстоятельствах.

В случае же виртуальных машин никакого эквивалента этому явлению нет: процессы одной (виртуальной) машины не видны из другой. Управление памятью требует меньшего объема кода, поскольку гипервизору не нужно учитывать ситуации, при которых машинам может потребоваться использовать память совместно — это просто нехарактерно для виртуальных машин. В результате гипервизоры обычно намного меньше и проще устроены, чем полноценные ядра. Ядро Linux включает более 20 миллионов строк кода (<https://oreil.ly/FHKhp>), а гипервизор Xen — всего 50 тысяч (<https://oreil.ly/1MWub>).

Чем меньше кода и ниже степень сложности, тем меньше поверхность атаки и количество уязвимостей, которыми мог бы воспользоваться злоумышленник. Поэтому считается, что у виртуальных машин надежные границы изоляции.

Тем не менее эксплойты в виртуальных машинах встречаются. Даршан Тан (Darshan Tank), Акшай Аггарвал (Akshai Aggarwal) и Нирбхай Чобей (Nirbhay Chaubey) (<https://oreil.ly/HCXBO>) предложили классификацию различных типов атак, а Национальный институт стандартов и технологий (NIST) опубликовал методические рекомендации по усилению защиты (https://oreil.ly/_nPbp) виртуализированных сред.

Недостатки виртуальных машин

Вероятно, я настолько красноречиво описала преимущества изоляции виртуальных машин, что вы недоумеваете: а зачем вообще использовать контейнеры? У виртуальных машин, по сравнению с контейнерами, есть несколько недостатков.

- Время загрузки виртуальной машины на несколько порядков больше, чем у контейнера. В конце концов, запуск контейнера — это всего лишь запуск нового процесса Linux, а не полномасштабная загрузка и инициализация VM. Относительно медленная загрузка виртуальных машин делает их менее удобными для масштабирования, не говоря уже о важности быстрой загрузки в случае частого выпуска новых версий ПО. Однако в настоящее время существует несколько легковесных «микро-VM» с очень коротким временем запуска. О них я расскажу в главе 10.
- Благодаря контейнерам разработчики могут быстро и эффективно реализовать принцип «собери один раз — запускай где угодно». Можно, конечно, хотя это и займет немало времени, собрать образ виртуальной машины целиком и запускать его на своем ноутбуке, но данный подход не получил широкого распространения среди разработчиков, в отличие от контейнеров.
- В современных облачных средах при аренде VM выбирается CPU и объем оперативной памяти, за которые необходимо вносить арендную плату вне зависимости от того, какая часть этих ресурсов фактически используется приложением.

При выборе между виртуальными машинами или контейнерами стоит учитывать множество факторов: производительность, цена, удобство, риски и прочность границ зон безопасности между приложениями.

Изоляция контейнеров по сравнению с изоляцией виртуальных машин

Как вы поняли из главы 4, контейнеры — это просто процессы Linux с ограничениями на доступные ресурсы. Ядро изолирует их с помощью механизмов пространств имен, контрольных групп и изменения корневого каталога, которые специально создавались для изоляции процессов. Однако сам факт совместного использования контейнерами ядра означает, что их базовая изоляция слабее по сравнению с виртуальными машинами.

Но ситуация не безнадежна! Существуют дополнительные средства безопасности и различные песочницы, которые позволяют значительно усилить изоляцию контейнеров. О них я расскажу в главе 10. Кроме того, есть весьма эффективные инструменты, базирующиеся на том, что контейнеры зачастую инкапсулируют микросервисы. Их мы рассмотрим в главе 15.

Резюме

Теперь у вас должно сложиться хорошее представление о том, что такое виртуальные машины. Вы узнали, почему изоляция виртуальных машин считается более надежной по сравнению с изоляцией контейнеров и почему контейнеры в целом не считаются достаточно безопасными для организации сложных мультиарендных сред. Эту разницу очень важно понимать при обсуждении безопасности контейнеров.

Тема безопасности виртуальных машин выходит за рамки книги, хотя мы и затронули вкратце вопрос о том, как повысить надежность настроек хоста контейнера, в разделе «Хосты и контейнеры» в главе 4.

Далее в книге вы увидите примеры, в которых более слабая (по сравнению с виртуальными машинами) изоляция контейнеров легко нарушается из-за некорректных настроек. Но сперва разберемся, как устроены образы контейнеров и как они могут существенно влиять на безопасность.

Образы контейнеров

Если вам случалось работать с Docker или Kubernetes, то вы уже знакомы с идеей образов контейнеров, которые хранятся в реестре. В этой главе мы разберемся, что представляют собой образы, посмотрим на их содержимое и обсудим их использование в таких средах выполнения, как Docker, containerd, podman или CRI-O.

Поняв, что такое образы контейнеров, вы сможете начать думать и о вопросах безопасности при их создании, хранении и извлечении — а с этими шагами связано множество векторов атак. Кроме того, я расскажу вам о практических рекомендациях, позволяющих гарантировать, что сборки и образы не подорвут безопасность системы в целом.

Корневая файловая система и конфигурация образов контейнеров

Образ контейнера состоит из двух частей: корневой файловой системы и конфигурации.

Если вы выполняли примеры из главы 4, то скачали копию файловой системы Alpine и использовали ее в качестве содержимого корневого каталога внутри контейнера. По сути, при запуске контейнера на основе его образа создается экземпляр контейнера, а образ включает корневую файловую систему. Если выполнить команду `docker run -it alpine sh` и сравнить результат с содержимым созданного вами вручную контейнера, вы увидите одинаковую структуру каталогов и файлов, а если версия Alpine такая же, то они совпадут в точности.

Если ваше знакомство с контейнерами состоялось через Docker, то вам уже привычна идея создания образов на основе инструкций в файлах Dockerfile. Часть команд Dockerfile (например, FROM, ADD, COPY и RUN) изменяет содержимое корневой файловой системы, включенной в образ. Другие команды, например

USER, PORT и ENV, влияют на информацию о настройках, хранимую в образе параллельно с корневой файловой системой. Просмотреть ее можно, выполнив для образа `docker inspect`. Эти настройки задают значения параметров среды выполнения, которые по умолчанию должны применяться при запуске образа. Так, переменные окружения задаются для выполняемого процесса контейнера с помощью команды ENV в Dockerfile.

Переопределение настроек в среде выполнения

В Docker можно переопределять конфигурацию в среде выполнения, воспользовавшись параметрами командной строки. Например, изменить существующую переменную окружения или задать новую можно с помощью команды `docker run -e <имя_переменной>=<новое_значение> ...`. Аналогично, чтобы переопределить идентификатор пользователя, заданный в образе, применяется параметр `--user`:

```
$ docker run --rm -it alpine whoami
root
$ docker run --rm -it --user 405 alpine whoami
guest
```

В Kubernetes конфигурацию контейнера можно задать в YAML-описании пода:

```
apiVersion: v1
kind: Pod
metadata:
  name: demo
spec:
  containers:
  - name: demo-container
    image: demo-reg.io/some-org/demo-image:1.0
    env:
    - name: DEMO_ENV
      value: "Переопределенное значение"
  securityContext:
    runAsUser: 1000
```

Предположим, что (вымышленный) образ контейнера `demo-image:1.0` был собран на основе файла Dockerfile, который мог содержать строку `ENV DEMO_ENV="Исходное значение"`. В этом YAML-описании переопределяется значение переменной `DEMO_ENV`, и если контейнер заносит значение данной переменной в журнал, то вы увидите там **Переопределенное значение**.

В этом примере также переопределяется идентификатор пользователя через параметр `securityContext.runAsUser`. Если бы в поде было несколько

контейнеров, этот параметр применялся бы к каждому из них. В документации Kubernetes (<https://kubernetes.io/docs/tasks/configure-pod-container>) описывается множество других параметров, которые можно определить для настройки контейнеров, работающих внутри пода.

Если в Kubernetes в качестве среды выполнения контейнера используется совместимая с OCI (Open Container Initiative) утилита, например `containerd/runc`, то значения из YAML-описания попадут в итоге в совместимый с OCI файл `config.json`. Поговорим подробнее об этих файлах контейнеров и инструментах, совместимых со стандартом OCI.

Стандарты OCI

Open Container Initiative (OCI) (<https://opencontainers.org/>) была создана для стандартизации контейнерных образов и контейнерных рантаймов, во многом с опорой на наработки, изначально появившиеся в Docker. В частности, одной из целей OCI было обеспечить такой же пользовательский опыт, к которому привыкли пользователи Docker, — например, возможность запускать образ с набором настроек по умолчанию. В 2017 году была утверждена спецификация OCI Image Format Spec 1.0, за ней в 2021-м последовала спецификация Distribution Spec 1.0, которая определила порядок отправки (`push`) образов в реестр и извлечение (`pull`) их оттуда. Сегодня этот стандарт широко используется в таких реестрах, как Harbor, Docker Hub, AWS ECR и GitHub Container Registry.



Спецификация OCI Distribution Spec позволяет хранить в реестре не только образы контейнеров, но и любое другое содержимое. В результате реестры стали использоваться для хранения артефактов других типов, связанных с контейнерами, включая чарты Helm, политики Open Policy Agent, SBOM (спецификации компонентов ПО) и аттестации (которые мы обсудим в главе 7) и многое другое.

Работать и просматривать образы OCI удобно с помощью утилиты Skopeo (<https://github.com/containers/skopeo>), которая умеет преобразовывать образ Docker в формат OCI:

```
$ skopeo copy docker://alpine:latest oci:alpine:latest
$ ls alpine
blobs index.json oci-layout
```



Далее в этой главе вы увидите, как получить аналогичные результаты с помощью инструментов Docker.

Файл `index.json` содержит *манифест* (описание) образа, включая уникальную контрольную сумму, которая его идентифицирует:

```
cat alpine/index.json | jq
{
  "schemaVersion": 2,
  "manifests": [
    {
      "mediaType": "application/vnd.oci.image.manifest.v1+json",
      "digest": "sha256:08001109a7d679fe33b04fa51d681bd40b975d8f5cea8c3e...",
      "size": 1022,
      "annotations": {
        "org.opencontainers.image.ref.name": "latest"
      }
    }
  ]
}
```

Эта контрольная сумма совпадает с одним из *блоков* в образе в формате OCI:

```
$ ls alpine/blobs/sha256/
08001109a7d679fe33b04fa51d681bd40b975d8f5cea8c3ef6c0eccb6a7338ce
cea2ff433c610f5363017404ce989632e12b953114fefc6f597a58e813c15d61
fe07684b16b82247c3539ed86a65ff37a76138ec25d380bd80c869a1a4c73236
```

Эти блоки представляют собой слои образа, о которых мы подробнее поговорим чуть позже в этой главе. Однако среды выполнения, совместимые с OCI, такие как `gunc`, не умеют работать напрямую с образами в таком формате. Сначала их необходимо распаковать в *комплект файловой системы* (filesystem bundle) (<https://github.com/opencontainers/runtime-spec/blob/main/bundle.md>). Рассмотрим пример, в котором для распаковки образа используется `umoci` (<https://github.com/opencontainers/umoci>):

```
$ sudo umoci unpack --image alpine:latest alpine-bundle
$ ls alpine-bundle
config.json
rootfs
sha256_08001109a7d679fe33b04fa51d681bd40b975d8f5cea8c3ef6c0eccb6a7338ce.mtree
umoci.json
$ ls alpine-bundle/rootfs
bin etc lib mnt proc run srv tmp var
dev home media opt root sbin sys usr
```

Как видите, этот комплект включает каталог `rootfs`, содержащий дистрибутив Alpine Linux. В него также входит файл `config.json`, в котором определены параметры среды выполнения. Среда выполнения создает экземпляр контейнера на основе этой корневой файловой системы и настроек.



Я запустила команду `umoci` с правами суперпользователя (`sudo`), но поскольку она поддерживает возможность запуска без `root`-прав (<https://umo.ci/quick-start/rootless>), то ее можно вызвать от имени непривилегированного пользователя, передав ей параметр `--rootless`.

При работе с `Docker` у вас нет прямого доступа к информации о настройках в виде файла, который можно было бы просмотреть с помощью команды `cat` или текстового редактора, но вы можете убедиться в его наличии, выполнив команду `docker image inspect`.

Конфигурация образа

Поскольку из глав 3 и 4 вы уже знаете, как создаются контейнеры, имеет смысл взглянуть на один из файлов `config.json`, ведь многое в нем покажется вам знакомым. Ниже представлен фрагмент такого файла, для примера:

```
"root": {
  "path": "rootfs"
},
"hostname": "umoci-default",
"mounts": [
  {
    "destination": "/proc",
    "type": "proc",
    "source": "proc"
  },
  ...
  {
    "destination": "/sys/fs/cgroup",
    "type": "cgroup",
    "source": "cgroup",
    "options": [
      "nosuid",
      "noexec",
      "nodev",
      "relatime",
      "ro"
    ]
  }
],
...
"namespaces": [
  {
    "type": "pid"
  },
  ...
]
```

```
{
  "type": "network"
},
{
  "type": "ipc"
},
{
  "type": "uts"
},
{
  "type": "mount"
}
]
```

Как видите, конфигурация включает описание всего, что утилита `runsc` должна сделать, чтобы создать контейнер, включая список точек монтирования для `/proc` и контрольных групп, а также пространства имен, которые необходимо сформировать.

Как вы поняли, образ состоит из двух частей: корневой файловой системы и информации о настройках. Теперь рассмотрим, как происходит сборка образов.

Сборка образов

Большинство пользователей собирают образы контейнеров с помощью команды `docker build`, которая формирует образ, следуя инструкциям из `Dockerfile`.

Традиционно запуск команды `docker build` — относительно простой и монолитный процесс, создающий образ контейнера, пригодного для использования на машинах с такой же архитектурой, как и на машине, на которой выполняется сборка. Недавно разработчики Docker из проекта Moby представили набор инструментов BuildKit. (Возможно, вы уже знаете, что в компании Docker переименовали открытый исходный код движка Docker в Moby, чтобы избежать путаницы из-за совпадения названия проекта с названием компании.) BuildKit предлагает более продвинутые возможности, включая поддержку режима, не требующего привилегий суперпользователя, создание образов для нескольких платформ, отправку образа в несколько реестров и многое другое. Начиная с версии 23, он используется в Docker по умолчанию.

Опасности команды `docker build`

В классической архитектуре Docker вызываемая утилита командной строки (`docker`) делает очень немного. По сути, она преобразует введенную команду в запрос API и отправляет его демону Docker через *socket Docker*. Любой процесс, имеющий доступ к сокету Docker, способен отправлять демону запросы API.

Демон Docker — это долгоживущий процесс, который фактически выполняет всю работу по запуску контейнеров и управлению ими и их образами. Как вы видели в главе 4, для создания контейнера демону необходимо уметь формировать пространства имен, поэтому он должен выполняться от имени суперпользователя. В этом легко убедиться с помощью команды `ps`:

```
$ ps -fc dockerd
UID          PID    PPID  C  STIME TTY          TIME CMD
root         22240      1   0 Jun04 ?        00:01:58 /usr/bin/dockerd ...
```

Пользователь может заставить демон выполнять произвольные команды, передав их в Dockerfile в команде `RUN`. Поскольку демон запускается от имени суперпользователя, любой, кто имеет право запускать сборку, фактически получает права суперпользователя на этой машине.

Представьте, что вы хотели бы выделить (виртуальную) машину для сборки образов контейнеров и сохранения их в реестре. При использовании стандартного подхода Docker на вашей машине должен быть запущен демон, возможности которого не ограничиваются сборкой образов и взаимодействием с реестром. При отсутствии дополнительных средств безопасности любой пользователь, наделенный полномочиями выполнять на такой машине команду `docker build`, сможет также выполнить `docker run` и запустить любую команду, какую только пожелает.

Причем дело не только в возможности выполнять произвольные команды, но и в том, что если он воспользуется этими полномочиями для злонамеренных действий, то выследить виновного будет нелегко. Даже если вы ведете журнал аудита действий пользователей, как наглядно показано в публикации Дэна Уолша (Dan Walsh) (<https://oreil.ly/KbD9T>), в нем будет фиксироваться идентификатор процесса демона, а не пользователя.

Чтобы снизить эти риски, были разработаны альтернативные подходы к созданию образов контейнеров без использования демона Docker:

- драйвер `docker-container` из набора инструментов BuildKit (<https://docs.docker.com/build/builders/drivers>) позволяет запускать процедуру сборки внутри контейнера, изолированного от хоста, на котором он выполняется;
- в настоящее время Docker поддерживает режим сборки без привилегий суперпользователя (<https://github.com/moby/buildkit/blob/master/docs/rootless.md>), в котором демон `buildkitd` запускается от имени обычного пользователя. На момент написания книги этот режим необходимо было включать явно;
- в числе прочих непривилегированных средств сборки — `podman` (<https://podman.io/>) и `buildah` (<https://buildah.io/>) от Red Hat. Пуджа Аббасси (Puja Abbassi) в своем блоге (<https://www.giantswarm.io/blog/building-container-images-with-podman-and-buildah>) описывает эти утилиты и сравнивает их с `docker build`;

- утилита Bazel (<https://bazel.build>) от компании Google умеет собирать не только образы контейнеров, но и многие другие типы артефактов. Она не использует Docker и отличается высокой детерминированностью генерируемых образов, что позволяет воспроизводить тот же самый образ на основе того же исходного кода;
- еще один инструмент, ориентированный на создание воспроизводимых сборок, — Nix (<https://nixos.org>), но освоить его очень непросто;
- создателям простых Go-приложений в контейнерах может пригодиться утилита ko (<https://ko.build>), а создателям Java-приложений — jib (<https://cloud.google.com/java/getting-started/jib>).



Некоторые рекомендации и примеры сборки контейнеров без привилегий суперпользователя можно найти на GitLab (https://docs.gitlab.com/ci/docker/using_buildkit/#build-images-in-rootless-mode).

Любой из этих инструментов можно запускать вручную из командной строки, но в рабочей среде обычно принято автоматизировать сборку в рамках конвейера непрерывной интеграции/непрерывного развертывания (CI/CD).

Слои образов

Вне зависимости от используемого инструмента абсолютное большинство сборок образов контейнеров описывается в Dockerfile. Этот файл задает последовательность инструкций, каждая из которых приводит либо к созданию слоя файловой системы, либо к изменению конфигурации образа. Все это прекрасно описано в документации Docker (<https://docs.docker.com/engine/storage/drivers/#images-and-layers>), но если вы хотите узнать больше, то, возможно, вас заинтересует моя статья, посвященная восстановлению Dockerfile на основе образа контейнера (<https://oreil.ly/2SSdJ>).

Конфиденциальные данные в слоях файловой системы

Любой, кто имеет доступ к образу контейнера, имеет и доступ ко всем включенным в этот образ файлам. С точки зрения безопасности нежелательно включать в образ конфиденциальные данные, например пароли или токены. (В главе 14 я расскажу, что делать с такими данными.)

Все слои хранятся отдельно, поэтому нужно быть осторожными: даже если в последующем слое данные удаляются, это не гарантирует их полного исчезновения. Dockerfile, представленный ниже, демонстрирует, как делать *не* нужно:

```
FROM alpine
RUN echo "top-secret" > /password.txt
RUN rm /password.txt
```

Один слой создает файл, а следующий его удаляет. Если собрать этот образ и затем его запустить, то никаких следов файла `password.txt` вы не увидите:

```
$ docker run --rm -it sensitive ls /password.txt
ls: /password.txt: No such file or directory
```

Однако не позволяйте ввести себя в заблуждение — конфиденциальные данные все равно попали в образ. Чтобы убедиться в этом, экспортируйте образ в файл `.tar` с помощью команды `docker save` и разархивируйте его:

```
$ docker save sensitive > sensitive.tar
$ mkdir sensitive
$ cd sensitive
$ tar -xf ../sensitive.tar
$ ls
blobs index.json manifest.json oci-layout repositories
```



Я использую версию Docker v28.3.2. Для совместимости со старыми инструментами она выводит метаданные как в стандартном формате OCI, так и в устаревшем формате Docker. В будущем или при использовании других инструментов вы, скорее всего, будете видеть только информацию в формате OCI. Мы можем использовать ее и просто игнорировать устаревшие данные из файлов `index.json` и `repositories`.

Эти файлы уже знакомы вам по предыдущей части главы:

- `oci-layout` — указывает версию спецификации OCI для данного образа;
- `manifest.json` — файл верхнего уровня с описанием образа. В нем указывается, в каком файле находится конфигурация, описываются все теги для данного образа и перечисляются все слои;
- `blobs` — каталог с файлами данных, из которых состоит образ.

Внутри `manifest.json` вы найдете путь к конфигурации, список тегов для образа и список файлов слоев, составляющих корневую файловую систему образа:

```
...
  "Config": "blobs/sha256/258d7...7f1a8",
  "RepoTags": [
    "sensitive:latest"
  ],
  "Layers": [
    "blobs/sha256/1231a...a69f1",
    "blobs/sha256/9e745...67031",
    "blobs/sha256/82316...4bd6e"
  ],
  ...
```

Конфигурация включает историю команд, выполнявшихся при формировании контейнера. Как можно видеть, здесь конфиденциальные данные `top-secret` отображаются на шаге выполнения команды `echo`:

```
$ cat blobs/sha256/258d* | jq '.history'
[
  {
    "created": "2025-05-30T16:20:41Z",
    "created_by": "ADD alpine-minirootfs-3.22.0-aarch64.tar.gz / # buildkit",
    "comment": "buildkit.dockerfile.v0"
  },
  {
    "created": "2025-05-30T16:20:41Z",
    "created_by": "CMD [\"/bin/sh\"]",
    "comment": "buildkit.dockerfile.v0",
    "empty_layer": true
  },
  {
    "created": "2025-07-26T05:02:59.245427668-05:00",
    "created_by": "RUN /bin/sh -c echo \"top-secret\" > /password.txt # buildkit",
    "comment": "buildkit.dockerfile.v0"
  },
  {
    "created": "2025-07-26T05:02:59.348379231-05:00",
    "created_by": "RUN /bin/sh -c rm /password.txt # buildkit",
    "comment": "buildkit.dockerfile.v0"
  }
]
```

Внутри каталога каждого из слоев находится еще один архив `.tar` с содержимым файловой системы этого слоя. Из каталога соответствующего слоя несложно извлечь файл `password.txt`:

```
$ cd blobs/sha256
$ tar -xvf 9e7*
etc/
password.txt
$ cat password.txt
top-secret
```

Как видите, даже если файл был удален в последующих слоях, любые данные, когда-либо существовавшие в одном из слоев, можно легко восстановить путем распаковки образа. Не включайте в слои ничего, чем вы не готовы поделиться со всеми, у кого есть доступ к образу.

В главе 8 мы обсудим сканирование образов контейнеров на наличие уязвимостей и познакомимся с некоторыми инструментами, которые могут не только сканировать, но также выявлять секретные данные, случайно включенные в слои образа. Если вы обнаружите один из своих секретов в образе, необходимо не только исправить сборку, чтобы в будущих версиях образа он

не фигурировал, но и заменить его, чтобы он стал бесполезным для тех, кто имел доступ к более старым версиям образа.

Иногда в процессе сборки необходимо использовать секреты, поэтому давайте посмотрим, как это делать безопасно, не оставляя их внутри образа.

Монтирование секретов с BuildKit

BuildKit предлагает альтернативный подход, позволяющий передавать конфиденциальные данные во время сборки так, что они никогда не записывались в слои образа. Указать идентификатор секрета вместе с файлом, содержащим конфиденциальную информацию, можно следующим образом:

```
$ docker build --secret id=<id>,src=secret.txt .
```

Секрет может быть предоставлен процессу сборки в виде файла, смонтированного в каталог `/run/secret/<id>`, а Dockerfile будет ссылаться на секрет там, где он необходим, например:

```
RUN --mount=type=secret,id=<id> <команда, которой требуется секрет>
```

Секрет подключается только для конкретного шага сборки и существует исключительно в оперативной памяти.

Например, предположим, что во время сборки вам нужен токен доступа для обращения к API. В таком случае Dockerfile будет содержать команду следующего вида:

```
RUN --mount=type=secret,id=API_TOKEN \  
    sh -c 'curl -H "Authorization: Bearer $(cat /run/secrets/API_TOKEN)" \  
    https://api.your-domain.com/data'
```

Затем вы можете временно сохранить токен в файл и получить к нему доступ во время сборки следующим образом:

```
$ echo <секретный токен> > token.txt  
$ docker build --secret id=API_TOKEN,src=token.txt -t myimage:v1.0.0 .
```

Разумеется, не стоит оставлять файл `token.txt` с секретным токеном на машине со сборкой, поэтому его следует удалить — при этом можно быть уверенными, что он не сохранился ни в одном из слоев `myimage:v1.0.0`.

Звучит красиво, но, к сожалению, преимущество монтирования секретов с BuildKit легко потерять, *выполнив команду, которая сохраняет значение секрета в образе!* Избегайте подобных шаблонов, когда секрет записывается в файл:

```
RUN --mount=type=secret,id=my-secret cat /run/secret/my-secret > out.txt
```

Такая команда создаст файл `out.txt`, содержащий конфиденциальные данные, который окажется внутри формируемого образа.



Более подробную информацию о монтировании секретов, включая передачу значений секретов в процесс сборки из переменных окружения или через соединения SSH, можно найти в документации Docker (<https://docs.docker.com/build/building/secrets>).

Монтирование секретов — это лишь одна из возможностей BuildKit, отсутствующих в оригинальном подходе к сборке с применением средств Docker. Еще одна полезная функция — возможность создавать образы для разных аппаратных архитектур.

Мультиплатформенные образы

Образы контейнеров могут быть собраны для разных аппаратных архитектур, включая распространенные варианты: `amd64` (обычно используется для Intel) и `arm64` (для систем на базе ARM). Мультиплатформенный образ содержит список манифестов, в котором указаны конкретные архитектуры. Для проверки мультиплатформенного образа можно использовать Docker:

```
$ docker manifest inspect alpine
{
  "schemaVersion": 2,
  "mediaType": "application/vnd.oci.image.index.v1+json",
  "manifests": [
    {
      "mediaType": "application/vnd.oci.image.manifest.v1+json",
      "size": 1022,
      "digest": "sha256:08001109a7d679fe33b04fa51d681bd40b975d8f5ce...",
      "platform": {
        "architecture": "amd64",
        "os": "linux"
      }
    },
    ...
    {
      "mediaType": "application/vnd.oci.image.manifest.v1+json",
      "size": 1025,
      "digest": "sha256:008448246686fe28544e36ba89e7bc7fbe6dad8a2cc...",
      "platform": {
        "architecture": "arm64",
        "os": "linux",
        "variant": "v8"
      }
    },
    ...
  ]
}
```

Манифест включает контрольную сумму, идентифицирующую образ для каждой поддерживаемой платформы. При загрузке образа среда выполнения контейнеров извлечет вариант, соответствующий платформе, на которой она работает.

Содержимое образов для каждой платформы не зависит друг от друга. Если в вашей организации используются образы на разных платформах, необходимо убедиться, что образы сканируются на наличие уязвимостей и небезопасных конфигураций для всех платформ, поскольку нет гарантии, что содержимое образов совпадает. О сканировании образов мы поговорим в главе 8.

Ранее в этой главе вы видели, что находится внутри образа контейнера, совместимого с OCI. А теперь вы знаете, что происходит, когда эти образы собираются на основе Dockerfile. Обсудим, как образы хранятся.

Хранение образов

Образы сохраняются в реестрах контейнеров. Если вы используете Docker, то, вероятно, вам случалось взаимодействовать с реестром Docker Hub (<https://hub.docker.com/>). Если же вы работаете с контейнерами с помощью облачных сервисов, то, возможно, знакомы с одним из их реестров, например AWS Elastic Container Registry или Google Container Registry. Существуют также коммерческие реестры и решения с открытым исходным кодом, такие как Harbor (<https://goharbor.io>).

Создание собственного реестра

Многие организации поддерживают собственные реестры или используют управляемые реестры своего поставщика облачных услуг и требуют, чтобы использовались только образы из разрешенных реестров. Развертывание собственного реестра (или собственного экземпляра управляемого реестра) дает больше контроля и прозрачности над тем, кто может отправлять и получать образы. Это также снижает вероятность DNS-атаки, позволяющей злоумышленнику подменить адрес реестра. Если реестр находится внутри VPC, то вероятность подобной атаки крайне мала. Большинство управляемых реестров (таких как AWS ECR, Microsoft ACR и Google GCR) поддерживают частные конечные точки VPC.

Следует также ограничить прямой доступ к хранилищу данных реестра. Например, реестр, работающий в AWS, может использовать Amazon S3 для хранения образов, и для соответствующих S3-корзин должны быть установлены строгие ограничения, чтобы злоумышленник не мог напрямую получить доступ к данным образов.

Сохранение и извлечение

О сохранении образа в реестре обычно говорят как о *добавлении* (буквально: «вталкивании», push), а о получении образа из реестра — как об *извлечении* (буквально: «вытаскивании», pull).

Все слои хранятся по отдельности, в виде больших двоичных объектов в реестре, идентифицируемых по хешу их содержимого. В целях экономии места желательно хранить только одну копию каждого объекта, на которую могут ссылаться различные образы. В реестре также хранится *манифест* образа, который определяет набор больших двоичных объектов слоев, составляющих образ. Хеш (или контрольная сумма) манифеста образа служит уникальным идентификатором всего образа и называется *дайджестом* (digest) образа. Если пересобрать образ и что-то в нем изменить, хеш также поменяется.

При использовании Docker просмотреть дайджесты образов, хранящихся локально на машине, можно с помощью следующей команды:

```
$ docker image ls --digests
REPOSITORY TAG DIGEST IMAGE ID CREATED SIZE
nginx latest sha256:50cf...8566 231d40e811cd 2 weeks ago 126MB
```

При отправке образа в реестр и извлечении из него можно использовать дайджест, чтобы точно указать конкретную сборку. Однако это не единственный способ сослаться на образ. Рассмотрим различные способы идентификации образов контейнеров.

Идентификация образов

Первая часть ссылки на образ — это URL реестра, в котором он хранится. (Если адрес реестра опущен, значит, образ хранится либо на локальной машине, либо в Docker Hub, в зависимости от контекста команды.)

Следующая часть ссылки — имя учетной записи пользователя или организации, которым принадлежит образ. Далее следует имя образа и затем — либо дайджест, идентифицирующий содержимое, либо удобочитаемый тег.

В итоге полный адрес образа выглядит примерно так:

```
<URL реестра>/<имя пользователя или название организации>/<репозиторий>
@sha256:<дайджест>
<URL реестра>/<имя пользователя или название организации>/<репозиторий>:<тег>
```

Если URL реестра не указан, то по умолчанию используется адрес Docker Hub — docker.io. На рис. 6.1 показан пример, как образ отображается в Docker Hub.

Извлечь образ можно с помощью одной из следующих команд (сокращены для компактности):

```
$ docker pull aquasec/trivy:0.66.0
$ docker pull aquasec/trivy:sha256:08697...dfb1e
```

Для человека неудобно ссылаться на образ по хешу, поэтому часто используются *теги* — произвольные метки, присваиваемые образам. Одному образу допускается присваивать произвольное число тегов, а один и тот же тег может быть переназначен с одного образа на другой. С помощью тегов часто указывают версии содержащегося в образе ПО — как в приведенном выше примере с версией 0.66.0.

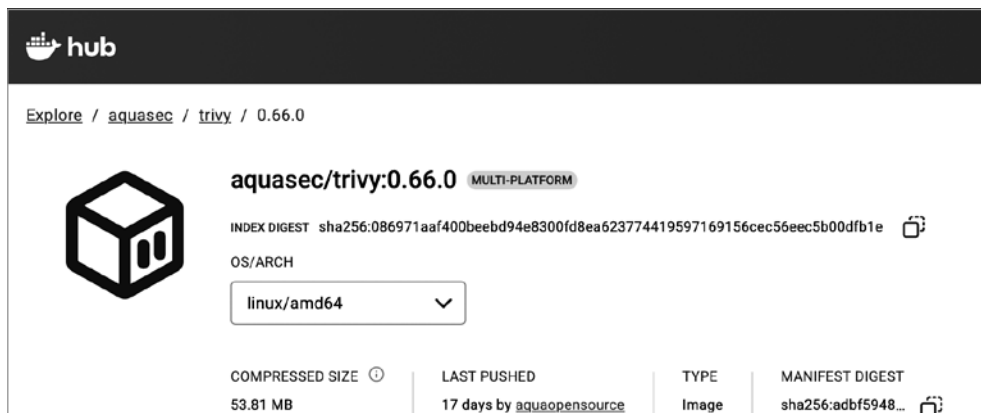


Рис. 6.1. Пример образа в Docker Hub

Поскольку теги можно переносить с образа на образ, нет гарантии, что обращение к образу по тегу даст одинаковый результат сегодня и завтра. А вот при ссылке по хешу вы всегда получаете один и тот же образ, поскольку хеш определяется содержимым образа. Любое изменение образа приводит к изменению хеша.

Именно такой эффект обычно и требуется. Например, можно ссылаться на образ по тегу, отражающему основной и дополнительный номера версии в схеме семантического версионирования (semantic versioning — semver). При выпуске новой версии специалисты, сопровождающие продукт, должны отметить эту версию соответствующим тегом с номером версии, чтобы при следующем извлечении образа вы получили актуальный продукт.

Однако иногда необходима уникальная ссылка на образ. Например, представьте сканирование образов на уязвимости (об этом говорится в главе 8).

Контроллер допуска проверяет, чтобы развертываемые образы прошли этап сканирования на уязвимости, а потому должен сверяться с базой уже про-сканированных образов. В случае ссылки на образы по тегу на информацию полагаться нельзя, ведь точно не известно, был ли образ изменен и нужно ли повторно его сканировать. Ссылка на образы по дайджесту — распространенное явление в автоматизированных конвейерах развертывания и считается хорошей практикой в рабочих средах.

В некоторых реестрах можно сделать теги неизменяемыми, что позволяет предотвратить добавление нового хеша к существующему тегу в реестре. Это избавляет от «путаницы тегов», но также препятствует повторному использованию тега при семантическом версионировании.

Резюме

В этой главе вы увидели, как формируются и хранятся образы, узнали достаточно о формате образов, чтобы понимать, как в них могут по ошибке попасть конфиденциальные данные, и выяснили, как предотвратить попадание конфиденциальных данных в образы, используя безопасные методы передачи секретов, необходимых в процессе создания. Мы также рассмотрели варианты ссылки на образы по тегам и по дайджесту.

При извлечении образа из реестра важно быть уверенными, что он содержит ожидаемый код и не включает ничего вредоносного. В главе 7 мы обсудим вопросы, связанные с безопасностью и происхождением образа и его содержимого, которые все вместе обычно называют *безопасностью цепочки поставок*.

Безопасность цепочки поставок

Любой, кто разворачивает образы контейнеров, должен быть уверен, что ПО внутри каждого образа безопасно для запуска и не было подменено на каком-либо этапе. В этой главе мы рассмотрим, как безопасно создавать и распространять образы контейнеров вместе с информацией, подтверждающей целостность и оригинальное происхождение образов.

Под безопасностью цепочки поставок подразумевается соответствие развертываемого и запускаемого ПО вашим ожиданиям за счет сборки из исходного кода, которому вы доверяете, с помощью системы сборки, которой вы доверяете. В цепочке существует множество потенциально слабых звеньев, от сборки и сохранения образа до его запуска, как показано на рис. 7.1.

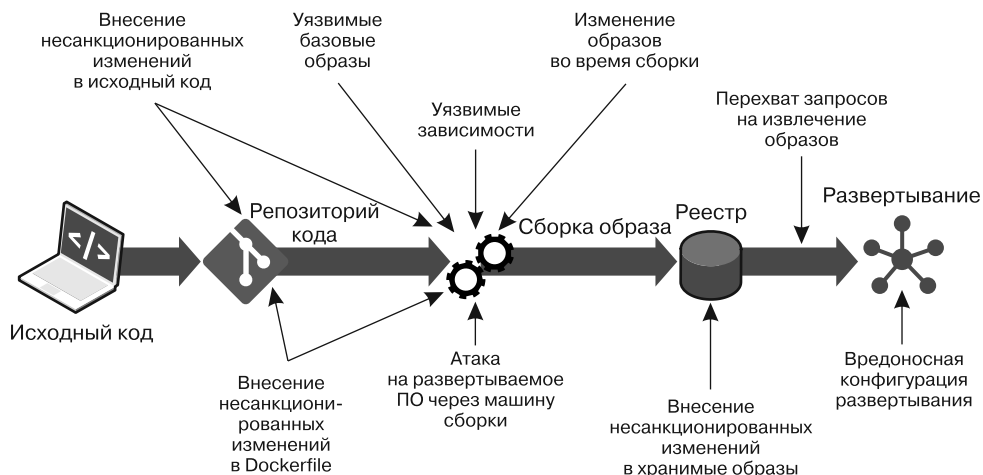


Рис. 7.1. Векторы атак на образы

Программные компоненты в образах контейнеров

Образы контейнеров могут содержать множество разных программных компонентов.

- Как было показано в главе 4, образ контейнера включает файловую систему, часто основанную на дистрибутиве Linux, содержащую все файлы и каталоги, входящие в этот дистрибутив.
- Дистрибутивы обычно поддерживают менеджер пакетов, такой как `apt` или `yum`, и в образ контейнера могут быть включены некоторые пакеты — в идеале (но не обязательно) только те, которые необходимы приложению.
- В зависимости от используемого языка программирования само приложение может иметь вид скомпилированного исполняемого файла или набора интерпретируемых сценариев.
- Часто требуются библиотеки, специфичные для конкретного языка программирования, например пакеты Rust, Ruby, Node или Python.
- В образ могут быть включены и другие файлы, например с настройками или данными.

Любой из этих программных компонентов может содержать уязвимый код, который может использовать злоумышленник.

Дистрибутивы, пакеты, библиотеки и исходный код приложений со временем изменяются. Более свежие версии любого ПО могут содержать исправления имеющихся уязвимостей, но также привносить и новые.

Разные компоненты, как правило, написаны разными разработчиками и получены из разных источников. Например, дистрибутив Linux может быть предоставлен такой организацией, как Alpine или Red Hat. Если вы используете базовый образ, скажем, с дистрибутивом Red Hat Enterprise Linux, то вы определенно захотите быть уверенными, что он действительно получен от Red Hat, а не от злоумышленника-самозванца. Аналогично важно знать, что пакеты и библиотеки, включенные в образ, получены от проверенных поставщиков.

Необходимы также соответствующие средства управления доступом к репозиториям исходного кода, чтобы гарантировать, что неавторизованные пользователи не смогут вносить несанкционированные изменения в ПО, которое встраивается в образы контейнеров.

Ваша компания или организация может использовать образы контейнеров, созданные ею самой и включающие собственные бизнес-приложения. При этом, вероятно, используются и образы контейнеров, созданные поставщиками или сторонними организациями, с общими инфраструктурными компонентами или инструментами. Возможно также, что вы сами формируете

образы контейнеров, которые распространяются и задействуются другими организациями, и тогда важно дать пользователям уверенность в безопасности ваших образов.

Информация о том, какие компоненты и какие их версии включены в образ, имеет решающее значение для выявления любых известных уязвимостей. Подробнее об этом мы поговорим в главе 8, а пока нас интересует возможность проверки и отслеживания каждого компонента на пути создания и доставки контейнера.

SLSA

Уровни безопасности в цепочке поставок компонентов программного обеспечения (Supply Chain Levels for Software Artifacts, SLSA) — это структура, разработанная фондом безопасности открытого исходного кода (OpenSSF) (<https://openssf.org>) и определяющая желательные характеристики для обеспечения безопасности цепочки поставок ПО. SLSA определяет три уровня, отражающих возрастающую уверенность в целостности, прослеживаемости и происхождении ПО, включая образы контейнеров. Рекомендации SLSA включают:

- воспроизводимость сборки;
- изолированность среды сборки;
- документ, позволяющий отслеживать процесс создания ПО и его зависимостей.

Информация о компонентах, входящих в состав программного артефакта, называется *спецификацией программного обеспечения* (Software Bill of Materials, SBOM). В ней перечислены компоненты, использованные при сборке, а SLSA дополнительно определяет информацию о происхождении, которая описывает, как и кем был создан артефакт.

Как указано на сайте SLSA (<https://slsa.dev/how-to/get-started>), «файл конфигурации сборки (например, рабочий процесс GitHub) соответствует требованиям уровня 1 SLSA. Он будет считаться неподписанным, произвольным документом о происхождении». Там же говорится: «Для достижения уровня 3 SLSA необходимо, чтобы:

- сборка выполнялась на хостинговой платформе, которая генерирует и подписывает документы, подтверждающие происхождение данных;
- этапы сборки не влияли друг на друга;
- предоставлялось подписанное свидетельство о происхождении, подлинность которого можно проверить».

SBOM и информация о происхождении создаются в процессе сборки образа контейнера. Пользователи образа наверняка захотят проверить эту информацию в ходе развертывания. Этап проверки предусмотрен и в рамках SLSA. Рассмотрим, каких рисков можно избежать благодаря SBOM.

Спецификация программного обеспечения

SBOM — это машиночитаемый перечень всех компонентов, входящих в образ контейнера, включая информацию об используемых версиях. Как будет показано в главе 8, SBOM позволяет автоматизировать процесс определения образов, нуждающихся в обновлении при обнаружении новой уязвимости. SBOM также содержит информацию о лицензиях на программные компоненты, что может быть полезно для соблюдения требований соответствия.

SBOM может сыграть решающую роль в борьбе с уязвимостями, связанными с путаницей зависимостей и «галлюцинациями» пакетов.

Путаница зависимостей

Путаница с зависимостями может возникнуть из-за использования зависимости с неподходящей версией — например из-за некорректного указания версии или из-за того, что загрузка производилась из неправильного источника (несоответствующего реестра, менеджера пакетов или кэша). Этого можно избежать, явно указав версию и местоположение зависимости.

Галлюцинации пакета

Проблема путаницы с зависимостями значительно обострилась с появлением кода, генерируемого искусственным интеллектом (ИИ). Исследование 2025 года¹ показало, что в коде, сгенерированном большими языковыми моделями (large language model, LLM), наблюдается тенденция к искажению имен импортируемых пакетов, часто из-за слепого следования предсказуемым шаблонам именования. Это, безусловно, проблема, так как сгенерированный код может не работать только потому, что пытается импортировать несуществующий пакет. Более серьезная проблема возникает, когда злоумышленник добавляет пакеты, имена которых часто искажаются моделями LLM, из-за чего сгенерированный код кажется работоспособным, но содержит уязвимые зависимости.

При сборке контейнеров часто приходится иметь дело с зависимостями, специфическими для языка программирования, на которые ссылается исходный код разработчиков, а также с зависимостями образов контейнеров, указанными в Dockerfile.

¹ *Spracklen J. et al. We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. arXiv, 2 марта 2025 года, <https://arxiv.org/abs/2406.10279>.*

Специфические зависимости в SBOM

В исходном коде довольно редко указываются точные версии используемых им зависимостей (например, команда `import antigavity` в Python¹ не определяет номер версии). Существуют различные подходы к фиксации правильных версий — например, lock-файлы, файлы `go.sum` в Go или `requirements.txt` в Python, — но они зависят от конкретного языка и их применение не является обязательным. Как было показано в главе 6, теги, назначаемые образам контейнеров, лишь приблизительно указывают номера версий. В процессе сборки все эти расплывчатые спецификации преобразуются в конкретные версии, которые и используются при создании образа.

Идеальная SBOM должна точно фиксировать, какие версии разрешено использовать в процессе сборки. Это легко организовать с помощью инструментов сборки, таких как Bazel или экосистема OWASP CycloneDX (<https://github.com/CycloneDX>), которая имеет расширения почти для каждого языка программирования. Например, при сборке проекта Java можно выполнить следующую команду:

```
mvn org.cyclonedx:cyclonedx-maven-plugin:makeAggregateBom
```

А при сборке проекта на Go:

```
cyclonedx-gomod mod -licenses -json -output sbom.json
```

Эти инструменты генерируют SBOM-файлы, включающие все обнаруженные зависимости, на основе таких файлов, как `pom.xml` или `go.mod` проекта соответственно. Расширения CycloneDX существуют и для многих других языков.

Если SBOM не генерируется во время сборки, ее можно создать с помощью инструментов, которые исследуют образы и определяют их содержимое, но в этом случае есть вероятность, что спецификации будут менее полными и точными, особенно когда речь идет о пакетах, специфичных для конкретного языка программирования. Создание SBOM, по сути, решает ту же задачу, что и сканеры уязвимостей, такие как `trivy` и `syft`, — в действительности оба инструмента сканирования способны генерировать не только отчеты об уязвимостях, но и спецификации. Наилучшим вариантом считается объединение SBOM для конкретного языка программирования и SBOM уровня контейнера, потому что в этом случае получается наиболее полная картина. Подробнее об уязвимостях и сканировании образов мы поговорим в главе 8, а о создании SBOM уровня контейнера — далее в этой главе.

¹ См. комикс Python на сайте XKCD: <https://xkcd.com/353>. — *Примеч. пер.*



В OpenSSF существует рабочая группа по обеспечению безопасности репозитория ПО (<https://github.com/ossf/wg-securing-software-repos>), которая занимается выработкой рекомендаций и методов, общих для разных сообществ менеджеров пакетов, с целью совершенствования процесса подписания цифровой подписью и предоставления информации о происхождении ПО с открытым исходным кодом.

Минимальные базовые образы

В первой строке `Dockerfile` находится инструкция `FROM`, ссылающаяся на базовый образ, который будет использоваться как основа для остальной части текущего образа. Как было показано в главе 6, образ включает файловую систему, а базовый образ — начальную точку для этой файловой системы, которая появится в первом слое образа.

Чем меньше базовый образ, тем ниже вероятность, что он содержит ненужный код, а значит, меньше поверхность атаки. Кроме того, небольшие образы быстрее передаются по сети. Существует несколько подходов к минимизации базового образа.

- Если возможно, рассмотрите сборку с нуля, начав с совершенно пустого образа (такой вариант, например, хорошо подходит для автономных исполняемых файлов). Здесь пригодится многоэтапная сборка, когда на начальном этапе (этапах) компиляции используется «полный» базовый образ, а затем полученные двоичные файлы копируются в базовый образ на заключительном этапе. (О многоэтапной сборке будет рассказываться далее в этой главе.)
- Используйте минимальный базовый образ, например образы без дистрибутива (`distroless`-образы) от компании Google (<https://github.com/GoogleContainerTools/distroless>) или образы с усиленной защитой от таких поставщиков, как Chainguard или Docker. Amazon предлагает минимальный образ контейнера (<https://oreil.ly/bJNl9>) для своего дистрибутива Amazon Linux, Microsoft предоставляет минимальные образы с Azure Linux (<https://oreil.ly/E9nXV>), а у Canonical есть проект под названием `chisel` (<https://github.com/canonical/chisel>), цель которого — создание образов Ubuntu с минимальным набором пакетов.
- Инструмент `Slim Toolkit` (<https://slimtoolkit.org>) позволяет проанализировать образ и выявить избыточные компоненты, которые можно удалить.

Использование зависимостей и базовых образов не единственный путь к появлению уязвимостей. На безопасность приложений влияет и написанный разработчиками код. Инструменты статического и динамического анализа безопасности, экспертная оценка, анализ безопасности и тестирование на проникновение — все это может помочь выявить уязвимости, появившиеся

в процессе разработки. Все вышеперечисленное относится не только к контейнеризированным, но и к обычным приложениям, и рекомендации SLSA тоже применимы к любому ПО. Однако, поскольку книга посвящена контейнерам, сосредоточимся на них и начнем с Dockerfile.

Безопасность Dockerfile

На этапе сборки файл Dockerfile превращается в образ контейнера. Количество возможных рисков для безопасности на данном этапе весьма велико.

Происхождение Dockerfile

Инструкции для сборки образа контейнера берутся из Dockerfile. Каждый этап сборки включает выполнение одной из этих инструкций, и если злоумышленник сумеет изменить Dockerfile, то сможет выполнить вредоносные действия, в том числе:

- включить в образ вредоносное ПО или утилиты для майнинга криптовалют;
- получить доступ к секретным данным, используемым на этапе сборки;
- получить полную информацию о топологии сети, доступной из инфраструктуры сборки;
- атаковать хост, на котором производится сборка.

Повторю очевидное: Dockerfile (как и любой другой исходный код) должен быть защищен средствами управления доступом, чтобы обезопасить его от выполнения злоумышленниками вредоносных шагов.

Содержимое Dockerfile также сильно влияет на безопасность полученного в результате сборки образа контейнера. Рассмотрим несколько практических мер, которые можно предпринять в отношении Dockerfile, чтобы повысить безопасность образа.

Практические рекомендации по безопасности Dockerfile

Приведенные ниже рекомендации повышают безопасность образа и сокращают шансы на взлом контейнеров, запущенных из этого образа.

Базовые образы

Первая строка Dockerfile содержит инструкцию `FROM`, в которой задается базовый образ для создания нового образа. При выборе базового образа следует учитывать:

- базовый образ должен извлекаться из доверенного реестра (см. раздел «Хранение образов» в главе 6);

- базовый образ должен содержать только самый необходимый минимум, как говорилось в разделе «Минимальные базовые образы» ранее;
- сторонние базовые образы могут включать вредоносный код, поэтому некоторые организации требуют использования только заранее одобренных («золотых») базовых образов.

Как было показано в главе 6, на версии образов можно ссылаться с помощью тегов или дайджестов (контрольных сумм), поэтому тщательно обдумайте, какой из способов вы будете реализовывать для ссылки на базовый образ. Во втором случае воспроизводимость сборки будет выше, но снизится вероятность получения новых версий базового образа с обновлениями безопасности. (С другой стороны, вы должны получать недостающие обновления с помощью сканирования собранного образа на уязвимости.) Избегайте путаницы зависимостей, явно указывая реестр, откуда должен извлекаться базовый образ.

Многоэтапная сборка

Используя многоэтапную сборку (multistage build), можно исключить ненужное содержимое из итогового образа (<https://docs.docker.com/build/building/multi-stage>). На начальном этапе в образ могут включаться все пакеты и программные средства, необходимые для сборки приложения, но в среде выполнения многие из них не нужны. Приведу такой пример: чтобы создать исполняемую программу на языке Go, нужен компилятор Go. Но контейнеру, в котором выполняется эта программа, доступ к компилятору Go не требуется. Здесь имеет смысл разбить сборку на несколько этапов: на первом производится компиляция и создается двоичный исполняемый файл, а на следующем этот файл просто копируется в меньший базовый образ, возможно даже в scratch-образ, представляющий собой пустую файловую систему. В итоге поверхность атаки разворачиваемого образа значительно уменьшается; помимо пользы для безопасности, сам образ также будет меньше, а значит, сократится время, требуемое на его извлечение.



В блоге корпорации Capital One можно найти несколько примеров многоэтапной сборки (<https://oreil.ly/CRMuY>), демонстрирующих, что на различных шагах многоэтапной сборки можно выполнять даже тестирование, не влияя при этом на содержимое итогового образа.

Непривилегированный пользователь

Инструкция `USER` в `Dockerfile` указывает, что учетная запись пользователя, с полномочиями которой по умолчанию запускаются контейнеры, основанные на этом образе, не обладает `root`-правами. В главе 11 я перечислю несколько веских причин, почему следует избегать запуска контейнеров от имени суперпользователя и указывать непривилегированного пользователя во всех `Dockerfile`, где это возможно.

Команды RUN

Нужно понимать: команда RUN Dockerfile позволяет выполнить любую произвольную команду. Если злоумышленнику удастся внести изменения в Dockerfile, содержащий настройки безопасности по умолчанию, то он сможет выполнить *любой код, какой только пожелает*. Если у вас есть хотя бы минимальные причины не доверять людям, запускающим сборку образов в вашей системе, то скажу прямо: вы фактически предоставили им возможность удаленного выполнения кода. Убедитесь, что полномочия на редактирование Dockerfile есть только у доверенных членов вашей команды, и внимательно следите за кодом, меняющим эти настройки. Возможно, имеет смысл даже ввести проверку или журнал аудита при добавлении или изменении команд RUN в Dockerfile.

Точки монтирования томов

Каталоги хоста часто монтируются к контейнеру с помощью точек монтирования томов, особенно для демонстрации и тестов. Как вы увидите в главе 11, важно проверять, чтобы инструкции в Dockerfile не монтировали к контейнеру каталоги с конфиденциальными данными, например /etc или /bin.

Не включайте в Dockerfile конфиденциальные данные

Более подробно конфиденциальные данные и секреты мы обсудим в главе 14, а пока просто имейте в виду, что включение в образ учетных данных, паролей и прочих секретов упрощает злоумышленникам доступ к ним.

Избегайте исполняемых файлов с установленным битом setuid

Как обсуждалось в главе 2, желательно не включать в образ исполняемые файлы с установленным битом `setuid`, поскольку они потенциально могут открывать путь к повышению привилегий.

Избегайте лишнего кода

Чем меньше в контейнере кода, тем меньше поверхность атаки. Не добавляйте в образ пакеты, библиотеки и исполняемые файлы, если они не являются строго необходимыми. По той же причине желательно использовать в качестве базового пустой образ или один из distroless-образов, ведь это резко сократит объем кода — а значит, и уязвимость вашего образа.

Включайте все, что действительно нужно контейнеру

Если в предыдущем пункте я призывала вас исключать из сборки лишний код, то здесь вывод иной: *включайте* в нее все, что требуется вашему приложению для работы. Если разрешить контейнеру устанавливать дополнительные пакеты в среде выполнения, то как проверить, что ни один из них не содержит вредоносного кода? Гораздо лучше выполнять установку и проверку всех пакетов на этапе сборки и создать неизменяемый образ. Подробнее о том, почему этот вариант предпочтительнее, рассказывается в подразделе «Неизменяемые контейнеры» в главе 8.

Избегайте путаницы зависимостей

Вот несколько способов избежать путаницы зависимостей в файлах Dockerfile.

- Как уже упоминалось ранее, базовый образ задается в инструкции FROM в Dockerfile. Указывайте версию базового образа явно по хешу (предпочтительнее) или хотя бы по тегу с номером версии — не полагайтесь на тег latest. Также явно укажите реестр, чтобы система не пыталась получить образ из неожиданного источника по умолчанию.
- Загружайте базовые образы только из источников, которым вы доверяете. Многие организации требуют извлекать образы из частного реестра, чтобы гарантировать использование одобренной версии. Изначально образы могут поступать из публичного реестра, затем проверяться службой безопасности и только потом сохраняться в частном реестре. Другой вариант — собирать базовые образы из исходного кода.
- Убедитесь, что менеджеры пакетов используют правильные реестры — например с помощью параметра --index-url в командах RUN pip install или set @my-org:registry в команде RUN npm config.
- Указывайте точные версии зависимостей пакетов в таких командах, как RUN apt-get install.
- Избегайте автоматического или неявного обновления зависимостей в процессе сборки. Например, задействуйте параметр --require-hashes при выполнении команды RUN pip install.
- В идеале все необходимые пакеты следует устанавливать явно — например, используя флаг --no-install-recommends в команде RUN apt-get install.

Явно указывая версии, вы гарантированно получите образ с точно определенным набором зависимостей и избежите путаницы зависимостей, однако у этого подхода есть и недостатки. Если не обновлять версии, можно пропустить важные обновления безопасности, которые были бы автоматически установлены при предъявлении менее строгих требований к номерам версий. С другой стороны, если ослабить это требование, то можно оказаться в ситуации, когда новая версия пакета «ломает обратную совместимость», из-за чего ваш код становится неработоспособным. Сканирование уязвимостей поможет определить, когда образ нуждается в важном обновлении безопасности, а тестирование позволит выявить ситуации нарушения обратной совместимости, однако ни один из этих методов не дает стопроцентной гарантии. Поэтому важно найти баланс между автоматическими обновлениями и явно задаваемыми версиями зависимостей, что весьма непросто.

Следуя этим рекомендациям, вы сможете создавать образы, которые сложнее взломать. А теперь давайте обсудим риск того, что злоумышленник найдет слабые места в системе сборки контейнеров.

Атаки на машину сборки

Компьютер, на котором производится сборка, с точки зрения безопасности интересует нас по двум основным причинам.

- Если у злоумышленника получится взломать машину сборки и выполнить на ней код, сможет ли он добраться до прочих частей системы? Как вы видели в подразделе «Опасности команды `docker build`» в главе 6, имеет смысл использовать утилиту сборки, не требующую привилегированного процесса-демона. Старайтесь задействовать инструменты сборки, не требующие привилегий суперпользователя.
- Способен ли злоумышленник повлиять на результаты сборки так, что в итоге будут собраны и запущены вредоносные образы? Последствия любого несанкционированного доступа, который влияет на инструкции `Dockerfile` или инициирует сборку непредусмотренных образов, могут оказаться катастрофическими. Например, если атакующий внесет изменения в собираемый код, он сможет и внедрить лазейки в контейнеры в рабочей среде.

Поскольку машины сборки генерируют код, который в дальнейшем будет выполняться в рабочем кластере, крайне важно защищать их от атак, как и сам кластер. Старайтесь уменьшить поверхность атаки, удаляя ненужные утилиты из машин сборки. Выполняйте сборку на отдельной (изолированной от рабочей среды) машине или в кластере машин, чтобы ограничить возможный эффект атаки на хост из сборки. Еще лучше, если это возможно, используйте временную инфраструктуру, создавая новые виртуальные машины для каждой сборки: злоумышленнику будет сложнее проникнуть в систему сборки, если его лишить возможности «сохранять» (потенциально вредоносные) файлы между сборками.



Многие сервисы сборки и платформы CI/CD, такие как GitHub Actions, GitLab CI/CD и Bitbucket Pipelines, поддерживают концепцию *исполнителей* (runners) — временных виртуальных машин, которые запускаются специально для выполнения сборки. GitHub предлагает машины-исполнители, размещенные в Microsoft Azure, но есть возможность предоставить собственную инфраструктуру в публичном или частном облаке. Задание может выполняться непосредственно на машине-исполнителе или внутри контейнера, запущенного на ней.

Филипп Гарбе (Philipp Garbe) в своей статье описал различные варианты настройки GitHub Actions с использованием машин-исполнителей, автоматически размещаемых на AWS CodeBuild (<https://garbe.io/blog/2025/07/28/github-actions-codebuild>).

Сборка с полномочиями непривилегированного пользователя лишает злоумышленника возможности запускать произвольный код на хосте, а для организации многоуровневой защиты следует также ограничить доступ к сети и облачным сервисам с машин сборки, чтобы предотвратить доступ к другим компонентам инфраструктуры.

Уменьшайте поверхность атаки, удаляя ненужные инструменты с машин сборки. Ограничивайте прямой доступ пользователей к этим машинам и защищайте их от несанкционированного доступа к сети с помощью VPC и межсетевых экранов.

Генерирование SBOM

Как обсуждалось ранее, хорошей практикой является сопровождать свои образы спецификацией SBOM, перечисляющей их содержимое. В идеале должен получиться перечень SBOM, относящийся к языку программирования вашего приложения, а также содержащий информацию о базовом образе и установленных пакетах ОС.

Лучше всего, если спецификация будет формироваться во время сборки, например, с помощью команды `docker build --sbom=true`. Есть также возможность сгенерировать спецификацию для существующего образа. Для этого обычно применяется несколько инструментов, среди которых `syft` (<https://github.com/anchore/syft>) и `trivy` (<https://trivy.dev>). Информация для SBOM часто генерируется в распространенных форматах SPDX или CycloneDX. Например, вот как можно сгенерировать SBOM в формате SPDX для последнего образа `nginx`:

```
$ trivy image --format spdx-json nginx
```

Эта команда выведет информацию о пакетах в образе, сведения о лицензировании, взаимосвязи между пакетами и данные об инструменте, который сформировал отчет. Когда я запустила команду выше, то получила вывод, содержащий более 8000 строк, поэтому я не буду приводить их все, но, чтобы вы могли получить некоторое представление, вот выдержка (с некоторыми удаленными или сокращенными строками) с описанием пакета `bash`, который Trivy выявил в контейнере `nginx`:

```
"name": "bash",  
"SPDXID": "SPDXRef-Package-a592647b46e04269",  
"versionInfo": "5.2.15-2+b8",  
"supplier": "Organization: Matthias Klose \u003cdoko@debian.org\u003e",  
"downloadLocation": "NONE",  
"filesAnalyzed": false,
```

```
"sourceInfo": "built package from: bash 5.2.15-2",
"licenseConcluded": "GPL-3.0-or-later AND GPL-3.0-only AND LicenseRef-4f0e9e1...",
"licenseDeclared": "GPL-3.0-or-later AND GPL-3.0-only AND LicenseRef-4f0e9e1...",
"externalRefs": [
  {
    "referenceCategory": "PACKAGE-MANAGER",
    "referenceType": "purl",
    "referenceLocator": "pkg:deb/debian/bash@5.2.15-2%2Bb8?arch=amd64\u0026distro=debian-12.11"
  }
],
"primaryPackagePurpose": "LIBRARY",
"annotations": [
  {
    "annotator": "Tool: trivy-0.63.0",
    "annotationDate": "2025-06-23T10:06:37Z",
    "annotationType": "OTHER",
    "comment": "LayerDiffID: sha256:7fb72a7d1a8e984..."
  }
],
```

Спецификации SBOM можно передавать на вход сканера уязвимостей для сопоставления с известными уязвимостями (об этом мы поговорим в главе 8), и индексировать, чтобы быстро выявлять компоненты, затронутые недавно обнаруженными уязвимостями. Кроме того, SBOM также можно использовать для проверки соответствия лицензионным требованиям. Например, если вы разрабатываете коммерческое ПО, то вас, вероятно, будет волновать необходимость включения лицензий GPL. Аналогично организация может обращаться к SBOM для проверки соблюдения политик, определяющих, какие компоненты разрешено использовать в образах.

Почти наверняка вам потребуется хранить SBOM вместе с образом, который она описывает. Вы можете выгрузить ее в реестр как OCI- артефакт со ссылкой на образ, либо подписать и прикрепить к образу. Давайте рассмотрим, как можно подписывать образы и другие артефакты.

Подписывание образов и программных компонентов

Подписывая образ, вы связываете с ним криптографическую сущность (аналогично подписыванию сертификатов, о котором мы поговорим в главе 11). Подпись позволяет убедиться, что образ создан доверенной стороной и не был изменен с после сборки. Другие компоненты, включая спецификации SBOM, тоже можно подписывать, чтобы подтвердить, что они созданы доверенным поставщиком.

Первоначально для подписания образов использовался Docker Content Trust, созданный на основе проекта с открытым исходным кодом `notary` (<https://github.com/notaryproject>). Однако подход `notary` считался слишком сложным и позже был заменен проектом версии 2, чей инструмент командной строки называется `notation` (<https://github.com/notaryproject/notation>). Этот проект поддерживается крупными представителями облачной индустрии, включая AWS, Microsoft и Docker.

Альтернативный подход, разработанный в Google, — инструмент `sigstore` (<https://www.sigstore.dev>), который сейчас принадлежит OpenSSF (дочернему фонду CNCF). Фактически он стал стандартным средством с открытым исходным кодом для подписи образов контейнеров и других артефактов в экосистеме Kubernetes и состоит из трех основных компонентов:

- `cosign` — используется для подписи и проверки файлов и артефактов, таких как образы контейнеров, чарты Helm и файлы SBOM;
- `fulcio` — центр сертификации, выдающий сертификаты с коротким сроком действия;
- `rekor` — записывает операции подписания в неизменяемый журнал.

Пожалуй, самое значительное преимущество Sigstore перед предыдущими подходами — поддержка *бесключевой подписи* (keyless signing). Вместо закрытых ключей, для которых обычно требуется целая инфраструктура, Sigstore использует временные сертификаты, автоматически выдаваемые на основе информации от известных поставщиков идентификации OIDC, такой как учетные записи Google, Microsoft или GitHub. В результате пользователям не нужно беспокоиться о смене и распространении ключей.

Если вы пожелаете прикрепить SBOM к образу перед его подписанием, то сделать это можно будет так:

```
$ cosign attach sbom --sbom sbom.json my-image:v0.0.1
```

Прежде чем подписать и сохранить образ в реестре, вы можете добавить к нему аттестат сборки.

Создание аттестатов

Проверка подписи подтверждает, что образ создан доверенной стороной, но как узнать, не вмешивался ли кто-либо в процесс сборки во время создания образа и его компонентов? Аттестат сборки (build attestation) описывает, как был создан образ, включая такую информацию, как:

- имя репозитория и номер коммита;
- среда сборки;
- версии инструментов сборки;
- параметры и вывод сборки;
- сведения о том, кто или что послужили причиной начала сборки и когда.

Аттестаты могут храниться в реестре контейнеров в виде метаданных OCI, сопровождающих образы, и проверяться при развертывании.

В рамках проекта SLSA существуют инструменты, генерирующие информацию о происхождении сборки: `slsa-github-generator` (<https://github.com/slsa-framework/slsagithub-generator-go>) в GitHub Actions и утилита командной строки `slsa-provenance`. Хорошей практикой считается подписывать аттестаты с помощью таких средств, как `cosign`.

Помимо инструментов, предоставляемых фондом OpenSSF в рамках проекта SLSA, существует еще один проект, который решает проблемы, связанные с цепочкой поставок образов контейнеров: `in-toto` (<https://in-toto.io>). Он гарантирует, что каждый из ожидаемых этапов сборки был выполнен полностью, в правильном порядке, доверенными людьми и дал точный результат при корректных входных данных. Фреймворк `in-toto` связывает несколько этапов и передает между ними метаданные, касающиеся безопасности. В результате гарантируется, что ПО в рабочей среде будет выполнять именно тот код, который разработчик отправил в репозиторий со своего ноутбука.

Вот как с помощью `cosign` прикрепить аттестат сборки к образу:

```
$ cosign attach attestation my-image:v0.0.1 --attestation
./my-image-sbom.att.json
```

Прикрепление SBOM и аттестатов считается необязательной, но хорошей практикой, повышающей безопасность и помогающей достичь более высокого уровня SLSA. После прикрепления всех необходимых файлов можно подписать сам образ:

```
$ cosign sign my-image:latest
```

Подписанные образы допускается хранить в реестре OCI.

А что делать, если вам понадобится образ контейнера от стороннего поставщика, — либо непосредственно как приложение, либо в качестве базового образа для своей сборки? В таком случае вы сможете убедиться, что образ получен от доверенного поставщика, выполнив команду `cosign verify`. Подробнее об этом я расскажу далее в этой главе.



Не могу не упомянуть один интересный публичный реестр: `ttl.sh`. В него можно анонимно выгружать образы для временного использования. Такая возможность может пригодиться в конвейерах CI/CD, где тестирование распараллелено между несколькими машинами, каждой из которых необходим доступ к только что созданному образу. Выгружая образ, вы должны указать продолжительность его хранения (не более 24 часов). Образы допускается маскировать универсальным уникальным идентификатором (Universally Unique Identifier, UUID), достаточно длинным, чтобы с высокой вероятностью быть уникальным. UUID немного затруднит злоумышленнику поиск образа, но этот реестр — общедоступный, поэтому я бы не рекомендовала использовать его для чего-либо, что действительно нужно сохранить в тайне.

Теперь, зная о подписывании образов, вы согласитесь, что подписывать образы перед отправкой в этот реестр и проверять целостность подписи после извлечения — хорошая идея.

Манифесты образов

После включения подписей и аттестатов в образ они становятся частью *манифеста образа*, о котором кратко рассказывалось в главе 6. Там же вы видели, что для просмотра манифеста используется команда `docker inspect`. Еще один полезный инструмент для просмотра манифестов образов — проект ORAS (<https://oras.land>) от CNCF. Желающие углубиться в эту тему найдут на веб-сайте проекта много информации о форматах манифестов.

В примере я использовала образ `rekor` от Sigstore из реестра контейнеров на GitHub, потому что он содержит много интересной информации. К сожалению, на момент написания книги многие стандартные и свободно доступные образы библиотек не содержали подписей и аттестатов.

Вот пример получения манифеста — на этот раз для образа Alpine, хранящегося в Docker Hub:

```
$ oras manifest fetch docker.io/library/alpine:latest | jq
{
  "manifests": [
    {
      "annotations": {
        "com.docker.official-images.bashbrew.arch": "amd64",
        "org.opencontainers.image.base.name": "scratch",
        "org.opencontainers.image.created": "2025-05-30T18:04:04Z",
        ...
        "org.opencontainers.image.version": "3.22.0"
      },
      "digest": "sha256:08001109a7d6...6a7338ce",
      "mediaType": "application/vnd.oci.image.manifest.v1+json",
    }
  ]
}
```

```

    "platform": {
      "architecture": "amd64",
      "os": "linux"
    },
    "size": 1022
  },
  {
    "annotations": {
      "com.docker.official-images.bashbrew.arch": "amd64",
      "vnd.docker.reference.digest": "sha256:08001109a7d6...6a7338ce",
      "vnd.docker.reference.type": "attestation-manifest" ❷
    },
    "digest": "sha256:bd4199eb785a...859f0827f",
    "mediaType": "application/vnd.oci.image.manifest.v1+json",
    "platform": {
      "architecture": "unknown",
      "os": "unknown"
    },
    "size": 838
  },
  ...
}

```

Очень похоже на результат команды `docker manifest inspect`, который вы уже видели. Для простоты я оставила в выводе только то, что относится к платформе `amd64`; выполнив эту команду у себя, вы увидите аналогичное содержимое и для других аппаратных архитектур. На этот раз я хочу обратить ваше внимание на аттестат, включенный в манифест.

❶ Первый пункт в этом списке ссылается на образ для платформы `amd64`, при этом его дайджест начинается с `080011`.

❷ Следующий пункт — это `attestation-manifest` для данного образа, который ссылается на него по дайджесту (контрольной сумме).

Давайте взглянем на этот манифест аттестата (для простоты я сократила дайджесты в нем):

```
$ oras copy docker.io/library/alpine@sha256:bd419...f0827f --to-oci-layout
alpine-attestation
```

```

✓ Copied application/vnd.in-toto+json
79.1/79.1 KB 100.00% 4ms
└─ sha256:5ff8d...7ea6a
✓ Copied application/vnd.in-toto+json
5.55/5.55 KB 100.00% 2ms
└─ sha256:0775b...09944
✓ Copied application/vnd.oci.image.config.v1+json
241/241 B 100.00% 2ms
└─ sha256:7a444...41d36
✓ Copied application/vnd.oci.image.manifest.v1+json

```

```
838/838 В 100.00% 3ms
└─ sha256:bd419...0827f
Copied [registry] docker.io/library/alpine@sha256:bd419...f0827f => [oci-layout]
alpine-attestation
Digest: sha256:bd419...f0827f
```

Как видите, первые два элемента в выводе относятся к аттестатам `in-toto`. Файлы были извлечены в формате ОЦИ (с которым вы уже сталкивались в главе 4) в каталог с именем `alpine-attestation`. Внутри этого каталога присутствует несколько объектов с двоичными данными, которые соответствуют дайджестам элементов:

```
$ file blobs/sha256/*
blobs/sha256/0775b...09944: ASCII text, with very long lines (5682), with no line
terminators
blobs/sha256/5ff8d...7ea6a: JSON data
blobs/sha256/7a444...41d36: JSON data
blobs/sha256/bd419...0827f: JSON data
```

Файл JSON с хешем SHA, начинающимся с `5ff8d`, является одним из аттестатов `in-toto`. Файл довольно объемный, поэтому я покажу лишь несколько фрагментов кода, чтобы вы могли получить представление о нем:

```
"_type": "https://in-toto.io/Statement/v0.1",
"predicateType": "https://spdx.dev/Document",
"subject": [
  {
    "name": "pkg:docker/alpine@3.22.0?platform=linux%2Famd64",
    "digest": {
      "sha256": "08001...338ce"
    }
  },
  ...
],
"predicate": {
  "spdxVersion": "SPDX-2.3",
  "dataLicense": "CC0-1.0",
  "SPDXID": "SPDXRef-DOCUMENT",
  "name": "sbom",
  "documentNamespace": "https://docker.com/docker-scout/fs/sbom-15dbb...9910858",
  "creationInfo": {
    "creators": [
      "Organization: Docker, Inc",
      "Tool: docker-scout-1.18.0",
      "Tool: buildkit-0.16.0-tianon"
    ],
    "created": "2025-05-30T18:04:21Z"
  },
  "packages": [
    ...
    {
      "name": "busybox-binsh",
      "SPDXID": "SPDXRef-Package-1cf4a...06f01",
      "versionInfo": "1.37.0-r18",
      ...
    }
  ]
}
```

```
"licenseDeclared": "GPL-2.0-only",
"description": "busybox ash /bin/sh",
"externalRefs": [
  {
    "referenceCategory": "PACKAGE-MANAGER",
    "referenceType": "purl",
    "referenceLocator": "pkg:apk/alpine/busybox-binsh@1.37.0-r18?os_name=alpine&os_version=3.22"
  }
]
```

...

Подробный разбор всех деталей этого аттестата выходит за рамки книги, однако из приведенного фрагмента видно, что он относится к пакету Alpine, включает SBOM, созданную с помощью инструментов `docker-scout` и `buildkit`, и среди перечисленных в спецификации пакетов присутствует командная оболочка `busybox`, распространяемая по лицензии GPL-2.0.

В повседневной работе вам вряд ли потребуется исследовать подобные аттестаты или подписи, но однажды знание этой информации может оказаться весьма полезным, особенно когда вам понадобится выяснить первопричину проблемы безопасности цепочки поставок.

Безопасность развертывания образов

Основной вопрос безопасности на этапе развертывания — как гарантировать извлечение и запуск того образа, который нужен, хотя в процессе так называемого *контроля допуска* (*admission control*) могут выполняться и дополнительные проверки.

Развертывание правильного образа

Как вы видели в разделе «Идентификация образов» главы 6, теги образов контейнеров не являются неизменяемыми — их можно переназначить на другие версии того же образа (хотя некоторые реестры, включая AWS Elastic Container Registry, поддерживают неизменяемые теги). Если же ссылаться на образ по дайджесту, а не тегу, то можно быть уверенными, что версия образа именно та, которая нужна. Однако если ваша система сборки присваивает образам теги на основе семантического версионирования и строго его соблюдает, этого может быть достаточно, поскольку не придется обновлять ссылки на образы при каждом мелком обновлении.

Если вы используете теги, то перед запуском необходимо всегда извлекать последнюю версию на случай возможного обновления. К счастью, это не требует слишком больших затрат ресурсов, поскольку сначала извлекается манифест, а слои образа скачиваются только в том случае, если они изменились.

В Kubernetes это поведение задается параметром `imagePullPolicy`. Задавать стратегию извлечения образов всякий раз не нужно, если вы ссылаетесь на них по дайджесту, поскольку любое обновление ведет к изменению дайджеста.

Вредоносная конфигурация развертывания системы

При использовании оркестратора контейнеры, составляющие каждое приложение, обычно описываются в файлах конфигурации — например, в YAML-файлах в случае Kubernetes. Проверять происхождение этих файлов не менее важно, чем проверять сами образы.

Очень тщательно проверяйте скачанные из Интернета YAML-файлы, прежде чем выполнять их в рабочем кластере. Учтите, что даже крошечные отличия — например замена одного символа в URL реестра — могут привести к запуску вредоносного образа в развернутой вами системе.

Проверка подлинности подписи и происхождения образа

Нет смысла подписывать образы, если эти подписи не проверяются! Проверка подписи образа гарантирует, что он получен из надежного источника, будь то ваш собственный конвейер сборки или поставщик. Проверка происхождения гарантирует, что все этапы процесса сборки образа выполнены должным образом. Если вы следуете стандартам SLSA и/или нормативным требованиям, то должны выполнять эти проверки.

В подразделе «Развертывание правильного образа» выше я говорила, что можно загружать образы по тегам, чтобы получать последние обновления для этой версии. Проверка подписи позволит вам убедиться, что это запланированное изменение и что вы не загрузили образ, измененный злоумышленником.

Проверить цифровую подпись образа можно с помощью `cosign` следующим образом:

```
cosign verify --key supplier.pub myimage:tag
```

В данном случае `supplier.pub` — это открытый ключ поставщика образа. Аналогично можно использовать команду `cosign verify-attestation`, чтобы проверить аттестат сборки. Проект SLSA также предоставляет инструмент `slsa-verifier` (<https://github.com/slsa-framework/slsa-verifier>) для проверки информации о происхождении.

Эти проверки можно включить в конвейер CI/CD как этап, предшествующий развертыванию, но также можно использовать контроллер допуска Kubernetes.

Контроль допуска

Контроллер допуска, такой как Kyverno (<https://kyverno.io>) или OPA Gatekeeper (<https://open-policy-agent.github.io/gatekeeper/website/docs>), может выполнять проверки непосредственно перед развертыванием ресурса в кластере. В Kubernetes контроль допуска позволяет проверять соответствие ресурсов любых типов заданным политикам. Но в этой главе мы рассмотрим только контроллер допуска, определяющий, разрешать ли запуск контейнера, в основе которого лежит конкретный образ, заданный политиками. Если тест контроля допуска не пройден, то контейнер не запускается.

Политики контроля допуска могут включать несколько важных проверок безопасности образа контейнера, прежде чем превратить последний в работающий контейнер.

- Был ли образ просканирован на предмет уязвимостей/вредоносного ПО/соответствия политикам?
- Получен ли образ из доверенного реестра?
- Подписан ли образ доверенной стороной?
- Одобрен ли образ для использования?
- Выполняется ли образ от имени суперпользователя?

Этот контроль гарантирует, что никто не сможет обойти проверки, выполненные на более ранних этапах. Например, нет смысла вводить в конвейер сборки образа сканирование на уязвимости, если оказывается, что у злоумышленников есть возможность указывать инструкции развертывания, ссылающиеся на непросканированные образы.

Резюме

В этой главе вы узнали, что среде выполнения контейнера необходима корневая файловая система и определенная информация о настройках. Конфигурацию можно переопределить с помощью параметров, передаваемых в среду выполнения или указываемых в манифестах Kubernetes. Некоторые из этих настроек конфигурации влияют на безопасность приложения. Кроме того, если не следовать практическим рекомендациям, представленным в подразделе «Практические рекомендации по безопасности Dockerfile» выше в этой главе, открывается множество путей для попадания в образы контейнеров вредоносного кода.

Инструменты обеспечения безопасности цепочки поставок позволяют подписывать образы, а также передавать информацию об их содержимом и процессе

создания. На этапе развертывания образа оркестраторы и утилиты безопасности предоставляют возможность осуществлять контроль допуска, а это отличная возможность для выполнения проверок безопасности этих образов, включая проверку подписей и информации о происхождении.

Образы контейнеров инкапсулируют код приложения и все зависимости от сторонних пакетов и библиотек. В следующей главе вы узнаете, что и эти зависимости могут содержать уязвимости, а также изучите инструменты для обнаружения и устранения этих уязвимостей.

Программные уязвимости в образах контейнеров

Исправление уязвимостей в ПО давно является важной частью поддержания безопасности развернутого кода. Эта задача стала еще более актуальной, потому что время, необходимое злоумышленнику для использования уязвимости, постоянно сокращается. Согласно отчету Mandiant за 2024 год (<https://cloud.google.com/blog/topics/threat-intelligence/time-to-exploit-trends-2023?hl=en>), этот срок оценивается всего в пять дней, а с учетом злонамеренного использования ИИ для изобретения и автоматизации атак он, вероятно, стал значительно меньше.

В мире контейнеров эта проблема не теряет своей значимости, и, как вы увидите в текущей главе, процесс исправления был полностью переосмыслен. Но сначала разберемся, что такое программные уязвимости и как информация о них публикуется и отслеживается.

Исследования уязвимостей

Уязвимость — это известный изъян в программном обеспечении, которым злоумышленник может воспользоваться, чтобы произвести какие-либо вредоносные действия. Как правило, чем сложнее ПО, тем выше вероятность, что в нем есть ошибки, открывающие возможность для атаки.

Когда в широко используемом программном обеспечении обнаруживается изъян, злоумышленники могут эксплуатировать его везде, где развернуто это ПО. Поэтому существует целая сфера исследований, посвященная поиску и публикации информации об уязвимостях в общедоступном ПО, особенно в пакетах для операционных систем и в библиотеках языков программирования. Наверное, вы слышали о некоторых наиболее катастрофических уязвимостях, таких как Shellshock, Meltdown и Heartbleed, имеющих не только название, но иногда даже и эмблему. Это своего рода рок-звезды мира уязвимостей, однако они составляют лишь крошечную долю из тысяч проблем, о которых сообщается ежегодно.

В наши дни новые уязвимости в существующем ПО находят не только люди, но и искусственный интеллект. В начале января 2025 года Microsoft сообщила, что платформа автоматического обнаружения уязвимостей unpatched.ai (<https://unpatched.ai>) выявила множество серьезных проблем. Широкую известность также получили соревнования по кибербезопасности с использованием ИИ (Artificial Intelligence Cyber Challenge; <https://aicyberchallenge.com>), проводимые на площадке DARPA (Defense Advanced Research Projects Agency — Агентство передовых оборонных исследований). Их цель — разработка инструментов на основе ИИ для поиска и устранения уязвимостей в коде инфраструктуры, включая ядро Linux, Jenkins и SQLite. Методы динамического тестирования с использованием ИИ, применяемые командой Google Open Source Security Team, с каждым годом выявляют все большее количество проблем.

К сожалению, ИИ используется не только для выявления реальных проблем, но также для создания множества фиктивных отчетов по безопасности (<https://oreil.ly/95a0w>) об уязвимостях, которых на самом деле нет. Анализ таких отчетов отнимает огромное количество времени у разработчиков проектов и, вероятно, приводит некоторых из них к «выгоранию» (https://oreil.ly/ScMT_). Если вы думаете об использовании ИИ для быстрого получения вознаграждения за обнаружение ошибок, то не спешите и подумайте еще раз.

После того как уязвимость будет выявлена, начинается гонка за выпуском исправлений, чтобы пользователи успели установить их прежде, чем ошибки кода задействуют злоумышленники. Если бы о слабых местах в ПО сразу же объявляли публично, это фактически развязало бы злоумышленникам руки. Во избежание этого была принята методология ответственного подхода к раскрытию проблем безопасности. Обнаружив уязвимость, аналитик в области безопасности связывается с разработчиком или поставщиком соответствующего ПО и договаривается о сроке, по истечении которого может публиковать полученные результаты. Эти сроки подгоняют поставщика, побуждая его как можно быстрее выпустить патч, поскольку и для него, и для пользователей лучше, если исправление будет доступно до публикации информации об уязвимости.

Каждой новой проблеме присваивается уникальный идентификатор, начинающийся с букв CVE (Common Vulnerabilities and Exposures — общедоступный перечень уязвимостей и угроз), за которыми следует год. Например, уязвимость Shellshock была открыта в 2014 году и официально называется CVE-2014-6271. Организация, которая заведует этими идентификаторами, называется MITRE (<https://mitre.org/>). Она координирует работу более 400 центров по нумерации CVE (CVE Numbering Authority, CNA; <https://www.cve.org/ProgramOrganization/CNAs>), имеющих право присваивать идентификаторы CVE в определенных областях. Некоторые крупные поставщики ПО, например Microsoft, Red Hat и Oracle, являются такими центрами и обладают правом присваивать идентификаторы уязвимостям в их собственных программных продуктах. GitHub получил статус CNA в конце 2019 года.

По мере выявления дефектов их регистрируют на сайте CVE (<https://cve.org>), а затем эти идентификаторы CVE используются в Национальной базе уязвимостей (National Vulnerability Database, NVD) (<https://nvd.nist.gov/>) для отслеживания пакетов и версий программ, затронутых каждой из угроз¹. На первый взгляд кажется, что на этом все и заканчивается: есть список всех затронутых версий пакетов, так что если у вас установлена одна из них, значит, ваша система уязвима для атаки. К сожалению, не все так просто, поскольку в зависимости от используемого дистрибутива Linux может существовать исправленная версия соответствующего пакета.

Как было показано в главе 7, лучшей практикой является сопровождение каждого образа спецификацией SBOM, в которой перечислены все программные компоненты и их версии. Когда обнаруживаются новые уязвимости, можно проверить содержимое SBOM и определить, какие образы затронуты.

Уязвимости, исправления и дистрибутивы

Возьмем для примера Shellshock — критическую уязвимость, затронувшую пакет GNU bash. На странице NVD, посвященной CVE-2014-6271 (<https://nvd.nist.gov/vuln/detail/CVE-2014-6271>), приведен длинный список уязвимых версий, от 1.14.0 до 4.3. Если вы работаете на очень старой версии Ubuntu, 12.04, и обнаружили, что версия bash на вашем сервере — 4.2-2ubuntu2.2, то можете подумать, что она уязвима, поскольку основана на оболочке bash 4.2, включенной в список NVD для Shellshock.

На самом же деле, согласно инструкции по безопасности Ubuntu для этой уязвимости (<https://oreil.ly/IEUqF>), данная конкретная версия уже содержит патч безопасности. Дело в том, что специалисты по сопровождению Ubuntu решили, что лучше задействовать исправление уязвимости и опубликовать корректную версию, чем заставлять всех работающих на 12.04 обновляться до новой промежуточной версии bash.

Чтобы получить объективное представление об уязвимости пакетов, установленных на сервере, необходимо заглянуть не только в NVD, но и в инструкции по безопасности для конкретного дистрибутива.

До сих пор в главе шла речь о пакетах (например, bash в предыдущем примере), распространяемых в двоичном виде через такие системы управления пакетами, как apt, yum, rpm или apk. Эти пакеты используются всеми приложениями,

¹ NVD организована в США, но широко используется во всем мире, однако существуют и другие базы данных уязвимостей и рекомендации по безопасности. В странах ЕС имеются свои базы данных, которые ведутся во Франции и Германии. Аналогичные базы данных есть также в Японии и Бразилии. Они в основном используют или по крайней мере ссылаются на систему нумерации CVE.

присутствующими в файловой системе, и данный факт приводит к бесчисленным проблемам на серверах и виртуальных машинах: приложение может зависеть от определенной версии пакета, несовместимой с другим приложением, которое должно работать на той же машине. Эта проблема управления зависимостями — одна из тех, в решении которых могут помочь контейнеры благодаря отдельной корневой файловой системе у каждого из них.

Уязвимости уровня приложения

Встречаются и уязвимости на уровне приложений. Большинство приложений работают со сторонними библиотеками, которые обычно устанавливаются с помощью системы управления пакетами, специфичными для конкретного языка программирования. В Node.js применяется `npm`, в Python — `pip`, в Java — `Maven` и т. д. В компилируемых языках, например Go, C и Rust, сторонние зависимости могут устанавливаться в виде совместно используемых библиотек или включаться в исполняемый файл во время сборки.

Сторонние библиотеки, установленные через эти утилиты, — еще один потенциальный источник уязвимостей. В главе 7 мы видели, что средства, специфичные для языков программирования, могут генерировать подробные файлы SBOM, помогающие выявлять уязвимости в зависимостях, устанавливаемых с помощью `pip`, `npm`, `Maven` и т. д.

У автономных исполняемых файлов по определению (что следует из самого слова «автономный») нет внешних зависимостей. У них могут быть зависимости от сторонних библиотек, но они встроены в эти исполняемые файлы. В таком случае можно создать образ контейнера на основе пустого базового образа, содержащего только нужный исполняемый файл.

Если у приложения нет зависимостей, то его нельзя просканировать на предмет известных уязвимостей. Однако в нем все равно могут быть ошибки, делающие его уязвимым для атак, и это мы обсудим в разделе «Уязвимости нулевого дня» далее в главе.

Управление рисками, связанными с уязвимостями

Работа с уязвимостями программного обеспечения — важная часть управления рисками. Любое более или менее сложное ПО, скорее всего, содержит некоторое количество уязвимостей, и существует риск, что через них будет осуществлена атака на приложение. Чтобы управлять этим риском, необходимо идентифицировать уязвимости, оценить степень их серьезности, распределить по степени приоритетности и наладить процессы их исправления либо снижения негативных последствий.

Наличие уязвимости само по себе не означает, что она актуальна или может быть использована злоумышленником. Предположим, что библиотека содержит две функции: `decent_code()` и `poorly_written()`. Приложение может импортировать эту библиотеку, потому что использует функцию `decent_code()`. При этом код `poorly_written()` все еще присутствует в приложении, однако если он никогда не вызывается, то заключенную в нем уязвимость невозможно эксплуатировать.

Уязвимости, имеющиеся в коде, но недоступные для использования, называют ложноположительными. Такие дефекты ПО создают дополнительную работу для команды безопасности по их проверке. Как раз в таких случаях на помощь приходит VEX.

Vulnerability Exploitability eXchange (VEX) — это машиночитаемый формат для описания влияния известных CVE. Документ VEX охватывает некоторое количество CVE и описывает, относится ли конкретный программный артефакт к одной из следующих категорий:

- **affected** (затронут) — уязвимость присутствует и может использоваться злоумышленниками;
- **not_affected** (не затронут) — уязвимость неприменима к данному артефакту;
- **fixed** (исправлен) — в этой версии уязвимость устранена;
- **under_investigation** (исследуется) — ведутся работы по выяснению, затронут ли данный артефакт уязвимостью.

Разработчик ПО может создать документ VEX, чтобы проинформировать пользователей своего продукта о статусе уязвимостей CVE. Документы VEX также можно подписывать для подтверждения их подлинности и хранить в реестрах ОСИ вместе с описываемыми в них образами.

Сканеры уязвимостей автоматизируют процесс идентификации уязвимостей и предоставляют информацию о степени серьезности проблем, а также о версиях пакетов ПО, в которых они были исправлены (если исправление было опубликовано). Результаты будут более точными, если предоставить сканеру информацию в формате VEX, которую он сможет использовать для фильтрации уязвимостей, не влияющих на продукт.

Сканирование на уязвимости

В Интернете можно найти огромное количество утилит для сканирования на уязвимости, охватывающих разнообразные методики, включая такие средства сканирования портов, как `nmap` и `nessus`, предназначенные для внешнего поиска слабых мест в системе, исследуя ее извне. Это ценный подход, но в данной

главе нас больше интересуют инструменты, с помощью которых можно найти бреши в ПО, установленном в корневой файловой системе. Аналогичный подход используется в утилитах, генерирующих файлы SBOM для образов контейнеров. Фактически получение перечня компонентов, входящих в состав образа, является необходимым шагом при сканировании уязвимостей, поэтому многие инструменты сканирования, такие как `trivy` (<https://trivy.dev>), были доработаны и теперь могут генерировать файлы SBOM.

Представьте, что у вас есть образ контейнера, в котором (пока) нет SBOM. Чтобы идентифицировать уязвимости, присутствующие в образе, сначала нужно выяснить, какое программное обеспечение в нем имеется. Существует несколько механизмов установки ПО:

- корневая файловая система, основанная на дистрибутиве Linux, который уже содержит уязвимости;
- системные пакеты, установленные через менеджер пакетов Linux (например, `rpm` или `apk`), а также пакеты, ориентированные на конкретный язык программирования и установленные с помощью таких утилит, как `pip` и `RubyGems`;
- программное обеспечение, установленное непосредственно с помощью `wget`, `curl` или даже FTP.

Некоторые сканеры уязвимостей запрашивают у системы управления пакетами список установленного программного обеспечения. В случае использования одной из подобных утилит следует избегать установки ПО напрямую, поскольку оно не будет просканировано на предмет уязвимостей.

Установленные пакеты

Как вы видели в главе 6, любой образ контейнера может включать дистрибутив Linux, возможно, с какими-либо уже установленными пакетами, помимо кода приложения. Может быть запущено много экземпляров контейнеров, причем каждый — со своей копией файловой системы образа контейнера, включая все потенциально уязвимые пакеты, содержащиеся в нем. На рис. 8.1 проиллюстрирована подобная ситуация: два экземпляра контейнера X и один экземпляр контейнера Y. Кроме того, показаны дополнительные пакеты, установленные непосредственно на хосте.

Установка пакетов прямо на хосте — обычное дело. На самом деле именно их исправления системным администраторам приходится устанавливать из сообщений безопасности. Для этого они нередко подключаются к каждому

хосту по SSH и устанавливают исправленный пакет. Однако в эпоху развития облачных технологий это не приветствуется, поскольку ручное изменение состояния машины подобным образом означает невозможность автоматического восстановления ее в то же состояние. Вместо этого лучше создать новый образ машины со свежими пакетами или обновить сценарии автоматизации, служащие для подготовки образов, чтобы новые установки включали актуальные пакеты.

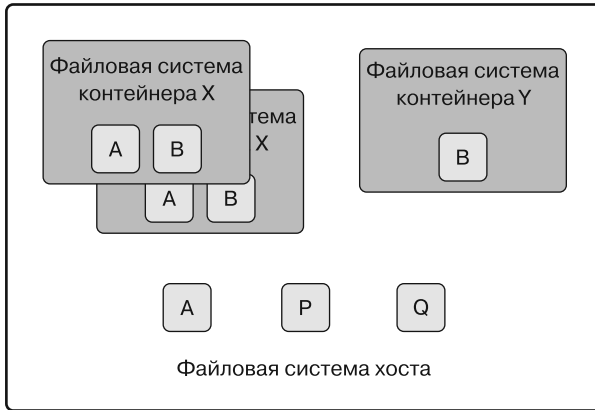


Рис. 8.1. Пакеты на хосте и в контейнерах

Сканирование образов контейнеров

Чтобы понять, не запущены ли в развернутой системе контейнеры с уязвимым ПО, необходимо просканировать в них все зависимости. Для этого существует несколько различных подходов.

Представьте утилиту, сканирующую все запущенные на хосте (или в системе, развернутой на нескольких хостах) контейнеры. В современных системах, развертываемых на облачных платформах, нередко можно увидеть сотни экземпляров контейнеров, созданных на основе одного образа контейнера, и просматривать одни и те же зависимости сотни раз было бы крайне неэффективно. Гораздо рациональнее проверять образ, на котором основаны все эти контейнеры.

Однако такой подход требует, чтобы в контейнерах присутствовало только программное обеспечение из образа контейнера и ничего более. Работающий в каждом из контейнеров код должен быть *неизменяемым* (immutable). Посмотрим, каковы преимущества таких неизменяемых контейнеров.

Неизменяемые контейнеры

Обычно ничто не мешает контейнеру после запуска скачивать дополнительное программное обеспечение и размещать его в своей файловой системе. И действительно, на заре развития контейнеров подобное поведение встречалось довольно часто, поскольку считалось удобным способом обновления ПО контейнера до последних версий без повторной сборки образа контейнера. Если до сих пор эта идея не приходила вам в голову, то забудьте ее сразу же, поскольку она считается исключительно неудачной по многим причинам, включая представленные ниже.

- Если контейнер загружает код в среде выполнения, разные экземпляры контейнера могут выполнять разные версии этого кода, причем отследить, в каком экземпляре работает та или иная версия, будет непросто. Без сохраненной версии кода из данного контейнера может быть сложно (или даже невозможно) воссоздать его точную копию, что создает проблемы при попытке воспроизвести ошибки, возникшие при эксплуатации.
- При изменении содержимого контейнера файл SBOM перестает отражать его реальное состояние. Любые результаты, основанные на файле SBOM и предназначенные для информирования о проблемах с лицензированием или уязвимостями в контейнерах, будут неточными.
- Становится труднее контролировать и гарантировать происхождение ПО, работающего в каждом из контейнеров, если его можно загрузить в произвольный момент времени и откуда угодно. Любые аттестаты происхождения ПО в образе контейнера практически теряют смысл, поскольку они больше не соответствуют фактически выполняемому коду.
- Сборку образа контейнера и его сохранение в реестре легко автоматизировать в конвейере CI/CD. Несложно добавить в тот же конвейер дополнительные проверки безопасности, например сканирование на уязвимости или проверку цепочки поставок ПО.

Многие рабочие системы рассматривают контейнеры как неизменяемые, просто в порядке практической рекомендации, но никакого контроля не осуществляется. Есть утилиты, способные автоматически обеспечить неизменяемость контейнеров с помощью запрета на запуск в контейнере файлов, отсутствовавших там на момент сканирования. Данная методика, о которой мы подробнее поговорим в главе 15, называется *предотвращением расхождений* (drift prevention).

Еще один способ реализовать неизменяемость — запустить контейнер с файловой системой, предназначенной только для чтения. Если коду приложения требуется сохранять данные в локальном хранилище, то можно смонтировать открытую для записи временную файловую систему. Здесь может понадобиться вносить изменения в приложение, чтобы оно записывало данные лишь в эту временную файловую систему.

Если контейнеры неизменяемые, то достаточно просканировать только образ, чтобы найти все уязвимости, которые могут встретиться во всех контейнерах, основанных на нем. Но, к сожалению, просканировать их однократно недостаточно. Обсудим, почему необходимо выполнять регулярные проверки.

Регулярное сканирование

Как уже обсуждалось в начале главы, по всему миру есть множество исследователей, занимающихся безопасностью, которые ищут неизвестные ранее уязвимости в программном коде. Иногда эти люди находят проблемы, существовавшие уже многие годы. Один из самых ярких примеров — HeartBleed — критическая уязвимость в широко используемом пакете OpenSSL, связанная с ошибкой в механизме запросов и ответов heartbeat, который поддерживает TLS-соединение в активном состоянии. Эта уязвимость, обнаруженная в апреле 2014 года, позволяла злоумышленнику отправлять heartbeat-запрос, составленный специальным образом, на небольшое количество данных в крупном буфере. Отсутствие проверки длины в коде OpenSSL приводило к тому, что в ответ возвращались те немногие запрошенные данные, а остальная часть буфера заполнялась содержимым используемой в текущий момент памяти, которое могло включать конфиденциальные данные, попадавшие таким образом к злоумышленнику. Было установлено, что эта уязвимость стала причиной серьезных утечек данных, связанных с потерей паролей, номеров социального страхования и медицинских данных.

Столь серьезные случаи, как HeartBleed, редки, но лучше всегда предполагать, что в любой сторонней зависимости рано или поздно будет выявлена новая брешь в безопасности. И, к сожалению, неизвестно, когда это произойдет. Даже если сам код не меняется, в его зависимостях могут обнаружиться новые уязвимости.

При регулярном сканировании образов контейнеров утилита проверки может сравнивать их содержимое с актуальной базой знаний уязвимостей (из NVD и прочих источников инструкций по безопасности). Очень часто все развернутые образы сканируют каждые 24 часа, а также стоит проверять новые образы при сборке в рамках автоматизированного конвейера CI/CD.

Средства сканирования

Существует множество утилит для сканирования образов контейнеров — от реализаций с открытым исходным кодом вроде Trivy (<https://github.com/aquasecurity/trivy>) и Grype (<https://github.com/anchore/grype>) до коммерческих решений таких компаний, как JFrog, Palo Alto и Aqua. Во многих реестрах образов контейнеров, например Docker Hub с Docker Scout (<https://docs.docker.com/scout>) и в проекте Harbor (<https://goharbor.io/>) от CNCF, а также в реестрах

от основных поставщиков облачных сервисов есть встроенные возможности сканирования.

Большинство этих инструментов (если не все) способны принимать спецификации в форматах SBOM и VEX для быстрого сканирования и выдавать более релевантные результаты, учитывающие не только присутствие уязвимости как таковой, но и возможность ее эксплуатации.

В среде Kubernetes оператор Trivy (<https://github.com/aquasecurity/trivy-operator>) может автоматически сканировать развертывания на наличие различных проблем, включая ошибки в зависимостях, генерируя метрики Prometheus, чтобы дать возможность выявлять проблемы с применением панелей мониторинга.



На сайте Liquid Reply опубликовано пошаговое руководство по установке Trivy и его оператора (<https://oreil.ly/KrGaA>), а также визуализации метрик Prometheus на панелях мониторинга Grafana.

К сожалению, выдаваемые различными сканерами результаты сильно разнятся, и имеет смысл обсудить почему.

Источники информации

Как уже говорилось в этой главе, существует несколько источников информации об уязвимостях, включая инструкции по безопасности различных дистрибутивов. У Red Hat их даже несколько: их лента новостей OVAL (<https://redhat.com/en/blog/evolving-oval>) включает только уязвимости, для которых уже есть исправления, тогда как Red Hat Security Data API (https://docs.redhat.com/en/documentation/red_hat_security_data_api/1.0) содержит уже опубликованные CVE, но пока не исправленные.

Если сканер не использует данные из ленты новостей по безопасности дистрибутива и основывается лишь на данных NVD, то будет, вероятно, выдавать множество ложноположительных результатов для образов, основанных на этом дистрибутиве. Если вы предпочитаете конкретный дистрибутив Linux в своих образах или программные решения наподобие distroless, то убедитесь, что сканер их поддерживает.

Устаревшие источники

Иногда персонал, отвечающий за сопровождение дистрибутивов, меняет способ оповещения об уязвимостях. Так случилось с Alpine: вместо информации об уязвимостях в `alpine-secdb` (<https://github.com/alpinelinux/alpine-secdb>) теперь применяется новая система в `aports` (<https://gitlab.alpinelinux.org/alpine/aports>). Все известные мне и активно развивающиеся проекты сканирования обновились и начали использовать метаданные `aports` (или собственные методы сбора

данных), но этот пример наглядно показал, что подобные структурные изменения в будущем могут потребовать значительных обновлений инструментов сканирования.

Не все уязвимости исправляются

Иногда персонал, отвечающий за сопровождение дистрибутивов, решает не исправлять конкретную уязвимость (например, потому, что риск пренебрежимо мал, а устранить проблему непросто, или потому, что способ взаимодействия с прочими пакетами на их платформе делает невозможной эксплуатацию уязвимости).

А раз уязвимость исправлять никто не собирается, то у разработчиков сканеров возникает философский вопрос: отображать ли ее в списке результатов, если ничего все равно сделать нельзя? Когда я работала в компании Aqua Security, мы не раз слышали от некоторых клиентов, что они не хотели бы видеть данную категорию результатов, поэтому мы предоставили пользователю возможность выбора. Все это свидетельствует о том, что не существует «правильных» наборов результатов, когда речь идет о сканировании на уязвимости.

Передача VEX в сканеры

Как я уже говорила, информация VEX может давать дополнительные сведения о том, действительно ли уязвимости могут быть использованы в конкретном продукте. Если сканер учитывает данные VEX, то он будет реже выдавать ложную тревогу.

Уязвимости подпакетов

Иногда система управления пакетами устанавливает пакет и сообщает о нем как о едином целом, однако на самом деле он состоит из нескольких подпакетов. Хороший пример: пакет `bind` в Ubuntu. Иногда он устанавливается с одним только подпакетом `docs`, включающим, как легко догадаться, лишь документацию. Некоторые сканеры предполагают, что если менеджер пакетов сообщает об установленном пакете, значит, установлен весь пакет (со всеми возможными подпакетами). Это может привести к ложноположительным результатам, когда сканер сообщает об уязвимостях, которых нет и быть не может, поскольку соответствующий подпакет не установлен.

Различия названий пакетов

Имя источника для пакета иногда включает двоичные файлы с совершенно другими именами. Например, в Debian пакет `shadow` (<https://tracker.debian.org/pkg/shadow>) содержит двоичные файлы `login`, `passwd` и `uidmap`. Если сканер это не учитывает, то может выдавать ложноотрицательные результаты.

Статически скомпонованные исполняемые файлы

В компилируемых языках программирования, таких как Go, C или Rust, часто применяются дополнительные библиотеки. Эти библиотеки могут загружаться динамически в среде выполнения или компоноваться в один исполняемый файл с кодом приложения. Статически скомпонованный автономный исполняемый файл допускается встраивать в образ контейнера, основанный на пустом базовом образе. Это означает, что в файловой системе такого образа не будет ничего, кроме исполняемого файла. В главе 7 я рекомендовала этот подход как способ минимизации поверхности атаки.

Однако этот выбор влияет на действия, которые должен предпринять сканер, чтобы корректно определить используемые библиотеки и их версии. Динамические библиотеки обычно устанавливаются с помощью менеджера пакетов, и в таких случаях сканер, как правило, имеет доступ к информации о версиях из метаданных пакетов, которые добавляются в образ контейнера в процессе его сборки. Однако никакие из этих сведений о версиях не будут включены в образ, если процедура сборки просто копирует в него один автономный исполняемый файл.

Но это не означает, что все потеряно! Исполняемый файл обычно содержит множество подсказок о модулях или библиотеках, включенных в него, например, в таблице символов ELF и в заголовках разделов. Существуют сканеры — например, `syft`, — способные анализировать эту информацию и на ее основе создавать SBOM со списком модулей или библиотек, встроенных в исполняемый файл.

Точность может быть еще выше, если компилятор поддерживает возможность создания SBOM по мере объединения компонентов. Например, в последних версиях Go информация о модулях встраивается в специальный раздел `.note.go.buildinfo` скомпилированного исполняемого файла. Инструменты сканирования могут просто сравнить информацию в нем со своим списком известных уязвимостей.

Сканирование мультиплатформенных образов

Как вы знаете из главы 6, образы контейнеров могут включать версии сразу для нескольких разных аппаратных архитектур, которые перечисляются в манифесте образа. Поскольку для каждой архитектуры необходимо свое программное обеспечение, уязвимости, существующие, например, в варианте для ARM, могут отличаться от уязвимостей в варианте для Intel.

Обычно сканеры извлекают и исследуют только версию образа для той архитектуры, на которой он запущен, однако некоторые инструменты позволяют задать параметр, чтобы получить результаты сканирования для другой платформы. Так, просканировать версию для ARM можно с помощью команды `trivy image --platform linux/arm64 <образ>`.

Дополнительные возможности сканирования

Многие сканеры образов способны выявлять и другие проблемы, помимо уязвимостей, например:

- известные вредоносные программы внутри образа — по сигнатурам вирусов или с применением эвристических методов обнаружения, таких как Yet Another Research Assistant (YARA) (<https://github.com/VirusTotal/yara>);
- исполняемые файлы с установленным битом `setuid` (который, как вы видели в главе 2, может позволить повысить привилегии);
- образы, конфигурация которых предусматривает выполнение от имени суперпользователя;
- секретные учетные данные, например токены или пароли;
- конфиденциальные данные вроде номеров платежных карт, номеров социального страхования и т. п.;
- нарушения политик (скажем, устаревшие зависимости, не соответствующие требованиям лицензии);
- неправильная конфигурация образа, например отсутствие проверки работоспособности;
- сравнение компонентов, включенных в образ, со списком в спецификации SBOM (это может помочь выявить несанкционированное добавление вредоносного компонента).

Ошибки сканеров

Надеюсь, материал данного раздела позволяет понять, что поиск уязвимостей не такая простая задача, как может показаться на первый взгляд. Поэтому весьма вероятно, что при использовании любого сканера будут встречаться ложноположительные или ложноотрицательные результаты из-за ошибок в сканере или изъянов в информации об уязвимостях, которую он задействует.

Тем не менее наличие сканера все же лучше, чем его отсутствие. Если у вас нет сканера или вы не используете его регулярно, то никак не узнаете, не станет ли ваше ПО легкой добычей злоумышленников. Время в этом смысле плохой лекарь: критическая уязвимость Shellshock была обнаружена в коде, существовавшем десятилетия. Если вы опираетесь на сложные наборы зависимостей, то следует исходить из того, что рано или поздно в них будут найдены уязвимости.

Ложноположительные результаты раздражают, но некоторые утилиты позволяют заносить отдельные отчеты об уязвимостях в «белый список», чтобы вы могли сами решить, следует ли на них реагировать.

Надеюсь, я убедила вас в преимуществах использования сканера. Далее рассмотрим возможные варианты его интеграции в ваш технологический процесс.

Сканирование в конвейере CI/CD

Пройдем по конвейеру CI/CD слева направо, где этап «разработка кода» — крайний слева, а этап «развертывание в рабочей среде» — крайний справа, как показано на рис. 8.2. В данном конвейере желательно устранять проблемы как можно раньше, потому что это проще и дешевле — точно так же, как поиск и исправление программных ошибок требует намного больше времени и затрат после развертывания, чем во время разработки.

В традиционной модели развертывания на хосте все работающее на нем ПО использует одни и те же пакеты. За их регулярное обновление путем установки патчей в организации обычно отвечает группа специалистов по компьютерной безопасности. Данная деятельность более или менее не зависит от этапов разработки и тестирования жизненного цикла приложения и располагается ближе к правому краю конвейера развертывания. Нередко возникают проблемы, связанные с совместным использованием одного и того же пакета различными приложениями, которым нужны разные его версии. Такие проблемы требуют продуманного управления зависимостями и в некоторых случаях изменений кода.

И напротив, как вы видели в главе 6, при развертывании на основе контейнеров каждый образ содержит собственные зависимости, поэтому контейнеры для различных приложений могут включать различные версии пакетов, если это необходимо. Не нужно волноваться о совместимости кода приложения и набора используемых зависимостей. Это с учетом существования утилит сканирования образов контейнеров позволяет «сдвинуть» управление уязвимостями влево по конвейеру.

Сканирование на уязвимости можно автоматизировать. Устранить найденную брешь разработчики могут с помощью обновления и повторной сборки образа контейнера приложения с включением в него исправленной версии. Группам специалистов по компьютерной безопасности больше не нужно делать это вручную.

Есть несколько этапов, на которых можно внедрить сканирование (см. рис. 8.2).

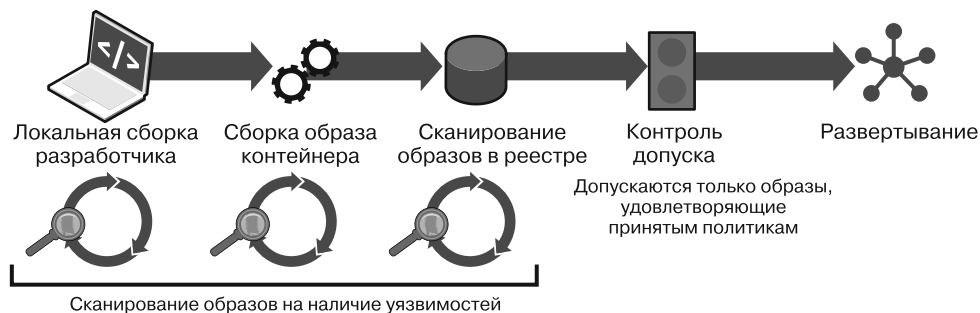


Рис. 8.2. Сканирование на уязвимости в конвейере CI/CD

Сканирование на стороне разработчика

Если используемый сканер можно развернуть на настольном компьютере, то отдельные разработчики способны сканировать локальные сборки образов на предмет проблем, благодаря чему получают шанс внести исправления до отправки исходного кода в репозиторий.

Сканирование при сборке

Рассмотрите возможность включения шага сканирования сразу же после сборки образа контейнера в вашем конвейере. В случае выявления уязвимостей, превышающих заданный порог серьезности, можно сообщить о «неуспехе» сборки, гарантируя тем самым, что она точно не будет развернута в рабочей среде. На рис. 8.3 приведена информация, выводимая проектом AWS CodeBuild при сборке образа на основе Dockerfile и дальнейшем его сканировании. В данном примере была обнаружена очень серьезная уязвимость, поэтому сборка остановилась.

Сканирование реестра

После того как образ будет собран, конвейер обычно помещает его в реестр образов (в идеале с прикрепленными и подписанными SBOM и аттестатом сборки). Имеет смысл регулярно сканировать образы на случай обнаружения новой уязвимости в одном из пакетов, используемых в образе, который давно не собирался повторно. Наличие полной спецификации может ускорить сканирование, поскольку нет необходимости заново проверять сам (неизменившийся) образ — достаточно сравнить содержимое образа из SBOM с базами данных уязвимостей.

The screenshot shows the AWS CodeBuild console for a build named '4d90542f0623'. The build status is 'FAILED'. The console output shows the build process, including the phase 'POST_BUILD' where the command 'trivy --no-progress --exit-code 1 --severity HIGH,CRITICAL --auto-refresh SREPOSITORY_URI:success' was executed. The output of this command shows a table of vulnerabilities found in the image.

LIBRARY	VULNERABILITY ID	SEVERITY	INSTALLED VERSION	FIXED VERSION	TITLE
musl	CVE-2019-14697	HIGH	1.1.22-r2	1.1.22-r3	musl libc through 1.1.23 has an x87 floating-point stack adjustment imbalance, related...

The build failed because of this critical vulnerability. The console output also shows the reason for the failure: 'Error while executing command: trivy --no-progress --exit-code 1 --severity HIGH,CRITICAL --auto-refresh SREPOSITORY_URI:success. Reason: exit status 1'.

Рис. 8.3. Пример остановки сборки из-за обнаружения очень серьезной уязвимости



В следующих статьях можно найти полезную информацию о включении разнообразных сканеров в различные конвейеры CI/CD:

- *Scanning images with Trivy in an AWS CodePipeline* (<https://oreil.ly/6ANm9>);
- *Container Scanning on GitLab* (<https://oreil.ly/okLcm>).

Если в вашей системе используются машины с разными аппаратными архитектурами, то вам также может пригодиться пример конвейера сборки и сканирования для нескольких архитектур, представленный Эндрю Блуманом (Andrew Blooman) на GitLab (<https://oreil.ly/6Wuyy>).

Вероятно, не стоит откладывать этап сканирования до момента развертывания по той простой причине, что вы станете сканировать каждый создаваемый экземпляр контейнера, хотя источником их всех является один и тот же образ. Если рассматривать контейнер как неизменяемый, то имеет смысл сканировать образ, а не контейнер.

Предотвращение запуска образов с уязвимостями

Проверять наличие серьезных уязвимостей с помощью сканера — это одно, но надо еще и гарантировать, что образы с уязвимостями не попадут в рабочую среду. Как было показано на рис. 8.2, это можно сделать на этапе контроля допуска, который мы рассматривали в главе 7. Если нет проверки, гарантирующей, что к развертыванию допускаются только просканированные образы, обойти сканер уязвимостей будет довольно легко.

Как правило, коммерческие сканеры уязвимостей продаются в составе более обширной платформы, связывающей контроль допуска с результатами сканирования. В системах, развертываемых на основе Kubernetes, можно использовать Kyverno или Open Policy Agent Gatekeeper для реализации своих проверок, которые могут включать проверку прохождения образами сканирования на уязвимости, и для установки пороговых значений оценок уязвимости (например, блокировать образы с известными CVE высокого или критического уровня). Как было показано в главе 7, также можно добавить проверки политик и требовать, чтобы образы были подписаны доверенными поставщиками.

Теперь представьте, что этап сканирования выявил уязвимые образы контейнеров. Что делать дальше?

Обновление образов

Определив, что некоторые ваши образы уязвимы и доступны исправленные версии зависимостей, вы должны пересобрать эти образы, используя обновленные пакеты, и повторно развернуть их. Насколько быстро это нужно сделать?

Многое зависит от конкретной ситуации, но обычно рекомендуется выделить несколько недель на развертывание обновленного или исправленного ПО. Например, в стратегии непрерывного мониторинга FedRAMP (<https://oreil.ly/EX1Ib>) говорится, что «высокий уровень риска для любых зависимостей должен быть понижен до умеренного за счет внедрения компенсирующих мер контроля в течение 30 дней», а в директиве CISA Binding Directive (<https://oreil.ly/o6TV9>) от 2019 года указано, что «критические уязвимости должны устраняться в течение 15 календарных дней с момента первоначального обнаружения».

Однако действительно ли стоит оставлять без внимания свои системы с критическими уязвимостями на две недели или более, особенно если учесть, что, по данным команды Mandiant¹, среднее время от обнаружения до эксплуатации уязвимости в 2023 году составляло всего пять дней? Только представьте, сколько атак можно предпринять за традиционный 30- или 45-дневный «цикл обновления».

С момента обнаружения новой исправленной уязвимости следует выполнить несколько шагов.

1. Просканировать образ или SBOM, чтобы выявить необходимость в обновлении.
2. Обновить код, lock-файлы зависимостей или скрипты до исправленной версии.
3. Пересобрать и протестировать образы контейнеров.
4. При необходимости развернуть обновленные версии в рабочей среде. Это может быть «поэтапное обновление», когда старые экземпляры контейнеров постепенно выводятся из эксплуатации и замещаются новыми, чтобы не вызвать сбоев в работе пользователей.

В этом случае особую важность приобретает автоматизация. Например, GitHub Dependabot (<https://docs.github.com/en/code-security/getting-started/dependabot-quick-start-guide>) может предлагать изменения в вашем коде через запрос на слияние (pull request) для обновления уязвимых пакетов в ваших репозиториях. Это еще одна быстро развивающаяся область, где ИИ способен оказать некоторую помощь (хотя я подозреваю, что ИИ может использоваться не только командами безопасности, но и злоумышленниками).

До выполнения этих шагов развертывание остается уязвимым, и совершенно очевидно, что чем быстрее вы развернете обновленный код, тем ниже вероятность серьезного взлома. К сожалению, сократить это время до нуля

¹ Команда Mandiant из компании Google публикует очень хороший, хотя и несколько пугающий ежегодный отчет о состоянии угроз (<https://cloud.google.com/blog/topics/threat-intelligence/time-to-exploit-trends-2023?hl=en>).

невозможно, даже если проводить тестирование не слишком тщательно! В ожидании готовности обновленных образов можно предпринять дополнительные меры и использовать eBPF для смягчения уязвимостей в существующих контейнерах. Их мы обсудим в главе 15.

До сих пор в этой главе мы обсуждали лишь известные уязвимости в зависимостях, необходимых для кода приложения, но обошли вниманием такую важную категорию, как *уязвимости нулевого дня*.

Уязвимости нулевого дня

В разделе «Сканирование на уязвимости» говорилось, как исследователи в сфере безопасности по всему миру ищут новые способы взлома имеющегося ПО. Разумеется, при обнаружении новой бреши исправление публикуется не сразу, а спустя какое-то время. До обнаружения исправления такая уязвимость называется *уязвимостью нулевого дня* (zero-day vulnerability), поскольку с момента публикации исправления еще не прошло ни одного дня. В отчете Mandiant говорится, что 70 % эксплуатируемых уязвимостей в 2023 году были уязвимостями нулевого дня.

И если в сторонних библиотеках содержатся программные ошибки, которыми может воспользоваться злоумышленник, то же самое справедливо для любого кода — включая создаваемые вами приложения. Обнаружить проблемы в коде помогают экспертная оценка, статический анализ и тестирование, но всегда есть вероятность, что некоторые проблемы останутся незамеченными. В зависимости от конкретной организации и ценности данных всегда могут найтись злоумышленники, которые сочтут, что поиск этих изъянов будет оправдан.

Хорошая новость: если информация об уязвимости не была опубликована, то абсолютное большинство потенциальных злоумышленников знает о ней не больше, чем вы.

Плохая новость: у опытных злоумышленников и правительственных организаций есть библиотеки пока еще не опубликованных уязвимостей, как мы знаем из разоблачений Эдварда Сноудена (<https://oreil.ly/Yz1tJ>). Как обсуждалось в начале главы, инструменты на основе ИИ помогают выявлять новые бреши в безопасности, и я готова поспорить, что о некоторых из них не сообщается и они остаются недоступными для использования злоумышленниками.

Никакой поиск в базе уязвимостей не поможет выявить слабые места, информация о которых еще не опубликована. В зависимости от типа и степени серьезности уязвимости защитить приложение и данные можно с помощью выполнения кода в песочнице, как описывается в главе 10. Лучший способ защититься от уязвимостей нулевого дня — обнаружить и предотвратить аномальное поведение в среде выполнения, о чем я расскажу в главе 15.

Резюме

В текущей главе вы прочитали об исследованиях уязвимостей и идентификаторах CVE, присваиваемых различным уязвимостям. Вы узнали, почему так важна информация о безопасности, относящаяся к конкретному дистрибутиву, и почему не стоит полагаться на одну только базу NVD. Кроме того, я рассказала вам, почему различные сканеры могут выдавать разные результаты, поэтому теперь вы лучше подготовлены к выбору используемых утилит. Вне зависимости от того, на какой сканер падет ваш взор, надеюсь, вы понимаете, что сканирование образов контейнеров должно быть включено в ваш конвейер CI/CD.

Некоторые инструменты сканирования, рассмотренные в этой главе, также способны проверять файлы, описывающие и настраивающие инфраструктуру, на которой работают ваши приложения. В главе 9, посвященной инфраструктуре как коду и GitOps, мы рассмотрим последствия автоматизации предоставления и настройки инфраструктуры с точки зрения безопасности.

Инфраструктура как код и GitOps

Программное обеспечение работает на компьютерах, и, когда в дело вовлечено несколько машин, мы начинаем использовать термин «инфраструктура», чтобы описать не только сами компьютеры, но и сеть, которая их соединяет, а также такие устройства, как хранилища данных, доступные этим компьютерам. То, как устроена эта инфраструктура, напрямую влияет на ее безопасность.

Книга посвящена безопасности контейнеров, поэтому я не буду вдаваться в подробности сетевой и компьютерной безопасности. Но в облачных средах, где обычно работают контейнеры, инструменты и процессы, используемые для настройки инфраструктуры, могут оказывать существенное влияние на безопасность этих контейнеров. В этой главе я сначала кратко объясню понятие «инфраструктура как код» (infrastructure as code, IaC), а затем опишу, как этот подход заложил основы GitOps, который, в свою очередь, обладает рядом очень важных характеристик безопасности.

IaC

Чтобы запустить ПО в облаке, необходимо подготовить инфраструктуру для его работы: создать виртуальные машины или настроить физические серверы. Вам придется выбрать операционную систему (или системы) для развертывания на этих машинах и соединить их, возможно, с помощью VPC, чтобы изолировать их от других машин и пользователей. Вам также может потребоваться настроить доступ к облачным сервисам, таким как базы данных или системы обмена сообщениями, к которым будет обращаться ваше ПО. В общедоступной облачной среде инфраструктура может включать управляемые сервисы Kubernetes или системы оркестрации контейнеров, такие как AWS Elastic Container Service.

Идея *инфраструктуры как кода* (infrastructure as code, IaC) заключается в том, что вместо ручного выполнения команд для развертывания инфраструктуры

используется набор файлов, описывающих ее компоненты: виртуальные машины, сеть и управляемые облачные сервисы, на которых будут работать ваши приложения. Эти файлы используются в качестве входных данных для таких инструментов, как Terraform/OpenTofu, CloudFormation или Pulumi, которые развертывают инфраструктуру в соответствии с определениями в этих файлах. Автоматизация развертывания вместо ручного ввода команд делает процесс более надежным и воспроизводимым.

Вы, вероятно, уже поняли, что файлы, описывающие конфигурацию инфраструктуры, можно — и нужно — хранить в системе управления версиями, такой как Git, которая предоставляет все обычные преимущества: историю изменений (для простой отмены неудачных модификаций), возможность совместной работы нескольких человек и проверки изменений, а также исторические записи развернутых компонентов, которые можно использовать для аудита или устранения неполадок.

Помимо автоматического масштабирования количества виртуальных машин, инфраструктура меняется редко. Для нового развертывания инфраструктуры (возможно, в новом географическом регионе или в качестве тестовой среды) пользователь обычно запускает команду или нажимает кнопку в административной консоли, чтобы инициировать автоматическое выделение ресурсов. Но облачное ПО имеет еще одно преимущество: возможность частого обновления. GitOps основывается на концепции IaC («инфраструктура как код»), применяя те же идеи к конфигурации рабочих нагрузок и обеспечивая автоматические динамические обновления.

GitOps

В 2017 году Алексис Ричардсон (Alexis Richardson) ввел термин GitOps, описывающий методологию, в которой вся конфигурационная информация о состоянии системы хранится в системе управления версиями, как и исходный код приложения. Когда пользователю требуется внести изменения в действующую систему, он не выполняет команды напрямую, а описывает желаемое состояние в коде (например, в YAML-файлах для Kubernetes). При этом автоматизированная система (так называемый GitOps-контроллер) обеспечивает обновление системы в соответствии с последним состоянием, определенным в репозитории. Примерами реализации такого подхода могут служить ArgoCD (<https://argoproj.github.io/cd>) и Flux (<https://fluxcd.io/flux>). Оба решения являются проектами с открытым исходным кодом и уже достаточно зрелые, чтобы получить статус проекта высокого качества в CNCF.

GitOps положительно влияет на безопасность. Пользователям больше не нужен прямой доступ к работающей системе, поскольку все действия производятся

опосредованно через систему управления версиями (обычно Git, как понятно из названия). Как показано на рис. 9.1, учетные данные пользователя дают доступ к системе управления версиями, но только у автоматизированного GitOps-контроллера есть полномочия для модификации работающей системы. А поскольку Git фиксирует все изменения, то каждая операция сопровождается записью в журнале аудита.

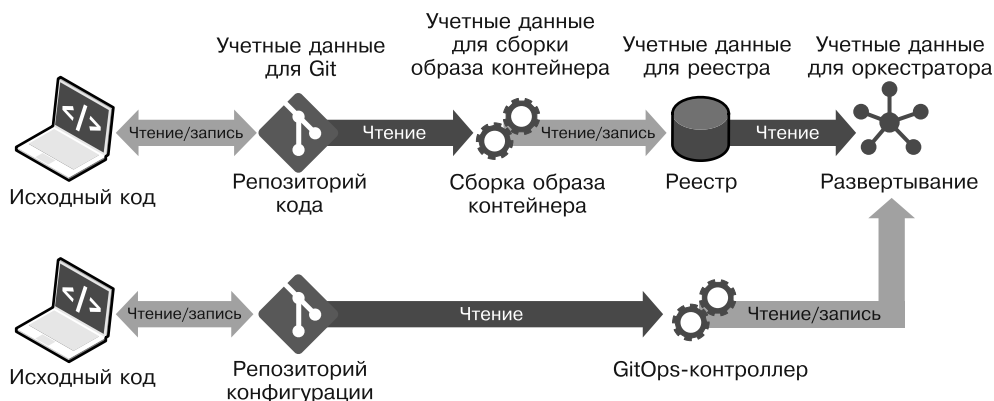


Рис. 9.1. GitOps

Методология GitOps продвигает следующие принципы.

Декларативность

Описание состояния системы в конфигурационных файлах имеет декларативный характер, то есть определяет желаемое состояние системы. Этот подход отличается от императивного кода, представляющего собой последовательность команд, которые должны быть выполнены для достижения нужного результата. Главное преимущество декларативного подхода — для него не имеет значения, в каком состоянии находилась система первоначально: инструменты сами определяют детали перехода из начального состояния в требуемое.

Управление версиями и неизменяемость

Если файлы хранятся в Git, существует история версий, которую можно использовать как журнал аудита, чтобы увидеть, какие изменения были внесены, кем и когда (а комментарии, сопровождающие коммиты, могут даже подсказать причину). Файл, хранящийся в репозитории Git, используется работающей системой без изменений, а системные изменения разрешены, только если они инициированы файлами из репозитория Git. Даже доверенным пользователям не разрешается изменять конфигурацию на лету: им придется предварительно сохранить свои изменения в репозитории Git.

Автоматическое извлечение

Инструменты GitOps автоматически извлекают необходимую информацию о состоянии репозитория Git без ручного ввода или управления.

Непрерывная синхронизация

Файлы, хранящиеся в Git, описывают желаемое состояние инфраструктуры и набор рабочих нагрузок, которые должны выполняться. Инструменты GitOps постоянно проверяют, соответствует ли фактическое состояние системы требуемому, и вносят правки для приведения работающей системы в соответствие с желаемым состоянием. Если состояние, хранящееся в Git, изменяется (например, в результате модификаций файла и его отправки в репозиторий), то, следуя принципу непрерывной синхронизации, инструменты GitOps в конечном счете приведут работающую систему в соответствие с этим преобразованием.



Дополнительную информацию о методологии GitOps и ее принципах вы найдете на сайте OpenGitOps (<https://opengitops.dev>).

Как правило, GitOps тесно связана с Kubernetes, поскольку Kubernetes имеет декларативные API и операторы, обеспечивающие непрерывную синхронизацию. Такие инструменты, как Crossplane и Cluster API, расширяют эту методологию, позволяя использовать операторы Kubernetes и для управления инфраструктурой.

Следствия для безопасности развертывания

Обе методологии — IaC и GitOps — обладают преимуществами с точки зрения воспроизводимости и автоматизации, но давайте рассмотрим их влияние на безопасность.

История версий

Как уже упоминалось, система управления версиями, подобно журналу аудита, позволяет узнать, кто и когда изменял файлы.

Управление изменениями

Только пользователи с соответствующими правами доступа к Git способны отправлять изменения в конфигурацию. Системы Git могут устанавливать дополнительные правила, например требовать подтверждения более чем одним человеком перед выполнением операции слияния изменения. Рекомендую использовать эту функцию, чтобы все изменения, в исходном коде приложения или в конфигурации инфраструктуры, проверялись в обязательном порядке.

Предотвращение расхождений (drift prevention)

Непрерывная синхронизация постоянно приводит работающую систему в соответствие с желаемой конфигурацией, поэтому системе трудно «отклониться» в сторону. Даже если кто-то сумеет вручную внести изменения (возможно, путем взлома системы), контроллер GitOps вскоре вернет систему в требуемое состояние.

Сканирование конфигурации

Существует несколько инструментов для автоматического сканирования и тестирования файлов IaC и/или GitOps на предмет безопасности. К ним относятся: checkov (<https://www.checkov.io>), conftest (<https://www.conftest.dev>), kubescape (<https://kubescape.io>) и trivy (<https://trivy.dev>). Подобно тому как можно настроить конвейер CI/CD, чтобы он не развертывал образы, не прошедшие сканирование, изменения в конфигурации также могут быть отклонены, если они не соответствуют правилам, используемым при сканировании.

Снижение требований к полномочиям

В модели GitOps людям не нужен прямой доступ на запись к кластерам Kubernetes, только инструменты должны иметь возможность записывать файлы ресурсов в кластер — именно в том виде, в каком они были получены из Git. Использование `kubectl` людьми можно ограничить операциями «только для чтения» с помощью механизма RBAC. Это уменьшит поверхность атаки и снизит вероятность случайного применения неправильной конфигурации. Отсутствие полномочий на запись также обеспечивает дополнительную поддержку модели неизменяемого желаемого состояния и уменьшает круг возможностей для отклонения от нормы.

Управление секретами

Как обсуждается в главе 14, секреты не следует хранить в системе управления версиями! Инструменты GitOps интегрируются с внешними средствами управления секретами, такими как Vault, Sealed Secrets или External Secrets Operator, и используют их для получения секретов по мере необходимости.

Откат

Если по какой-то причине что-то пойдет не так, то инструменты GitOps позволят легко откатиться к версии, которая ранее работала корректно.

Конфигурация как документация

Конфигурационные файлы, хранящиеся в Git, являются источником достоверной информации о том, как должна работать система. Благодаря этому отпадает необходимость в отдельной документации, которая легко может устареть. Все изменения связаны с коммитом, соответствующим сообщением и/или запросом на слияние, объясняющим внесенные изменения.

В модели GitOps больше нет необходимости напрямую настраивать то, что работает в кластере Kubernetes. Это повышает уровень безопасности самого кластера, но смещает акцент на инструменты GitOps и репозитории Git. В частности, поскольку конфигурация хранится в виде кода в файлах, эти файлы подвержены тем же видам атак *на цепочку поставок*, что и исходный код приложения, как сказано в главе 6.

- Наиболее очевидный риск связан с *внедрением вредоносных манифестов*, когда злоумышленник, получивший доступ к репозиторию Git, может контролировать развертываемые компоненты. Если атакующий имеет возможность изменять конфигурационные файлы, он сможет включить рабочие нагрузки по своему выбору, изменить разрешения RBAC или внедрить sidecar-контейнеры для эксфильтрации (кражи) данных. Если этому не воспрепятствуют другие уровни защиты, то контроллер GitOps добросовестно развернет любое состояние, описанное в репозитории, включая и вредоносные изменения.
- Атака, основанная на путанице зависимостей и описанная в главе 7, в контексте GitOps может применяться к дополнительным типам зависимостей. Так, конвейер GitOps способен разрешить ссылки на модули Terraform или чарты Helm во время развертывания, поэтому, если конвейер удастся обмануть, он загрузит вредоносные версии зависимостей из общедоступных реестров.

Далее рассмотрим рекомендации по обеспечению безопасности GitOps, которые помогут защититься от этих атак.

Рекомендации по обеспечению безопасности GitOps

Ниже перечислены практики, которые легко внедрить с помощью таких инструментов, как GitHub или GitLab.

- Убедитесь, что все коммиты подписаны, чтобы иметь возможность подтвердить автора изменений: это разумный шаг не только для репозитория, используемых для управления IaC или GitOps, но и для исходного кода любого типа.
- Кроме того, должны подписываться теги релизов, поскольку в случае проблем может потребоваться откатиться к заведомо работоспособной версии, и желательно, чтобы это было подтверждено доверенным и уполномоченным лицом.

- Используя правила защиты веток Git, можно потребовать одобрения запросов на слияние (pull requests) как минимум одним рецензентом, помимо автора изменений, особенно если речь идет о политиках безопасности или конфигурациях RBAC. Запросы на слияние также содержат запись об авторе изменения и причинах, а также информацию об одобрении.
- Следует запретить принудительный push (`git push --force`), потому что это может переписать историю и снизить надежность журналов аудита. Кроме того, если будут удаляться коммиты других разработчиков, теоретически это может запутать контроллер GitOps, такой как Flux или ArgoCD.
- Для входа пользователей в инструменты Git требуйте многофакторную аутентификацию. Еще лучше (учитывая, что SMS-коды и другие факторы могут использоваться для фишинга) ввести обязательную беспарольную аутентификацию Fast Identity Online (FIDO), основанную на криптографических учетных данных, привязанных к устройству пользователя.
- Избегайте использования долгосрочных персональных токенов доступа для рабочих процессов CI/CD, поскольку они уязвимы для утечки при повторном или неправомерном применении и не обновляются автоматически. Вместо этого задание CI/CD должно аутентифицироваться с помощью OIDC (OpenID Connect) (<https://docs.github.com/en/actions/concepts/security/openid-connect>), а для получения секретов, отправки образов или развертывания кода предпочтительнее задействовать кратковременные учетные данные.
- Перед слиянием необходимо подтвердить успешное завершение проверок, проводимых инструментами сканирования.
- Для доступа к репозиториям должен применяться принцип наименьших привилегий, чтобы только доверенные пользователи могли отправлять, объединять или просматривать изменения. GitHub и GitLab дают такую возможность. Обратите внимание, что контроллерам GitOps (таким как Flux или ArgoCD) необходимы разрешения на запись в кластер (-ы) Kubernetes, но для работы с репозиторием Git им достаточно полномочий только на чтение.
- Репозитории для манифестов IaC/GitOps должны быть отделены от репозитория с исходным кодом приложения. Это ограничит масштабы угрозы в случае взлома репозитория и упростит реализацию принципа наименьших привилегий.
- Помимо журналов аудита, GitHub и GitLab предлагают дополнительные платные функции безопасности, которые также стоит рассмотреть.
- В среде выполнения рекомендуется использовать операторы доступа и/или инструменты обеспечения безопасности для обнаружения и защиты от развертывания неожиданного ПО.

Резюме

В этой главе мы обсудили, почему информация об инфраструктуре и конфигурационные файлы должны храниться в системе управления версиями, а также рассмотрели преимущества и последствия для безопасности при использовании инструментов IaC и GitOps, с помощью которых эти конфигурации применяются к развертыванию. Надеюсь, информация по рекомендуемым практикам пригодится вам, когда вы будете настраивать репозитории Git для работы с инструментами GitOps.

Как и в главе 7, здесь рассматривались приемы, помогающие предотвратить развертывание небезопасных образов контейнеров и конфигураций, но, исходя из принципа многоуровневой защиты, не следует уповать, что эти меры нельзя будет обойти. В главе 10 мы рассмотрим усиление изоляции работающего контейнера, чтобы усложнить эксплуатацию уязвимостей злоумышленником.

Усиление изоляции контейнеров

В главах 3 и 4 вы видели, как контейнеры обеспечивают разделение рабочих заданий, даже если они выполняются на одном хосте. В этой главе я расскажу о некоторых более совершенных утилитах и методиках усиления изоляции выполняемых заданий.

Представьте, что у вас есть две рабочие нагрузки, которые не должны мешать друг другу. Один из вариантов — изолировать их так, чтобы они не знали о существовании друг друга; фактически именно это делают контейнеры и виртуальные машины.

Другой подход — ограничить доступные им действия так, чтобы даже если одна из них знала о другой, то никак не могла бы повлиять на нее. Изоляция приложений с ограниченным доступом к ресурсам называется *помещением в песочницу* (sandboxing).

Контейнер, в котором запускается приложение, играет, по сути, роль песочницы. При каждом запуске контейнера известно, какой код приложения будет выполняться внутри него. В случае взлома приложения злоумышленник может попытаться запустить код, поведение которого выходит за нормальные для этого приложения рамки. Механизм песочницы позволяет ограничить разрешаемые данному коду действия, таким образом лишая злоумышленника возможности повлиять на систему.

Некоторые из этих подходов к изоляции процессов предполагают применение профиля при запуске контейнера, определяющего операции, доступные и недоступные контейнеру. Существуют также инструменты безопасности среды выполнения на основе eBPF, которые эффективно изолируют контейнеры, используя профиль, который можно обновлять или применять во время работы контейнера. Мы поговорим о них в главе 15. Но сначала рассмотрим механизм *seccomp*.

Механизм seccomp

В разделе «Системные вызовы» главы 2 вы видели, что системные вызовы предоставляют приложению интерфейс, с помощью которого оно может обращаться к ядру системы с просьбой выполнить определенные действия. Seccomp — механизм ядра Linux, ограничивающий набор системных вызовов, которые приложение может выполнять.

В 2005 году, когда seccomp (сокращение от secure computing mode — «безопасный режим вычислений») впервые появился в ядре Linux, он означал, что переведенный в этот режим процесс может выполнять только несколько системных вызовов:

- `sigreturn` (возврат из обработчика сигнала);
- `exit` (завершение выполнения процесса);
- `read` и `write`, но лишь при использовании дескрипторов файлов, уже открытых до перехода в безопасный режим.

Недоверенный код можно было спокойно выполнять в этом режиме, не опасаясь вредоносных действий. К сожалению, был и побочный эффект: огромное количество программ вообще не могли сделать ничего полезного. Такая песочница оказалась слишком ограниченной.

В 2012 году в ядро был добавлен новый подход, *seccomp-bpf*, определяющий допустимость конкретного системного вызова на основе фильтров пакетов Беркли (Berkeley Packet Filters, BPF) с учетом профиля seccomp данного процесса. У каждого процесса может быть свой профиль.



Фильтры пакетов Беркли являются предшественниками eBPF, которые мы обсудим в последующих главах.

Фильтр seccomp BPF может проверить код операции и параметры вызова и на их основе принять решение, разрешен ли вызов в соответствии с профилем. На самом деле все несколько сложнее: профиль указывает, что делать, если системный вызов соответствует конкретному фильтру, причем возможные действия включают возврат ошибки, завершение процесса или вызов трассировщика. Но в большинстве сценариев в мире контейнеров профиль либо разрешает вызов, либо возвращает ошибку, поэтому его можно считать своего рода «белым» (или «черным») списком системных вызовов.

Все вышеописанное может пригодиться в мире контейнеров, поскольку в действительности некоторые системные вызовы контейнеризированному

приложению выполнять незачем, разве что в исключительно редких ситуациях. Например, нежелательно, чтобы контейнеризированное приложение могло менять системное время на хосте, поэтому имеет смысл запретить ему доступ к системным вызовам `clock_adjtime` и `clock_settime`. Маловероятно, что контейнерам понадобится менять модули ядра, и потому им нет необходимости вызывать `create_module`, `delete_module` или `init_module`. В ядре Linux существует сервис `keyring`, который не изолирован пространством имен, вследствие чего лучше запретить контейнерам обращаться к `request_key` и `keyctl`.

Стандартный профиль `seccomp` для Docker (<https://docs.docker.com/engine/security/seccomp>) блокирует более 40 из 400 с лишним системных вызовов (включая все вышеперечисленные) без каких-либо отрицательных последствий для абсолютного большинства контейнеризированных приложений. Это отличный профиль для использования по умолчанию, разве что у вас есть особые причины не задействовать его.

Начиная с версии 1.22, Kubernetes поддерживает возможность настройки параметра `seccompProfile` в разделе `podSecurityContext` конфигурации рабочей нагрузки. Если в этом параметре задать `RuntimeDefault`, то будет применен профиль по умолчанию для среды выполнения контейнера (например, `containerd` использует профиль `Moby/Docker`). Возможно, вы захотите ограничить контейнер еще меньшим кругом системных вызовов — в идеальном мире у каждого приложения был бы свой профиль, разрешающий ему только необходимый набор системных вызовов. Существует несколько различных подходов к созданию подобных профилей.

- С помощью команды `strace` можно отследить все системные вызовы, выполняемые приложением. В своей статье (<https://oreil.ly/ROIHh>) Джесс Фразель (Jess Frazelle) описала, как на основе такого подхода она сгенерировала и протестировала стандартный профиль `seccomp` для Docker.
- Для развертывания в Kubernetes существует оператор профилей безопасности (Security Profiles Operator; <https://oreil.ly/MUOGw>), который может составить список системных вызовов, используемых приложением, и затем применять его в качестве профиля. Этот инструмент способен генерировать профили `AppArmor` и `SELinux`, а также `seccomp` — мы обсудим их чуть позже.
- Если создание профилей `seccomp` кажется вам слишком трудоемкой задачей, то, возможно, вам стоит взглянуть на коммерческие утилиты, призванные обеспечить безопасность контейнеров. Некоторые из них умеют автоматически генерировать пользовательские профили `seccomp` на основе наблюдений за отдельными рабочими нагрузками.

Помните, что профили `seccomp` постоянно развиваются вместе с Linux. С момента выхода первого издания книги в ядро было добавлено около 100 системных вызовов. Как правило, разработчики приложений не пользуются

системными вызовами напрямую, потому что они абстрагированы библиотеками языков программирования, и обновление этих библиотек, скорее всего, приведет к изменению списка системных вызовов, используемых приложением. Слишком строгий профиль `seccomp` может запретить доступ к вполне легальному новому системному вызову, поэтому при каждом обновлении версии ядра ОС, включающей новые системные вызовы, может потребоваться обновить профили.



Если вам интересны технологии, лежащие в основе `strace`, посмотрите мой доклад на GopherCon 2017 (<https://www.youtube.com/watch?v=01w7viEZzXQ>), в котором я создала простейшую реализацию `strace` с помощью всего нескольких строк на языке Go.

Модуль AppArmor

AppArmor (<https://gitlab.com/apparmor>; сокращение от Application Armor) — один из нескольких модулей безопасности Linux (Linux security modules, LSM), которые можно включить в ядро Linux. Чтобы проверить, какие LSM доступны на машине, взгляните на содержимое файла `/sys/kernel/security/lsm`.

В AppArmor профиль можно связать с исполняемым файлом и описать, что этому файлу можно, а что — нет, на языке привилегий и прав доступа к файлу. Напомню, что о них мы говорили в главе 2.

AppArmor поддерживают многие среды выполнения контейнеров, включая Docker, `containerd` и CRI-O.

AppArmor и прочие LSM реализуют *мандатное управление доступом* (mandatory access control, MAC)¹. Оно настраивается главным администратором, после чего остальные пользователи не могут никоим образом изменить права доступа или передать их другим пользователям. В Linux, напротив, управление доступом является *избирательным* (discretionary access control, DAC) в том смысле, что если моя учетная запись является владельцем файла, то я могу предоставить доступ к нему другому пользователю (если эта возможность не переопределена с помощью мандатного управления доступом) или запретить даже моей собственной учетной записи что-либо записывать в этот файл, чтобы случайно не испортить что-то. Благодаря мандатному управлению доступом администратор способен гораздо тоньше контролировать происходящее в системе, причем отдельные пользователи не смогут переопределить заданные им настройки.

¹ Иногда в русскоязычной литературе называется также принудительным контролем доступа. — *Примеч. пер.*

AppArmor включает режим жалоб (*complain mode*), в котором можно запускать исполняемый файл, используя заданный профиль, и все нарушения будут записываться в журнал. Предполагается, что на основе этих записей обновляется профиль, пока не перестанут появляться новые нарушения — после этого можно начинать обеспечивать соблюдение профиля. Когда профиль готов, его устанавливают в каталог `/etc/apparmor` и запускают утилиту `apparmor_parser` для его загрузки. Просмотреть загруженные профили можно в `/sys/kernel/security/apparmor/profiles` или выполнив команду `sudo apparmor_status`.

Если запустить контейнер командой `docker run --security-opt="apparmor:<имя_профиля>" ...`, то его поведение будет ограничено вариантами, разрешенными профилем. По умолчанию Docker применяет профиль AppArmor, который блокирует выполнение различных операций, таких как вызов `ptrace`, внутри контейнера. Однако вы, скорее всего, не увидите этот профиль в `/etc/apparmor.d`, потому что Docker создает его в памяти и передает в `apparmor_parser` при запуске демона Docker.

Увидеть, какой профиль AppArmor применяется к запущенному контейнеру, можно в выводе команды `docker inspect <ID_контейнера>`, который выглядит примерно так:

```
"AppArmorProfile": "docker-default"
```

Чтобы применить какой-либо профиль AppArmor к контейнеру в модуле (*pod*) Kubernetes, необходимо добавить аннотации (<https://kubernetes.io/docs/tutorials/security/apparmor/>). Упомянутый выше оператор профилей безопасности может создавать и применять профили AppArmor, специфичные для конкретной рабочей нагрузки.

Модуль SELinux

SELinux (Security-Enhanced Linux — Linux с улучшенной безопасностью) — еще один тип LSM, но уже разработанный Red Hat, хотя в Википедии утверждается (https://oreil.ly/gV7_e), что свои истоки он ведет от проектов Агентства национальной безопасности США. Если вы используете дистрибутив Red Hat (RHEL, Fedora или Centos Stream), то весьма вероятно, что SELinux уже активизирован.

SELinux позволяет ограничивать доступные процессам действия с точки зрения взаимодействий с файлами и прочими процессами. Каждый процесс выполняется в каком-либо *домене* SELinux (можете считать его своего рода контекстом работы процесса), а у каждого файла есть свой тип. Для просмотра информации SELinux для файла воспользуйтесь командой `ls -lZ`. Можно также добавить параметр `-Z` в команду `ps`, чтобы получить сведения SELinux для процессов.

Ключевое отличие между правами доступа SELinux и обычными правами доступа DAC Linux (которые вы видели в главе 2) состоит в том, что в SELinux права никак не связаны с учетной записью пользователя — они описываются исключительно метками. Тем не менее эти системы прав работают совместно, и потому любое действие должно быть разрешено как DAC, так SELinux.

Для того чтобы обеспечивать реализацию политик, необходимо сначала маркировать все файлы на машине метками SELinux. Эти политики указывают права доступа процесса из конкретного домена к файлам конкретного типа. На практике это означает следующее: можно сделать так, что приложение будет иметь доступ только к собственным файлам, а у остальных процессов никакого доступа к ним не будет. Взлом приложения в этом случае повлияет лишь на ограниченное количество файлов, даже если обычные права DAC его разрешают. При включенном SELinux появляется режим, в котором политики не применяются принудительно, а их нарушения просто заносятся в журнал (аналогично тому, что мы видели в AppArmor).

Чтобы создать удачный профиль SELinux приложения, нужно знать все файлы, к которым ему может потребоваться доступ как во время нормальной работы, так и в случае возникновения ошибок, поэтому данную задачу лучше возложить на разработчика приложения. Некоторые поставщики ПО предоставляют профили для своих приложений.

SELinux тесно интегрирован в среды выполнения контейнеров `podman` и `CRI-O`, которые поддерживаются Red Hat. В этих средах каждый контейнер работает в собственном домене SELinux, а файловые тома могут помечаться флагом `:z` или `:Z`, чтобы автоматически выполнять маркировку содержимого и разрешить доступ к нему для контейнеров.



Узнать больше о SELinux можно из прекрасного руководства от DigitalOcean (<https://oreil.ly/2Hx6b>), а также наглядного руководства от Дэна Уолша (Dan Walsh) (<https://oreil.ly/jmhC->).

Все механизмы безопасности, которые мы ранее обсудили, — `seccomp`, AppArmor и SELinux — предназначены для низкоуровневого управления поведением процессов. Генерация подробного профиля, в смысле точного описания множества необходимых системных вызовов и привилегий, — задача непростая, причем незначительное изменение приложения может потребовать существенных изменений профиля. Накладные расходы на поддержание профилей в актуальном состоянии могут оказаться довольно большими, вдобавок люди по своей природе склонны либо использовать достаточно либеральные профили, либо вообще их отключать. Если у вас нет возможностей генерировать профили для всех своих приложений, то имейте в виду, что стандартные профили `seccomp` и AppArmor для Docker работают вполне неплохо.

Стоит отметить, впрочем, что, хотя все эти механизмы и позволяют ограничить действия, доступные приложениям в пользовательском пространстве, не следует забывать о совместно используемом ими ядре. От уязвимости в самом ядре, такой как Dirty COW (https://en.wikipedia.org/wiki/Dirty_COW), вас не спасут никакие из этих утилит (если только они не блокируют все возможные пути выполнения, позволяющие получить доступ к уязвимости ядра).

До сих пор в главе обсуждались применяемые к контейнерам механизмы безопасности, с помощью которых можно ограничить действия, разрешаемые контейнерам. Рассмотрим теперь методики песочниц, обеспечивающие промежуточный уровень изоляции между контейнерами и виртуальными машинами, начиная с gVisor.

Песочница gVisor

Песочница gVisor от компании Google ограничивает контейнер, перехватывая системные вызовы примерно так же, как гипервизор перехватывает системные вызовы гостевой виртуальной машины. Она реализует значительный набор системных вызовов Linux в пользовательском пространстве с помощью паравиртуализации. Как вы видели в главе 5, паравиртуализация означает повторную реализацию инструкций, которые иначе выполнялись бы ядром хоста.

Для этого компонент gVisor, называемый Sentry, перехватывает поступающие от приложения системные вызовы. Sentry жестко ограничен с помощью `seccomp`, вследствие чего сам не может обращаться к ресурсам файловой системы. Если возникает необходимость выполнить системные вызовы, связанные с доступом к файлам, он передает их для выполнения отдельному процессу — `Gofor`.

Даже системные вызовы, не осуществляющие доступ к файловой системе, не передаются непосредственно ядру хоста, а реализуются заново внутри Sentry. По сути, это гостевое ядро, работающее в пользовательском пространстве.

Проект gVisor (https://gvisor.dev/docs/user_guide/quick_start/oci) содержит исполняемый файл `runsc`, совместимый с пакетами в формате OCI, очень похожий на обычную среду выполнения OCI `runc`, с которой мы познакомились в главе 6. Запуск контейнера с помощью `runsc` позволяет легко увидеть процессы gVisor. В следующем примере я использую тот же пакет для Alpine Linux, что и в разделе «Стандарты OCI» главы 6:

```
$ cd alpine-bundle
$ sudo runsc run sh
```

В другом окне терминала можете воспользоваться командой `runsc list`, чтобы просмотреть контейнеры, созданные `runsc`:

```
$ sudo runsc list
ID  PID    STATUS  BUNDLE                                CREATED                                OWNER
sh  32258  running /home/liz/alpine-bundle             2019-08-26T13:51:21                  root
```

Запустите команду `sleep` внутри контейнера на достаточно длительное время, чтобы посмотреть на ее выполнение из другого окна терминала. Команда `runsc ps <ID_контейнера>` отображает запущенные внутри контейнера процессы:

```
$ runsc ps sh
UID  PID    PPID    C      TTY      STIME    TIME    CMD
5000  1      0       0      pts/0    11:49    0s      sh
5000  2      1       0      pts/0    11:49    10ms     sleep
```

Пока все идет так, как ожидалось, но если взглянуть на процессы с точки зрения хоста, ситуация сразу станет гораздо интереснее (вывод команды был отредактирован, чтобы подчеркнуть некоторые нюансы):

```
$ ps fax
PID  TTY  STAT TIME COMMAND
...
26162 pts/1 Ss   0:00 \_ sudo runsc run sh
26163 pts/1 Sl+  0:00 \_ runsc run sh
26170 ?    Ssl  0:00 \_ runsc-gofer --root=/var/run/runsc gofer
                --bundle=/home/liz/alpine-bundle ...
26175 ?    Ssl  0:14 \_ runsc-sandbox --root=/var/run/runsc boot
                --apply-caps=false ...
26205 ?    Ss   0:00 \_ [exe]
26211 ?    S    0:00 \_ [exe]
26213 ?    SN   0:00 | \_ [exe]
26228 ?    S    0:00 \_ [exe]
26229 ?    SN   0:00 | \_ [exe]
26230 ?    S    0:00 \_ [exe]
26231 ?    SN   0:00 \_ [exe]
...
```

Здесь мы видим процесс `runsc run`, породивший два процесса: один для `Gofer`, а второй — для `runsc-sandbox`, который в документации `gVisor` называется `Sentry`. У песочницы есть несколько дочерних и внучатых процессов. Если вывести информацию о них с точки зрения хоста, то обнаружится нечто интересное: все они запускают исполняемый файл `runsc`. Для краткости в следующем примере показаны только дочерний и внучатый процессы:

```
$ sudo ls -l /proc/26230/exe
lrwxrwxrwx 1 root root 0 Jul 29 11:57 /proc/26230/exe -> /usr/bin/runsc
$ sudo ls -l /proc/26231/exe
lrwxrwxrwx 1 root root 0 Jul 29 11:57 /proc/26231/exe -> /usr/bin/runsc
```

Что примечательно, ни один из этих процессов не ссылается на исполняемый файл `sleep`, который, как мы знаем (поскольку можем увидеть его с помощью команды `runsc ps`), работает внутри контейнера. Найти исполняемый файл `sleep` напрямую с хоста также не получается:

```
$ sudo ps -eaf | grep sleep
liz 3554 3171 0 14:26 pts/2    00:00:00 grep --color=auto sleep
```

Подобная «невидимость» процессов, работающих внутри gVisor, гораздо больше напоминает поведение обычной виртуальной машины, чем контейнера. Это обеспечивает дополнительную защиту процессов, работающих внутри песочницы: даже если злоумышленник получит права суперпользователя на хосте, граница между хостом и работающими процессами остается относительно непроницаемой. По крайней мере, так должно было быть, если бы не сама команда `runsc`! Она предоставляет подкоманду `exec`, с помощью которой суперпользователь хоста способен производить различные операции внутри работающего контейнера:

```
$ sudo runsc exec sh ps
PID  USER    TIME  COMMAND
  1   root     0:00  /bin/sh
 21   root     0:00  sleep 100
 22   root     0:00  ps
```

И хотя подобная изоляция выглядит достаточно прочной, ее существенно ограничивают два фактора.

- Во-первых, в gVisor реализованы далеко не все системные вызовы Linux. На сайте проекта есть руководство по совместимости (https://gvisor.dev/docs/user_guide/compatibility), где отмечается, что многие языки программирования анализируют доступные системные вызовы и могут создавать альтернативные варианты в среде выполнения, поэтому большинство приложений будет работать в gVisor. В настоящее время gVisor успешно функционирует даже на графических (GPU) и тензорных (TPU) процессорах, которые широко используются в задачах машинного обучения.
- Во-вторых, возможна потеря производительности. На сайте проекта gVisor опубликовано руководство по производительности (https://gvisor.dev/docs/architecture_guide/performance), позволяющее изучить этот вопрос более подробно. По сути, gVisor намеренно выбирает более строгую модель безопасности (https://gvisor.dev/docs/architecture_guide/security), жертвуя производительностью. Для gVisor есть режим платформы на основе KVM (https://gvisor.dev/docs/architecture_guide/platforms/#kvm), способный обеспечить лучшую производительность при развертывании на «голом железе».

Если вы используете облачную платформу Google, то с легкостью сможете получить доступ к gVisor. Эту песочницу также можно применять в стандартном кластере Kubernetes.

Как вы видели в этом разделе, gVisor предоставляет механизм изоляции, напоминающий скорее виртуальную машину, чем обычный контейнер. Однако gVisor влияет только на способ обращения приложения к системным вызовам. Для изоляции контейнера по-прежнему используются пространства имен, контрольные группы и смена корневого каталога.

В оставшейся части главы мы обсудим подходы, в которых контейнеризированные приложения выполняются с помощью изоляции виртуальных машин.

Среда выполнения контейнеров Kata Containers

Как вы видели в главе 4, при запуске обычного контейнера среда выполнения создает новый процесс на хосте. Основная задумка Kata Containers (<https://katacontainers.io/>) состоит в том, чтобы запускать контейнеры внутри отдельной виртуальной машины. Подобный подход позволяет выполнять приложения из обычных образов контейнеров в формате OCI при полной изоляции виртуальной машины. Проект Kata Containers поддерживается фондом Open Infra Foundation.

Для каждого контейнера Kata создает отдельную виртуальную машину, используя микро-VMM — легковесный гипервизор, такой как Firecracker, QEMU или Cloud Hypervisor — эти технологии мы рассмотрим чуть позже.

Как и gVisor, Kata Containers предполагает компромисс между безопасностью и производительностью. Для многих развертываний, особенно там, где рабочие нагрузки, по сути, являются доверенными (например, все они создаются и эксплуатируются одной и той же компанией), дополнительная изоляция требует больше памяти и времени CPU, производительность снижается, а такие функции, как общие тома или поддержка графических процессоров, могут быть недоступны.

Легковесные микровиртуальные машины

Как вы видели в разделе «Недостатки виртуальных машин» главы 5, обычные виртуальные машины медленно запускаются, вследствие чего плохо подходят для краткосрочных рабочих заданий, обычно выполняемых в контейнерах. Но что, если бы виртуальная машина могла запускаться чрезвычайно быстро?

В настоящее время существует несколько вариантов минималистичных виртуальных машин, часто называемых легковесными VM или микро-VM, которые предлагают преимущества безопасной изоляции через гипервизор и не используют общее ядро, но разработаны специально для контейнеров и обладают коротким временем запуска.

Firecracker (<https://firecracker-microvm.github.io/>) и Cloud Hypervisor (<https://www.cloudhypervisor.org>) — это минималистичные виртуальные машины, написанные на языке Rust, который обеспечивает безопасную работу с памятью, помогая избежать уязвимостей и достичь времени запуска около 100 мс. В Edera (<https://oreil.ly/QDoTT>) применен аналогичный подход — в ней используется гипервизор на основе Xen, но в значительной степени переписанный на Rust.

Недавно компания Apple запустила проект с открытым исходным кодом Apple Containerization (<https://github.com/apple/containerization>), который позволяет запускать контейнеры в легковесных виртуальных машинах Linux внутри гипервизора, написанного на Swift.

Эти микро-ВММ способны запускать виртуальные машины так быстро, потому что в них отсутствует функциональность, обычно имеющаяся в ядре, но не требующаяся в контейнере. Обнаружение устройств — один из самых медленных этапов загрузки системы, но контейнеризированные приложения редко используют много устройств. Основная экономия достигается за счет минимальной модели устройств, в которой есть только все самое необходимое.

Между этими проектами существуют некоторые различия в философии и контексте.

- Firecracker изначально разрабатывалась в AWS и используется для запуска рабочих нагрузок Lambda. Она предоставляет функциональность, минимально необходимую для запуска и изоляции контейнерных рабочих нагрузок, и обеспечивает максимально быстрый запуск.
- Cloud Hypervisor способна обслуживать более сложные рабочие нагрузки, например использующие вложенную виртуализацию, Windows как гостевую ОС и поддержку графических процессоров.
- Edera реализует запуск контейнеров в облегченных «зонах», подобных виртуальным машинам, с акцентом на безопасности, предлагая более строгую изоляцию, чем у обычных контейнеров.
- Apple Containerization позволяет запускать контейнеры в собственных легковесных виртуальных машинах на Mac, не требуя использования виртуальной машины Linux в качестве хоста.

Как и в большинстве технологических решений, здесь есть свои компромиссы. Более строгая изоляция, характерная для виртуальных машин, обеспечивает более надежную защиту. Каждый контейнер имеет собственное ядро, поэтому

выход за пределы контейнера равносильен выходу за пределы виртуальной машины. Недостаток заключается в том, что без общего ядра инструменты инфраструктуры на основе eBPF теряют контроль над контейнерами и становятся ближе к модели sidecar-контейнеров с отдельным экземпляром для каждого контейнера/виртуальной машины. Влияние на производительность для многих рабочих нагрузок, вероятно, будет минимальным, но иногда может быть вполне заметным.



Компания Edera сравнила различные подходы к виртуализации в своей статье (https://oreil.ly/_FSmN).

В завершение стоит упомянуть еще один подход к изоляции с еще более радикальным вариантом сокращения размера гостевой ОС — Unikernels.

Unikernels

В образах виртуальных машин выполняются универсальные операционные системы, которые можно использовать повторно для любого приложения. Само собой разумеется, что приложения вряд ли будут задействовать все функциональные возможности ОС. Исключение неиспользуемых компонентов позволяет сократить поверхность атаки.

Основная идея Unikernels состоит в создании специализированного образа машины, состоящего из приложения и необходимых ему частей операционной системы. Этот образ машины может выполняться непосредственно на гипервизоре, с тем же уровнем изоляции, что и у обычных виртуальных машин, однако с более коротким временем начальной загрузки по сравнению с Firecracker.

Каждое приложение необходимо скомпилировать в образ Unikernels, включающий все необходимое ему для работы. А затем гипервизор сможет загружать эту машину как обычный образ виртуальной машины Linux.

Методики Unikernels для контейнеров применяются в проекте Nabla (<https://nabla-containers.github.io/>) компании IBM. В настоящее время его развитие практически остановилось. В контейнерах Nabla используется крайне ограниченный набор всего из семи системных вызовов, контроль за которыми осуществляется с помощью профиля seccomp. Все прочие системные вызовы приложения обрабатываются в библиотечном компоненте OS Unikernels. Благодаря обращению лишь к небольшой доле ядра контейнеры Nabla сокращают поверхность атаки.

Unikraft — еще один проект на основе Unikernels, развиваемый под эгидой Linux Foundation и ориентированный на облачные приложения.

Следует отметить, что Unikernels — это не контейнеры, но они предоставляют иной способ изоляции приложений друг от друга на общем хосте.

Резюме

В этой главе вы увидели, что существует множество способов изолировать экземпляры приложения друг от друга, которые в известной степени можно рассматривать как контейнер.

- Один из вариантов — использовать обычные контейнеры с дополнительными механизмами безопасности для усиления исходной изоляции контейнеров: `seccomp`, `AppArmor`, `SELinux`. Все они проверены на практике, но при этом известны своей сложностью с реализацией эффективного управления.
- В случаях, когда требуется более строгая изоляция между контейнерами, допускается использовать микро-VMM, хотя это может привести к потере производительности.
- Наконец, есть и третья категория методов изоляции, таких как `gVisor`, которые занимают промежуточное положение между контейнерами и виртуальными машинами.

Какой вариант лучше для ваших приложений, зависит от того, какие риски вы готовы принять. Оцените, какие возможности предоставляет ваше публичное облако или сервис с полным управлением. Следует также учитывать доступность инструментов обеспечения безопасности в среде выполнения (о них мы поговорим в главе 15), которые способны предложить гибкий и динамичный подход к изоляции, более подходящий для ваших развертываний. Они могут служить альтернативой статическим профилям изоляции или использоваться совместно с ними для создания многоуровневой защиты.

Вне зависимости от выбранной вами среды выполнения контейнеров и степени обеспечиваемой ею изоляции, всегда существуют способы ее взломать. В главе 11 вы узнаете, как именно.

Нарушение изоляции контейнеров

В главе 4 вы узнали, как устроен контейнер и каким образом он получает ограниченное представление о машине, на которой работает. В этой главе вы увидите, насколько просто настроить контейнеры так, что эта изоляция фактически перестает работать.

Иногда это делается умышленно, для достижения определенных целей, например, чтобы частично делегировать сетевую функциональность вспомогательному контейнеру (далее будем называть его *sidecar*-контейнером). В остальных же случаях обсуждаемые в текущей главе идеи серьезно подорвут безопасность ваших приложений!

Для начала поговорим о поведении в мире контейнеров, вероятно по умолчанию наиболее опасном: о выполнении контейнеров от имени суперпользователя.

Выполнение контейнеров по умолчанию от имени суперпользователя

Если в образе контейнера пользователь не указан или не прописан вами вручную, то по умолчанию контейнер будет запускаться от имени суперпользователя. И если вы не настроили пространства имен пользователей, то полномочия суперпользователя будут действовать не только внутри контейнера, но и на хосте.



В данном примере предполагается, что вы используете команду `docker` от Docker. Если у вас установлен `podman`, то вы, возможно, последовали совету создать псевдоним для `docker`, чтобы в действительности выполнялась команда `podman`. Поведение `podman` относительно суперпользователей довольно сильно отличается. Далее в главе мы поговорим об этих отличиях, а пока просто учтите, что с `podman` пример работать не будет.

Запустите командную оболочку от имени рядового пользователя внутри контейнера Alpine, задействуя `docker`, и проверьте его идентификатор:

```
$ whoami
liz
$ docker run -it alpine sh
/ $ whoami
root
```

Хотя контейнер создавался с помощью команды `docker`, выполнявшейся от имени обычного пользователя, внутри контейнера используется учетная запись `root`. Теперь проверим, совпадает ли эта учетная запись с учетной записью `root` на хосте, открыв второй терминал на той же машине и выполнив внутри контейнера команду `sleep`:

```
/ $ sleep 100
```

Проверьте идентификатор пользователя во втором окне:

```
$ ps -fC sleep
UID          PID    PPID  C  STIME TTY          TIME CMD
root         30619  30557  0  16:44 pts/0    00:00:00 sleep 100
```

С точки зрения хоста владельцем этого процесса является `root`; `root` внутри контейнера является пользователем `root` и на хосте.

Если для запуска используется `runc`, а не `docker`, то тот же пример будет менее убедительным (за исключением контейнеров, не требующих привилегий суперпользователя, о которых мы поговорим чуть позже), поскольку для запуска контейнера нужно обладать `root`-правами на хосте. Дело в том, что только у суперпользователя есть необходимые привилегии для создания пространств имен. В Docker контейнеры создает демон Docker, выполняемый от имени суперпользователя.

То, что в Docker контейнеры выполняются от имени суперпользователя, даже когда запускаются рядовым пользователем, — одна из форм повышения привилегий. Сама по себе работа контейнера от имени суперпользователя не является проблемой, но вызывает вопросы, связанные с безопасностью. Если атакующий сможет выйти за рамки контейнера, работающего от имени суперпользователя, то получит свободный доступ ко всему, что есть на машине. Вы же не хотите оказаться всего в одном шаге от захвата хоста злоумышленником?

К счастью, контейнеры можно запускать и от имени рядовых пользователей. Для этого можно задать его идентификатор или использовать вышеупомянутые контейнеры без привилегий суперпользователя. Рассмотрим оба варианта.

Переопределение идентификатора пользователя

Переопределить UID в среде выполнения можно, указав идентификатор пользователя для контейнера.

В `runc` для этого необходимо изменить файл `config.json` внутри комплекта. Попробуйте, например, поменять `process.user.uid` следующим образом:

```
...
"process": {
  "terminal": true,
  "user": {
    "uid": 5000,
    ...
  }
  ...
}
```

Теперь среда выполнения подхватит указанный UID и будет использовать его для запуска процесса контейнера:

```
$ sudo runc run sh
$ whoami
whoami: unknown uid 5000
$ sleep 100
```

И хотя мы используем `sudo` для запуска команды от имени `root`, идентификатор пользователя контейнера — `5000`, в чем можно убедиться со стороны хоста:

```
$ ps -fc sleep
UID      PID  PPID  C  STIME TTY          TIME CMD
5000     26909 26893  0 16:16 pts/0    00:00:00 sleep 50
```

Как вы видели в главе 6, в комплект совместимого с OCI образа входит как корневая файловая система образа, так и сведения о конфигурации среды выполнения. Та же информация упаковывается в образы Docker. Переопределить настройки пользователя можно с помощью параметра `--user`:

```
$ docker run -it --user 5000 ubuntu bash
I have no name!@b7ca6ec82aa4:/$
```

Поменять встроенный в образ Docker идентификатор пользователя можно с помощью команды `USER` в соответствующем Dockerfile. Но абсолютное большинство образов контейнеров в общедоступных репозиториях настроены на использование `root`, поскольку параметр конфигурации `USER` в них не задан. А если идентификатор пользователя не задан, то по умолчанию контейнер выполняется от имени суперпользователя.

Без новых привилегий

В главе 2 вы познакомились с флагом `setuid`, который позволяет другому пользователю, запускающему файл, присвоить себе идентификатор владельца этого файла.

Рассмотрим пример использования этого механизма для переопределения пользователя, указанного в следующем Dockerfile:

```
FROM ubuntu:24.04
RUN cp /usr/bin/bash /tmp/mybash      ❶
RUN chmod 4755 /tmp/mybash           ❷
RUN useradd -ms /tmp/mybash myuser    ❸
USER myuser                           ❹
ENTRYPOINT ["/tmp/mybash"]           ❺
```

- ❶ Копирует исполняемый файл `bash` в каталог `/tmp` в образе контейнера.
- ❷ Цифра 4 в команде `chmod` устанавливает бит `setuid` для этого файла, а набор привилегий 755 позволяет любому пользователю читать и выполнять файл, но записывать в него — только владельцу.
- ❸ Создает нового пользователя с именем `myuser`.
- ❹ Настраивает запуск контейнера от имени `myuser`.
- ❺ Определяет точку входа для запуска копии `bash` с битом `setuid`.

Запуск этого образа контейнера открывает оболочку `bash` от имени `myuser`:

```
$ docker run -it nopriv
mybash-5.2$ whoami
myuser
mybash-5.2$ id
uid=1001(myuser) gid=1001(myuser) groups=1001(myuser)
```

Проверив характеристики исполняемого файла, можно увидеть, что он принадлежит пользователю `root` и имеет флаг `setuid`:

```
mybash-5.2$ ls -l /tmp/mybash
-rwsr-xr-x 1 root root 1446024 May  8 13:16 /tmp/mybash
```

Учитывая это, можно было бы ожидать, что командная оболочка запустится от имени `root`, но, как отмечалось в главе 2, оболочка `bash` проверяет UID и возвращается к привилегиям пользователя, запустившего ее, чтобы избежать этого простого способа повышения привилегий.

Однако `bash` принимает флаг `-p`, который позволяет обойти эту проверку и запустить программу от имени владельца файла. Вы можете передать этот флаг в команде, которая выполняется при запуске контейнера, и получить совершенно другой результат:

```
$ docker run -it nopriv -p
mybash-5.2# whoami
root
mybash-5.2# id
uid=1001(myuser) gid=1001(myuser) euid=0(root) groups=1001(myuser)
```

В выводе команды `id` видно, что, хотя файл запущен от имени пользователя `myuser`, он получил эффективный идентификатор (`euid`) пользователя `root`. Поскольку в этом примере используется `bash`, в команде необходимо указать флаг `-p`, чтобы предотвратить возврат к исходному пользователю. Если бы в `ENTRYPOINT` в образе был указан другой исполняемый файл с флагом `setuid`, то простой запуск контейнера переопределил бы значение директивы `USER` и контейнер работал бы от имени `root`.

Флаг `setuid` легко упустить из виду¹ и предположить, что будет действовать директива `USER`, поэтому может быть нелишним добавить дополнительный уровень защиты в виде параметра `no-new-privileges` команды запуска, чтобы предотвратить подобное повышение привилегий:

```
$ docker run -it --security-opt no-new-privileges nopriv -p
myuser@e731e9ac6d6c:/$ whoami
myuser
myuser@e731e9ac6d6c:/$ id
uid=1001(myuser) gid=1001(myuser) groups=1001(myuser)
```

В этом примере пользователь, от имени которого должен запускаться контейнер, указывается во время сборки с помощью директивы `USER`, однако результат будет тем же, если запустить контейнер с параметром `--user`.

В Kubernetes того же эффекта можно добиться, определив параметр `securityContext.allowPrivilegeEscalation: false` в спецификации контейнера. Это оптимальное значение по умолчанию, которое следует применять ко всем приложениям, если только нет веских причин разрешать повышение привилегий внутри пода.

Требование выполнения от имени суперпользователя внутри контейнера

Существует множество широко используемых образов контейнеров, которые инкапсулируют популярное ПО, изначально предназначенное для запуска непосредственно на сервере. Возьмем, например, обратный прокси-сервер и балансировщик нагрузки Nginx; он появился задолго до того, как Docker

¹ Как отмечалось в главе 8, некоторые сканеры образов обнаруживают исполняемые файлы с флагом `setuid`.

обрел популярность, но сейчас доступен в виде образа контейнера на Docker Hub (https://hub.docker.com/_/nginx).

Стандартный контейнер Nginx по умолчанию настроен на выполнение от имени суперпользователя, по крайней мере по состоянию на момент написания книги. Если вы запустите контейнер `nginx` и посмотрите на работающие в нем процессы, то увидите, что главный процесс выполняется от имени суперпользователя:

```
$ docker run -d --name nginx nginx
4562ab6630747983e6d9c59d839aef95728b22a48f7aff3ad6b466dd70ebd0fe
$ docker top nginx
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	24945	24920	0	22:35	?	00:00:00	nginx: master process nginx
message+	24992	24945	0	22:35	?	00:00:00	nginx: worker process
message+	24993	24945	0	22:35	?	00:00:00	nginx: worker process
message+	24994	24945	0	22:35	?	00:00:00	nginx: worker process
message+	24995	24945	0	22:35	?	00:00:00	nginx: worker process



Основная цель этого примера — показать процесс, работающий от имени `root`, но что означает символ `+` рядом с UID рабочих процессов? Прежде всего он означает, что имя длиннее восьми символов и начинается с `message`. Вывод команды `docker top` показывает имена пользователей, как они определены на хосте, а не внутри контейнера. На моем хосте список пользователей в файле `/etc/passwd` выглядит примерно так:

```
$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
...
messagebus:x:101:101:./nonexistent:/usr/sbin/nologin
...
lizr:x:501:1000:lizr:/home/lizr.linux:/bin/bash
...
```

В описании официального образа Nginx (https://hub.docker.com/_/nginx#user-and-group-id) отмечается, что, начиная с версии 1.17.0, для рабочих процессов используется идентификатор рядового пользователя с ID 101. Пользователь с ID 101 на этой машине — `messagebus`, что соответствует выводу команды `docker top`.

Вполне логично, что при выполнении на сервере код `nginx` запускается от имени суперпользователя. По умолчанию он принимает запросы через традиционный веб-порт `80`. А для открытия портов с номером меньше `1024` необходима привилегия `CAP_NET_BIND_SERVICE` (см. главу 2), и самый простой способ гарантировать ее наличие — запускать `nginx` от имени суперпользователя. Однако в контейнере подобное требование особого смысла не имеет, потому что благодаря отображению портов код `nginx` может прослушивать любой порт, если перенаправить его (при необходимости) на порт `80` хоста.

Вместо запуска от имени `root` можно задействовать `setcap` и добавить привилегию `CAP_NET_BIND_SERVICE` в исполняемый файл `nginx` во время сборки образа контейнера. В таком случае в `Dockerfile` допускается указывать пользователя без привилегий `root`, при этом исполняемый файл все равно сможет подключиться к порту с номером из нижнего диапазона. Возможно, разработчики Nginx решили этого не делать, потому что `nginx` не будет работать, если привилегии будут отозваны средой выполнения контейнера (например, `docker --cap-drop=ALL`). Кроме того, для поддержки привилегий потребуется установить пакет `libcap2-bin`, в котором находится утилита `setcap`, по крайней мере во время сборки образа.

Есть и другой способ: разрешить рядовым пользователям подключаться к любому порту, присвоив с помощью `sysctl` значение `0` параметру ядра `net.ipv4.ip_unprivileged_port_start`. В настоящее время в Docker данный механизм реализован по умолчанию, а в Kubernetes можно настроить этот и другие параметры `sysctl` (<https://oreil.ly/aIhYp>) для отдельных рабочих нагрузок с помощью `podSecurityContext`.

Многие поставщики признали, что выполнение от имени суперпользователя — это проблема с точки зрения безопасности, и предоставляют образы Docker, работающие от имени обычных, непривилегированных пользователей. Например, существует официальный образ Nginx для непривилегированного доступа (<https://hub.docker.com/r/nginxinc/nginx-unprivileged>), который несколько отличается от образа для `root`, например, использованием порта `8080` по умолчанию. Это позволяет ему корректно работать даже в среде выполнения контейнеров, которая не поддерживает использование `sysctl` для выдачи разрешения любому пользователю открывать порты с низкими номерами.

Если бы Nginx не предоставлял такой возможности, было бы относительно просто создать образ Nginx, способный работать от имени рядового пользователя (пример можно найти здесь: <https://oreil.ly/UFmcG>). Для других приложений это может оказаться задачей посложнее и потребовать более обширных изменений кода, чем несколько исправлений в `Dockerfile` и конфигурации. К счастью, существуют организации, включая Bitnami (<https://github.com/bitnami/containers>), Chainguard (<https://www.chainguard.dev>), и образы `distroless` от Google (<https://github.com/GoogleContainerTools/distroless>), которые взяли на себя труд по созданию и поддержанию в актуальном состоянии набора образов контейнеров, не требующих `root`, для многих популярных приложений. Некоторые поставщики ПО предлагают непривилегированные версии своих дистрибутивов (как это делает Nginx, о чем упоминалось в предыдущем абзаце).

Открытие портов с номерами из нижнего диапазона не единственная причина, по которой программному обеспечению может потребоваться запуск от имени `root`. Еще одна популярная операция, требующая привилегий, — установка ПО.

Привилегии root для установки программного обеспечения

Еще одна причина, по которой образы контейнеров иногда требуют запуска от имени суперпользователя, — необходимость установки программного обеспечения с помощью менеджеров пакетов, таких как `yum` и `apt`. Это вполне оправданно на этапе сборки образа контейнера, но в `Dockerfile` следующим шагом может быть команда `USER`, позволяющая запускать образ от имени рядового пользователя. На самом деле это стоит делать в рамках многоэтапной сборки (подробнее об этом рассказывается в главе 7).

Я настоятельно рекомендую не давать разрешение контейнерам на установку дополнительного ПО в среде выполнения. И тому есть несколько причин.

- Нерациональность: в случае установки всего необходимого ПО во время сборки достаточно сделать это один раз, а не повторять всю процедуру при каждом создании нового экземпляра контейнера.
- Пакеты, устанавливаемые в среде выполнения, не подвергаются сканированию на уязвимости (см. главу 8).
- Вероятно, более серьезная, чем отсутствие сканирования на уязвимости, проблема: становится сложно отследить, какие версии пакетов установлены в каждом конкретном запущенном экземпляре контейнера. Поэтому, даже если у вас появится информация о какой-то уязвимости, вы не будете знать, какие контейнеры необходимо удалить и развернуть заново с исправлениями.
- В зависимости от приложения, контейнер можно запустить в режиме «только для чтения» (с помощью параметра `--read-only` команды `docker run` или установив режим монтирования `readOnly` в спецификации пода Kubernetes), что значительно усложнит для злоумышленника установку кода.
- Добавление пакетов в среде выполнения означает, что контейнер не рассматривается как неизменяемый. О преимуществах неизменяемых контейнеров с точки зрения безопасности рассказывается в подразделе «Неизменяемые контейнеры» в главе 8.

Привилегии для eBPF и модулей ядра

Еще одно действие, которое можно выполнить только от имени суперпользователя, — изменение ядра с помощью его модулей или eBPF. Оба подхода позволяют динамически загружать код в ядро Linux во время выполнения для расширения его функциональности. Как известно, все контейнеры на хосте используют одно и то же ядро, поэтому добавление в ядро, например, инструментов мониторинга или безопасности означает, что они могут немедленно увидеть и повлиять на все контейнеризированные приложения на этом хосте.

Проблема с модулями ядра в том, что они не проходят столь же тщательное тестирование сообществом и проверку на безопасность, как остальные части

ядра Linux. Пользователи опасаются устанавливать модули ядра из-за риска возникновения ошибки, которая может привести к сбою и отключению всей системы.

В отличие от модулей программы eBPF гарантированно безопасны для запуска в ядре благодаря процессу проверки eBPF, который анализирует каждый возможный путь выполнения программы и отказывается загружать ее в ядро, если существует хотя бы малейшая вероятность сбоя. Это делает eBPF очень мощной платформой для создания инструментов инфраструктуры, особенно в контейнеризированных средах.



Аббревиатура eBPF раньше расшифровывалась как Extended Berkeley Packet Filter (расширенный пакетный фильтр Беркли), но это вызывало путаницу, потому что возможности eBPF гораздо больше простой фильтрации пакетов, и сообщество решило, что понятие eBPF лучше рассматривать как самостоятельное название (<https://oreil.ly/9Y2WJ>), а не как аббревиатуру.

Если вы хотите углубиться в изучение eBPF, можете начать со слайдов из моего доклада о раскрытии потенциала ядра с помощью eBPF (<https://oreil.ly/OPcR3>). Обязательно посмотрите увлекательный документальный фильм о eBPF (<https://ebpfdocumentary.com>), рассказывающий историю появления проекта. Огромный объем информации и ресурсов вы найдете на сайте проекта eBPF (<https://ebpf.io>) и на сайте Брендана Грегга (Brendan Gregg) (<https://brendangregg.com>), а если у вас появится желание писать свои программы на eBPF, то можете прочитать мою книгу *Learning eBPF*¹.

Очевидно, что для загрузки модулей ядра или программ eBPF в ядро требуются значительные привилегии. Для загрузки модулей необходима привилегия `CAP_SYS_MODULE`, а для запуска программ eBPF — как минимум `CAP_BPF`². В зависимости от действий, выполняемых программой, могут понадобиться и другие привилегии eBPF, например:

- `CAP_NET_ADMIN` — для подключения к сетевому стеку;
- `CAP_PERFMON` — для подключения к обработчикам ядра, связанным с мониторингом производительности;
- `CAP_SYS_RESOURCE` — для увеличения лимитов, что необходимо для отображения памяти eBPF;
- `CAP_SYS_ADMIN` — для монтирования файловых систем;
- `CAP_SYS_PTRACE` — для трассировки пользовательских процессов.

¹ Райс Л. Изучаем eBPF: программирование ядра Linux для улучшения безопасности, сетевых функций и наблюдаемости. — Астана, 2024.

² Начиная с Linux 5.8. До этого для запуска программ eBPF требовалась привилегия `CAP_SYS_ADMIN`.

Инструментам, работающим в пространстве ядра, также могут потребоваться определенные файловые системы хоста, смонтированные в контейнер (например, `/lib/modules` для модулей ядра или `/sys/fs/bpf` для eBPF).

Иными словами, если вы решите запускать инструменты на основе eBPF (или модули ядра), вам придется предоставить им дополнительные привилегии. Однако преимущества могут значительно перевесить риски, поскольку это открывает возможности для использования средств обеспечения безопасности на основе eBPF, которые мы обсудим в главе 15.

По возможности старайтесь запускать код своих приложений от имени рядовых пользователей или задействовать для него пространства имен пользователей (как показано в разделе «Пространство имен пользователей» главы 4), чтобы суперпользователь внутри контейнера отличался от суперпользователя хоста. Один из удобных практических способов применения пространств имен — *контейнеры, не требующие привилегий суперпользователя* (rootless), если ваша система их поддерживает.

Контейнеры, не требующие привилегий суперпользователя

Если вы выполняли примеры из главы 4, то знаете, что для некоторых действий, необходимых при создании контейнера, требуются привилегии суперпользователя. Такой вариант часто не подходит в традиционных средах с общим доступом к машине, поскольку на одной и той же машине могут работать несколько пользователей. Примером может служить компьютерная система университета, в которой на совместно используемой машине или в кластере машин создаются учетные записи для студентов и преподавателей. Системный администратор, само собой, откажется предоставлять всем подряд привилегии суперпользователя для создания контейнеров и будет прав, ведь такие привилегии позволят делать все что угодно (случайно или намеренно) с кодом и данными других пользователей.

Инициатива Rootless Containers (<https://rootlesscontainers.rs>) способствовала включению в ядро изменений, необходимых для запуска контейнеров без привилегий суперпользователя. Теперь Docker и containerd полностью поддерживают режим выполнения без root-прав. Реализация контейнеров podman уже давно поддерживает возможность выполнения rootless-контейнеров и в отличие от Docker не использует привилегированный процесс демона. Именно поэтому примеры в начале главы выполняются по-разному, если docker является псевдонимом для podman.

В Kubernetes существует различие между запуском без привилегий суперпользователя контейнерных рабочих нагрузок и компонентов Kubernetes (<https://oreil.ly/KN6G5>). Запуск рабочих нагрузок от имени рядового пользователя осуществляется с помощью podSecurityContext (<https://oreil.ly/GgS4k>); на момент написания книги поддержка запуска компонентов Kubernetes без привилегий суперпользователя все еще находилась на стадии альфа-тестирования, хотя

в таком состоянии она доступна с версии Kubernetes 1.22 (притом что актуальная версия уже 1.33!).



В системе Docker для запуска контейнеров не обязательно иметь привилегии суперпользователя, достаточно быть членом группы `docker`, имеющей право отправлять команды демону Docker через сокет Docker. Стоит отметить, что эти возможности эквивалентны привилегиям суперпользователя на хосте. Любой член данной группы может запускать контейнеры, и, как вы уже знаете, по умолчанию они будут выполняться от имени суперпользователя. А если он смонтирует корневой каталог хоста с помощью команды `docker run -v /:/host <образ>`, то получит и полный доступ к корневой файловой системе хоста.

Контейнеры, не требующие привилегий суперпользователя, активно используют функциональность пространств имен пользователей, о которой вы прочли в разделе «Пространство имен пользователей» главы 4. Идентификатор рядового пользователя на хосте можно связать с суперпользователем в контейнере. Если злоумышленнику и удастся неким образом выйти за рамки контейнера, то он не получит автоматически привилегий суперпользователя, так что с точки зрения безопасности это значительный прогресс.

Узнать больше о `root`-правах внутри и вне контейнера `podman` можно в статье Скотта Маккарти (Scott McCarty) (https://oreil.ly/67k_C).

Впрочем, `rootless`-контейнеры не панацея. Не всякий образ, успешно работающий в обычном контейнере, запущенном с привилегиями суперпользователя, будет вести себя так же в контейнере, не требующем прав суперпользователя, хотя с точки зрения контейнера он запускается от его имени.

Как указано в документации (<https://oreil.ly/iZiaw>) по пространствам имен пользователей, они изолируют не только идентификаторы пользователей и групп, но и другие атрибуты, включая привилегии. Другими словами, можно добавлять или убирать привилегии процесса в пространстве имен пользователя, причем они будут применяться лишь в пределах данного пространства имен. Поэтому добавленная привилегия для `rootless`-контейнера будет применяться исключительно в этом контейнере, но не в том случае, если контейнер должен взаимодействовать с другими ресурсам хоста.

Дэн Уолш (Dan Walsh) привел в своем блоге (<https://oreil.ly/1fwZP>) несколько неплохих примеров. Один из них относится к привязке портов с номерами ниже `1024`, требующей привилегии `CAP_NET_BIND_SERVICE`. Если запустить обычный контейнер с этой привилегией (присутствующей по умолчанию, если контейнер запущен от имени суперпользователя) и при этом использовать пространство имен сети, такой контейнер сможет привязаться к любому порту хоста. Контейнер, не имеющий привилегий суперпользователя, но обладающий привилегией `CAP_NET_BIND_SERVICE` и использующий сеть совместно

с хостом, не сможет привязаться к портам с номерами ниже 1024, поскольку привилегия неприменима вне пространства имен пользователя контейнера.

По большому счету, ограничение привилегий пространствами имен — хорошая вещь, ведь она позволяет контейнеризированным процессам выполняться как будто от имени суперпользователя, но без возможности производить действия, требующие привилегий на системном уровне, например изменять время или перезагружать машину. Абсолютное большинство приложений, которые могут работать в обычном контейнере, смогут успешно работать и в rootless-контейнере.

В случае применения rootless-контейнеров процесс, который изнутри контейнера выглядит как запущенный от имени суперпользователя, с точки зрения хоста считается запущенным от имени обычного пользователя. Что любопытно, вследствие этого, в частности, права доступа к файлам у контейнера, не требующего привилегий суперпользователя, не обязательно такие же, какими были бы без переназначения пользователей. Чтобы обойти данную проблему, файловая система должна поддерживать подмену владельцев файлов и групп в пространстве имен пользователя. (На момент написания книги не все файловые системы могут похвастаться такой поддержкой.)

Сама по себе работа от имени суперпользователя внутри контейнера не проблема, ведь злоумышленнику еще надо как-то выйти за рамки контейнера. Уязвимости, позволяющие выйти за пределы контейнера, время от времени находят и, вероятно, будут находить в будущем, однако преодолеть границы контейнера можно не только через уязвимость. Далее в главе вы увидите, как небезопасные настройки контейнеров сильно упрощают атакующему получение доступа к данным без всяких уязвимостей. В сочетании с контейнерами, запущенными от имени суперпользователя, подобные неудачные конфигурации провоцируют весьма опасную ситуацию.

Благодаря переопределению идентификаторов пользователей и контейнерам, не требующим привилегий суперпользователя, можно уберечь систему от выполнения контейнеров от имени суперпользователя. Неважно, как вы этого добьетесь, но лучше избегать подобных ситуаций.

Флаг `--privileged` и привилегии

Docker и прочие среды выполнения контейнеров позволяют указывать флаг `--privileged` при запуске контейнера. Эндрю Мартин (Andrew Martin) назвал этот флаг «наиболее опасным флагом в истории вычислительной техники», и не без оснований: его возможности чрезвычайно широки, вдобавок его очень часто используют неправильно.

Многие считают, что флаг `--privileged` означает запуск контейнера от имени суперпользователя, но вы ведь уже знаете, что контейнеры по умолчанию

выполняются с `root`-правами. Так какими же еще привилегиями этот флаг наделяет контейнер¹?

Ответ: хотя в Docker процесс выполняется от имени пользователя `root` по умолчанию, значительная группа обычных привилегий пользователя `root` не предоставляется автоматически. (Чтобы освежить в памяти, что такое привилегии, загляните в раздел «Привилегии Linux» главы 2.)

С помощью утилиты `capsh` можно легко посмотреть, какие привилегии получает контейнер, сначала в контейнере без флага `--privileged`, а затем — с ним (далее в примерах я опустила часть вывода для большей ясности).

Для наглядности посмотрим на результат в том случае, когда все привилегии сброшены и затем одна добавлена:

```
$ docker run --rm -it --cap-drop=all --cap-add=CAP_NET_BIND_SERVICE
alpine sh -c 'apk add -U libcap; capsh --print'
```

```
Current: cap_net_bind_service=ep
Bounding set =cap_net_bind_service
```



В этих примерах делается то, от чего я ранее предостерегала: пакет `libcap` устанавливается в среде выполнения. Не делайте этого в рабочих контейнерах!

Этот контейнер обладает — и может обладать — только одной привилегией. Теперь посмотрим, какой набор привилегий предоставляет Docker по умолчанию:

```
$ docker run --rm -it alpine sh -c 'apk add -U libcap; capsh --print'
```

```
Current: cap_chown,cap_dac_override,cap_fowner,cap_fsetid,cap_kill,cap_setgid,
cap_setuid,cap_setpcap,cap_net_bind_service,cap_net_raw,cap_sys_chroot,
cap_mknod,cap_audit_write,cap_setfcap=ep
Bounding set =cap_chown,cap_dac_override,cap_fowner,cap_fsetid,cap_kill,
cap_setgid,cap_setuid,cap_setpcap,cap_net_bind_service,cap_net_raw,
cap_sys_chroot,cap_mknod,cap_audit_write,cap_setfcap
```

Набор привилегий, предоставляемых по умолчанию, выглядит внушительно. Точный список привилегий, предоставляемых без флага `--privileged`, зависит от конкретной реализации. Стандарт OCI определяет набор по умолчанию (<https://oreil.ly/ryVjj>), предоставляемый `runc`.

¹ Режим `privileged` — это не только атрибут лабораторных примеров для демонстрации взлома. В реальной инфраструктуре его могут включать для служебных контейнеров: агентов управления, средств мониторинга, сканеров безопасности и инструментов, работающих с контейнерной средой выполнения. Однако это не означает, что такой режим безопасен или допустим по умолчанию. Использование `privileged` должно быть явно обоснованно и учтено, его нужно регулярно проверять, а по возможности заменять более точечной выдачей необходимых прав. — *Примеч. науч. ред.*

Теперь посмотрим, что получится, если добавить флаг `--privileged`:

```
$ docker run --rm -it --privileged alpine sh -c 'apk add -U libcap; capsh --print'
```

```
Current: =ep
Bounding set =cap_chown,cap_dac_override,cap_dac_read_search,cap_fowner,
cap_fsetid,cap_kill,cap_setgid,cap_setuid,cap_setpcap,cap_linux_immutable,
cap_net_bind_service,cap_net_broadcast,cap_net_admin,cap_net_raw,cap_ipc_lock,
cap_ipc_owner,cap_sys_module,cap_sys_rawio,cap_sys_chroot,cap_sys_ptrace,
cap_sys_pacct,cap_sys_admin,cap_sys_boot,cap_sys_nice,cap_sys_resource,
cap_sys_time,cap_sys_tty_config,cap_mknod,cap_lease,cap_audit_write,
cap_audit_control,cap_setfcap,cap_mac_override,cap_mac_admin,cap_syslog,
cap_wake_alarm,cap_block_suspend,cap_audit_read,cap_perfmon,cap_bpf,
cap_checkpoint_restore
```

Как рассказывалось в главе 2, некоторые инструменты Linux при запуске от имени `root` с полным набором (неявно предоставленных) привилегий отображают пустой список. Именно поэтому текущий набор привилегий (`Current`), который вы видите здесь, пуст. В наборе присвоенных (`Bounding set`) привилегий явно упоминается `CAP_SYS_ADMIN` — всего один этот флаг предоставляет доступ к огромному числу привилегированных действий, включая операции с пространствами имен и монтирование файловых систем. Аналогично флаг `--privileged` добавил привилегии `CAP_NET_ADMIN` (для управления сетевым стеком) и `CAP_BPF` (для загрузки программ eBPF).



Эрик Чан (Eric Chiang) описал в своей статье (<https://oreil.ly/4f4QO>) опасности использования флага `--privileged` и привел пример выхода из контейнера в файловую систему хоста путем монтирования устройства из каталога `/dev` к файловой системе контейнера.

Флаг `--privileged` появился в Docker ради сценария «Docker внутри Docker». Он широко применяется в инструментах сборки и в системах CI/CD, работающих в виде контейнеров, которым требуется доступ к демону Docker для сборки образов контейнеров с помощью Docker. Но, как отмечается в упомянутой статье (<https://oreil.ly/-ULQo>), механизм «Docker в Docker», да и вообще флаг `--privileged`, следует использовать осмотрительно.

Есть и менее очевидная причина, по которой флаг `--privileged` столь опасен, — поскольку люди часто считают, что контейнерам необходимо предоставлять привилегии суперпользователя, то делают и обратный ошибочный вывод: запущенный без этого флага контейнер не является процессом, работающим от имени суперпользователя. Если у вас все еще остались сомнения, вернитесь в раздел «Выполнение контейнеров по умолчанию от имени суперпользователя» этой главы.

Даже если у вас есть основания запускать контейнеры с флагом `--privileged`, я рекомендую вам применять средства контроля или по крайней мере проводить

аудит, чтобы гарантировать использование данного флага только теми контейнерами, которым он действительно нужен. Вместо этого лучше по возможности задавать отдельные привилегии.

Давайте воспользуемся `capable` — инструментом командной строки на основе eBPF, который распространяется в составе пакета `libbpf-tools`, — и с его помощью попробуем отследить события `cap_capable` и посмотреть, какие привилегии данный контейнер запрашивает у ядра. Запустите `capable` в одном терминале и контейнер `nginx` во втором:

```
$ docker run -it --rm nginx
```

После запуска команда `capable` выведет информацию примерно следующего вида (некоторые фрагменты опущены для краткости):

TIME	UID	PID	COMM	CAP	NAME	AUDIT	VERDICT
18:06:24	0	23745	nginx	21	CAP_SYS_ADMIN	1	allow
18:06:24	0	23745	nginx	21	CAP_SYS_ADMIN	1	allow
18:06:24	0	23745	nginx	2	CAP_DAC_READ_SEARCH	1	allow
18:06:24	0	23745	nginx	2	CAP_DAC_READ_SEARCH	1	allow
18:06:24	0	23745	nginx	1	CAP_DAC_OVERRIDE	1	allow
18:06:24	0	23745	nginx	21	CAP_SYS_ADMIN	1	allow
18:06:24	0	23792	nginx	6	CAP_SETGID	1	allow
18:06:24	0	23792	nginx	7	CAP_SETUID	1	allow

Выяснив, какие привилегии нужны контейнеру, вы, следуя принципу минимума привилегий, можете указать в среде выполнения лишь необходимые. Рекомендую сначала убрать все привилегии, а затем добавить обратно только требуемые, вот так:

```
$ docker run --cap-drop=all --cap-add=<npriv1> --cap-add=<npriv2> <образ> ...
```

Теперь вы знаете обо всех опасностях флага `--privileged` и о возможностях, позволяющих точно задать привилегии для контейнеров. Рассмотрим теперь другой способ обойти изоляцию контейнеров — монтирование каталогов с конфиденциальными данными с хоста.

Монтирование каталогов с конфиденциальными данными

С помощью флага `-v` можно смонтировать каталог хоста в контейнер и получить из контейнера доступ к нему. При этом ничто не мешает вам смонтировать корневой каталог хоста:

```
$ touch /ROOT_FOR_HOST
$ docker run -it -v /:/hostroot ubuntu bash
```

```

root@91083a4eca7d:/$ ls /
bin  dev  home  lib  media  opt  root  sbin  sys  usr
boot  etc  hostroot  lib64  mnt  proc  run  srv  tmp  var
root@91083a4eca7d:/$ ls /hostroot/
ROOT_FOR_HOST  etc  lib  media  root  srv  vagrant
bin  home  lib64  mnt  run  sys  var
...

```

В примере используется стандартный образ контейнера Ubuntu, поэтому он запускается от имени `root`. Злоумышленник, взломавший такой контейнер, фактически получает права суперпользователя на хосте и полный доступ ко всей файловой системе.

Монтирование всей файловой системы — пример скорее патологический, но существует и множество других ситуаций разной степени изощренности.

- Монтирование `/etc` позволит модифицировать файл `/etc/shadow` хоста изнутри контейнера, а также вмешиваться в работу заданий `cron`, `init` и `systemd`.
- Монтирование `/bin` и тому подобных каталогов, например `/usr/bin` или `/usr/sbin`, позволит контейнеру записывать исполняемые файлы в каталог хоста, в том числе перезаписывать уже существующие исполняемые файлы.
- Монтирование каталогов с журналами хоста может открыть для злоумышленника возможность изменять журналы, а значит, и скрывать следы своих действий.
- В среде Kubernetes монтирование `/var/log` предоставляет доступ ко всей файловой системе хоста любому пользователю, у которого есть доступ к команде `kubectl logs`. А все потому, что файлы журналов контейнера представляют собой символические ссылки из каталога `/var/log` в прочие места файловой системы, но ничто не мешает контейнеру создать символическую ссылку на любой другой файл. Больше информации об этой интересной разновидности выхода за пределы контейнера можно найти в статье по адресу <https://oreil.ly/gN7no>. Для реализации этого сценария требуется запуск от имени `root`, а значит, данную проблему можно решить с помощью команды `runAsNonRoot`. Также никогда не следует разрешать запись в `/var/log` через `hostPath`.
- В Kubernetes можно пометить операцию монтирования тома как `readOnly`, но будьте осторожны! Если внутри такого каталога на хосте есть другие точки монтирования, то смонтированные в них разделы могут быть доступны для записи. Чтобы гарантировать, что внутри контейнера все смонтированные разделы будут доступны только для чтения, установите параметр `recursiveReadOnly` в значение `enabled`. Данный шаг подробно объясняет Акихиро Суда (Akihiro Suda) в своей статье (<https://oreil.ly/Op3NP>). Согласно документации (<https://oreil.ly/adKhW>), Docker пытается делать это по умолчанию.

Файловая система уязвима, только если контейнер запускается от имени `root` (или другого привилегированного пользователя). Но при применении основных приемов безопасности, таких как запуск от имени рядового пользователя или использование пространств имен, стандартные права доступа к файлам Linux помогут защитить большинство (если не все) конфиденциальных файлов.

Монтирование сокета Docker

В среде Docker всю работу фактически выполняет процесс-демон Docker. При запуске утилиты командной строки `docker` она отправляет демону инструкции через сокет Docker, расположенный в `/var/run/docker.sock`. Любой объект, имеющий доступ к этому сокету, тоже может отправлять инструкции демону Docker. Демон выполняется от имени суперпользователя и способен легко произвести для вас сборку и запуск любого ПО, включая — как вы видели — запуск контейнера от имени суперпользователя на хосте. Следовательно, доступ к сокету Docker, по существу, эквивалентен наличию `root`-прав на хосте.

Сокет Docker очень часто монтируют в системах непрерывной интеграции (CI) наподобие Jenkins, где он необходим для отправки Docker инструкций по запуску сборки образов, в виде составной части конвейера. Это вполне допустимое его применение, которое, однако, создает потенциальное уязвимое место, на радость злоумышленнику. Пользователь, у которого есть возможность изменять `Jenkinsfile`, может заставить Docker выполнять нужные ему команды, в том числе предоставляющие пользователю доступ к кластеру с `root`-правами. Поэтому крайне не рекомендуется использовать в рабочей среде конвейер CI/CD, монтирующий сокет Docker.

Совместное использование пространств имен контейнером и его хостом

Иногда возникают основания использовать в контейнере те же пространства имен, что и его хост. Допустим, процессу в контейнере Docker нужен доступ к информации о процессе с хоста. В Docker это можно сделать с помощью параметра `--pid=host`.

Напомню, что все контейнеризированные процессы видны на хосте; следовательно, совместное использование пространства имен процессов с контейнером позволяет ему видеть другие контейнеризированные процессы. Следующий пример начинается с выполнения длительной операции `sleep` внутри одного контейнера; данный процесс оказывается виден из другого контейнера, запущенного с параметром `--pid=host`:

```
$ docker run --name sleep --rm -d alpine sleep 1000
fa19f51fe07fca8d60454cf8ee32b7e8b7b60b73128e13f6a01751c601280110
$ docker run --pid=host --name alpine --rm -it alpine sh
$ ps | grep sleep
30575 root      0:00 sleep 1000
30738 root      0:00 grep sleep
```

Еще более поразительный факт: выполнение `kill -9 <pid>` из второго контейнера позволяет прервать `sleep` в первом!

Ранее я показывала несколько вариантов того, как совместное использование пространств имен или томов контейнерами (либо контейнером и его хостом) может ослабить изоляцию контейнера и нарушить безопасность. Но это никоим образом не означает, что обмен информацией с контейнерами — *всегда* плохая идея. В завершение главы рассмотрим `sidecar`-контейнеры, которые не случайно используются так широко.

Sidcar-контейнеры

Sidcar-контейнеры намеренно предоставляется доступ к одному из пространств имен контейнера приложения для делегирования ему части функций данного приложения. В микросервисной архитектуре встречается функциональность, которую необходимо применять повторно во всех микросервисах системы, поэтому нередко подобную функциональность помещают в образы `sidecar`-контейнеров для удобства повторного использования.

До появления `sidecar`-контейнеров можно было реализовать многократно используемый код в виде библиотек. Такой код легко импортировать в приложения, написанные на одном языке, но ситуация осложняется, если команды разработчиков пишут приложения на нескольких языках¹. `Sidcar`-контейнеры решают эту проблему: поскольку функциональность упакована в контейнер, она может быть реализована на любом языке, как показано на рис. 11.1. Кроме того, `sidecar`-контейнеры допускается использовать для дооснащения контейнеров, предоставляемых сторонними разработчиками.

Вот несколько распространенных примеров.

- Сервисная сетка (`service mesh`) использует `sidecar`-контейнеры, которые берут на себя сетевые функции от имени контейнера приложения. Сервисная сетка может, например, обеспечить использование во всех сетевых подклю-

¹ В принципе, библиотеки, написанные на одном языке, можно повторно использовать в программах, написанных на других языках, если имеются специализированные средства, реализующие интерфейс для целевых библиотек.

чениях протокола TLS. Делегирование этой функциональности sidecar-контейнеру означает, что всегда, когда основной контейнер разворачивается вместе со вспомогательным, он будет устанавливать только защищенные TLS-соединения — командам разработчиков не придется тратить время и включать эту возможность в код приложения. (Дальнейшее обсуждение сервисных сеток вы найдете в разделе «Сервисная сетка» главы 12.)

- Sidecar-контейнеры для мониторинга позволяют задавать параметры журналирования, трассировки и сбора метрик. Например, Prometheus (<https://oreil.ly/Jn10W>) и OpenTelemetry (<https://oreil.ly/0HwpE>) поддерживают sidecar-контейнеры для экспорта данных наблюдений.
- Sidecar-контейнеры безопасности позволяют контролировать, какие исполняемые файлы и сетевые подключения разрешены внутри контейнера приложения (например, см. мою статью в блоге (<https://oreil.ly/oHAEk>) про обеспечение безопасности контейнеров AWS Fargate с помощью MicroEnforcer от компании Aqua в sidecar-контейнерах).

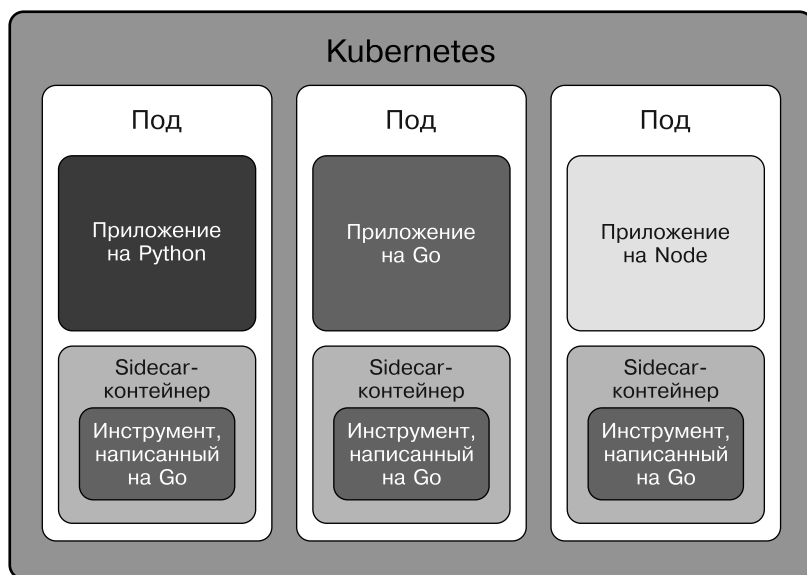


Рис. 11.1. Sidecar-контейнеры позволяют использовать инструменты, написанные на другом языке

Это лишь некоторые примеры применения sidecar-контейнеров, которые на вполне законных основаниях используют общие пространства имен с контейнерами приложений.

Развертывание sidecar-контейнеров

В Kubernetes sidecar-контейнеры получили широкое распространение: спецификация пода может включать несколько контейнеров, которые используют общее сетевое пространство имен, общие тома, а также планируются и перезапускаются одновременно. Многие инструменты, основанные на sidecar-контейнерах, такие как Istio, Linkerd и Vault, изменяют спецификацию пода приложения в процессе развертывания благодаря объекту `MutatingAdmissionWebhook`. Он перехватывает запросы на создание подов и добавляет определение sidecar-контейнера в спецификацию пода.

AWS ECS и Fargate тоже поддерживают sidecar-контейнеры, поскольку они настраиваются с помощью определений задач `TaskDefinition`, которые могут включать несколько контейнеров. Они создаются с общим сетевым пространством имен и способны совместно использовать тома файловой системы.

В Docker нет встроенной поддержки sidecar-контейнеров, однако есть возможность добиться чего-то подобного с помощью Docker Compose для определения нескольких контейнеров с общим сетевым пространством имен.

Ограничения sidecar-контейнеров

Sidecar-контейнеры — полезный инструмент для внедрения функциональности, но и они имеют свои недостатки. Совершенно очевидно, что в этой модели увеличивается количество развертываемых контейнеров — один sidecar-контейнер на каждый обычный контейнер приложения — потребляющих дополнительные ресурсы. Потребление ресурсов увеличивается еще и потому, что контейнеры и поды сознательно изолируются друг от друга и обмен информацией между ними намеренно затруднен; поэтому sidecar-контейнерам требуются собственные копии общих конфигурационных данных, таблиц маршрутизации и т. п.

Жизненный цикл sidecar-контейнера тесно связан с контейнером рабочей нагрузки, и если нужно обновить sidecar-контейнер, то придется перезапустить и основной контейнер. Иногда могут возникать проблемы, связанные с порядком запуска и инициализации sidecar-контейнеров и контейнеров приложений, что усложняет использование инструментов из sidecar-контейнеров в рабочей среде. Эти ограничения подтолкнули отрасль к переходу на модели без sidecar-контейнеров для инфраструктурных инструментов, например с использованием eBPF для дооснащения всей (виртуальной) машины на уровне ядра. К инструментам на основе eBPF, особенно в контексте безопасности контейнеров, мы вернемся в главе 15.

Контейнеры для отладки

В Kubernetes *контейнеры для отладки* (debug containers) — это временные контейнеры, которые можно подключить к работающему поду с помощью команды `kubectl debug`. Они используются для устранения неполадок или диагностики.

Такие контейнеры могут быть очень полезными, но, как вы наверняка догадались, они предоставляют возможность выполнения произвольного кода, поэтому ими легко злоупотребить. К ним следует относиться как к привилегированным инструментам, подверженным высокому риску. Если решите задействовать их в рабочем кластере, то убедитесь, что доступ к ним имеют только доверенные пользователи, обязательно используйте соответствующие средства контроля доступа и обеспечьте тщательное ведение журналов аудита. Нелишним будет предусмотреть автоматическое удаление таких контейнеров через определенный период времени.

Резюме

В этой главе мы рассмотрели несколько вариантов неудачных настроек, которые приводят к нарушению изоляции контейнеров.

Все конфигурационные параметры существуют не просто так. Например, монтирование каталогов хоста в контейнер может оказаться исключительно полезным и иногда может пригодиться возможность запуска контейнера от имени суперпользователя или даже с дополнительными привилегиями, предоставляемыми флагом `--privileged`. Однако если вас волнует безопасность, то лучше прибегать к этим потенциально небезопасным возможностям как можно реже и применять утилиты для выявления случаев их использования.

Если вы работаете в мультиарендной среде, следует быть особенно внимательными к контейнерам с подобными потенциально небезопасными настройками. Любой контейнер с флагом `--privileged` будет иметь полный доступ ко всем прочим контейнерам на том же хосте, вне зависимости от довольно несерьезных средств контроля, например от того, работают ли они в одном или разных пространствах имен Kubernetes.

В разделе «Sidecar-контейнеры» я упоминала контейнеры, реализующие поддержку сервисной сетки, которым можно делегировать часть сетевой функциональности. Теперь самое время обсудить передачу данных в контейнерах по сети.

Сетевая безопасность контейнеров

Внешние атаки на развертываемую систему производятся по сети, поэтому, чтобы обеспечить безопасность приложений и данных, необходимо иметь хотя бы базовые представления о сетевых технологиях. Мы не станем обсуждать эту сложную тему во всех подробностях (иначе книга оказалась бы намного больше), но я опишу главные элементы продуманной ментальной модели, которую можно использовать для организации передачи данных по сети при развертывании контейнеров.

Начну с обзора межсетевых экранов для контейнеров, позволяющих реализовывать сетевую безопасность на основе намного более детализированного подхода, по сравнению с традиционными программами фильтрации трафика.

Далее вас ждет обзор семиуровневой сетевой модели, о которой необходимо знать, чтобы понимать, на каком уровне работает тот или иной механизм сетевой безопасности. После этого мы обсудим, как реализуется сетевая безопасность контейнеров с использованием нескольких подходов. Попутно рассмотрим сетевые политики Kubernetes на разных уровнях сетевой модели и некоторые приемы применения этих политик.

Межсетевые экраны для контейнеров и микросегментация

Контейнеры часто сочетаются с микросервисной архитектурой, в которой приложения разбиваются на маленькие компоненты, развертываемые независимо друг от друга. С точки зрения безопасности преимущества такого подхода несомненны, ведь для небольшого компонента намного легче определить «нормальное» поведение. Каждый контейнер при этом, вероятно, будет взаимодействовать лишь с ограниченным количеством других контейнеров, и только части контейнеров нужен будет доступ к внешнему миру.

Например, рассмотрим разбитое на микросервисы приложение интернет-магазина. Один из его компонентов может отвечать за поиск товаров: принимать запросы от конечных пользователей и производить поиск в базе товаров.

Контейнерам, составляющим этот микросервис, не требуется обмениваться информацией, скажем, с платежным шлюзом. Рисунок 12.1 иллюстрирует данный пример.

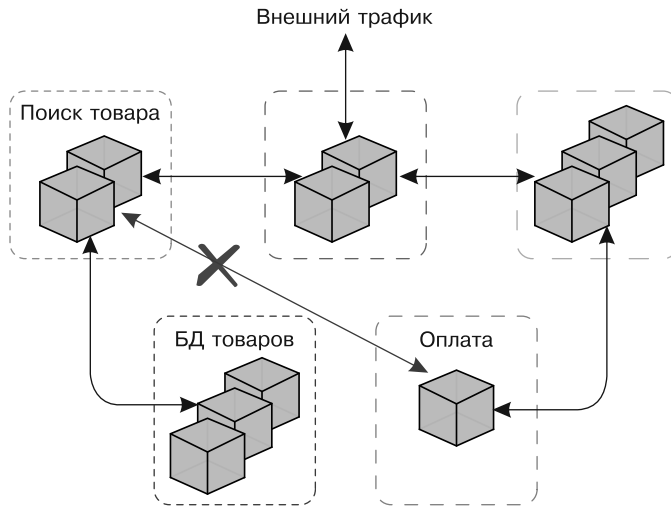


Рис. 12.1. Микросегментация

Межсетевые экраны для контейнеров могут ограничивать трафик, поступающий в набор контейнеров и исходящий из него. В оркестраторах, например Kubernetes, термин «межсетевой экран для контейнера» используется редко, обычно говорят о *микросегментации*, когда сетевые политики определяются с точки зрения рабочих нагрузок и сервисов, а не связей между отдельными контейнерами. В обоих случаях идея заключается в ограничении входящего/исходящего сетевого трафика контейнера так, чтобы он мог передаваться только разрешенным узлам.

В дистрибутиве Kubernetes с открытым исходным кодом отсутствуют встроенные средства организации сетевого взаимодействия — вместо них используется интерфейс, называемый интерфейсом сетей контейнеров (Container Network Interface, CNI). Существует несколько проектов и продуктов, которые могут обеспечить необходимую сетевую функциональность на другой стороне этого интерфейса. Они известны как плагины CNI или просто CNI. Для ограничения трафика между рабочими нагрузками Kubernetes реализует концепцию сетевых политик (Network Policies), хотя, как вы увидите далее в этой главе, *CNI не требуется, чтобы обеспечить соблюдение этих политик*. Однако, если вы хотите, чтобы они работали, вам нужно выбрать подходящий CNI!

В Docker отсутствует встроенная концепция сетевых политик, за исключением определения отдельных (виртуальных) сетей для изоляции контейнеров друг от друга, хотя существуют коммерческие решения, ориентированные на возможности межсетевых экранов для контейнеров без использования Kubernetes.

Инструменты сетевой безопасности контейнеров, такие как традиционные межсетевые экраны, также обычно уведомляют о попытках установить не разрешенные правилами соединения, тем самым предоставляя полезную информацию для дальнейшего расследования возможных атак.

Их можно использовать вместе с другими утилитами сетевой безопасности, которые встречаются и при обычном варианте развертывания системы. Например:

- среду выполнения контейнеров нередко развертывают в VPC, что изолирует хосты от остального мира;
- с помощью межсетевого экрана можно контролировать входящий и исходящий трафик всего кластера;
- ограничивать трафик на уровне 7 можно с помощью API-шлюзов, или межсетевых экранов веб-приложений (Web Application Firewall, WAF);
- можно шифровать сетевой трафик, чтобы исключить возможность его перехвата (см. главу 13).

Ни один из этих подходов не нов. В сочетании со средствами безопасности, учитывающими специфику контейнеров, они обеспечивают дополнительный уровень защиты.

Прежде чем перейти к нюансам реализации защиты контейнеров с применением межсетевых экранов, вспомним устройство семиуровневой сетевой модели и проследим путь IP-пакетов по сети.

Сетевая модель OSI

Сетевая модель взаимодействия открытых систем (Open Systems Interconnection, OSI), опубликованная в 1984 году, описывает многоуровневую модель обмена информацией по сети, которую и сегодня часто упоминают, хотя, как вы видите из рис. 12.2, не у всех из ее семи уровней есть эквиваленты в IP-сетях.

Уровень 7	Прикладной	HTTP
Уровень 6	Представления	
Уровень 5	Сеансовый	
Уровень 4	Транспортный	TCP
Уровень 3	Сетевой	IP
Уровень 2	Канальный	Например, Ethernet
Уровень 1	Физический	Поток битов

Рис. 12.2. Модель OSI

Вот основные уровни IP-сетей.

- Уровень 7 — прикладной (уровень приложений). Представьте приложение, которое выполняет веб-запрос или отправляет запрос к RESTful API, — и поймете, что происходит на уровне 7. Такой запрос обычно направляется по URL, и в целях его маршрутизации к месту назначения доменное имя преобразуется в IP-адрес с помощью протокола разрешения доменных имен (Domain Name Resolution), реализуемого сервисом доменных имен (Domain Name Service, DNS).
- Уровень 4 — транспортный. Обычно данные передаются по нему в виде TCP или UDP-пакетов. На этом уровне используются номера портов.
- Уровень 3 — уровень, на котором перемещаются IP-пакеты и работают IP-маршрутизаторы. Каждой IP-сети выделяется множество IP-адресов, и, когда контейнер присоединяется к ней, ему назначается один из этих IP-адресов. В данной главе нам неважно, используется ли в IP-сети протокол IPv4 или IPv6, — можете считать это несущественным нюансом реализации.
- Уровень 2 — здесь пакеты данных направляются к конечным точкам, соединенным с физическим или виртуальным интерфейсом (которые мы обсудим чуть позже). Существует несколько протоколов уровня 2, включая Ethernet, WiFi и, если мысленно перенестись в прошлое, Token Ring. (WiFi тут немного все запутывает, поскольку охватывает как уровень 2, так и уровень 1.) В этой главе я буду рассматривать только протокол Ethernet — именно он в основном используется для передачи данных по сети в контейнерах. На уровне 2 интерфейсы идентифицируются с помощью MAC-адресов¹.
- Уровень 1 называется физическим, хотя учтите, что интерфейсы на этом уровне могут быть и виртуальными. У физического компьютера есть физическое сетевое устройство, к которому подключен кабель или беспроводной передатчик. В разделе «Знакомство с VMM» главы 5 отмечалось, что VMM предоставляет гостевому ядру доступ к виртуальным устройствам, которые отображаются в реальные устройства. При наличии сетевого интерфейса, скажем, на виртуальном узле EC2 в AWS вы получаете доступ к одному из этих виртуальных интерфейсов. Сетевые интерфейсы контейнеров на уровне 1 тоже обычно виртуальные. Подключаясь к сети, контейнер получает для нее интерфейс уровня 1.



Более подробное объяснение можно найти в описании сетевой модели OSI от Cloudflare (<https://oreil.ly/VADPU>).

¹ Аббревиатура MAC в контексте сетевых технологий расшифровывается как Media Access Control («управление доступом к среде передачи»), однако для себя ее вполне можно воспринимать как обозначение аппаратного адреса; не путайте с Mandatory Access Control («обязательный контроль доступа»).

Теперь посмотрим, что происходит на каждом из этих уровней, когда приложение хочет отправить сообщение.

Отправка IP-пакета

Представьте приложение, которое собирается отправить сообщение на целевой URL. Поскольку речь идет о приложении, то из предыдущего определения следует, что это происходит на уровне 7.

Первый шаг — поиск в DNS IP-адреса, соответствующего имени хоста в этом URL. Определение адреса может осуществляться локально (например, с использованием файла `/etc/hosts` на вашем ноутбуке) или путем обращения к удаленному сервису DNS, расположенному по заданному в настройках IP-адресу. (Если приложению известен IP-адрес получателя сообщения, а не URL, то этап с DNS пропускается.)

Получив целевой IP-адрес, на уровне 3 сетевого стека принимается решение о маршрутизации, состоящее из двух частей.

1. От целевого пункта нас может отделять несколько транзитных узлов IP-сети. Зная целевой IP-адрес, нужно определить IP-адрес следующего транзитного узла.
2. Нужно определить, какой сетевой интерфейс соответствует IP-адресу этого следующего транзитного узла.

Далее пакет необходимо преобразовать в кадры Ethernet, а IP-адрес следующего транзитного участка сопоставить с соответствующим MAC-адресом. Для этого используется протокол разрешения адресов (Address Resolution Protocol, ARP), который соотносит IP-адреса и MAC-адреса. Если сетевому стеку еще неизвестен MAC-адрес, который соответствует следующему IP-адресу (он может уже содержаться в кэше ARP), то он определяется с помощью ARP.

После того как сетевой стек получит MAC-адрес следующего транзитного узла, сообщение отправляется через интерфейс. В зависимости от реализации сети для этого может использоваться прямое соединение, либо интерфейс может подключаться к *мосту* (bridge).

Чтобы понять, что такое мост, представьте физическое устройство, к которому подключено несколько Ethernet-кабелей. Другой конец каждого кабеля подключен к сетевой карте устройства, например компьютера. Физической сетевой карте соответствует уникальный MAC-адрес, зашитый в нее производителем. Мост выясняет MAC-адреса всех устройств, расположенных на концах кабелей, подключенных к его интерфейсу. Все подключенные к мосту устройства

могут отправлять через него друг другу пакеты. При передаче данных по сети между контейнерами мосты реализованы программно, а не аппаратно, а роль Ethernet-кабелей играют виртуальные Ethernet-интерфейсы. Таким образом, сообщение поступает на мост, который на основе MAC-адреса следующего транзитного участка определяет, в какой интерфейс его направить.

Когда сообщение достигает другого конца Ethernet-соединения, IP-пакет извлекается и переводится обратно на уровень 3. Данные инкапсулируются с заголовками на различных уровнях сетевого стека, как показано на рис. 12.3.

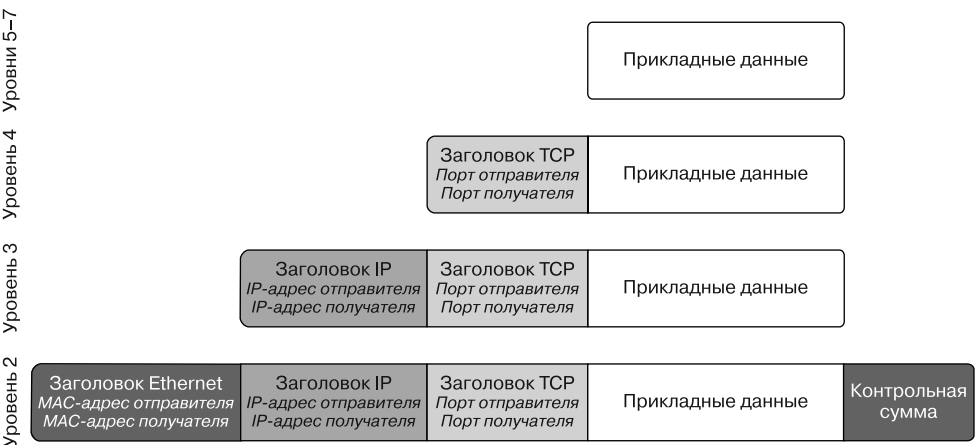


Рис. 12.3. Сетевые заголовки

Если это конечный пункт назначения пакета, то данные передаются приложению-получателю. Впрочем, если для пакета это всего лишь очередной транзитный узел, то сетевому стеку приходится снова принимать решение о том, куда отправлять пакет далее.

В данном пояснении опущены некоторые детали (например, механизм работы ARP и способ определения IP-адреса следующего транзитного узла), но этого должно быть достаточно для понимания передачи данных по сети в контейнерах.

IP-адреса контейнеров

В предыдущем разделе мы говорили о том, как трафик достигает места назначения на основе IP-адреса. IP-адреса контейнеров могут совпадать с IP-адресами их хостов, но для них может применяться и отдельный сетевой стек, работающий в их собственном пространстве имен сети. О таких пространствах мы говорили в главе 4. Поскольку велика вероятность, что вы запускаете свои

контейнеры под управлением Kubernetes, посмотрим, как IP-адреса используются в Kubernetes.

У каждого пода в Kubernetes свой IP-адрес. Если под содержит более одного контейнера, то, как вы понимаете, все они используют один и тот же IP-адрес, благодаря тому что делят одно и то же пространство имен. Каждому из узлов выделяется свой диапазон адресов (блок CIDR), а когда под распределяется на узел, то ему присваивается один из адресов этого диапазона.



Стоит отметить, что узлам далеко не всегда заранее выделяются диапазоны адресов. Например, в AWS подключаемый модуль управления IP-адресами динамически распределяет IP-адреса по подам Kubernetes, из соответствующего VPC диапазона.

Kubernetes требует, чтобы поды в кластере могли связываться друг с другом напрямую, без какого-либо преобразования сетевых адресов (Network Address Translation, NAT). В некоторых случаях NAT отображает IP-адреса таким образом, что отправитель видит IP-адрес получателя, который на самом деле не является фактическим адресом целевого устройства. (Именно благодаря этому IPv4 используется гораздо дольше, чем планировалось изначально. Хотя количество устройств, адресуемых по протоколу IP, намного превысило количество адресов, доступных в пространстве адресов IPv4, NAT позволяет повторно задействовать большинство IP-адресов в частных сетях.) В Kubernetes стратегии безопасности сети и разбиение на сегменты могут мешать подам взаимодействовать друг с другом, но если уж два пода обмениваются информацией, они видят IP-адреса друг друга напрямую, без какого-либо преобразования NAT. Однако NAT между подами и внешним миром вполне возможен.

Сервисы Kubernetes представляют собой некую разновидность NAT. Сервис Kubernetes — самостоятельный ресурс, которому присваивается свой IP-адрес. Впрочем, это всего лишь IP-адрес — у такого сервиса нет никаких интерфейсов, он не принимает и не отправляет трафик. Этот адрес используется только для маршрутизации. Каждому сервису соответствует несколько подов, фактически выполняющих вместо него всю работу, поэтому пакеты, посылаемые на IP-адрес сервиса, приходится пересылать одному из этих подов. Вскоре мы увидим, как это происходит.



В традиционных сетевых технологиях наряду с NAT вам может встретиться термин PAT (port address translation — «преобразование адресов портов»). В мире облачных технологий и контейнеров чаще говорят о сопоставлении портов, когда номер порта хоста преобразуется в номер порта внутри контейнера.

Сетевая изоляция

На всякий случай уточню, что компоненты могут взаимодействовать друг с другом, только если подключены к одной сети. В обычных средах на основе хостов можно было изолировать приложения друг от друга, размещая их в разных виртуальных локальных сетях.

В мире контейнеров Docker значительно упрощает настройку большого количества сетей благодаря команде `docker network`, однако это плохо вписывается в модель Kubernetes, где любой под может (если не учитывать сетевые стратегии и настройки безопасности) обращаться по IP-адресу к другому поду.

Кроме того, обратите внимание: в Kubernetes управляющие компоненты запускаются в подах и подключаются к той же сети, что и прикладные поды. Если вы пришли в Kubernetes из телекоммуникационной отрасли, это может вас удивить: там десятилетиями стремились отделить плоскость управления (control plane) от плоскости данных (data plane).

В Kubernetes же передача данных по сети между контейнерами основывается на сетевых политиках (<https://oreil.ly/sHh4p>), работающих на уровнях 3 и 4, а также на уровне 7 в зависимости от используемого сетевого плагина. Сначала я опишу традиционный подход, а затем объясню, почему eBPF, с которым вы познакомились в главе 9, является лучшим вариантом реализации.

Маршрутизация на уровнях 3, 4 и правила

Как вы уже знаете, основная задача маршрутизации на уровне 3 — выбор следующего транзитного узла для IP-пакета. Это решение основывается на наборе правил, которые описывают, какой интерфейс следует использовать для достижения тех или иных адресов. Но это лишь малая часть возможностей правил уровня 3: на нем можно, например, игнорировать пакеты или выполнять различные операции с IP-адресами, скажем, в целях реализации балансировки нагрузки, NAT, межсетевых экранов и стратегий безопасности сети. Правила также могут применяться на уровне 4 для учета номеров портов.

Традиционно эти правила опираются на механизм ядра Linux под названием `netfilter` (<https://netfilter.org/>), но в настоящее время большинство развертываний используют или переходят к реализациям на основе eBPF, о чем вы узнаете позже в этой главе.

Механизм `netfilter` — это система фильтрации пакетов, впервые появившаяся в версии 2.4 ядра Linux. Здесь используется набор правил, описывающий, что

делать с пакетом в зависимости от его адреса отправителя и адреса получателя. Настраивать правила **netfilter** в пользовательском пространстве можно несколькими способами, из которых наибольшее распространение получила утилита **iptables**.

Утилита **iptables**

Утилита **iptables** — это традиционный способ настройки правил обработки IP-пакетов, которые обрабатываются ядром с помощью **netfilter**. Существует несколько типов таблиц, однако в контексте сетевого взаимодействия контейнеров наибольший интерес представляют **filter** и **nat**:

- таблица **filter** используется, чтобы определить, что делать с пакетом — отбросить его или передать далее;
- таблица **nat** отвечает за преобразования адресов.

Суперпользователь может просмотреть текущий набор правил конкретного вида через команду **iptables -t <тип таблицы> -L**.

Правила **netfilter**, задаваемые с помощью **iptables**, очень удобны для обеспечения безопасности. Утилита **iptables** используется на нескольких межсетевых экранах для контейнеров, а также в сетевых плагидах Kubernetes для формирования хитроумных правил сетевых политик, реализуемых через правила **netfilter**. Мы еще вернемся к этому вопросу в разделе «Сетевые политики» далее в этой главе. Но сначала займемся правилами, которые задаются с применением **iptables**.

Как упоминалось ранее, сервис в Kubernetes — это абстракция, которая сопоставляет имя сервиса с набором подов. На каждом узле есть компонент *kube-proxy*, управляющий балансировкой нагрузки трафика между всеми подами, предоставляющими сервис, используя правила **iptables**. Клиент может отправить запрос, указав доменное имя сервиса, которое разрешается в IP-адрес с помощью DNS. Когда поступает пакет, предназначенный для этого IP-адреса сервиса, срабатывает правило **iptables**, сопоставляющее адрес получателя и заменяющее его на адрес одного из подов.

Если набор подов для этого сервиса меняется, то правила **iptables** перезаписываются соответствующим образом на всех хостах. К сожалению для поклонников **iptables**, в среде Kubernetes поды динамически появляются и исчезают, и эта перезапись правил оказалась узким местом, что в итоге привело к внедрению более совершенных технологий, таких как eBPF. Тем не менее знакомство с некоторыми правилами **iptables** может помочь вам понять логику балансировки нагрузки и сетевых политик *kube-proxy*, поэтому разберем их подробнее.

Просмотреть правила `iptables` для сервиса довольно просто. Рассмотрим кластер Kubernetes, в котором используется две реплики `nginx`, соответствующие сервису (для наглядности я удалила некоторые поля из вывода):

```
$ kubectl get svc,pods -o wide
NAME                                TYPE             CLUSTER-IP      PORT(S)
service/kubernetes                  ClusterIP         10.96.0.1        443/TCP
service/my-nginx                    NodePort          10.100.132.10    8080:32144/TCP

NAME                                READY    STATUS    IP
pod/my-nginx-75897978cd-n5rdv      1/1      Running   10.32.0.4
pod/my-nginx-75897978cd-ncnfk      1/1      Running   10.32.0.3
```

Взглянуть на текущие правила преобразования адресов можно с помощью команды `iptables -t nat -L`. Вероятно, при этом будет выведено немало информации, но найти в ней интересующие нас части, которые относятся к `nginx`, не так уж сложно. Для начала рассмотрим правило, соответствующее сервису `my-nginx`, который запущен по IP-адресу `10.100.132.10`. Как видите, оно входит в цепочку `KUBE-SERVICES`, что логично, поскольку оно относится к сервису:

```
Chain KUBE-SERVICES (2 references)
target                prot opt source      destination
...
KUBE-SVC-SV7AMNAGZFKZEMQ4 tcp -- anywhere   10.100.132.10 /* default/my-
nginx:http cluster IP */ tcp dpt:http-alt
...
```

Это правило задает целевую цепочку, встречающуюся далее в правилах `iptables`:

```
Chain KUBE-SVC-SV7AMNAGZFKZEMQ4 (2 references)
target                prot opt source      destination
KUBE-SEP-XZGVMMRRSKK6PWWN all -- anywhere   anywhere      statistic mode
random probability 0.500000000000
KUBE-SEP-PUXUHP3DTPPX72C all -- anywhere   anywhere
```

По фрагменту `random probability 0.5` можно предположить, что трафик равномерно распределяется между двумя получателями. Логика становится понятной, когда мы видим, что для этих получателей есть правила, соответствующие IP-адресам подов (`10.32.0.3` и `10.32.0.4`):

```
Chain KUBE-SEP-XZGVMMRRSKK6PWWN (1 references)
target                prot opt source      destination
KUBE-MARK-MASQ        all -- 10.32.0.3    anywhere
DNAT                  tcp -- anywhere     anywhere      tcp to:10.32.0.3:80
...
Chain KUBE-SEP-PUXUHP3DTPPX72C (1 references)
target                prot opt source      destination
KUBE-MARK-MASQ        all -- 10.32.0.4    anywhere
DNAT                  tcp -- anywhere     anywhere      tcp to:10.32.0.4:80
```

Основная проблема при работе с `iptables` заключается в падении производительности по мере того, как система записывает и перезаписывает наборы сложных правил. Фактически использование `iptables` в `kube-proxy` было определено как узкое место в производительности, которое дает о себе знать при масштабировании Kubernetes. В одной из публикаций (<https://oreil.ly/xOyqb>) отмечается, что 2000 сервисов по 10 подов каждый означает 20 000 дополнительных правил `iptables` на каждом узле.

Одной из реализаций, которые могли бы помочь решить эту проблему, был IPVS (IP Virtual Server — программный коммутатор 4 уровня модели OSI). При использовании в `kube-proxy` он показал более высокую производительность (сравнение производительности `iptables` и IPVS вы найдете на сайте Project Calico (<https://oreil.ly/xGOON>)). Однако IPVS так и не получил широкого распространения по ряду причин: для работы ему нужны модули ядра `ip_vs` (которые могут быть ненадежными), он не был принят поставщиками облачных сервисов, а экосистема инструментов, привычных для многих операторов, которые используют `iptables`, была с ним совместима.

Другим более современным подходом является использование `nftables`, и даже существует проект по его использованию для увеличения производительности `kube-proxy` (<https://oreil.ly/ygT5->). Однако сообщество Kubernetes уже перешло к решениям на основе eBPF, которые могут предложить гораздо больше, чем простые правила фильтрации и балансировка нагрузки, имея при этом лучшую производительность, наблюдаемость и управление политиками.



Тема производительности выходит за рамки книги, но, если вы хотите увидеть, как eBPF обеспечивает производительность сетей контейнеров, сопоставимую с сетью хоста, настоятельно рекомендую посмотреть выступление Даниэля Боркмана (Daniel Borkmann) под названием *Turning Up Performance to 11* (<https://www.youtube.com/watch?v=AVEBcZc0YsQ>).

Все эти подходы — IPVS, `iptables`, `nftables` и eBPF — имеют одну общую черту: они работают внутри ядра. Напомню, что ядро является общим для всех контейнеров на хосте, а это значит, что при использовании этих механизмов для реализации политик безопасности они применяются на уровне хоста, а не внутри каждого контейнера.

Как было сказано в главе 9, eBPF используется многими инструментами инфраструктуры — не только в сетевых технологиях, но и для обеспечения наблюдаемости, безопасности и многого другого. Я показала вам правила `iptables` для балансировки нагрузки сервиса `nginx` между двумя подами. Давайте теперь посмотрим, как выглядит аналогичная реализация на основе eBPF.

eBPF

Сетевой плагин Cilium (<https://cilium.io>; проект, развиваемый под эгидой CNCF) изначально разрабатывался с использованием eBPF. Плагин Calico (<https://github.com/projectcalico/calico>) тоже имеет вариант на основе eBPF. В этом примере я буду использовать Cilium, однако в Calico используется тот же подход к реализации политик на уровне 3 и 4.

При использовании Cilium в kube-проху отсутствуют правила `iptables` для балансировки нагрузки сервисов. Вместо этого плагин Cilium устанавливает программы eBPF непосредственно в ядро, которые перехватывают сетевые пакеты. Он также поддерживает в ядре структуры данных, называемые *eBPF-картами*, где хранится такая информация, как соответствие IP-адресов и портов сервисов подам.

Когда под отправляет сетевой пакет в сервис, eBPF-программа, установленная плагином Cilium, выполняет следующие действия.

1. Перехватывает пакет внутри сетевого стека и определяет IP-адрес и порт получателя.
2. Выполняет поиск сервиса в eBPF-карте, чтобы определить поды, поддерживающие этот сервис.
3. Выбирает под, используя алгоритм балансировки нагрузки (например, случайный, циклический, Maglev).
4. Подставляет в пакет IP-адрес и порт получателя, соответствующие выбранному поду.
5. Отправляет пакет.

Все это происходит исключительно внутри ядра без выполнения дорогостоящих переходов в пользовательское пространство.

Вместо проверки правил `iptables` для оценки работы балансировки нагрузки сервисов вы можете изучить данные из eBPF-карты, созданной Cilium, используя утилиту командной строки `cilium-dbg`. Например:

```
$ cilium-dbg bpf lb list
```

SERVICE ADDRESS	BACKEND ID	BACKEND ADDRESS
10.100.132.10:8080	1	10.32.0.3:80
	2	10.32.0.4:80

Теперь, когда у вас есть представление об обработке сетевых пакетов с помощью `iptables` и программ eBPF, посмотрим, как эти технологии используются для реализации сетевых политик безопасности.

Сетевые политики

Сетевые политики в Kubernetes определяют, какой трафик может поступать в различные поды и исходить от них. Если политика запрещает отправку/получение конкретного сообщения, то сеть должна либо отказать в установлении соединения, либо игнорировать пакеты, которые соответствуют данному сообщению. В примере с приложением для электронной коммерции, рассмотренном в начале главы, одна из политик могла бы запрещать трафик, посылаемый контейнером поиска товаров в сервис платежей.

Политики, применяемые на уровнях 3 и 4, могут определяться в контексте портов, IP-адресов или, что встречается чаще, на основе меток, присваиваемых подам. (К политикам прикладного уровня мы вернемся чуть позже.)

В традиционной среде приложение обычно устанавливается на компьютер, и затем IP-адрес этого компьютера служит для идентификации приложения. Если на одной машине работает несколько приложений, они будут использовать разные порты на уровне 4. В таких средах применение политик на основе IP-адресов и портов выглядит логично. Однако в мире Kubernetes поды часто существуют очень недолго, а назначенные им IP-адреса могут повторно использоваться при удалении и создании подов. Таким образом, становится невозможно однозначно идентифицировать рабочую нагрузку по IP-адресу и номеру порта.

Однако в Kubernetes есть другой механизм идентификации — метки, которые могут использоваться для самых разных целей, например для определения связи между подом и сервисом или для идентификации приложения. В более общем случае метки позволяют идентифицировать рабочую нагрузку, выполняемую внутри данного пода. Сетевые политики определяют правила, разрешающие или запрещающие трафик между рабочими нагрузками и, возможно, другими объектами, таким как получатели за пределами кластера.

Ниже представлен простой объект **NetworkPolicy**, который разрешает доступ к сервису **my-nginx** только подам с меткой **access=true**:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: access-nginx
spec:
  podSelector:
    matchLabels:
      app: my-nginx
  ingress:
    - from:
      - podSelector:
          matchLabels:
            access: "true"
```

Принудительное применение сетевых политик осуществляется не основным компонентом Kubernetes, а сетевым плагином (<https://oreil.ly/VmVCc>). При этом в качестве механизма могут быть задействованы `iptables`, `eBPF` или даже что-то другое (в будущем)!

Реализация политики на уровнях 3 и 4 с использованием `iptables`

Создание такой сетевой политики с использованием сетевого плагина Weave приводит к добавлению следующего правила `iptables` в таблицу `filter`:

```
Chain WEAVE-NPC-INGRESS (1 references)
target     prot opt source                destination
ACCEPT     all  --  anywhere              anywhere           match-set weave-{U;]TI.1|Md
RzDhN7$NRn[t)d src match-set weave-vC070kAfB$if8}PFMX{V9Mv2m dst
/* pods: namespace: default, selector: access=true ->
pods: namespace: default, selector: app=my-nginx (ingress) */
```

Правило `match-set` не предназначено для чтения человеком, однако комментарий (между `/*` и `*/`) вполне соответствует нашим ожиданиям: это правило разрешает трафик из подов в пространстве имен по умолчанию с меткой `access=true`, посылаемый в поды в пространстве имен по умолчанию с меткой `app=my-nginx`.

Теперь, когда вы увидели, как Kubernetes использует правила `iptables` для применения сетевых политик, попробуем описать собственное правило. Я сделаю это на только что установленной системе Ubuntu, чтобы список правил был пуст:

```
$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
```

Настроим утилиту `netcat` так, чтобы она отвечала на запросы, поступающие на порт `8000`:

```
$ while true; do echo "hello world" | nc -l 8000 -N; done
```

В другом окне терминала попробуем отправить запрос на этот порт:

```
$ curl localhost:8000
hello world
```

Теперь создадим правило, которое будет отклонять входящий трафик, поступающий на порт 8000:

```
$ sudo iptables -I INPUT -j REJECT -p tcp --dport=8000
$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
REJECT     tcp  --  anywhere              anywhere        tcp dpt:8000 reject-with icmp-
port-unreachable

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
```

Как вы уже, наверное, поняли, теперь команда `curl` не получит ответа:

```
$ curl localhost:8000
curl: (7) Failed to connect to localhost port 8000: Connection refused
```

Этот пример демонстрирует использование `iptables` для ограничения трафика. Вы можете представить создание множества подобных правил для ограничения трафика между контейнерами, но я не рекомендую вам делать это вручную, разве что ваше развертывание небольшое и статичное. Но даже для небольших развертываний Kubernetes придется вручную написать довольно много правил. Чтобы вы понимали масштаб, у меня есть один узел Kubernetes с сетевым плагином Calico и всего несколькими запущенными прикладными подами без каких-либо сетевых стратегий, и команда `iptables -L` выводит более 300 строк правил, определенных в таблице `filter`.

Реализация политики на уровнях 3 и 4 с использованием eBPF

Только что вы увидели, как сетевой плагин может задействовать правила `iptables` для сетевой политики, разрешающей доступ к сервису `my-nginx` только подам с меткой `access=true`. Если реализовать ту же политику с помощью Cilium, то вместо правил `iptables` будут созданы программы и eBPF-карты.

Для этого Cilium создает *идентификатор* пода на основе хеша его меток. Вот пример набора идентификаторов, которые отслеживает Cilium:

```
$ cilium-dbg identity list

ID      LABELS
123     k8s:app=my-nginx
45      k8s:access=true
...
```

Политики преобразуются в правила разрешения и запрета для входящего и исходящего трафика в конечной точке. Правило, которое мы рассматривали, преобразуется следующим образом:

```
$ cilium-dbg bpf policy get 123
Ingress:
  From Identity 45: ALLOW
  All other: DROP
```

Данная политика представляет правило для входящего трафика, применяемое к идентификатору 123, который соответствует приложению `app=my-nginx`. Она пропускает только трафик, поступающий от идентификатора 45, соответствующего параметру `access=true`.

Правила политики хранятся в eBPF-картах в виде хеш-таблицы, что обеспечивает очень быстрый поиск внутри ядра, когда eBPF-программа Cilium в сетевом стеке перехватывает пакет, предназначенный (в данном случае) для конечной точки с идентификатором 123.

Рассмотренные нами политики содержат правила для сетевых уровней 3 и 4. Некоторые решения также могут обеспечивать соблюдение требований на прикладном уровне — 7.

Политики на уровне 7

Политики на уровнях 3 и 4 могут разрешать или запрещать трафик между заданной парой конечных точек, однако такое правило будет отбрасывать весь трафик, соответствующий политике. Политики на уровне 7 могут определять, какой трафик разрешать или блокировать, исходя из характеристик протокола прикладного уровня. Для HTTP-трафика политика способна дать разрешение только для определенных URL-адресов, методов или конкретных заголовков. DNS-трафик может фильтроваться по доменному имени или типу запроса (A, AAAA, MX и т. д.). Для GRPC-трафика возможна блокировка по имени метода, сервиса или на основе содержимого заголовков.

Cilium, Calico и Antrea поддерживают сетевые политики на уровне 7, реализованные путем интеграции прокси-сервера Envoy (<https://www.envoyproxy.io>). Применение правил происходит в пользовательском пространстве внутри прокси-сервера.



Впервые я увидела Cilium в действии в презентации Томаса Графа (Thomas Graf) на конференции DockerCon 2017, где он показал демо на основе «Звездных войн» (<https://oreil.ly/dbcHg>). Туда входили политики 3 и 4 уровней, которые разрешают доступ к Звезде Смерти только кораблям Империи, а также политики 7 уровня, блокирующие опасные действия кораблей Империи, например HTTP-запросы методом PUT в API Звезды Смерти по пути `/v1/exhaust-port`. Вы можете попробовать повторить пример у себя в кластере Kubernetes, следуя документации (https://oreil.ly/Erqm_).

Одно из преимуществ подхода к сетевым политикам в Cilium состоит в том, что политики на уровнях 3, 4, 7 и даже политики аутентификации можно определить как единый ресурс:

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: mutual-auth-echo
spec:
  endpointSelector:
    matchLabels:
      app: echo
  ingress:
    - fromEndpoints:
        - matchLabels:
            app: pod-worker
      authentication:
        mode: "required"
      toPorts:
        - ports:
            - port: "3000"
              protocol: TCP
      rules:
        http:
          - method: "GET"
            path: "/headers"
```

- ❶ Данная политика применяется ко всем подам с меткой `app=echo`.
- ❷ Эта политика применяется ко входящему трафику от подов с меткой `app=podworker`.
- ❸ Политика требует взаимной аутентификации трафика между этими конечными точками, то есть нужна криптографическая идентификация обоих подов до разрешения обмена данными. Аутентификация рассматривается в главе 13.
- ❹ Разрешается только TCP-трафик на порт 3000.
- ❺ Это должен быть HTTP-запрос методом GET к пути `/headers`.

Простота объединения требований всех политик облегчает эксплуатацию и устранение неполадок.

Программные решения для сетевых политик

В Kubernetes существуют объекты `NetworkPolicy`, хотя, как уже упоминалось ранее, сам Kubernetes их применяет. Они работают только при наличии поддерживающего их сетевого плагина (<https://oreil.ly/CBy9U>). К сожалению, если плагин CNI не реализует поддержку сетевых политик Kubernetes, они просто

игнорируются. Обязательно убедитесь, что используемый вами CNI применяет эти политики. Я слышала немало печальных историй о том, как операторы тратили уйму времени на настройку политик, не осознавая, что делают это впустую.

Отдельные сетевые плагины позволяют перейти на платную версию, которая отличается большей гибкостью или более удобным управлением. Некоторые коммерческие платформы безопасности контейнеров включают межсетевые экраны для контейнеров, которые позволяют добиться практически тех же результатов, но не устанавливаются непосредственно в виде сетевых плагинов Kubernetes. Иногда они самостоятельно обучаются распознавать обычный трафик, чтобы затем автоматически формировать политики.

Если используемый вами плагин CNI не поддерживает политики для уровня 7, это может стать поводом, чтобы обратить свое внимание в сторону сервисной сетки.

Сервисная сетка

В разворачиваниях Kubernetes, помимо плагина CNI, можно использовать *сервисную сетку (service mesh)*, предоставляющую дополнительные средства управления и возможности на прикладном уровне. Понятие «сервисная сетка» может иметь разный смысл для разных людей и включать обнаружение сервисов, балансировку нагрузки на уровне 7, канареечные разворачивания и функции мониторинга. Но в рамках книги мы рассмотрим, пожалуй, наиболее фундаментальную возможность — обеспечение безопасного сетевого соединения между контейнеризированными рабочими нагрузками. Она состоит из двух частей: создания аутентифицированных соединений с шифрованием трафика, которые рассматриваются в главе 13, и ограничения сетевого трафика на прикладном уровне путем применения политик уровня 7. В большинстве реализаций сервисной сетки эти политики отделены от политик уровней 3 и 4, применяемых CNI.



В статье, опубликованной на DigitalOcean (<https://oreil.ly/0Fq5A>), приводится довольно полный обзор других возможностей сервисных сетей, таких как канареечное разворачивание, которые не связаны ни с сетями, ни с безопасностью.

Примерами проектов сервисных сетей могут служить Istio (<https://istio.io>) и Linkerd (<https://linkerd.io>), а также управляемые решения от поставщиков облачных услуг, такие как AWS ECS Service Connect. Cilium CNI предоставляет множество возможностей, которыми обычно обладают сервисные сетки, но без дополнительной плоскости управления.

Вот пример политики авторизации Istio, предназначенной для ограничения HTTP-трафика:

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: echo
spec:
  selector:
    matchLabels:
      app: echo
  action: ALLOW
  rules:
    - from:
        - source:
            principals: ["cluster.local/ns/default/sa/pod-worker"]
      to:
        - operation:
            methods: ["GET"]
```

❶ Как и в предыдущем примере с политиками на уровне 7, здесь политика применяется к подам с меткой `app=echo`.

❷ Эта политика разрешает прохождение трафика.

❸ Запросы разрешаются только в том случае, если они исходят из рабочей нагрузки, имеющей сертификат X.509, идентифицирующий ее по учетной записи сервиса `pod-worker`. Подробнее о сертификатах и идентификаторах мы поговорим в главе 13.

❹ Разрешены только HTTP-запросы методом `GET`.

До недавнего времени большинство сервисных сеток реализовывались с использованием `sidecar`-контейнеров, имеющих некоторые ограничения, которые обсуждались в главе 9. В настоящее время существуют варианты сервисных сеток, не задействующие `sidecar`-контейнеры (<https://oreil.ly/V8NDQ>), включая Istio Ambient Mesh, а также поддержка возможностей сервисных сеток в Cilium, не требующая запуска `sidecar`-контейнера для каждого пода.

Если вы используете или рассматриваете реализацию на основе `sidecar`-контейнеров, следует учитывать некоторые дополнительные ограничения, связанные с безопасностью.

- `Sidecar`-контейнер может обеспечить безопасность только для подов, в которые он фактически внедрен. Если он отсутствует, то он ничего не сможет сделать. Тщательно тестируйте и/или проверяйте конфигурации, чтобы убедиться, что контейнеры с сеткой применяются ко всем рабочим нагрузкам, где они необходимы.

- Контейнер сервисной сетки размещается рядом с контейнерами приложений внутри пода. В случае взлома контейнера приложения злоумышленник может попытаться обойти или изменить политики, применяемые контейнером сервисной сетки. Контейнер сервисной сетки и контейнер приложения используют одно и то же пространство имен сети, поэтому рекомендуется отзывать привилегию `CAP_NET_ADMIN` у контейнеров приложений, чтобы в случае взлома одного из них злоумышленник не смог изменить общий сетевой стек.
- Жизненный цикл `sidecar`-контейнера тесно связан с жизненным циклом пода. Это означает, что для обновления сервисной сетки придется перезапустить весь под.

Политики в сервисной сетке определяются на уровне сервиса, поэтому целесообразно использовать принцип многоуровневой защиты, дополнив его средствами обеспечения безопасности сети контейнеров или среды выполнения, чтобы ограничить трафик, поступающий непосредственно к контейнерам, а не на IP-адрес сервиса.

Практические рекомендации для сетевых стратегий

Какие бы утилиты вы ни использовали для создания, управления и применения сетевых политик, существуют несколько рекомендаций.

Отказ по умолчанию

Следуя принципу минимума привилегий, задавайте политики для всех пространств имен, которые бы отклоняли входящий трафик по умолчанию (<https://oreil.ly/L6OjC>), и лишь затем добавляйте политики, пропускающие действительно необходимый трафик.

Отказ по умолчанию для исходящего трафика

Политики исходящего трафика относятся к исходящим запросам вашего пода. В случае взлома контейнера злоумышленник может начать зондировать окружающую среду через сеть. Настройте политики для всех пространств имен, которые по умолчанию отклоняют исходящий трафик (<https://oreil.ly/RmeUT>), а затем добавьте политики, пропускающие ожидаемый исходящий трафик.

Ограничение трафика между подами

Поды обычно маркируются в соответствии с приложением. Ограничивайте трафик с помощью политик, чтобы он мог передаваться лишь между разрешенными приложениями, а также используйте политики для разрешения трафика только из подов с соответствующими метками.

Ограничение списка портов

Ограничивайте прием трафика так, чтобы он принимался только на определенных портах для каждого приложения.



Ахмет Алп Балкан (Ahmet Alp Balkan) приводит список полезных примеров сетевых стратегий (<https://github.com/ahmetb/kubernetes-network-policy-recipes>).

Резюме

В этой главе вы видели, как контейнеры позволяют весьма скрупулезно настроить межсетевой экран внутри развернутой системы, что помогает обеспечить соблюдение нескольких принципов безопасности:

- разделение обязанностей/принцип минимума привилегий — контейнерам предоставляется лишь ограниченная возможность обмена информацией;
- ограничение радиуса поражения — скомпрометированный контейнер не сможет атаковать своих соседей;
- многоуровневая защита — за счет сочетания сетевых политик на уровнях 3, 4 и 7 с обычными межсетевыми экранами в масштабах всего кластера.

Я уже упоминала, что вам, возможно, стоит рассмотреть вопрос о шифровании сетевого трафика, перемещаемого между контейнерами. В главе 13 я расскажу о некоторых подходах к обеспечению безопасности и шифрованию трафика и попытаюсь пролить свет на роль ключей и сертификатов при установлении защищенных соединений.

Защищенное соединение между компонентами

В любой распределенной системе есть компоненты, которым нужно взаимодействовать друг с другом. В мире облачных технологий этими компонентами могут быть контейнеры, которые обмениваются сообщениями друг с другом или с другими внутренними или внешними компонентами. В этой главе вы увидите, как защищенные соединения позволяют компонентам безопасно отправлять друг другу зашифрованные сообщения. Вы также узнаете, как компоненты идентифицируют друг друга и устанавливают защищенные соединения между собой, чтобы исключить вредоносные элементы из обмена информацией.

Если вы знакомы с принципами работы ключей и сертификатов, то можете спокойно пропустить эту главу, поскольку в ней нет ничего относящегося именно к контейнерам. Я включила ее в книгу лишь потому, что обсуждаемые здесь вопросы вызывают затруднения у многих, кто при изучении контейнеров и облачных инструментов столкнулся с ними впервые.

Если вы отвечаете за администрирование облачной инфраструктуры, то, вероятно, вам придется настраивать сертификаты, ключи и центры сертификации для Kubernetes, etcd и других компонентов. Эти задачи печально известны своей сложностью, а в инструкциях по установке обычно объясняется только то, *что* нужно сделать, не объясняя, *почему* или *как*. Текущая глава поможет вам понять роли, которые играют все эти элементы.

Начнем с того, что вообще подразумевается под защищенными соединениями.

Защищенные соединения

В повседневной жизни мы постоянно сталкиваемся с защищенными соединениями, например при работе в браузере. Если вы зашли, скажем, в онлайн-банк и не увидели маленький зеленый замочек в адресной строке, значит, соединение

не защищено и лучше не вводить там свои учетные данные. Формирование защищенного соединения с сайтом банка состоит из двух этапов.

- Во-первых, необходимо убедиться, что сайт, на который вы зашли, действительно принадлежит вашему банку. Браузер убеждается в подлинности сайта путем проверки его сертификата. Этот этап называется *аутентификацией*.
- Второй этап — *шифрование*. При доступе к банковской информации важно гарантировать, что никакие сторонние лица не смогут перехватить (или, что еще хуже, изменить) передаваемую по этому каналу информацию.

Вероятно, вы знаете, что для защищенных соединений с сайтами используется протокол *HTTPS* (HTTP-Secure — «защищенный HTTP»). Как следует из названия, это обычное HTTP-соединение, только защищенное на транспортном уровне с помощью протокола, изобретательно названного протоколом *безопасности транспортного уровня* (transport layer security, TLS).

Если вы думали, что S обозначает SSL, то есть Secure Sockets Layer — «уровень защищенных сокетов», то не слишком ошиблись. Транспортный уровень представляет собой уровень обмена информацией между парами сетевых сокетов, а TLS — новое название протокола, который когда-то назывался SSL. Первые спецификации SSL были опубликованы компанией Netscape в 1995 году в виде версии 2 (в первоначальной версии 1 содержалось слишком много изъянов, поэтому ее так и не выпустили). К 1999 году в IETF (Internet Engineering Task Force — Инженерный совет Интернета) был создан стандарт TLS v1.0, в значительной степени основанный на SSL v3.0 от Netscape, а сейчас преимущественно применяется TLS v1.3.

Неважно, называете ли вы его SSL или TLS, суть остается одной: защищенные соединения формируются в этом протоколе на основе сертификатов. Непонятно, почему 20 лет спустя после перехода на TLS их все еще называют SSL-сертификатами. Правильнее всего называть их сертификатами X.509.

TLS не единственный протокол для создания безопасных соединений, и, как вы увидите далее в этой главе, для шифрования трафика в контейнерах довольно часто используются такие протоколы, как IPSec и WireGuard. Все эти протоколы предполагают обмен идентификационной информацией и шифрование трафика с применением криптографического алгоритма.

С помощью сертификатов X.509 можно обмениваться информацией как о владельце, так и о ключах шифрования. Давайте подробнее рассмотрим, что представляют собой эти сертификаты и как они работают.



Существует несколько утилит для генерации ключей, сертификатов и центров сертификации, в числе которых `ssh-keygen`, `openssl keygen` и `minica`. В докладе «Руководство по защищенным соединениям для программиста на Go» (*A Go Programmer's Guide to Secure Connections*, <https://youtu.be/OF3TM-b890E>) я продемонстрировала, как пользоваться `minica`, а также пошагово описала, что происходит, когда клиент устанавливает TLS-соединение с сервером.

Сертификаты X.509

Термин X.509 — это название стандарта Международного союза электросвязи (International Telecommunication Union, ITU), в котором описаны эти сертификаты. Сертификат представляет собой элемент структурированных данных, содержащий информацию о его владельце, а также открытый ключ шифрования, предназначенный для обмена информацией с его владельцем. Этот открытый ключ входит в пару ключей «открытый — закрытый» (рис. 13.1).

Как показано на рис. 13.1, важнейшими элементами информации в сертификате являются:

- название сущности, которую идентифицирует данный сертификат, то есть *субъекта* (subject). Название субъекта обычно представляет собой доменное имя. На практике в сертификатах обычно есть поле «Другие названия субъекта», благодаря которому сертификат может идентифицировать субъект по нескольким именам;
- открытый ключ субъекта;
- название центра сертификации, выдавшего сертификат (мы вернемся к этому вопросу позднее, в подразделе «Центры сертификации»);
- срок действия сертификата, то есть дата и время, после которых сертификат считается недействительным.



Рис. 13.1. Сертификат

Пары ключей «открытый — закрытый»

Открытые и закрытые ключи представляют технологию асимметричного шифрования, когда данные шифруются с помощью открытого ключа, но расшифровать их можно только с помощью закрытого ключа. Как ясно из названия, открытым ключом можно делиться с кем угодно. Каждому открытому ключу соответствует закрытый, который владелец никогда не должен раскрывать.



Формальное математическое описание способов шифрования и расшифровки выходит за рамки книги, но в статье на сайте Medium (<https://oreil.ly/Tbhvd>) я собрала несколько рекомендуемых источников информации по этому вопросу.

Сначала генерируется закрытый ключ, а затем вычисляется соответствующий ему открытый. Пару ключей «открытый — закрытый» можно использовать для двух полезных целей.

- Как показано на рис. 13.2, с помощью открытого ключа можно шифровать сообщения, расшифровать которые способен только тот, у кого есть соответствующий закрытый ключ.

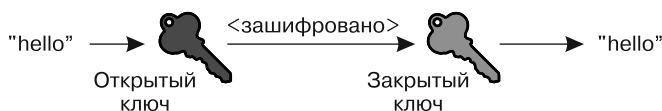


Рис. 13.2. Шифрование

- Закрытый ключ можно использовать для создания цифровой подписи сообщения, с помощью которой любой, у кого есть соответствующий открытый ключ, сможет убедиться, что сообщение поступило от владельца закрытого ключа (рис. 13.3).

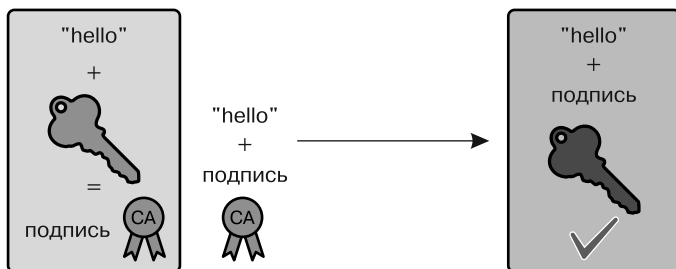


Рис. 13.3. Цифровая подпись

При установлении защищенных соединений возможности пар ключей «открытый — закрытый» используются как для шифрования, так и создания подписей.

Представьте, что вы хотите обмениваться со мной сообщениями. Я генерирую пару ключей и передаю вам открытый ключ, чтобы вы могли отправлять мне зашифрованные сообщения. Но если я отправлю вам открытый ключ, то как вы узнаете, что он действительно пришел от меня, а не от самозванца? Чтобы вы могли удостовериться, что я та, за кого себя выдаю, необходима третья сторона, которой вы доверяете и которая за меня поручится. Эти функции выполняет *центр сертификации* (ЦС).

Центры сертификации

Центр сертификации — доверенный орган, подписывающий сертификат и тем самым подтверждающий правильность информации о владельце, которая содержится в этом сертификате. Следует доверять только сертификатам, подписанным центрами, в которых вы уверены.

Клиент, инициирующий TLS-соединение, получает сертификат с другого конца соединения и проверяет его, чтобы убедиться, что его собеседник — тот, за кого себя выдает. Например, при открытии веб-соединения с банком браузер удостоверяется, что сертификат соответствует URL банка, а также проверяет, какой ЦС подписал этот сертификат.

Другие компоненты также должны иметь возможность безопасно идентифицировать центры сертификации, поэтому им соответствует свой сертификат. Но этот сертификат также должен быть подписан каким-то центром, «личность» которого тоже нужно проверить, для чего требуется еще один сертификат — и так до бесконечности! В конце концов, должен быть какой-то сертификат, которому мы можем просто доверять.

На практике цепочка заканчивается так называемым *самоподписанным сертификатом* — сертификатом X.509, который ЦС подписал для себя. Другими словами, этот сертификат представляет того же, чей закрытый ключ использовался для подписания сертификата. Если вы доверяете данному лицу, то можете полагаться и на сертификат. На рис. 13.4 приведена цепочка сертификатов, в которой сертификат Энн подписан Бобом, а сертификат Боба подписан Кэрлом. Цепочка оканчивается на самоподписанном сертификате Кэрол.

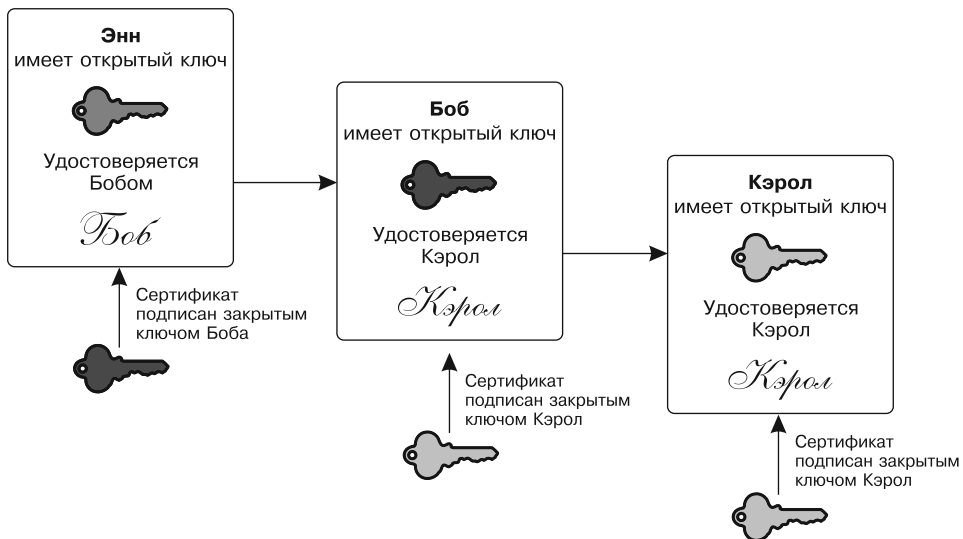


Рис. 13.4. Цепочка сертификатов

Браузеры поставляются с набором сертификатов известных доверенных центров сертификации — их называют корневыми ЦС. Ваш браузер будет доверять любому сертификату (или цепочке сертификатов), подписанному одним из этих корневых центров. Если же сертификат сайта не подписан одним из доверенных ЦС браузера, то он отметит сайт как небезопасный (в большинстве современных браузеров при этом пользователю будет выдано предупреждение или сообщение об ошибке).

При создании сайта, к которому пользователи должны подключаться через Интернет, вам понадобится сертификат, подписанный доверенным общедоступным ЦС. Существует несколько поставщиков сертификатов, выступающих в роли таких центров и генерирующих сертификаты за определенную плату. Кроме того, можно получить сертификат бесплатно от Let's Encrypt (<https://letsencrypt.org/>).

При настройке компонентов распределенных систем наподобие Kubernetes или etcd можно указать набор ЦС, используемых для проверки сертификатов. Если систему контролируете только вы, не имеет значения, доверяют ли этим компонентам сторонние пользователи (или их браузеры), — главное, чтобы компоненты доверяли друг другу. А поскольку это частная система, задействовать общедоступные доверенные ЦС не обязательно: можно просто создать собственные с самоподписанными сертификатами.

В корпоративной среде инфраструктура открытых ключей (Public Key Infrastructure, PKI) часто поддерживает возможности центра сертификации и, как правило, управляет политиками сертификатов, их продлением и аннулированием.

Независимо от того, используете ли вы собственный или общедоступный ЦС, необходимо сообщить ему, какие сертификаты вы хотели бы сгенерировать. Это делается с помощью запроса на подписание сертификата.

Запросы на подписание сертификатов

Запрос на подписание сертификата (Certificate Signing Request, CSR) — это файл, включающий:

- открытый ключ, который нужно включить в сертификат;
- доменное имя (имена), с которым (-и) должен работать этот сертификат;
- сведения о владельце данного сертификата (например, название компании или организации).

Вы уже знаете, что сертификат X.509 включает в себя всю эту информацию, а также подпись ЦС, поэтому совершенно логично, что именно эти данные вы должны отправить в запросе на подписание сертификата, чтобы получить сертификат X.509 от центра сертификации.

Создать новую пару ключей и CSR за один шаг можно с помощью таких утилит, как `openssl`. Возможно, это кажется запутанным, но `openssl` может принимать закрытый ключ в качестве входных данных для генерации CSR. Соответствующий этому сертификату компонент задействует закрытый ключ для расшифровки и подписывания сообщений (как вы скоро увидите), но никогда не использует сам открытый ключ. Последний нужен прочим компонентам, с которыми он обменивается информацией, и они получают этот открытый ключ из сертификата. Самому же компоненту нужен закрытый ключ, а также сертификат для отправки другим компонентам.

Теперь, когда вы уже понимаете, что такое сертификаты, обсудим, как они используются для TLS-соединений.

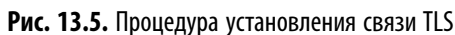
TLS-соединения

Инициатором соединений транспортного уровня является компонент, называемый *клиентом*, а сторона, с которой он обменивается информацией, называется *сервером*. Вполне возможно, что такая взаимосвязь «клиент — сервер» имеет место только на транспортном уровне, а на более высоких уровнях ранг компонентов, которые обмениваются информацией, будет одинаковым.

Клиент открывает сокет и запрашивает сетевое соединение с сервером. Если соединение защищено, то клиент запрашивает у сервера сертификат. Как вы уже знаете, сертификат содержит два важных элемента информации: название сервера и его открытый ключ.

Смысл в том, чтобы клиент мог выяснить, стоит ли доверять серверу. Клиент проверяет, подписан ли сертификат сервера доверенным ЦС, и если это так, то серверу можно доверять. Затем клиент может задействовать открытый ключ сервера для шифрования отправляемых серверу сообщений. Клиент и сервер приходят к соглашению относительно использования симметричного ключа, который будет применяться для дальнейших сообщений, передаваемых по этому соединению, — быстроедействие при этом выше, чем при использовании асимметричной пары из открытого и закрытого ключей. Данный поток сообщений показан на рис. 13.5.

Вероятно, вам встречался термин «пропуск проверки» (`skip verify`). Он описывает вариант, когда клиент на транспортном уровне пропускает проверку того, что сертификат подписан известным ему ЦС. Клиент при этом просто предполагает, что сервер — тот, за кого себя выдает. Это удобно при разработке, поскольку не требуется настраивать клиент с информацией о центрах сертификации — можно просто использовать самоподписанные сертификаты. При этом взаимодействие между компонентами остается зашифрованным, однако нет уверенности, что эти компоненты не «самозванцы», поэтому не используйте вариант пропуска проверки сертификатов в рабочей среде!



Если речь идет о сайте, на котором у вас есть учетная запись, например о сайте банка, то важно, чтобы сервер проверял, с кем имеет дело, перед выдачей, скажем, информации о состоянии банковского счета. В классической модели «клиент — сервер» при входе на сайт банка эта задача решается с помощью аутентификации на уровне 7. Вы указываете имя пользователя и пароль, возможно, с многофакторной аутентификацией в виде ввода кода, отправленного в текстовом сообщении, или одноразового пароля, сгенерированного Yubikey или мобильным приложением — Authy, 1Password или Google Auth.

Другой способ подтвердить «личность» клиента — воспользоваться еще одним сертификатом X.509. Поток сообщений на рис. 13.5 демонстрирует обмен сертификатами между клиентом и сервером — эту возможность можно включить на стороне сервера. С помощью сертификата сервер подтверждает клиенту свою «личность». А можно ли сделать обратное? Да, и это называется *взаимным TLS* (mutual TLS, mTLS).

Соединения TLS и mTLS широко применяются для обмена данными между клиентами и серверами. В контейнерах (или подах Kubernetes) также допускается использовать и TLS, и mTLS (или оба протокола сразу). Однако контейнеры работают на хостах, и альтернативным методом шифрования

трафика между контейнерами является шифрование всего сетевого трафика между хостами, на которых они работают, с использованием таких технологий, как WireGuard или IPSec.

WireGuard и IPSec

Эти две технологии отличаются способом реализации, но суть их одна: они шифруют IP-трафик (уровень 3), создавая защищенный туннель между конечными точками и инкапсулируя трафик, проходящий через туннель. Обе технологии широко используются в виртуальных частных сетях (VPN), которые вы могли использовать для безопасного подключения компьютеров к сети своего работодателя или для создания видимости, что устройство находится в другой стране.

В WireGuard сетевой интерфейс создается на устройстве обычно с именем `wg0` (или `wg1`, `wg2` и т. д., если их несколько). Идея создания сетевого интерфейса была представлена в главе 4, где было показано, как контейнеры подключаются к хосту через виртуальное Ethernet-соединение. WireGuard задействует UDP-туннель для инкапсуляции и шифрования IP-пакетов. Трафик может направляться в этот интерфейс (или в любой другой).

Интерфейс WireGuard настраивается с помощью закрытого ключа, списка узлов, открытых ключей для каждого из них, а также IP-адресов, которым разрешено отправлять трафик с этого узла. Вот пример такой конфигурации, взятый с сайта WireGuard:

```
[Interface]
PrivateKey = yAnz5TF+1XXJte14tji3zlMNq+hd2rYUIgJBgB3fBmk=
ListenPort = 51820
```

```
[Peer]
PublicKey = xTIBA5rboUvnH4htodjb6e697QjLERT1NAB4mZqp8Dg=
AllowedIPs = 10.192.122.3/32, 10.192.124.1/24
```

```
[Peer]
PublicKey = TrMvSoP4jYQlY6RIzBgbssQqY3vxI2Pi+y71l0WwXX0=
AllowedIPs = 10.192.122.4/32, 192.168.0.0/16
```

```
[Peer]
PublicKey = gN65BkIKy1eCE9pP1wdc8ROUtkHLf2PfAqYdyYBz6EA=
AllowedIPs = 10.10.10.230/32
```

Пакеты шифруются с использованием открытого ключа получателя и инкапсулируются в UDP-пакет. Получатель расшифровывает пакет с помощью своего закрытого ключа и проверяет, разрешено ли принимать данные с IP-адреса отправителя, — если нет, то пакет отбрасывается.

Протокол WireGuard считается очень безопасным (по крайней мере до наступления эры постквантовой криптографии) и простым в настройке, поэтому отлично подходит для прозрачного шифрования трафика между узлами. Однако WireGuard не соответствует стандарту FIPS (Federal Information Processing Standards — Федеральные стандарты обработки информации) не потому, что он технически недостаточно безопасен, а потому, что разработчики не без оснований не хотят проходить бюрократическую процедуру сертификации своих алгоритмов на соответствие FIPS. В регулируемых средах важно иметь возможность поставить галочку напротив пункта о соответствии, поэтому в таких ситуациях вам, возможно, стоит посмотреть в сторону IPSec¹.

В IPSec нет выделенного сетевого интерфейса и политики безопасности настраиваются на существующих интерфейсах. Эта конфигурация включает параметры безопасности самого хоста, его закрытый ключ, а также открытые ключи и IP-адреса узлов и сетей, которым разрешено передавать трафик через туннель IPSec. IPSec также позволяет выбирать алгоритмы шифрования и аутентификации, поэтому первым шагом является установление «безопасной связи», которая определяет параметры безопасности, используемые для взаимодействия с узлами, включая алгоритмы шифрования и аутентификации.

После настройки IPSec на сетевом интерфейсе он начинает перехватывать отправляемые или принимаемые IP-пакеты и применяет к ним политики безопасности IPSec, шифруя и аутентифицируя пакеты, а также проверяя IP-адрес отправителя и другие параметры безопасности.

IPSec может работать в двух режимах: транспортном или туннельном, которые определяют порядок обработки и шифрования пакетов. Туннельный режим считается более безопасным, поскольку он шифрует заголовок IP-пакета, включающий IP-адреса отправителя и получателя, и может использоваться там, где пакеты проходят через устройства NAT. Транспортный режим, как правило, обеспечивает лучшую производительность и идеально подходит для частных сетей, где IP-адреса не считаются конфиденциальной информацией.

Сетевые технологии WireGuard и IPSec широко используются в контейнерных развертываниях для поддержки принципа нулевого доверия.

Принцип нулевого доверия

Принцип нулевого доверия — это подход, предполагающий, что весь сетевой трафик, независимо от того, откуда он поступает, изнутри или извне, потенциально является вредоносным и что злоумышленники могут иметь доступ

¹ WolfSSL (<https://oreil.ly/DuYfq>) планирует заменить криптографический пакет WireGuard алгоритмами, соответствующими стандарту FIPS.

к сети. Для снижения рисков, связанных с этим предположением, весь трафик должен шифроваться и аутентифицироваться.

Такие алгоритмы, как WireGuard и IPSec, могут играть решающую роль в сетях с нулевым доверием, обеспечивая безопасную связь между устройствами. В сетях, где контейнеры работают на хостах, подключенных к сети с нулевым доверием, трафик автоматически шифруется при передаче от одного хоста к другому, а для аутентификации используются идентификаторы хостов. Для многих развертываний, включая управляемые среды, этого достаточно, чтобы соответствовать требованиям безопасности.



В сети Kubernetes, как было показано в главе 12, связь между подами обеспечивается плагином CNI. Некоторые CNI, включая Cilium и Calico, можно настроить на использование IPSec или WireGuard, что гарантирует шифрование всего трафика между хостами.

Если вы не управляете всей сетью, к которой подключены контейнеры (например, VPC или VPN-соединениями между кластерами), или не доверяете всем развернутым в ней контейнерам, одного шифрования сетевого трафика может быть недостаточно. В этом случае можно аутентифицировать и шифровать трафик между отдельными контейнерами (или подами в Kubernetes).

Защищенные соединения между контейнерами

До сих пор мы не обсуждали специфику контейнеров, но теперь давайте рассмотрим ситуации, в которых вам может пригодиться понимание ключей, сертификатов и центров сертификации.

- При установке или администрировании Kubernetes или других компонентов распределенных систем можно столкнуться с разными вариантами использования защищенных соединений. Инструменты установки, такие как `kubeadm`, упрощают применение TLS между компонентами плоскости управления и автоматически настраивают сертификаты. Но трафик между контейнерами и внешним миром они никак не защищают.
- Вам как разработчику может понадобиться написать прикладной код для организации защищенных соединений с другими компонентами (работающий в контейнере или нет). В этом случае вашему прикладному коду понадобится доступ к сертификатам, которые вам придется создать.
- Вместо того чтобы писать собственный код для организации защищенных соединений, можно воспользоваться сервисной сеткой. Сервисные сетки обсуждались в главе 12, и чуть позже в этой главе я расскажу, как они обеспечивают защищенные соединения между контейнерами, но сначала поговорим об отзыве сертификатов.

Отзыв сертификатов

Представьте, что злоумышленник каким-то образом раздобыл закрытый ключ. Теперь он может выдавать себя за владельца этого ключа, поскольку способен расшифровывать сообщения, зашифрованные открытым ключом, вложенным в соответствующие сертификаты. Чтобы это предотвратить, необходимо иметь способ немедленно аннулировать сертификаты, а не ждать истечения срока их действия.

Подобное аннулирование называется отзывом сертификатов (*certificate revocation*), и для него используется специальный *список отозванных сертификатов* (*certificate revocation list, CRL*), содержащий сертификаты, которые более не следует принимать.

Старайтесь не распространять субъекты (и их сертификаты) сразу на несколько компонентов или пользователей. Может показаться, что управление отдельными субъектами и сертификатами для каждого компонента — это дополнительная нагрузка, но это позволяет отзывать сертификаты субъектов по отдельности, не генерируя заново сертификаты для остальных (легитимных) пользователей. Кроме того, такой подход дает возможность разделять ответственность, предоставляя каждому субъекту свой набор прав.



В Kubernetes сертификаты используются компонентом kubelet на всех узлах для аутентификации его в сервере API и подтверждения, что он действительно является авторизованным kubelet. Эти сертификаты подписаны ЦС кластера и могут быть обновлены автоматически.

Сертификаты также служат одним из механизмов аутентификации клиентов в сервере API Kubernetes (<https://oreil.ly/0DuGQ>).

На момент написания книги Kubernetes не поддерживает отзыв сертификатов (<https://oreil.ly/RU3ga>). После выдачи сертификата его невозможно аннулировать до истечения срока действия. По этой причине рекомендуется применять сертификаты с коротким сроком действия и автоматическую ротацию. Можно также воспользоваться возможностями RBAC, чтобы запретить доступ к API клиентов, связанных с такими сертификатами (но это не мешает использованию сертификата для установления TLS-соединения).

Сервисные сетки и шифрование трафика

Вы уже познакомились с сервисными сетками и увидели, как их использовать для применения сетевых политик на прикладном уровне, но я лишь вскользь упомянула, как они обеспечивают аутентификацию и шифрование трафика между рабочими нагрузками. Давайте разберемся в этом подробнее.

Как уже упоминалось, в сервисных сетях долгое время использовалась модель на основе sidecar-контейнеров, согласно которой в каждый под (или по

крайней мере в каждый под, подключающийся к сервисной сетке) внедряется прокси-компонент. Он перехватывает входящие и исходящие сетевые соединения. Когда под А инициирует соединение с подом Б, прокси А выступает в роли конечной точки для этого соединения, устанавливает mTLS-соединение с прокси Б и пересылает данные по этому соединению.

Однако такая модель имеет ряд недостатков, особенно сильно проявляющихся при масштабировании. Если запустить 1000 подов, то появится 1000 дополнительных sidecar-контейнеров с прокси-компонентами, каждый из которых потребляет ресурсы, например память для таблицы маршрутизации, и увеличивает нагрузку на CPU. Трафик вынужден проходить дополнительный путь между прокси-компонентами и контейнерами приложений, что увеличивает задержку. Но, пожалуй, самое важное: сервисные сетки на основе sidecar-контейнеров привносят дополнительную сложность. Sidecar-контейнеры необходимо правильно внедрять, и, как упоминалось выше, они тесно связаны с жизненным циклом пода — поэтому для обновления сервисной сетки приходится перезапускать прикладные поды.

В результате многие команды решили использовать решения без sidecar-контейнеров, такие как Cilium (<https://cilium.io>) или Istio Ambient Mode (<https://istio.io/latest/docs/ambient>), более эффективные, с меньшим временем развертывания и более простые в эксплуатации, однако сохраняющие при этом основные возможности сервисных сеток, такие как аутентификация рабочих нагрузок, шифрование соединений, поддержка политик на уровне 7 и наблюдаемость.



Томас Граф (Thomas Graf) описал эволюцию сервисных сеток от модели с sidecar-контейнерами к модели без них (<https://oreil.ly/es7tp>), а в документации Istio приводится хорошее сравнение режимов с sidecar-контейнерами и без них (<https://oreil.ly/Y1FFx>).

Для аутентификации контейнера или пода в сервисной сетке необходим сертификат X.509, представляющий конкретную рабочую нагрузку. Как вы уже знаете, выпуск сертификатов X.509 происходит в несколько этапов, поэтому создание сертификатов и управление идентификацией для тысяч рабочих нагрузок — довольно сложная задача. Одна из инициатив, призванных это упростить, — SPIFFE.

SPIFFE

Аббревиатура SPIFFE расшифровывается как Secure Production Identity Framework For Everyone (инфраструктура безопасной идентификации рабочих нагрузок для всех). Это набор стандартов, определяющий способы выпуска и управления идентификационными данными, известными как SVID (SPIFFE Verifiable Identity Documents — проверяемые идентификационные документы

SPIFFE), которые на практике обычно представляют собой сертификаты X.509, хотя также поддерживаются JSON Web Token (JWT). Ключевая идея заключается в отделении идентификации рабочей нагрузки от идентификации компьютера или от традиционных подходов вроде IP-адресов, даже в таких динамических средах, как Kubernetes, где рабочие нагрузки недолговечны и часто перезапускаются на разных хостах.

Среда выполнения SPIFFE (SPIFFE Runtime Environment, SPIRE) — это реализация спецификации SPIFFE с открытым исходным кодом. Она выступает в роли центра сертификации рабочих нагрузок, автоматически выдавая SVID на основе таких характеристик, как дайджесты образов контейнеров или метки подов Kubernetes. SPIRE гарантирует, что только авторизованные рабочие нагрузки получают действительные идентификаторы, которые обновляются автоматически и безопасно.

Как отмечалось ранее в этой главе, для настройки mTLS-соединения требуется закрытый ключ и сертификат X.509. Рассмотрим подробнее, как происходит управление ключами и сертификатами с помощью SPIFFE/SPIRE:

- на каждом узле работает агент SPIRE, предоставляющий доступ к SPIRE Workload API через сокет, так что к нему могут подключаться только локальные процессы;
- когда запускается рабочая нагрузка (под в Kubernetes или любой процесс в других средах), она отправляет запрос агенту SPIRE через сокет Workload API;
- агент SPIRE использует *плагины аттестации* для определения типа рабочей нагрузки и ее идентификатора SPIFFE;
- для допустимой рабочей нагрузки агент SPIRE генерирует пару с открытым и закрытым ключами и отправляет запрос на подписание сертификата серверу SPIRE;
- сервер выступает в роли центра сертификации и возвращает SVID, содержащий сертификат X.509 для рабочей нагрузки;
- агент передает сертификат и закрытый ключ рабочей нагрузке, которая теперь может использовать их для установления соединений TLS или mTLS.

Этот процесс позволяет аутентифицировать рабочую нагрузку без необходимости вручную настраивать ключи и сертификаты.

Существуют различные варианты плагинов, обеспечивающих аттестацию рабочих нагрузок для SPIRE. Как правило, плагин идентифицирует рабочую нагрузку, используя ее PID, и собирает информацию об этом процессе, чтобы определить характеристики рабочей нагрузки. Объединив знания из предыдущих глав, можно представить простую реализацию, использующую `/proc/<pid>/exe`

для определения исполняемого файла, запущенного процессом. В Kubernetes существует инструмент аттестации рабочих нагрузок, который анализирует контрольные группы, связанные с процессом, чтобы определить, к какому поду он относится, а затем обращается к Kubernetes API для получения метаданных пода, таких как пространство имен, учетная запись сервиса и метки. Порядок отображения этих метаданных в идентификатор SPIFFE определяется настройками сервера SPIRE (<https://oreil.ly/B2wZR>).

Отдельные приложения могут использовать Workload API или сервисную сетку, чтобы безопасно получить SVID от имени рабочих нагрузок, так что вся операция будет полностью прозрачна для кода приложения.

Внешний трафик

До сих пор в главе речь шла о шифровании трафика между контейнерами, но во многих развертываниях также необходимо шифровать трафик, который проходит между контейнером и некоторой внешней конечной точкой:

- входящие запросы могут поступать, например, от браузеров из публичного Интернета или от клиентов внутри частной сети, но из-за пределов кластера контейнеров;
- контейнеры могут отправлять исходящие запросы к сервисам за пределами контейнера, например к сторонним платежным шлюзам, внешним хранилищам, таким как AWS S3, или внутренним базам данных.

Безопасность такого внешнего трафика тоже необходимо обеспечить.

Входящий трафик

Внешний трафик обычно проходит через некоторую точку входа и лишь потом попадает в контейнеризированное приложение. Это может быть балансировщик нагрузки, находящийся между внешним миром и кластером с контейнерами, либо Ingress API или Gateway API в кластере Kubernetes, либо обратный прокси в конфигурации Docker Compose. Все они выполняют схожие функции: определяют, какому контейнеру передать внешний запрос, и при необходимости способны завершать TLS-соединения.

Существует несколько вариантов завершения TLS-соединения:

- оборвать все входящие TLS-соединения на входном уровне и перенаправлять запросы в контейнер как обычный HTTP-трафик. Если используются прозрачные методы шифрования, такие как WireGuard или IPSec, трафик внутри сети контейнера останется зашифрованным;

- задействовать протокол TLS Passthrough, при котором TLS-соединение устанавливается до конечной точки в контейнере (или его прокси-компонента в сервисной сетке). Это может потребоваться, если в развертывании присутствуют ненадежные рабочие нагрузки, например в мультиарендной среде;
- завершать соединения на входном уровне и повторно шифровать трафик, а между входным уровнем и контейнерами использовать mTLS.

В то же время круг источников внешнего трафика может быть ограничен. Это ограничение можно реализовать с помощью обычного межсетевого экрана, настройки компонента **Ingress** или даже применения сетевой политики в самом контейнере.

Исходящий трафик

По умолчанию контейнеры способны устанавливать исходящие соединения с любыми доступными адресами. Круг таких адресов можно ограничить с помощью сетевой политики внутри контейнера и/или правил групп безопасности (security group) в публичном облаке. Чтобы затруднить злоумышленнику кражу данных, рекомендуется полностью блокировать исходящий трафик от рабочих нагрузок, если он им явно не требуется.

Кроме того, для защиты внешних сервисов, к которым должны подключаться ваши контейнеры, хорошим решением будет задействовать межсетевой экран. *Шлюз исходящего трафика* может сделать так, чтобы трафик, исходящий от контейнеризированной рабочей нагрузки, выглядел как поступающий с предсказуемого IP-адреса, что упростит определение правил межсетевого экрана. Подобные шлюзы имеются и в Cilium, и в Istio, работающих в кластере Kubernetes. В публичном облаке аналогичного эффекта можно добиться на уровне инфраструктуры с помощью NAT-шлюзов и правил таблиц маршрутизации.

Наблюдаемость сети и журналирование

Как и в традиционных развертываниях, в кластерах контейнеров желательно отслеживать и регистрировать входящие и исходящие соединения — на случай появления неожиданных получателей или резкого роста трафика, что может указывать на вредоносную активность. Некоторые инструменты, способные помочь в этом, мы рассмотрим в главе 15.

Многие CNI и сервисные сетки предоставляют инструменты мониторинга вроде Hubble от Cilium (<https://oreil.ly/89tWd>) или Kiali от Istio (<https://kiali.io>), которые помогают в отладке проблем с подключением и предоставляют информацию о потенциальных угрозах.

Резюме

Чтобы защититься от атаки типа «человек посередине» (man-in-the-middle), необходимо удостовериться в защищенности сетевых соединений между отдельными программными компонентами. Один из подходов — прибегнуть к методам организации сетевого взаимодействия с нулевым доверием, обеспечивающим постоянное шифрование трафика. Прозрачное шифрование между узлами с использованием WireGuard или IPSec, как правило, легко настраивается и не требует изменений в контейнерах или приложениях.

Иногда может понадобиться дополнительная аутентификация отдельных рабочих нагрузок. Прикладной код, работающий в контейнерах, может самостоятельно инициировать и/или обрывать TLS-соединения, а сервисная сетка — аутентифицировать рабочие нагрузки. Следует также подумать о том, как обезопасить входящий и исходящий трафик в контейнеризированной среде.

Для любого компонента, способного выступать в роли конечной точки TLS-соединения, вам понадобятся три элемента:

- закрытый ключ, который следует хранить в секрете и никогда никому не давать;
- сертификат для свободного распространения, с помощью которого другие компоненты могут проверить его «личность»;
- сертификаты от одного или нескольких доверенных ЦС для проверки сертификатов, полученных от других компонентов.

Контейнеризированные рабочие нагрузки могут сами выступать в роли конечных точек TLS или передавать эту функциональность другим компонентам, например сервисной сетке.

Теперь у вас должны сформироваться четкое представление о роли ключей, сертификатах и ЦС. Эти знания пригодятся вам при настройке компонентов и отладке проблем.

Если вы доверяете соединениям между контейнерами и умеете идентифицировать компоненты на другом конце соединения, то можете начать передавать между контейнерами *конфиденциальные данные*. Но для этого вам нужно научиться безопасно передавать в контейнеры конфиденциальные значения, о чем мы и поговорим в главе 14.

Передача в контейнеры секретных данных

Коду приложения для работы нередко требуются какие-либо учетные данные, например пароль для доступа к базе данных или токен для взаимодействия с определенным API. Учетные (секретные) данные существуют специально для того, чтобы ограничить доступ к ресурсам, например к базе данных и API. Важно обеспечить секретность этой информации и в соответствии с принципом минимума привилегий предоставлять доступ к ним лишь тем людям или компонентам, которым они действительно нужны.

Мы начнем эту главу с разбора желаемых свойств секретных данных (секретов), затем рассмотрим варианты передачи их в контейнеры, а завершим ее обсуждением нативной поддержки секретов в Kubernetes.

Свойства секретных данных

Самое очевидное свойство секретных данных — они должны оставаться секретными, то есть доступными только для тех людей (или сущностей), для которых они предназначены. Обычно такая конфиденциальность достигается за счет шифрования и передачи ключа для расшифровки лишь тем, кто имеет соответствующие привилегии.

Секреты необходимо хранить в зашифрованном виде, чтобы их не смогли считать все пользователи или сущности, имеющие доступ к хранилищу данных. При перемещении секретов из хранилища в место их применения их также необходимо шифровать, чтобы никто их не «подсмотрел» по пути. Старайтесь никогда не записывать секреты на диск в незашифрованном виде. Если приложению понадобится незашифрованная версия, то ее лучше хранить исключительно в оперативной памяти.

Имейте в виду, что просто зашифровать секреты недостаточно. Получателю нужно знать, как расшифровать полученные данные, а значит, должен существовать ключ расшифровки. Он сам по себе тоже является секретом, и его

нужно каким-то образом передать получателю, что возвращает нас к исходному вопросу о безопасной передаче секретов теперь уже на следующем уровне.

Кроме того, необходимо иметь возможность *отзывать* (revoke) секреты, то есть аннулировать их в том случае, если появились основания не доверять им. Такое может произойти, если вы узнали (или заподозрили), что доступ к ним получил кто-то посторонний. Вдобавок секретные данные иногда приходится отзывать исключительно по производственной необходимости, например, когда кто-то из разработчиков увольняется.

Важно также иметь возможность *заменять* (rotate) секреты. Совсем не факт, что вы сможете быстро узнать об их утечке, поэтому необходимо менять их время от времени, чтобы злоумышленник, получивший доступ к учетным данным, рано или поздно обнаружил, что они уже недействительны. Общеизвестно, что заставлять людей часто менять пароли — плохая идея (<https://oreil.ly/ETKEZ>), но программные компоненты способны справиться с частой сменой учетных данных.

Желательно, чтобы жизненный цикл секретов не зависел от жизненного цикла компонента, который их использует. Это удобно, поскольку в таком случае не нужно заново собирать и распространять компонент при изменении секретных данных.

Как правило, круг людей, которые должны иметь доступ к секретам, намного уже круга людей, которые имеют доступ к исходному коду приложения, использующего эти данные, развертывают или администрируют систему. Например, в банке разработчикам вряд ли нужно предоставлять доступ к данным, позволяющим просматривать банковские счета. Нередко учетные данные доступны людям только для записи: после того как секреты будут сгенерированы (зачастую автоматически, случайным образом), никаких законных оснований снова их просматривать может и не быть.

Доступ к секретным данным следует ограничить не только для людей. Желательно, чтобы его имели только те компоненты, которым он действительно нужен. А поскольку мы говорим о контейнерах, это значит, что секреты должны видеть лишь те контейнеры, которым они требуются для работы.

Теперь, когда мы обсудили желаемые свойства секретов, посмотрим на возможные механизмы их передачи в код приложения, выполняющегося в контейнере.

Передача информации в контейнер

Принимая во внимание, что контейнер изначально задуман как изолированная от окружающего мира сущность, неудивительно, что возможностей передачи информации — в том числе секретных данных — в работающий контейнер не так уж и много:

- данные можно включить в образ контейнера в виде файла в корневой файловой системе образа;

- в конфигурации, сопутствующей образу, можно определить переменные окружения (см. главу 6, где рассказывается о том, что образ состоит из корневой файловой системы и информации о настройках);
- контейнер способен получать информацию через сетевой интерфейс;
- допускается задавать или переопределять переменные окружения в момент запуска контейнера (например, добавляя параметры `-e` в команду `docker run`);
- контейнер может монтировать тома с хоста и считывать информацию из томов.

Рассмотрим каждый из этих вариантов по очереди.

Хранение секретных данных в образе контейнера

Первые два из упомянутых вариантов не подходят для передачи секретов, поскольку требуют их включения в образ во время сборки. Так сделать, конечно, можно, но это считается плохой практикой.

- Секреты будут видны всем, у кого есть доступ к исходному коду образа. Конечно, можно хранить эти данные в коде в зашифрованном виде, а не открытым текстом, но тогда придется как-то передать еще один секрет, чтобы контейнер мог расшифровать первый. И какой же механизм использовать для передачи этих вторых секретных данных?
- Секреты нельзя изменить, не собрав повторно образ контейнера, однако лучше, чтобы эти процессы были независимы. Более того, централизованная, автоматизированная система управления секретными данными (например, CyberArk или Hashicorp Vault) не сможет контролировать жизненный цикл секретов, жестко «защитых» в код.

К сожалению, встраивание секретов в исходный код — очень распространенная практика. Во-первых, не все разработчики знают, что это плохая идея; во-вторых, довольно легко вставить секретные данные непосредственно в код как временное решение на этапе разработки или тестирования, однако многие потом забывают их удалить.

Некоторые инструменты сканирования образов (обсуждаются в главе 8) могут помочь обнаружить секреты, жестко «защитые» в образ контейнера, чтобы затем их удалить и использовать более эффективный механизм!

А раз передача секретных данных во время сборки даже не обсуждается, то все остальные варианты связаны с передачей их во время запуска или работы контейнера.

Передача секретных данных по сети

Третий вариант — передача секретов через сетевой интерфейс. Обычно для управления секретами используются сервисы вроде AWS Secrets Manager или HashiCorp Vault, а код приложения выполняет соответствующие сетевые запросы, чтобы извлечь секреты из такого сервиса.

Однако для обеспечения безопасности секрета сетевой трафик между контейнером и сервисом должен быть зашифрован, что создает определенные проблемы на начальном этапе: как было показано в главе 13, контейнеру необходимы секретные учетные данные, чтобы установить защищенное сетевое соединение.

Для решения этого вопроса можно использовать прозрачное шифрование или переложить эту часть задачи на сервисную сетку, однако самой сервисной сетке все равно потребуется доступ к учетным данным, передаваемым через какой-то другой механизм.

Передача секретных данных в переменных окружения

Четвертый вариант — передача секретов с помощью переменных окружения — тоже обычно не приветствуется по нескольким причинам.

- Во многих языках программирования и фреймворках фатальный сбой приводит к выгрузке дампа отладочной информации, которая может включать все переменные окружения. И если эти сведения попадут в систему журналирования, то все, у кого имеется доступ к журналам, увидят секретные данные, передаваемые в переменных окружения.
- Если выполнить для контейнера команду `docker inspect` (или аналогичную ей), то можно увидеть все переменные окружения, заданные как во время сборки, так и во время выполнения. А администраторам, которым по вполне уважительным причинам необходимо просматривать свойства контейнера, вовсе не обязательно иметь доступ к секретам.
- Любой, у кого есть доступ к хосту, способен выполнить команду `cat /proc/<ID процесса>/environ`, чтобы увидеть все переменные окружения для этого процесса (подробности см. в разделе «Секретные данные доступны суперпользователю хоста» далее в главе).

Ниже представлен пример извлечения переменных окружения из образа контейнера:

```
$ docker image inspect --format '{{.Config.Env}}' nginx
[PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
NGINX_VERSION=1.27.5 NJS_VERSION=0.8.10 NJS_RELEASE=1~bookworm
PKG_RELEASE=1~bookworm DYNPKG_RELEASE=1~bookworm]
```

Можно также легко просмотреть переменные окружения в среде выполнения. Как показано в следующем примере, выводимые результаты включают все определения, указанные в команде `run` (в данном случае `EXTRA_ENV`):

```
$ docker run -e EXTRA_ENV=HELLO --rm -d nginx
13bcf3c571268f697f1e562a49e8d545d78aae65b0a102d2da78596b655e2f9a
$ docker container inspect --format '{{.Config.Env}}' 13bcf
[EXTRA_ENV=HELLO PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
NGINX_VERSION=1.27.5 NJS_VERSION=0.8.10 NJS_RELEASE=1~bookworm
PKG_RELEASE=1~bookworm DYNPKG_RELEASE=1~bookworm]
```

Манифест *The Twelve-Factor App* (12-факторное приложение; <https://12factor.net/ru/config>) поощрял разработчиков передавать параметры настроек через переменные окружения, поэтому на практике можно встретить сторонние контейнеры, которые ожидают именно такой способ конфигурации, включающий и некие секретные значения. Снизить риски при передаче секретов через переменные окружения можно несколькими способами (их целесообразность зависит от вашего профиля рисков):

- обработав выходные журналы: удалив или выполнив обфускацию секретных значений;
- изменив контейнер приложения (или используя sidcar-контейнер) так, чтобы он извлекал секретные данные из защищенного хранилища (наподобие Hashicorp Vault, CyberArk Conjur или систем управления секретными данными/криптографическими ключами поставщиков облачных сервисов). Некоторые коммерческие решения по обеспечению безопасности предоставляют подобную интеграцию.

AWS Fargate — пример управляемого контейнерного сервиса, поддерживающего передачу секретов с помощью переменных окружения. Вместо того чтобы указывать секретное значение непосредственно в конфигурацию задачи Fargate, определение задачи может ссылаться на секреты, безопасно хранящиеся в зашифрованном виде в AWS Secrets Manager. Это означает, что само определение задачи не содержит конфиденциальных данных (что в противном случае было бы аналогично хранению чувствительных данных в исходном коде образа контейнера). Тем не менее к моменту, когда контейнеризированное приложение, работающее в Fargate, увидит значение, полученное из сервиса Secrets Manager, это будет незашифрованная переменная окружения.

И последнее, что стоит отметить по поводу передачи секретов через переменные окружения: окружение для процесса формируется однократно — в момент создания процесса. И вам не удастся заново задать параметры окружения контейнера снаружи, если вы захотите изменить секретные данные.

Передача секретных данных через файлы

Лучший способ передавать секретные данные — записывать их в файлы, к которым у контейнера есть доступ через смонтированный том. Желательно, чтобы этот том представлял собой временный каталог, хранящийся в оперативной памяти, а не записываемый на диск. Например, секреты Docker Swarm (<https://oreil.ly/aUB8v>) и Kubernetes можно монтировать в контейнеры с помощью файловой системы в памяти. В сочетании с защищенным хранилищем секретов это гарантирует, что такие данные никогда не будут храниться в незашифрованном виде.

А поскольку файл монтируется с хоста в контейнер, его можно в любой момент обновить на стороне хоста, не перезапуская контейнер. И если приложение знает, как извлечь новые секретные данные из файла, когда старые перестали работать, то можно менять их, не прибегая к перезапуску контейнеров. Требование к приложениям отслеживать обновление секретов легко реализовать с помощью механизма `inotify` в Linux, посредством которого файловая система способна отправлять событие процессу, чтобы сообщить об изменении файла. (Механизм `inotify` также пригодится инструментам безопасности, которые могут подписываться на события, связанные с доступом к конфиденциальным файлам.)

Секретные данные в Kubernetes

Хорошая новость для тех, кто использует Kubernetes, — его встроенная поддержка секретов (<https://oreil.ly/xIgQs>) удовлетворяет многим критериям, описанным в начале главы:

- секреты Kubernetes создаются в виде независимых ресурсов, так что никак не привязаны к жизненному циклу использующего их приложения;
- секреты можно хранить (вместе с другими данными ресурсов) в `etcd` в виде значений в кодировке `base64`. Данные в `etcd` по умолчанию не шифруются, но Kubernetes имеет такую функцию (<https://oreil.ly/aIDnX>); ее можно включить для шифрования секретов (и любых других ресурсов по вашему выбору, которые вы считаете конфиденциальными). При использовании управляемого сервиса Kubernetes вы, скорее всего, обнаружите, что шифрование либо включено по умолчанию, либо легко настраивается (можно также зашифровать все хранилище данных `etcd`, но это делается редко, поскольку Kubernetes предлагает шифрование ресурсов на уровне сервера API, что обычно проще в управлении);
- секреты передаются между контейнерами в зашифрованном виде. Для этого необходимо установить защищенные соединения между компонентами Kubernetes (например, TLS-соединения между сервером API и хранилищем данных `etcd`), хотя этот вариант используется по умолчанию в большинстве случаев;
- Kubernetes поддерживает передачу секретов как через файлы, так и через переменные окружения, с монтированием секретов в виде файлов во временную файловую систему, которая хранится исключительно в оперативной памяти и никогда не записывается на диск;
- при желании можно настроить RBAC (role-based access control, управление доступом на основе ролей) Kubernetes так, чтобы пользователи могли создавать секреты, но не могли затем читать их, то есть им предоставляются права только на запись.

Помимо встроенной поддержки передачи секретов, Kubernetes имеет драйвер Secrets Store Container Storage Interface (CSI), который можно использовать вместо встроенных объектов `Secret` в Kubernetes.

Драйвер Secrets Store CSI

Это расширение позволяет извлекать секретные данные непосредственно из сервиса управления секретами (например, Vault или из службы управления ключами поставщика облачных услуг) в среде выполнения и монтировать их в поды в виде файлов. Эти секретные данные никогда не сохраняются в `etcd` и никогда не попадают в переменные окружения.

Чтобы начать использовать этот подход, возможно, потребуется обновить приложения, чтобы они читали секреты из нужных файлов, а также перезапускать их при смене ключей (если только они не могут обнаруживать обновления с помощью `inotify`).

Можно синхронизировать эти ресурсы с объектами `Secret` в Kubernetes, хотя это, казалось бы, сводит на нет весь смысл использования драйвера Secrets Store. Однако возможность синхронизации может быть полезна во время миграции, когда некоторым приложениям требуется старый способ работы с объектами `Secret`, например, потому, что они читают данные из переменных окружения. Кроме того, некоторые ресурсы могут ссылаться на объекты `Secret` (например, ресурс `Ingress` может искать сертификаты TLS в ресурсе `Secret`).

External Secrets Operator

Другой подход заключается в использовании External Secrets Operator (<https://external-secrets.io>) — оператора Kubernetes с открытым исходным кодом, который получает секретные данные из внешних сервисов управления секретами, таких как AWS Secrets Manager, HashiCorp Vault или Google Secret Manager, и синхронизирует их с ресурсами `Secret` в Kubernetes. При этом подходе не нужно вносить изменения в приложения, потому что они могут получать секретные данные обычным для Kubernetes способом — через переменные окружения или смонтированные файлы. Однако здесь используются встроенные в Kubernetes объекты `Secret`, которые хранятся в `etcd`, в отличие от подхода с применением драйвера Secrets Store CSI, где они никогда не попадают в хранилище кластера.

Ротация секретных данных в Kubernetes

Задача ротации (смены) секретов состоит из:

- ротации секретного значения Kubernetes, передаваемого приложению;
- ротации ключей в ресурсе `EncryptionConfig`, используемом для шифрования ресурсов `Secret` в `etcd`.

При работе с обычными объектами `Secret` их значения можно обновить с помощью `kubectl`, после чего, как правило, требуется перезапустить соответствующие поды, чтобы приложение начало применять новые значения. Сервисы управления секретными данными (например, Vault или AWS Secrets Manager) упрощают этот процесс: при использовании External Secrets Operator секретные значения обновляются автоматически из внешнего сервиса, правда, приложения все равно приходится перезапускать, чтобы они получили эти новые значения (если только приложения не настроены на постоянное отслеживание изменений).

Ротация ключей, используемых для шифрования ресурсов `Secret`, — это многоэтапный процесс (<https://oreil.ly/GaNMj>), который включает изменение ресурса `EncryptionConfig` и перезапуск сервера (серверов) API как минимум дважды.

1. Добавить новый ключ в качестве второй записи в ресурс `EncryptionConfig`.
2. Перезапустить серверы API, чтобы прочитать новый `EncryptionConfig`. У них есть доступ к новому ключу, и они могут использовать его для расшифровки, если столкнутся с ресурсом, который не удастся расшифровать с помощью старого ключа.
3. Поменять ключи местами, чтобы новый ключ стал первой записью в ресурсе `EncryptionConfig`.
4. Перезапустить серверы API, чтобы повторно прочитать ресурсы `EncryptionConfig` и начать использовать новый ключ для шифрования.
5. Заменить все существующие ресурсы `Secret`, чтобы они были зашифрованы новым ключом.
6. Обновить `EncryptionConfig` и удалить старый ключ.

Как показывает мой опыт, большинство организаций выбирает стороннее коммерческое решение для хранения секретов — либо от своего поставщика облачных сервисов (например, AWS Key Management System или ее эквиваленты на платформах Azure и GCP), либо от таких компаний, как Hashicorp или CyberArk. Эти решения имеют ряд преимуществ.

- Специализированную систему управления секретными данными можно использовать сразу для нескольких кластеров. Секретные значения допускается менять независимо от жизненного цикла кластеров приложений.
- Эти программные решения помогают организациям стандартизировать хранение секретов, а также внедрить использование единых практических рекомендаций по управлению и согласованному журналированию/аудиту секретных данных.



Все крупные поставщики облачных услуг документируют свои рекомендации по шифрованию секретных данных в Kubernetes:

- документация AWS по использованию Key Management Service с EKS (<https://oreil.ly/SoQUL>);
- документация Microsoft по использованию Key Management Service с AKS (<https://oreil.ly/VHlRd>);
- документация Google по использованию Cloud Key Management с GKE (<https://oreil.ly/BgGX2>).

Секретные данные доступны суперпользователю хоста

Вне зависимости от того, передаются ли секретные данные в контейнер в виде смонтированного файла или переменной окружения, суперпользователь хоста может при желании получить к ним доступ.

Если секреты хранятся в файле, значит, он находится где-то в файловой системе хоста. Даже если он располагается во временном каталоге, суперпользователь хоста сможет получить к нему доступ. Для примера попробуйте вывести список временных файловых систем, смонтированных на узле Kubernetes. Вы увидите примерно следующее:

```
$ sudo mount -t tmpfs
...
tmpfs on /var/lib/kubelet/pods/691c...5f8a/volumes/kubernetes.io~projected
/kube-api-access-lbgtt type tmpfs (rw,relatime,size=8024812k,noswap)
...
```

Исходя из названий каталогов, приведенных в этом выводе, суперпользователь без труда получит доступ к хранящимся в них файлам с секретными данными:

```
$ sudo ls -l $DIR
total 0
lrwxrwxrwx 1 root root 13 Jul 30 12:05 ca.crt -> ..data/ca.crt
lrwxrwxrwx 1 root root 16 Jul 30 12:05 namespace -> ..data/namespace
lrwxrwxrwx 1 root root 12 Jul 30 12:05 token -> ..data/token

$ sudo cat $DIR/ca.crt
-----BEGIN CERTIFICATE-----
MIIDBTCCAe2gAwIBAgIIeRwAgMJOFy4wDQYJKoZIhvcNAQELBQAwFTETMBEGA1UE
...
aRW9Jb5JqekJ
-----END CERTIFICATE-----
```

Извлечь секреты из переменных окружения для суперпользователя ничуть не сложнее. Продемонстрирую это, запустив контейнер с помощью Docker в командной строке и передав в него переменную окружения:

```
$ docker run --rm -it -e SECRET=mysecret ubuntu sh
# env
...
SECRET=mysecret
...
```

В данном контейнере запущен `sh`, и в другом терминале можно узнать идентификатор процесса для этого исполняемого файла:

```
$ ps -C sh
  PID TTY          TIME CMD
17322 pts/0    00:00:00 sh
```

В главе 4 вы видели, что в каталоге `/proc` содержится много интересной информации о процессах, включая все переменные окружения, находящиеся в `/proc/<ID процесса>/environ`:

```
$ sudo cat /proc/17322/environ
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=2cc99c98ba5a
TERM=xterm
SECRET=mysecret
HOME=/root
```

Как видите, этот способ позволяет прочесть любые секретные данные, передаваемые через переменные окружения. А если вам кажется, что лучше будет сначала зашифровать секретные данные, то задумайтесь, как передать в контейнер ключ для их расшифровки, который также необходимо хранить в секрете?

Совсем не лишним будет еще раз подчеркнуть: любой, кто обладает привилегиями суперпользователя на хосте, имеет полный доступ ко всему на хосте, включая все контейнеры и их секретные данные. Именно поэтому так важно не допускать несанкционированного доступа записи суперпользователя в развернутой системе и именно поэтому работа от имени `root` в контейнере настолько опасна: поскольку суперпользователь в контейнере фактически является суперпользователем на хосте, до компрометации всей системы остается всего один шаг.

Резюме

Если вы внимательно прочитали все изложенное в книге ранее, то наверняка уже имеете неплохое представление о том, как работают контейнеры и как передавать между ними секретную информацию. Вы уже видели множество способов эксплуатации уязвимостей контейнеров и немало средств их защиты.

Последний механизм защиты, который мы обсудим в следующей главе, — это *защита среды выполнения* (runtime protection).

Защита среды выполнения контейнеров

В главе 10 мы рассмотрели некоторые подходы к усилению изоляции контейнеров с помощью профилей безопасности, таких как `seccomp`, `AppArmor` и `SELinux`. Все они функционируют в среде выполнения, поэтому способствуют защите контейнеров в процессе работы и ограничивают набор доступных им действий. В этой главе рассказывается о более продвинутых инструментах, обладающих более гибкими возможностями обнаружения подозрительной активности в среде выполнения, а также использующих политики, которые можно настраивать для отдельных рабочих нагрузок.

Термин «наблюдаемость безопасности» (*security observability*) относится к генерации журналов и метрик, а также к инструментам, помогающим командам понимать события, связанные с безопасностью и происходящие в развернутой системе. Современные инструменты наблюдаемости безопасности, действующие в среде выполнения, генерируют данные, которые включают информацию об идентификации контейнеров или окружений Kubernetes, что значительно упрощает сопоставление подозрительных событий с конкретными контейнеризированными рабочими нагрузками. Некоторые инструменты позволяют выборочно *применять* политики, предотвращающие подозрительную активность.

В этой главе вы сначала увидите, как использовать контейнеризацию для создания политик, специфичных для конкретных рабочих нагрузок, и познакомитесь с некоторыми типами поведения, которые инструменты могут отслеживать и ограничивать. Затем вы изучите технологические средства принудительного соблюдения поведения в среде выполнения и некоторые из доступных инструментов.

Контейнеры отлично подходят для *микросервисных* архитектур. Разработчик приложения может разбить сложную программную систему на маленькие самодостаточные фрагменты кода, масштабируемые независимо друг от друга, каждый — в виде образа контейнера.

Разбиение большой системы на маленькие компоненты с четко очерченными интерфейсами упрощает проектирование, написание кода и тестирование отдельных компонентов. Кроме того, это также упрощает обеспечение безопасности.

Политики использования контейнерных образов в среде запуска

Если образ контейнера содержит код микросервиса, выполняющего ровно одну небольшую функцию, то относительно легко понять, что этот микросервис должен делать. Код микросервиса встроен в образ контейнера, и можно сформировать соответствующий этому образу профиль (или политику) среды запуска, определяющий, что именно ему разрешено делать.

Все контейнеры, созданные на основе данного образа, должны вести себя одинаково, поэтому логично описать профиль желаемого поведения для образа, а затем использовать его для контроля входящего/исходящего трафика всех контейнеров, основанных на этом образе.



В системах, развертываемых в Kubernetes (deployments), безопасность в среде выполнения можно контролировать между всеми подами. Под, по сути, представляет собой набор контейнеров с общим пространством имен, поэтому и механизмы обеспечения безопасности в среде выполнения остаются теми же.

В качестве примера я воспользуюсь платформой интернет-магазина, представленной в разделе «Межсетевые экраны для контейнеров и микросегментация» главы 12. Она имеет микросервис поиска товаров, который принимает веб-запросы по ключевому слову (или по первым нескольким символам), вводимому покупателем на сайте интернет-магазина. Задача микросервиса — отыскать в базе товаров элементы, соответствующие запросу, и вернуть результаты. Начнем с ожидаемого сетевого трафика для этого микросервиса.

Сетевой трафик

Из описания микросервиса поиска товаров понятно, что его контейнеры должны получать веб-запросы от балансировщика нагрузки или объекта Ingress и отвечать на них, а также открывать соединение с базой данных товаров. Помимо распространенных функций платформы, например журналирования или проверок состояния, не существует причин, почему этот сервис мог бы обрабатывать или порождать какой-либо прочий сетевой трафик.

Не так уж сложно составить политику, определяющую допустимый для этого сервиса трафик, а затем на ее основе описать правила, соблюдение которых обеспечивается на сетевом уровне, как вы видели в главе 12. Такие правила могут требовать, чтобы входящие запросы были разрешены только от объекта `Ingress` или от балансировщика нагрузки, а исходящий трафик допускался только к сервису базы данных товаров. Политика уровня 7 могла бы дополнительно определять, что эти запросы должны быть HTTP-запросами методом `GET`.

Существуют инструменты безопасности, способные работать в режиме записи, выполнять мониторинг входящих/исходящих сообщений сервиса за некий период времени и автоматически формировать профиль типичного потока трафика. Такой профиль можно затем преобразовать в сетевые политики.

Однако не только сетевой трафик подходит для наблюдения и профилирования.

Исполняемые файлы

Сколько исполняемых программ должно работать в этом микросервисе поиска товаров? В качестве примера допустим, что его код написан на языке Go в виде одного исполняемого файла `productsearch`. Если вы посмотрите список исполняемых файлов, работающих внутри соответствующих контейнеров поиска товаров, то вы увидите только `productsearch`. Если увидите что-то еще — это аномалия и, вероятно, признак атаки.

Даже если сервис написан на интерпретируемом языке вроде Python или Ruby, можно делать определенные выводы о том, какие варианты допустимы, а какие — нет. Приходится ли сервису «выходить в командную оболочку» для запуска других команд? Если нет, то исполняемые файлы `bash`, `sh` или `zsh`, запущенные в контейнере поиска товаров, должны вас насторожить.

Все вышеописанное предполагает, что контейнеры считаются неизменяемыми и вы не открываете командную оболочку в контейнерах. С точки зрения безопасности практически нет разницы между злоумышленником, получившим доступ к командной оболочке через уязвимость приложения, и открытием администратором консоли для выполнения технического обслуживания системы. Как обсуждалось в подразделе «Неизменяемые контейнеры» в главе 8, подобное считается плохой практикой!

Все исполняемые файлы и зависимости, необходимые приложению, должны быть включены в этот образ. Мы уже обсуждали это, когда говорили об обнаружении уязвимостей: невозможно просканировать код на наличие уязвимостей, если его нет в образе, поэтому важно включать туда все, что подлежит сканированию. Интерпретация контейнеров как неизменяемых сущностей дает еще один мощный инструмент, позволяющий обнаруживать инъекции

кода в среде выполнения: *предотвращение расхождений* (drift prevention). Всякий раз, когда контейнер пытается запустить новый процесс, это должно быть разрешено, только если исполняемый файл существовал в образе на момент сканирования. Этот подход, использующий не просто список имен, но также цифровые отпечатки файлов, полученные на этапе сканирования, может предотвратить попытку злоумышленника замаскировать внедренный исполняемый файл под легитимный.

В большинстве случаев мы запускаем программы, хранящиеся в файлах, но существует также и такое понятие, как *бесфайловое выполнение* (fileless execution). В этом случае инструкции программы хранятся в памяти и никогда не записываются на диск. Данный метод часто используется вредоносными программами для обхода проверок антивирусами, которые сканируют файлы по соответствующим сигнатурам. Современные средства обеспечения безопасности в среде выполнения могут обнаруживать попытки запуска кода как из памяти, так и из файлов.

Доступ к файлам

Наш гипотетический микросервис поиска товаров практически не обращается к файлам, за исключением записи каких-то событий в журнал и, возможно, чтения секретных ключей для аутентификации в базе данных товаров.

Но это относится только к тем файлам, к которым разработчик микросервиса обращается явно. В зависимости от языка программирования, лежащего в основе сервиса, в среде выполнения может использоваться несколько разделяемых библиотек и файлов. Например, возьмем простой исполняемый файл, такой как *cat*, и с помощью инструмента *opensnoop* посмотрим, к каким файлам обращается эта утилита:

PID	COMM	FD	ERR	PATH
45883	cat	3	0	/etc/ld.so.cache
45883	cat	3	0	/lib/aarch64-linux-gnu/libc.so.6
45883	cat	3	0	/usr/lib/locale/locale-archive
... еще 20 попыток доступа к другим файлам ...				
45883	cat	-1	2	/usr/lib/locale/C.UTF-8/LC_CTYPE
45883	cat	3	0	/usr/lib/locale/C.utf8/LC_CTYPE
45883	cat	3	0	myfile

Политика, ограничивающая набор файлов, к которым контейнер может получить доступ, должна учитывать все подобные вспомогательные файлы. Данный список файлов довольно длинный, поэтому даже опытный программист мог бы случайно пропустить часть из них, если бы создавал политику вручную, однако некоторые инструменты безопасности позволяют автоматически создавать полный профиль работающего контейнера и затем оповещать о попытках доступа к файлам, выходящим за рамки профиля, и/или блокировать их.

Другой, более универсальный подход заключается в определении набора файлов, которые считаются конфиденциальными и не должны использоваться (по крайней мере для записи) никакими рабочими нагрузками. Целесообразно регистрировать все обращения к некоторым файлам, например к `/etc/shadow`, в котором хранится информация о паролях. Можно даже предусмотреть блокировку любых попыток записи в этот файл. В среде Kubernetes также следует защитить каталог `/etc/kubernetes/manifests` от несанкционированной записи, чтобы предотвратить создание нежелательных статических подов.

Идентификаторы пользователей и групп

Как обсуждалось в главе 6, можно задать идентификатор пользователя, от имени которого будут запускаться процессы внутри контейнера, и это еще один аспект, который позволяет контролировать инструменты безопасности в среде выполнения. (Надеюсь, в профилях своих приложений вы не используете суперпользователя — см. раздел «Выполнение контейнеров по умолчанию от имени суперпользователя» главы 11.)

Как правило, контейнеру, выполняющему только одну задачу, достаточно работать от имени одного пользователя. Если этот контейнер использует другую учетную запись, то это сразу должно вызвать подозрения. Если же какой-либо процесс неожиданно запускается от имени суперпользователя, то подобное повышение привилегий — еще больший повод для беспокойства.

Для контроля за идентификаторами пользователей, которые должны использоваться в контейнерах, можно применить сканирование и контроль доступа. Для организации многоуровневой защиты задействуйте инструмент обеспечения безопасности в среде выполнения, который также способен обнаруживать попытки процесса изменить пользователя или группу, от имени которого он работает.

Применение ИИ для генерации политик в среде выполнения

Когда я писала первое издание, идея поручить ИИ проанализировать ПО и сгенерировать для него профиль безопасности казалась научной фантастикой. Сейчас, работая над вторым изданием, я замечаю стремительный рост инструментов программирования на основе больших языковых моделей (LLM). На данный момент результаты использования ИИ все еще весьма неоднозначны, но выглядят многообещающими и быстро улучшаются. Я верю, что вскоре средства на основе ИИ смогут определять, какие файлы (исполняемые и неисполняемые), привилегии и сетевые соединения необходимы для успешной работы образа контейнера. И я не удивлюсь, если к моменту, когда вы читаете эти строки, такие инструменты уже появились!

В настоящее время существуют компании, обещающие автономную сегментацию и распределенную защиту от эксплойтов путем применения политик, сгенерированных автоматически на основе наблюдаемого поведения и исследований уязвимостей.

Независимо от того, создаются ли профили и политики вручную или автоматически, вам понадобится инструмент, способный обнаруживать, когда процесс в контейнере делает что-то выходящее за рамки ожидаемого поведения. Поэтому рассмотрим технологические решения, позволяющие выявлять и даже предотвращать действия, нарушающие политики.

Технологические решения для безопасности в среде выполнения

За прошедшие годы использовалось несколько различных технологических решений для создания инструментов защиты в среде выполнения.

LD_PRELOAD

Как было показано в подразделе «Доступ к файлам», команда `cat` открывает несколько файлов. Если вернуться к показанному мной выводу, то можно заметить, что первый файл — `/etc/ld.so.cache`. Это кэш *динамических библиотек* — зависимостей, которые загружаются исполняемым файлом при запуске. `LD_PRELOAD` — давно известная технология, с помощью которой инструмент внедряет собственный код вместо использования обычных разделяемых динамических библиотек.

Инструмент обеспечения безопасности в среде выполнения может задействовать `LD_PRELOAD` для добавления средств мониторинга, которые запускаются до того, как приложение вызовет код из стандартной библиотеки. Обычно этот прием применяется для переопределения функций в стандартной библиотеке C, включая системные вызовы.

К сожалению, у этого подхода есть несколько недостатков.

- Он работает только с динамически скомпонованными приложениями. Злоумышленник может скомпилировать свой код как автономный исполняемый файл и обойти любые средства мониторинга, основанные на предварительной загрузке. Ваши собственные исполняемые файлы тоже могут быть автономными двоичными файлами, особенно если они написаны на Rust или Go, поэтому инструменты мониторинга, базирующиеся на технологии предварительной загрузки, окажутся полностью «слепыми» к их действиям.
- Вредоносное приложение может изменять настройки `LD_PRELOAD`, чтобы обойти инструменты, использующие этот механизм.

- В контейнерной среде разные приложения могут использовать различные реализации стандартных библиотек C (например, `glibc` или `musl`) и даже собственные версии этих библиотек. Написать обертки `LD_PRELOAD`, устойчивые ко всем этим вариантам, непросто.

На начальном этапе технология `LD_PRELOAD` демонстрировала хорошие результаты и была положена в основу первых коммерческих инструментов обеспечения безопасности контейнеров. Однако с появлением более совершенных методов, таких как `eBPF`, и с распространением приложений на `Go` и `Rust` `LD_PRELOAD` утратила свою надежность и уже не может считаться надежной базой для современных средств защиты.

ptrace

`ptrace` — это системный вызов Linux, позволяющий одному процессу наблюдать за другим и даже управлять им. Он широко используется как фундамент для инструментов отладки и трассировки. С его помощью можно легко определить, например, какие системные вызовы выполняет пользовательское приложение. Выглядит как отличная основа для инструментов обеспечения безопасности, не так ли? К сожалению, этот подход тоже имеет свои проблемы.

- Каждая точка останова, на которую реагирует `ptrace`, запускает переключение контекста с отслеживаемого процесса на процесс, осуществляющий трассировку. Это создает значительные издержки, когда требуется отслеживать сколько-нибудь значительную активность.
- Отслеживаемый процесс способен легко определить, что его отслеживают, и злоумышленники не упустят случая воспользоваться этой информацией.
- `ptrace` можно использовать для извлечения данных из памяти приложения или внедрения в него вредоносного кода, поэтому существует специальная привилегия `CAP_SYS_PTRACE`, ограничивающая применение этого вызова. Обычно она не предоставляется контейнерам из-за риска для безопасности. Если вы добавляете эту привилегию, то обязательно предусмотрите другие средства защиты, чтобы предотвратить ее использование вредоносными процессами. (По умолчанию `ptrace` не работает в контейнерах без полномочий суперпользователя из-за отсутствия привилегии `CAP_SYS_PTRACE`.)
- Во многих дистрибутивах Linux на системный вызов `ptrace` наложено ограничение, позволяющее использовать его только для слежения за дочерними процессами. Определить наличие этого ограничения можно по значению 1 в файле `/proc/sys/kernel/yama/ptrace_scope`.

В настоящее время `ptrace` широко применяется для отладки, но он не является серьезным конкурентом на рынке инструментов обеспечения безопасности контейнеров.



Несколько лет назад я выступила с докладом под названием «Отладчики с нуля» (*Debuggers from Scratch*; <https://www.youtube.com/watch?v=TBrv17QyUE0>), в котором продемонстрировала, как можно использовать `ptrace` для установки точек останова в целях отладки.

seccomp, AppArmor и SELinux

С этими технологиями вы познакомились в главе 10, где они были представлены как средства усиления изоляции контейнеров. Они занимают важное место в подходе к организации многоуровневой защиты в среде выполнения, но уступают современным инструментам.

- Все эти подходы основаны на предопределенных статических политиках с ограниченными возможностями фильтрации: `seccomp` способен фильтровать системные вызовы по номеру и с помощью очень простого сопоставления аргументов, а `AppArmor` и `SELinux` контролируют доступ на основе путей в файловой системе или меток типов. Поскольку они разрабатывались еще до появления контейнеров, будет очень сложно выразить политики вида «разрешить это действие в контейнере А и запретить в контейнере В».
- Они не обеспечивают наблюдаемость безопасности — в лучшем случае журнал покажет, что какое-то действие было заблокировано. Корреляция с идентификаторами контейнеров или `Kubernetes` полностью отсутствует, а генерируемые журналы не предоставляют контекста, поэтому эти инструменты не приносят пользы при реагировании на инциденты.
- Процесс внутри контейнера может полностью соответствовать своему статическому профилю и при этом оставаться вредоносным. Повышение привилегий, бесфайловое выполнение или горизонтальное перемещение практически невозможно обнаружить с помощью этих инструментов.

Одно из преимуществ этих методов — все они работают в ядре, откуда имеют доступ ко всем контейнерам, запущенным на компьютере. Давайте далее рассмотрим преимущество использования ядра в качестве точки наблюдения за контейнерами.

Обеспечение безопасности в среде выполнения на основе ядра

Как было показано в разделе «Процессы контейнера с точки зрения хоста» главы 4, все контейнеры на хосте используют одно ядро. Приложения работают в непривилегированном пользовательском пространстве и выполняют системные вызовы через ядро, когда им нужно получить доступ к файлу, отправить сообщение по сети или выделить память. Кроме того, ядро отвечает за координацию существующих и создание новых процессов, а также за проверку и предоставление им привилегий.

Все эти действия представляют определенный интерес с точки зрения безопасности. Инструменты обеспечения безопасности в среде выполнения могут определять политики, указывающие, может ли тот или иной контейнер обращаться к конкретным файлам, устанавливать сетевые соединения с конкретными конечными точками или запрашивать дополнительные привилегии Linux. Поскольку все эти действия затрагивают ядро, оно является идеальным местом для размещения инструментов обеспечения безопасности в среде выполнения (рис. 15.1).

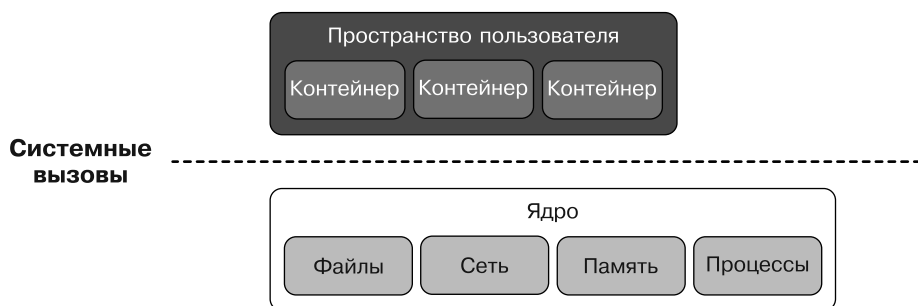


Рис. 15.1. Приложения в контейнерах используют общее ядро

Как показано на рис. 15.1, контейнеры используют общее ядро, которое координирует работу процессов, проверяет их полномочия и привилегии и обрабатывает операции, такие как доступ к файлам, сети и памяти. Ядро является идеальной точкой наблюдения для инструментов безопасности, обеспечивая видимость всех контейнеризированных приложений, работающих на хосте. Более того, инструменты на основе ядра могут иметь жизненный цикл, не зависящий от управляемых ими контейнеров. Это огромное преимущество по сравнению с моделью *sidecar*-контейнеров, с которой вы познакомились в главе 12 в контексте сервисных сеток.

Исторически единственным способом расширения ядра было использование его модулей. Главная проблема модулей ядра — их хрупкость: сбой модуля ядра может привести к остановке всей системы, в которой он работает. То же верно и в отношении самого ядра, но обычно ядро проходит серьезную проверку и доработку, прежде чем попасть в дистрибутив Linux, развертываемый в рабочих средах. Для сравнения, на момент написания книги (2025 год) широко применялся дистрибутив Red Hat Enterprise Linux 9, основанный на ядре 5.14, которое было выпущено в 2021 году. Однако любой конкретный модуль ядра используется лишь крошечной частью пользователей Linux, поэтому велика вероятность, что в нем остаются ошибки, которые могут привести к сбою. По понятным причинам многие организации с осторожностью относятся к использованию модулей ядра или вовсе запрещают взаимодействие с ними.

Надо отметить, что в последние годы появилась технология eBPF, которая позволяет создавать инструменты, работающие в ядре, но неспособные вызвать его сбой, поскольку, как вы знаете из главы 11, программы eBPF проверяются при загрузке. Рассмотрим, как эта технология применяется в инструментах обеспечения безопасности в среде выполнения.

eBPF для обеспечения безопасности в среде выполнения

Технология eBPF позволяет разработчику создавать программы, которые динамически загружаются в ядро и перехватывают его события. Эти программы могут передавать информацию о событиях в пользовательское пространство, а в последние годы возможности eBPF расширились настолько, что появились инструменты безопасности, способные блокировать события или завершать вредоносные процессы.

Самым простым примером мониторинга безопасности является наблюдение за программами в процессе их выполнения. Рассмотрим контейнер `nginx`. В обычных условиях ожидается, что внутри такого контейнера присутствуют только процессы `nginx`. Существует несколько инструментов на основе eBPF, которые могут отображать действующие процессы и позволяют запускать пользовательские программы внутри ядра.

В этом примере мы используем `execsnoop` из пакета `libbpf-tools`. (С утилитой `carable` из этого же пакета вы познакомились в главе 11.) Запускать `execsnoop` нужно от имени суперпользователя, чтобы получить соответствующие привилегии Linux для загрузки кода eBPF в ядро:

```
$ sudo execsnoop
```

После запуска `execsnoop` запустим контейнер `nginx` в другом терминале:

```
$ docker run --rm -d --name nginx
```

В первом терминале появится вывод команды `execsnoop`, показывающий множество новых процессов, запускаемых в контейнере `nginx`. Попробуйте понять, что это за процессы, воспользовавшись знаниями из главы 4 о том, как создаются контейнеры. Вот вывод (немного отредактированный для ясности):

```
PCOMM      PID    PPID   ARGS
docker      43495  38722  /usr/bin/docker run --rm -d --name nginx nginx
containerd-shim 43505  744   /usr/local/bin/containerd-shim-runc-v2 ①
  -namespace moby -address /run/containerd/containerd.sock -publish-binary
  /usr/local/bin/containerd -id 27bfa...e2357 start
containerd-shim 43513  43505 /usr/local/bin/containerd-shim-runc-v2
  -namespace moby -id 27bfa...e2357 -address /run/containerd/containerd.sock
runc        43524  43513 /usr/local/bin/runc ... create ②
  --bundle /run/containerd/io.containerd.runtime.v2.task/moby/27bfa...e2357
  --pid-file /run/containerd/io.task/moby/27bfa...e2357/init.pid 27bfa...e2357
...
```

```

iptables          43543  3127  /usr/sbin/iptables --wait -t raw -C PREROUTING ❸
    -d 172.17.0.3 ! -i docker0 -j DROP
iptables          43545  3127  /usr/sbin/iptables --wait -t raw -A PREROUTING
    -d 172.17.0.3 ! -i docker0 -j DROP
...
runc              43555  43513 /usr/local/bin/runc ... start 27bfa...e2357 ❹
docker-entrypoint 43537  43513 /docker-entrypoint.sh nginx -g daemon off; ❺
find              43561  43537 /usr/bin/find /docker-entrypoint.d/ ... ❻
...
nginx            43537  43513 /usr/sbin/nginx -g daemon off; ❼

```

Все эти процессы были запущены командой `docker`, которая только что была выполнена во втором терминале.

❶ Команда `containerd-shim` была запущена родительским процессом с идентификатором 744 (если вы попробуете выполнить то же самое у себя, то идентификаторы ваших процессов будут отличаться!). Запустив команду `ps 744`, я увидела, что этим родительским процессом является `containerd`. Процесс `containerd-shim` выступает в роли посредника между демоном `containerd` и `runc`, который позволяет процессу в контейнере продолжать работу даже в случае сбоя или перезапуска `containerd`.

❷ Далее выполняется команда `runc create`, которая создает контейнер из указанного пакета. Заглянув в каталог `/run/containerd/io.containerd.runtime.v2.task/moby/27bfa...e2357`, я увидела пакет OCI контейнера с корневой файловой системой и файлом `config.json`. Запуск команды `cat` для просмотра содержимого файла `init.pid` показал мне идентификатор процесса 43537, который можно увидеть еще раз чуть позже.

❸ Процесс 3127 — это демон Docker в данной системе, и, как видите, он настраивает правила `iptables` для нового контейнера. Ничего удивительного, что команда `docker inspect nginx` показала мне IP-адрес этого контейнера 172.17.0.3.

❹ Далее следует команда `runc start`, запускающая в контейнере...

❺ ...Команду `docker-entrypoint.sh nginx`. Эта команда точно соответствует тому, что указано в `process.args` внутри файла `config.json` контейнера, и у нее тот же идентификатор процесса 43537, который я нашла в файле `init.pid`.

❻ Скрипт `docker-entrypoint.sh` запускается командой `find`, которая отыскивает все конфигурационные скрипты `nginx`. В моем случае это привело к запуску еще 16 процессов...

❼ ...Прежде чем наконец запустился демон `nginx`.

Из этого примера можно сделать два ключевых вывода. Во-первых, обратите внимание, что утилита `hexesnoop`, работающая на хосте, способна видеть процессы, запущенные внутри контейнеров. Несмотря на то что контейнер

изолирован с помощью пространств имен и контрольных групп, он все равно использует ядро хоста, поэтому инструменты на основе eBPF, работающие в ядре, также имеют доступ к контейнеру. Это одна из причин, почему eBPF является такой привлекательной технологией для создания инфраструктурных инструментов для контейнерных сред.

Во-вторых, на начальном этапе внутри контейнера выполняются процессы инициализации, но через некоторое время остается только `nginx`. Это распространенная схема, и ее важно учитывать при разработке политик безопасности среды выполнения. В данном случае на этапе инициализации необходимо выполнить поиск, но запуск той же программы `find` позже может указывать на то, что злоумышленник что-то ищет в файловой системе контейнера.

Если теперь выполнить внутри контейнера другую команду, например `docker exec -it nginx ls`, то исполняемый файл отобразится в выводе `execsnoop`:

```
ls 43889 43877 0 /usr/bin/ls
```

Представьте, что злоумышленник внедрил программу для майнинга криптовалюты в один из ваших контейнеров. Когда запускается исполняемый файл этой программы, было бы желательно, чтобы система сообщила об этом событии (а возможно, даже предотвратила запуск), но не хотелось бы получить массу уведомлений каждый раз, когда `nginx` проходит этап инициализации или запускает обычный исполняемый файл `nginx`. Инструменты обеспечения безопасности в среде выполнения могут осуществлять подобное наблюдение, отслеживая запуск исполняемых файлов и сравнивая их с политикой для рабочих нагрузки.

До настоящего момента в этой главе обсуждались политики, определяющие характер поведения контейнера. Теперь рассмотрим инструменты, которые могут сообщать о подозрительном поведении контейнера или предотвращать его.

Инструменты обеспечения безопасности контейнеров в среде выполнения

По причинам, описанным ранее в главе, современные инструменты обеспечения безопасности контейнеров в среде выполнения основаны на eBPF. В мире открытого исходного кода одним из наиболее распространенных средств защиты на момент написания книги остается Falco, однако Tetragon, разрабатываемый в рамках проекта Cilium, уже начал набирать популярность. Помимо прочего, стоит упомянуть Tracee, а также инструмент Inspektor Gadget, который можно использовать для мониторинга безопасности в Kubernetes. Существуют и коммерческие решения для обеспечения безопасности.

Falco

Falco (<https://falco.org>) — это зрелый проект CNCF со статусом graduated, предлагающий возможность обнаружения событий с использованием eBPF или модуля ядра (рис. 15.2).

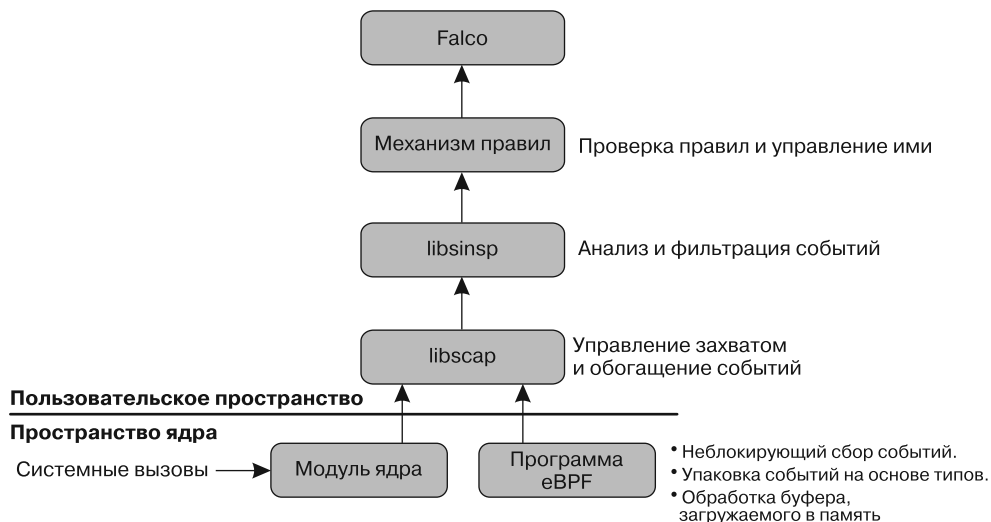


Рис. 15.2. Компоненты Falco¹

Falco имеет большой набор политик, разработанных сообществом и известных как *правила* (rules), например, для обнаружения событий, нарушающих стандарты NIST (<https://falco.org/blog/falco-nist-controls>) или PCI/DSS (<https://falco.org/blog/falco-pci-controls>). В Kubernetes события обогащаются дополнительной информацией: именами модулей и контейнеров, метками, аннотациями и пространствами имен Kubernetes, что позволяет по событиям находить соответствующие рабочие нагрузки.

Примечательно, что, судя по схеме Falco, события определяются в ядре, но анализируются и сравниваются с правилами в пользовательском пространстве.

В Falco нет встроенного механизма принудительного применения политик — максимум можно использовать инструмент пользовательского пространства для реагирования на нарушение правил, например для завершения процесса-нарушителя.

¹ На основе изображения, предоставленного компанией Falco (<https://falco.org/blog/falco-monitoring-new-syscalls>).

Компания Sysdig, создавшая проект Falco, в своем продукте Sysdig Secure предлагает его корпоративную версию, включающую пользовательский интерфейс и средства управления политиками.

Cilium Tetragon

Tetragon (<https://tetragon.io>) — это инструмент обеспечения безопасности в среде выполнения, разработанный в рамках проекта Cilium CNCF, который имеет долгую историю совместной разработки с eBPF. Подобно Falco, Tetragon генерирует информацию о событиях, дополненную данными о контейнерах Kubernetes, а также включает сведения о времени и происхождении процессов, что чрезвычайно важно для расследований и определения причин произошедшего.

Помимо мониторинга безопасности в режиме реального времени, Tetragon предлагает некоторые передовые возможности eBPF.

Фильтрация внутри ядра

Tetragon имеет работающий в пользовательском пространстве агент для сбора данных о событиях с последующей передачей их в базу данных или в систему управления информацией и событиями безопасности (security information and event management, SIEM). Но в отличие от Falco сопоставление с политиками происходит внутри ядра. Например, вам может понадобиться политика, которая сообщает о попытках доступа к определенным файлам. В Tetragon сравнение с интересующими именами файлов происходит внутри ядра, поэтому в пользовательское пространство попадают только значимые события. Это существенно повышает производительность.

API LSM

Правила Tetragon могут задействовать API модуля безопасности Linux (Linux Security Module, LSM). Это тот же API, который используется в таких инструментах, как AppArmor, чтобы обеспечить соблюдение статических политик безопасности, однако в сочетании с eBPF он позволяет реализовывать более сложные динамические политики (например, применять их к новым контейнерам по мере создания).

Принудительное применение

Политики Tetragon могут определять, следует ли сообщать о событии, изменять возвращаемое значение, чтобы обеспечить соблюдение правил или даже завершать процесс-нарушитель.

Политики Tetragon записываются в виде пользовательских ресурсов Kubernetes в YAML-файлах. Это упрощает управление ими в среде Kubernetes, но Tetragon также способен работать на физических или виртуальных

машинах, и с другими контейнерами, не связанными с Kubernetes. Вот пример политики, использующей SIGKILL для принудительного применения мер противодействия:

```
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "example"
spec:
  kprobes:
    - call: "security_file_permission"           ❶
      syscall: false
      args:                                     ❷
        - index: 0
          type: "file"
        - index: 1
          type: "int" # 0x04 is MAY_READ, 0x02 is MAY_WRITE
      selectors:
        - matchArgs:                           ❸
            - index: 0
              operator: "Equal"
              values:
                - "/tmp/liz"
          matchActions:                          ❹
            - action: Sigkill
```

❶ Эта политика привязана к функции ядра с именем `security_file_permission()`, которая является частью LSM API и вызывается ядром каждый раз, когда процесс хочет получить доступ к файлу для чтения или записи. Она используется для определения, следует ли разрешить операцию.

❷ Эта функция принимает два аргумента. Первый — указатель на структуру данных в ядре, описывающей файл, а второй — значение, определяющее тип доступа к файлу — на чтение или запись.

❸ Политика определяет несколько селекторов, используемых для фильтрации событий. В этом случае Tetragon будет реагировать на событие, только если файл, указанный в первом параметре, имеет имя `/tmp/liz`.

❹ Если селекторы совпадают, то Tetragon отправляет сигнал SIGKILL, чтобы завершить процесс, предпринявший попытку доступа к файлу (а также сообщает о событии в пользовательское пространство).

Когда приложение, возможно скомпрометированное, пытается обратиться к этому файлу, запускается eBPF-программа из пакета Tetragon. Выполняясь в пространстве ядра, код eBPF сравнивает событие с политикой и, если все условия выполняются, посылает сигнал SIGKILL. Это гарантирует, что действие, выходящее за рамки политики, никогда не будет выполнено. Эту последовательность событий иллюстрирует рис. 15.3.



Рис. 15.3. Tetragon может обеспечивать соблюдение политик безопасности в среде выполнения, синхронно отправляя сигнал SIGKILL из ядра

Tetragon также генерирует метрики в формате Prometheus (<https://tetragon.io/docs/installation/metrics>), которые можно визуализировать с помощью таких инструментов, как Grafana или Splunk, чтобы легче было определить, возникли ли проблемы и когда, а также выявить рабочие нагрузки, вызвавшие нарушение политик.

Мой работодатель, компания Isovalent (ныне входящая в состав Cisco), предоставляет корпоративную версию Tetragon.

Tracee

Tracee (<https://github.com/aquasecurity/tracee>) — еще один проект с открытым исходным кодом на основе eBPF от моего бывшего работодателя, компании Aqua Security. Как и Falco, он обеспечивает обнаружение событий, но без принудительного блокирования и, так же как и Tetragon, реализует некоторую фильтрацию на уровне ядра, хотя и с менее широкими возможностями.

Inspektor Gadget

Inspektor Gadget — еще один проект CNCF, предоставляющий набор гаджетов на основе eBPF для наблюдения за рабочими нагрузками Kubernetes и их отладки. Он не обладает возможностями обнаружения или принудительного применения мер противодействия, как Tetragon или Falco, но имеет ряд

полезных гаджетов, которые можно использовать для повышения безопасности, включая:

- мониторинг выполнения;
- мониторинг привилегий;
- трассировку сети;
- аудит доступа к файлам;
- трассировку системных вызовов.

Гаджеты допускается подключить к работающему поду Kubernetes, чтобы выяснить, к каким файлам осуществляется доступ или какие исполняемые файлы запускаются, что может быть полезно при расследовании инцидентов в режиме реального времени. Гаджеты также можно использовать для формирования базового профиля об ожидаемом поведении рабочей нагрузки и на его основе разработать политику Tetragon, правило Falco или политику seccomp.

Предотвращение или оповещение

При использовании любого инструмента для обеспечения безопасности в среде выполнения возникает один важный вопрос: какие действия должен предпринять этот инструмент при обнаружении аномального поведения? Что лучше: завершить процесс или просто уведомить пользователя о событии и позволить ему или ИИ принять решение? Это непростой вопрос, и ответ во многом зависит от особенностей рабочих нагрузок.

- Если в случае предупреждения автоматически останавливать процесс или удалять контейнер, то не повлияет ли это на работу сервиса? А как быть, если обнаружится несколько таких экземпляров? А что, если это ложная тревога?
- Если за создание нового экземпляра отвечает оркестратор, как быть, если вновь запущенный экземпляр подвергнется той же атаке? Может возникнуть порочный круг, в котором запускается контейнер, обнаруживается неправильное поведение, инструмент безопасности прерывает выполнение контейнера, после чего оркестратор запускает его заново (вспомните, как Kubernetes создает/уничтожает поды, чтобы обеспечить нужное количество реплик).
- Если версия контейнера новая, то можно ли откатиться к предыдущей?

Не существует единственно правильного ответа, когда речь идет об автоматической обработке предупреждений безопасности. Какая бы задержка ни потребовалась, злоумышленнику *может* хватить времени, чтобы нанести вред. Представьте, что программа eBPF сгенерировала событие, указывающее на попытку получения доступа к конфиденциальному файлу. Благодаря

синхронному принудительному применению мер противодействия Tetragon способен пресечь эту попытку до того, как она произойдет, но при использовании других инструментов вам придется ждать, пока компонент пользовательского пространства завершит процесс-нарушитель, и за это время часть конфиденциальных данных может быть уже похищена. Этот риск еще выше, если перед принятием мер требуется консультация с человеком. В данном случае профилактика лучше лечения.

Если используемые вами инструменты безопасности действительно способны предотвратить нежелательное поведение внутри контейнера, то есть надежда, что контейнер сможет продолжить работу. Представьте, что злоумышленник взломал контейнер поиска товаров и пытается запустить там программу для майнинга криптовалюты. Этот исполняемый файл не входит в профиль допустимого поведения, поэтому утилита обеспечения безопасности в среде выполнения просто не позволит его запустить. «Хорошие» процессы продолжают работать как ни в чем не бывало, но атака с майнингом криптовалюты будет предотвращена.

Вполне вероятно, что в ответ на разные типы потенциально вредоносной активности понадобятся разные варианты реагирования, поскольку последствия предотвращения такой активности могут различаться. Например, отбрасывание сетевого пакета в рамках обеспечения сетевой политики просто означает, что принимающее ПО его никогда не увидит и в результате пакет не сможет¹ вызвать ошибку в коде. С другой стороны, представьте инструмент, который предотвращает попытки доступа к файлам, возвращая процессам код ошибки. В идеале все приложения должны уметь корректно обрабатывать подобные ошибки, но разработчики могли это не предусмотреть, из-за чего приложение аварийно завершает работу. Подобные ситуации быстро выявляются во время тестирования, но данный конкретный участок кода мог остаться непротестированным, так как выполняется в очень редких случаях. Учитывая это, можно положиться на журналирование большинства попыток обращений к файлам и блокировать доступ только к наиболее чувствительным файлам, где возможный сбой приложения будет иметь меньшие последствия, чем утечка информации.

Когда возникают подозрительные или злонамеренные события, приятно знать, что они предотвращаются, но еще лучше, если вы сможете определить, была ли атака преднамеренной и как она произошла. Чтобы исследовать инцидент, понадобятся данные, предшествовавшие подозрительной активности. Инструменты eBPF, с которыми вы познакомились в этой главе, могут

¹ «Не сможет» — слишком сильно сказано. Теоретически может существовать плохо написанный код, который вызовет сбой, если не получит сообщение в течение определенного промежутка времени, но я надеюсь, что подобные ошибки будут исправлены до того, как код попадет в рабочую среду.

оказаться бесценными источниками подобной информации (как, например, журнал запуска программ и доступа к файлам в Tetragon или Tracее, работающих в режиме журналирования). Стоит также рассмотреть возможность не завершать подозрительный контейнер, а изолировать его.

Карантин контейнеров

Вместо полного уничтожения подозрительного контейнера можно приостановить его работу, чтобы сохранить информацию о состоянии, которая может пригодиться для анализа. Вот как это сделать с помощью Docker:

- команда `docker pause` приостанавливает работу контейнера и все его процессы, но не завершает их принудительно;
- при желании можно выполнить команду `docker commit`, чтобы создать копию файловой системы контейнера для последующего анализа;
- если в контейнер смонтированы тома, то также может потребоваться сделать снимок этих томов.

Kubernetes не имеет встроенного механизма приостановки подов, однако имеются другие методы:

- изолировать под с помощью сетевой политики;
- для проведения исследований допускается подключиться к поду через команду `kubectl debug`;
- можно создать контрольные точки для анализа контейнеров (<https://oreil.ly/tkxQG>) с использованием Kubelet Checkpoint API (https://oreil.ly/_pCHd; на момент написания книги была доступна только бета-версия);
- пометить узел, на котором запущен подозрительный под, специальной меткой, чтобы новые поды не размещались на этом узле.

Если есть основания полагать, что контейнер скомпрометирован, разумно как минимум приостановить или изолировать его, чтобы у злоумышленника было меньше возможностей для кражи данных или влияния на другие рабочие нагрузки.

Меры по снижению уязвимости

Новые уязвимости ПО обнаруживаются постоянно, и, даже если исправление выпускается практически сразу, может потребоваться некоторое время для сборки, тестирования и развертывания образов контейнеров. В этот период очень полезно отслеживать, использовалась ли уязвимость в вашем

развертывании, и даже предотвращать ее эксплуатацию. Инструменты безопасности на основе eBPF, такие как Tetragon, делают это возможным.

Ключевой в этом случае является способность eBPF динамически загружать программы и конфигурацию в ядро. Это означает, что можно развернуть новую политику во всей системе и она немедленно начнет защищать все рабочие нагрузки (контейнерные и не только).

Определение политики зависит от уязвимости. В качестве примера рассмотрим бэкдор xzutils 2024 года (<https://oreil.ly/KTNou>) — уязвимость в библиотеке `liblzma`, которая могла быть использована для удаленного выполнения кода через SSH. Чтобы демон SSH запустил код (уязвимой) библиотеки, его необходимо загрузить в память. Следующая политика Tetragon сгенерирует вывод, если обнаружит уязвимую версию библиотеки, к которой обращается демон SSH:

```
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "cve-2024-3094-xz-ssh"
spec:
  kprobes:
    - call: "security_mmap_file"                                ❶
  ...
  args:
    - index: 0                                                  ❷
      type: "file"
  ...
  selectors:
    - matchBinaries:                                           ❸
      - operator: "In"
        values:
          - "/usr/sbin/sshd"
    matchArgs:
      - index: 0                                                ❹
        values:
          - "liblzma.so.5.6.0"
          - "liblzma.so.5.6.1"
    matchActions:
      - action: Post                                           ❺
        rateLimit: "1m"
```

❶ Программа eBPF, управляющая этой политикой, привязана к функции `security_mmap_file()`, которая является частью API модуля безопасности Linux (Linux Security Module, LSM). Вы видели некоторые возможности LSM в главе 10, тем не менее напомним, что с помощью программ eBPF, привязанных к этому API, можно создавать динамические политики. Эта функция дает LSM — или программе eBPF — возможность одобрить или отклонить отображение файла в память.

- ❷ В первом аргументе этой функции передается файл, загружаемый в память.
- ❸ Селектор `matchBinaries` сообщает инструменту Tetragon, что событие представляет интерес, только если процесс, выполняющий загрузку в память, — это `/usr/sbin/sshd`, то есть демон SSH.
- ❹ Это событие представляет интерес, только если первый аргумент — файл одной из уязвимых версий библиотеки.
- ❺ Если событие соответствует всем условиям селектора, то Tetragon сгенерирует отчет о нем, ограничив количество событий одним в минуту, чтобы не перегружать данными тех, кто будет заниматься расследованием. (При желании Tetragon также может не только сообщить о попытке доступа, но и заблокировать ее.)

Поскольку фильтрация (сопоставление двоичных файлов и имен файлов) выполняется программами eBPF непосредственно в ядре, такая политика практически не влияет на производительность.



Демонстрацию этого приема можно увидеть в записи моего доклада на конференции ContainerDays (<https://www.youtube.com/watch?v=m50WKCPjXuI>).

Резюме

Средства наблюдаемости и обеспечения безопасности в среде выполнения значительно продвинулись со времени выхода первого издания книги, и во многом благодаря развитию eBPF. Возможность обнаружения событий в среде выполнения на очень детальном уровне, как описано в этой главе, делает специализированные утилиты по обеспечению безопасности контейнеров весьма перспективными, особенно если учесть, что это можно делать прозрачно для приложений и с минимальным влиянием на производительность.

Мы приближаемся к концу книги. В последней главе рассмотрим десять главных рисков для безопасности — список, составленный OWASP, — и обсудим уменьшение последствий от них применительно к развертыванию контейнеров.

Контейнеры и десять главных рисков по версии OWASP

Если вы занимаетесь безопасностью, то почти наверняка слышали об OWASP (<https://owasp.org/>) — открытом проекте обеспечения безопасности веб-приложений (Open Web Application Security Project); возможно, вы даже состоите в их сообществе. Эта организация периодически публикует список десяти главных рисков для безопасности веб-приложений.

И хотя не все приложения, контейнеризированные или нет, относятся к веб-приложениям, данный источник прекрасно подходит для понимания того, каких атак следует опасаться в первую очередь. Вдобавок на сайте OWASP вы найдете прекрасные пояснения к перечисленным атакам и советы по их предотвращению. В этой главе я расскажу о десяти текущих главных рисках (<https://owasp.org/www-project-top-ten>)¹, имеющих отношение к безопасности контейнеров.

Ошибки в управлении доступом

Эта категория относится к злоупотреблению излишними привилегиями, предоставленными пользователям или компонентам. Существует несколько специальных методик, позволяющих применять принцип минимума привилегий к контейнерам, как обсуждалось в главе 11:

- не запускайте контейнеры от имени суперпользователя;
- ограничивайте предоставляемые контейнерам привилегии;
- в Kubernetes для ограничения прав доступа используйте RBAC;
- по возможности задействуйте контейнеры, не требующие привилегий суперпользователя.

¹ На момент подготовки русскоязычного издания список OWASP Top 10 был обновлен: <https://owasp.org/Top10/2025/>. — *Примеч. ред.*

Можно также использовать инструменты обеспечения безопасности в среде выполнения, как обсуждалось в главе 15, чтобы ограничить круг действий, доступных контейнеру. Указанные подходы позволяют ограничить масштаб возможного ущерба при атаке, однако ни один из них не относится к привилегиям пользователей *на прикладном уровне*, поэтому все равно нужно следовать всем тем же советам, действующим и при развертывании обычных систем.

Недостаточно надежное шифрование

Чрезвычайно важно надежно защищать любые личные, финансовые и прочие чувствительные данные, к которым ваше приложение имеет доступ. Независимо, используется контейнеризация или нет, конфиденциальная информация должна всегда храниться и передаваться в зашифрованном с помощью криптостойкого алгоритма виде. По мере роста вычислительных возможностей старые алгоритмы начинают поддаваться взлому путем прямого подбора, а значит, перестают считаться надежными. Кроме того, с увеличением эффективности квантовых вычислений возникает опасение, что некоторые широко применяемые алгоритмы, такие как RSA и криптография на основе эллиптических кривых, станут бесполезными. Обязательно следите за развитием технологий и используйте актуальные алгоритмы, рекомендованные авторитетными организациями, такими как NIST или Национальный центр кибербезопасности Великобритании (National Cyber Security Centre, NCSC).

Но, поскольку конфиденциальные данные зашифрованы, приложению понадобятся учетные данные для доступа к ним. Следуя принципам минимума привилегий и разделения обязанностей, предоставляйте доступ к учетным данным только тем контейнерам, которым они действительно нужны. Безопасная передача секретных данных в контейнеры рассматривалась в главе 14.

В зависимости от конкретного сценария использования вам может потребоваться обеспечить шифрование сетевого трафика между контейнерами, как обсуждалось в главе 13.

Не забывайте сканировать образы контейнеров на предмет ключей шифрования, паролей и прочих конфиденциальных данных.

Инъекции кода

Если в вашем коде есть «дыра», открывающая путь для инъекций, то злоумышленник может воспользоваться ею, чтобы выполнить команды, замаскированные под данные. Лучшее всего этот сценарий иллюстрирует нетленный персонаж xkcd — Little Bobby Tables (<https://xkcd.com/327>).

Ничего относящегося исключительно к контейнерам здесь нет, хотя сканирование образов контейнеров позволяет обнаружить в зависимостях известные

уязвимости подобного типа. Анализируйте и тестируйте код своих приложений, как советует OWASP.

Небезопасный дизайн

Если проект системы имеет недостатки с точки зрения безопасности, то даже идеальная ее реализация будет содержать уязвимости. Контейнерные системы, построенные на основе микросервисов, безусловно, имеют преимущество перед монолитными, потому что разбивают систему на составляющие компоненты, а анализ моделей угроз в отдельных небольших блоках несравненно проще. Если вы используете широко распространенные инструменты, такие как Kubernetes, то также должны воспользоваться последними рекомендациями по обеспечению безопасности (например, реализовать принцип нулевого доверия между контейнерами и обеспечить доступ по схеме минимальных привилегий).

Неправильные настройки безопасности

В основе многих атак лежат ошибки, допущенные при настройке систем. В списке OWASP Top 10 приводятся такие примеры, как небезопасные или неполные настройки, общедоступные облачные хранилища и слишком подробные сообщения об ошибках, содержащие конфиденциальную информацию. Последствия всего этого применительно к контейнерам и развертываниям в облачной среде можно смягчить следующим образом.

- Используйте, например, контрольные показатели Центра интернет-безопасности (Center for Internet Security, CIS), чтобы оценить, настроена ли ваша система в соответствии с практическими рекомендациями. Там есть тесты для Docker и Kubernetes, а также для хостов под управлением Linux. Возможно, в вашей конкретной среде нет смысла следовать всем рекомендациям, но это прекрасная отправная точка для оценки системы. Многие меры обеспечения безопасности заложены в управляемые сервисы Kubernetes по умолчанию¹.
- Если вы используете общедоступный облачный сервис, то обязательно проверьте конфигурации на наличие таких проблем, как использование общедоступных хранилищ или неэффективных политик паролей. Gartner называет эти проверки управлением уровнем безопасности облака (Cloud Security Posture Management, CSPM). Поиск в Интернете по этому термину покажет множество инструментов для автоматизации таких проверок. Существуют также инструменты с открытым исходным кодом вроде Cloud Custodian (<https://cloudcustodian.io>), Prowler (<https://github.com/prowler-cloud/prowler>) и CloudSploit (<https://cloudsploit.com>).

¹ CIS-рекомендации нельзя применять слепо! На текущий момент последняя версия CIS Docker 1.8.0 рекомендует включать live restore. — *Примеч. науч. ред.*

- Как обсуждалось в главе 14, передача секретных данных через переменные окружения чревата их раскрытием в журналах, поэтому я не рекомендую использовать их для хранения конфиденциальной информации.
- Если вы используете публичный реестр контейнеров, например Docker Hub или Quay, убедитесь, что доступ к частным образам контролируется должным образом. Приватный реестр контейнеров может быть более безопасной альтернативой, хотя в любом случае доступ к образам должен быть защищен.
- Для ограничения сетевого трафика задействуйте сетевые политики, как обсуждалось в главе 12.

Стоит также обратить внимание на информацию о конфигурации, входящую в каждый образ контейнера. Этот вопрос мы обсуждали в главе 6 вместе с практическими рекомендациями по безопасному созданию образов.

Использование уязвимых или устаревших компонентов

Надеюсь, к этому времени вы уже догадываетесь, что я посоветую: применяйте сканер образов для поиска известных уязвимостей в образах контейнеров. Если вы создаете образы самостоятельно, то используйте минимальные базовые образы, чтобы уменьшить поверхность атаки.

Вам также пригодится процесс/набор инструментов:

- для повторной сборки образов контейнеров с включением в них актуальных версий пакетов;
- выявления и замены запущенных контейнеров, основанных на образах, содержащих уязвимости.

А в период, пока исправления применяются, используйте инструменты обеспечения безопасности в среде выполнения, которые могут смягчить последствия уязвимостей.

Взлом аутентификации и идентификации

К этой категории относится взлом аутентификации и похищение учетных данных. На прикладном уровне для контейнеризированных приложений действуют те же рекомендации, что и для монолитных в обычных системах, но здесь есть некоторые особенности, связанные именно с контейнерами:

- с учетными данными, необходимыми каждому контейнеру, следует обращаться как с секретными. Их нужно хранить и передавать в запущенные контейнеры со всеми мерами предосторожности, как обсуждалось в главе 14;

- разбиение приложения на множество контейнеризированных компонентов означает, что они должны как-то идентифицировать друг друга, обычно с помощью сертификатов, и взаимодействовать по защищенным соединениям. Эту задачу могут брать на себя сами контейнеры, а можно делегировать ее сервисной сетке или же применять прозрачное шифрование (см. главу 13).

Ненадежность цепочки поставок

К этой категории относится недостаточная защищенность цепочки поставок. Как обсуждалось в главе 7, эта проблема решается использованием подписанных образов контейнеров, проверкой спецификаций ПО (Software Bill of Materials, SBOM) и защитой конвейеров сборки CI/CD.

В эту категорию также входит несоблюдение требований по проверке целостности данных. Одним из примеров является небезопасная десериализация, когда злоумышленник «подсовывает» приложению для интерпретации специальный объект, в результате чего оно предоставляет пользователю дополнительные привилегии или меняет свое поведение каким-либо образом. (Я была свидетелем подобной атаки в 2011 году в качестве клиента Citibank, когда у банка обнаружилась уязвимость (<https://oreil.ly/EsgO7>), которая позволяла авторизованному пользователю получить доступ к информации о счетах других клиентов, просто изменив URL.)

Как правило, обработка данных приложением не зависит от того, работает оно в контейнерах или нет.

Небезопасные журналирование и мониторинг

В отчете IBM «Стоимость утечки данных в 2025 году» (*Cost of a Data Breach Report 2025*, <https://www.ibm.com/reports/data-breach>) приводится ужасающая статистика: в среднем на выявление утечки уходит 181 день, а на ее устранение — еще 60. При должном уровне наблюдаемости и оповещений о нетипичном поведении этот срок можно значительно сократить.

Используйте централизованную систему журналирования, такую как Fluentd, ELK Stack или SIEM, или средства журналирования, предлагаемые вашим поставщиком облачных услуг, чтобы гарантировать сохранность журналов контейнеризированных приложений при удалении контейнера.

В рабочей среде следует фиксировать события, связанные с контейнерами:

- запуск и остановку контейнера, включая идентификаторы образа и пользователя;
- доступ к секретным данным;
- любые изменения привилегий;

- вмешательство в содержимое контейнера (может указывать на инъекции кода, см. подраздел «Неизменяемые контейнеры» в главе 8);
- входящие и исходящие сетевые соединения;
- монтирование томов (для анализа смонтированных томов, в которых могут содержаться конфиденциальные данные, как рассказывается в разделе «Монтирование каталогов с конфиденциальными данными» главы 11);
- действия, завершившиеся неудачей, например попытки открыть сетевые соединения, записать данные в файлы или изменить права доступа пользователя, поскольку они могут указывать на то, что злоумышленник проводит разведку системы (см. главу 15, где рассказывается об инструментах обеспечения безопасности в среде выполнения).

Наиболее серьезные коммерческие инструменты обеспечения безопасности контейнеров интегрируются с корпоративными системами SIEM (security information and event management, управление информацией и событиями, относящимися к безопасности). Эти инструменты способны не только отслеживать атаки и сообщать о них постфактум, но и предотвращать их на основе профилей времени выполнения, как обсуждалось в главе 15.

Подделка запросов на стороне сервера

На мой взгляд, в списке OWASP Top 10, посвященном контейнерам, эта категория могла бы занять более высокое место из-за рисков получения контроля над системой через принуждение приложения отправлять запросы внутренним сервисам метаданных или к API плоскости управления. Например, злоумышленник может использовать уязвимость SSRF в приложении для отправки запросов в Kubernetes API, сервисы метаданных поставщика облачных услуг или базу данных, доступную внутри развертывания. Эти сервисы должны быть доступны только доверенным компонентам. Снизить вероятность подобных атак можно с помощью сетевых политик (см. главу 12) и мониторинга в среде выполнения (см. главу 15).

Резюме

Список десяти главных рисков по версии OWASP — полезный источник информации, позволяющий повысить защиту подключенных к Интернету приложений от наиболее распространенных типов атак.

Безопасность должна быть заложена в код приложения и в инфраструктуру, в которой работают эти приложения. Я не могу обещать, что советы из книги обеспечат абсолютную безопасность, но их применение поможет повысить общий уровень защищенности вашей системы.

Заключение

Поздравляю, вы дочитали книгу до конца!

Прежде всего надеюсь, что у вас сформировалось ясное представление о контейнерах. Это пригодится вам при обеспечении безопасности систем, развертываемых на их основе. Кроме того, вы теперь обладаете знаниями о доступных вариантах изоляции, если стандартные контейнеры не обеспечивают достаточного уровня разграничения рабочих нагрузок в вашей среде.

Я также надеюсь, что вам теперь понятно, как контейнеры взаимодействуют друг с другом и внешним миром. Передача данных по сети — обширная тема, но главное, что вы должны для себя уяснить: контейнеры являются объектом не только развертывания, но и безопасности. Существует множество способов ограничить трафик так, чтобы между контейнерами и внешней средой происходило лишь ожидаемое взаимодействие.

Вы познакомились с методами создания образов контейнеров и обнаружения известных уязвимостей или попыток изменения образов. Вы понимаете, почему важно рассматривать контейнеры как неизменяемые сущности, и видели, как инструменты обеспечения безопасности в среде выполнения могут выявлять и даже предотвращать подозрительную или вредоносную активность.

Думаю, вы осознали, насколько полезна многоуровневая защита в случаях несанкционированного доступа. Если даже злоумышленнику удалось воспользоваться уязвимостью в системе, на его пути возникнут другие преграды. Чем больше уровней защиты, тем меньше вероятность успешной атаки.

Как вы видели в главе 16, для контейнеров существуют специфические меры предотвращения угроз, применимые к наиболее распространенным атакам из списка OWASP. Однако этот список охватывает далеко не все уязвимости вашей системы. Теперь, когда вы почти дочитали книгу до конца, рекомендую вам освежить в памяти список векторов атак на контейнеры из раздела «Модель угроз для контейнеров» главы 1. В приложении вы также найдете список вопросов, которые позволят проанализировать, какие места вашей развернутой системы наиболее уязвимы и где следует усилить меры безопасности.

Надеюсь, информация, изложенная в книге, поможет вам защитить развертываемые вами системы при любых обстоятельствах. Если вы подверглись атаке, то независимо от ее исхода расскажите мне об этом. Я буду рада любой обратной связи: отзывам, комментариям, рассказам о подобных случаях — все это вы можете направлять на страничку <https://containersecurity.tech>. Меня также можно найти на LinkedIn (<https://linkedin.com/in/lizrice>), GitHub (<https://github.com/lizrice>) и в социальных сетях (<https://bsky.app/profile/lizrice.com>).

ПРИЛОЖЕНИЕ

Чек-лист по безопасности

Здесь перечислены важные вопросы, которые стоит себе задать при выборе оптимального способа обеспечения безопасности контейнеризированной системы. Возможно, в вашей среде не имеет смысла применять *все* эти рекомендации, но хотя бы задуматься о них вовсе не помешает. Разумеется, данный список отнюдь не исчерпывающий!

- Все ли ваши контейнеры запущены от имени непривилегированного пользователя? См. раздел «Выполнение контейнеров по умолчанию от имени суперпользователя» главы 11.
- Запускаете ли вы какие-нибудь контейнеры с флагом `--privileged`? Сбрасываете ли привилегии, которые не нужны для каждого образа? См. раздел «Флаг `--privileged` и привилегии» главы 11.
- Запускаете ли вы контейнеры в режиме «только для чтения», если есть такая возможность? См. подраздел «Неизменяемые контейнеры» в главе 8.
- Проверяете ли вы наличие конфиденциальных данных в каталогах, смонтированных с хоста? Установлен ли для них режим «только для чтения»? А как насчет сокета Docker? См. разделы «Монтирование каталогов с конфиденциальными данными» и «Монтирование сокета Docker» главы 11.
- Запускаете ли вы конвейер CI/CD в рабочем кластере? Имеет ли он повышенные привилегии или доступ к сокету Docker? См. подраздел «Опасности команды `docker build`» в главе 6.
- Создаете ли вы подписанные образы с использованием SBOM и аттестаций сборки? Проверяете ли подписи перед развертыванием контейнеров? См. главу 7, где рассказывается о безопасности цепочки поставок.
- Сканируете ли вы свои образы контейнеров на предмет уязвимостей? Налажен ли у вас процесс и есть ли инструментарий для повторной сборки и развертывания контейнеров на случай, если в базовом образе были найдены уязвимости? См. главу 8, где рассказывается об уязвимостях ПО в образах.

- Используете ли вы профили `seccomp`, `AppArmor` или `SELinux`? Для начала подойдут профили `Docker` по умолчанию; еще лучше — создавать «сжатый» профиль для каждого приложения. См. главу 10, где рассказывается об усилении изоляции контейнеров и раздел «Политики использования контейнерных образов в среде запуска» главы 15.
- Используете ли вы сетевые политики для ограничения трафика между компонентами? См. главу 12, где рассказывается о безопасности сети контейнеров.
- Шифруете ли вы трафик между компонентами? Шифрование можно реализовать прозрачно на уровне сети или с помощью сервисной сетки. См. разделы «Принцип нулевого доверия» и «Защищенные соединения между контейнерами» главы 13.
- Используете ли вы современные инструменты обеспечения безопасности в среде выполнения для защиты от несанкционированного доступа к файлам, программам, сетевому трафику или от повышения привилегий, а также для устранения известных уязвимостей? См. главу 15, где рассказывается об инструментах обеспечения безопасности в среде выполнения.
- Какой базовый образ вы используете? Есть ли у вас возможность воспользоваться пустым или `distroless`-образом? Можете ли вы сократить содержимое своих образов, чтобы уменьшить поверхность атаки? См. подраздел «Практические рекомендации по безопасности `Dockerfile`» в главе 7.
- Требуете ли вы использования неизменяемых контейнеров? Другими словами, есть ли у вас гарантии, что весь исполняемый код добавляется в образ контейнера во время сборки, а не во время выполнения? См. подраздел «Неизменяемые контейнеры» в главе 8.
- Установили ли вы ограничения на использование ресурсов для своих контейнеров? См. раздел «Контрольные группы для контейнеров» главы 3.
- Применяете ли вы контроль допуска, чтобы гарантировать, что в рабочей среде контейнеры будут создаваться только на основе проверенных образов? См. подраздел «Контроль допуска» в главе 7.
- Передаете ли вы секретные данные между контейнерами через временную файловую систему? Шифруются ли ваши секретные данные во время хранения и передачи? Используете ли вы систему управления секретами для их хранения и замены? См. главу 14, где рассказывается о передаче секретных данных в контейнеры.
- Используете ли вы хосты исключительно для выполнения контейнеров, отделяя их от остальных приложений? Установлены ли на операционных системах ваших хостов все последние выпуски обновлений безопасности? Рассмотрите возможность установки на них специальной ОС для хостов. См. раздел «Хосты и контейнеры» главы 4.

- Настроены ли ваши хосты и контейнеры в соответствии с передовыми методами обеспечения безопасности, такими как стандарты CIS для Linux, Docker и Kubernetes? См. раздел «Неправильные настройки безопасности» главы 16.
- Применяете ли вы подход GitOps? Если да, то соблюдаете ли рекомендации по обеспечению безопасности репозитория Git, в которых хранятся конфигурационные файлы? См. главу 9.
- Собираете ли вы журналы в удаленное централизованное хранилище для последующего аудита в случае утечки данных? Содержат ли эти журналы подробную информацию о том, как и когда выполнялись процессы? См. главу 15.

Все эти вопросы актуальны и для развертываний в Kubernetes, однако существуют и другие уязвимости, выходящие за рамки книги. Сервисы Kubernetes, управляемые поставщиками облачных услуг, предполагают также использование инфраструктуры контроля за идентификацией и доступом, которую следует тщательно настроить. В документации Kubernetes (<https://kubernetes.io/docs/concepts/security>) подробно рассматриваются вопросы безопасности. Хорошее общее представление дает матрица угроз Kubernetes, составленная Microsoft (<https://microsoft.github.io/Threat-Matrix-for-Kubernetes/about>), а AWS предлагает руководство по безопасности в EKS (<https://docs.aws.amazon.com/eks/latest/best-practices/security.html>), большая часть которого применима независимо от того, где размещен кластер Kubernetes. Также можно использовать инструмент CSPM для регулярной проверки параметров безопасности базовой облачной инфраструктуры.

Об авторе

Лиз Райс (Liz Rice) — главный специалист по открытым технологиям в компании Isovalent, специализирующейся на eBPF и сетевой безопасности и основавшей проект Cilium. Ныне Isovalent входит в состав Cisco. Лиз занимала множество должностей в Cloud Native Computing Foundation (CNCF): была членом управляющего совета, председателем комитета по техническому надзору и сопредседателем конференций KubeCon + CloudNativeCon. Она также является автором книги *Изучаем eBPF*¹ и первого издания «Безопасность контейнеров»². В свободное от работы время Лиз катается на велосипеде в местах с более приятной, чем в ее родном Лондоне, погодой, и пишет музыку под псевдонимом Insider Nine.

¹ Райс Л. Изучаем eBPF: программирование ядра Linux для улучшения безопасности, сетевых функций и наблюдаемости.

² Райс Л. Безопасность контейнеров. Фундаментальный подход к защите контейнеризированных приложений.

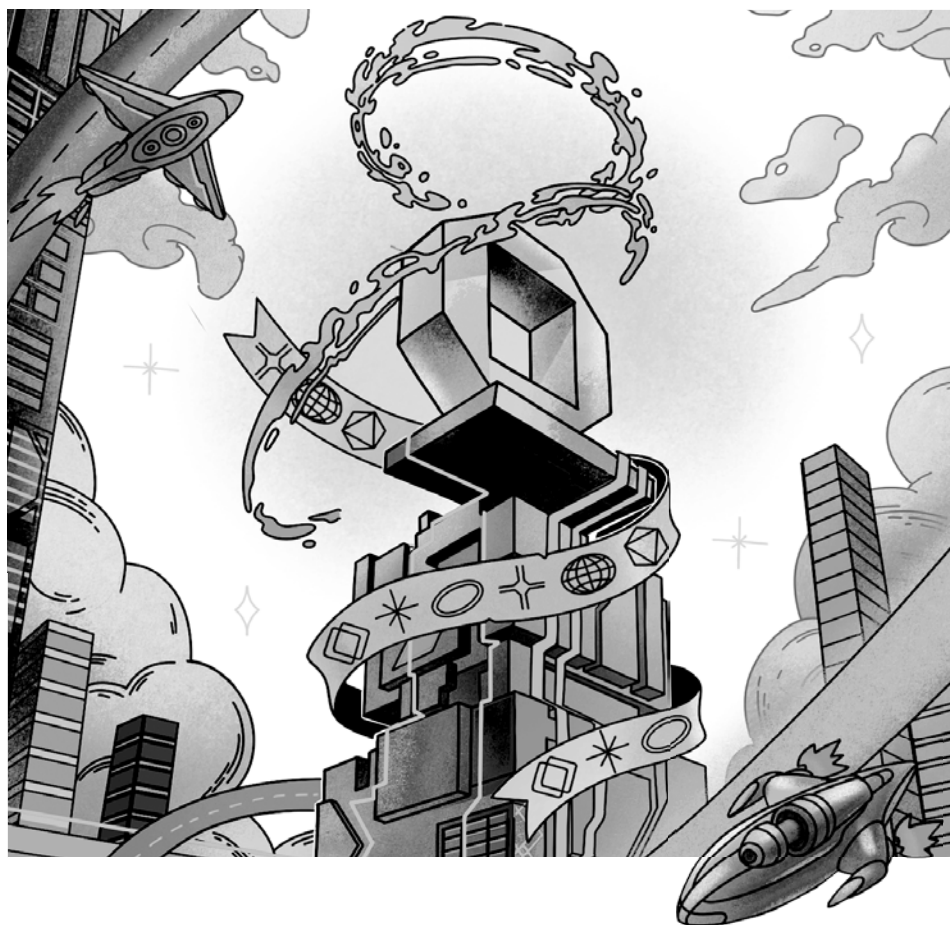
Иллюстрация на обложке

На обложке изображен представитель семейства кольчужных сомов, или локариевых (*Loricariidae*), также известных как сомики-нижнероты, или плеcostомусы, по названию вида *Hypostomus plecostomus*. Эти рыбы населяют пресноводные реки и ручьи Коста-Рики, Панамы и Южной Америки. Кольчужные сомы легко приспосабливаются к самым разным средам обитания: медленным и быстрым потокам, каналам, прудам, озерам, эстуариям и даже домашним аквариумам.

Существует более 680 видов кольчужных сомов разнообразных расцветок, форм и размеров. Их характерные особенности: гибкие костяные пластины, отличающие их от других сомов, сплюснутое тело и расположенный внизу рот-присоска, с помощью которого они питаются, дышат и прикрепляются к различным поверхностям. В зависимости от внешних условий их размер может составлять от десяти сантиметров до метра. Кольчужные сомы практически всеядны, в их рацион входят водоросли, беспозвоночные, мелкие двустворчатые моллюски, дафнии, черви, личинки насекомых и детрит; известен даже род (*Panaque*), питающийся деревом. Для локариевых характерна забота о потомстве: многие виды роют длинные норы вдоль берега, куда самки откладывают икринки. Самцы охраняют их до тех пор, пока не вылупятся мальки.

Кольчужные сомы ведут ночной оседлый образ жизни, но при попадании в новую среду склонны расселяться по территории и вытеснять местные популяции рыб. Помимо бронированного тела, кольчужные сомы в процессе эволюции развили пищеварительную систему, способную служить дополнительным органом дыхания. При необходимости они могут дышать атмосферным воздухом и существовать вне воды в течение более чем 20 часов!

Автор иллюстрации на обложке — Карен Монтгомери (Karen Montgomery), за основу взята черно-белая гравюра из «Зоологии» Джорджа Шоу (George Shaw).



Эту книгу мы выпустили вместе с Orion soft

Orion soft создает ПО для ИТ-инфраструктуры Enterprise-бизнеса. Главные принципы разработки, которым отвечает каждый продукт компании, — стабильность, простота использования и безопасность.

В продуктовую экосистему Orion soft входит zVirt — платформа виртуализации серверов, дисков и сетей с одной из крупнейших баз инсталляций в России.

Orion soft стремится постоянно развивать как свои компетенции, так и российское ИТ-сообщество, собирая и распространяя лучшие современные практики разработки.

Лиз Райс

**Безопасность контейнеров. Фундаментальный подход
к защите контейнеризированных приложений
2-е издание**

Перевел с английского А. Киселев

Научные редакторы С. Задорожный, С. Метелёв

Изготовитель: ТОО «Спринт Бук». Место нахождения и фактический адрес:
010000, Казахстан, город Астана, район Алматы, проспект Рахымжан Кошкарбаев, д. 10/1, н. п. 18.

Дата изготовления: 07.2026. Наименование: книжная продукция. Срок годности: не ограничен.

Подписано в печать 25.05.26. Формат 70×100/16. Бумага офсетная. Усл. п. л. 21,930. Тираж 500. Заказ 0000.